

Network Computing

(058172)



POLITECNICO
MILANO 1863

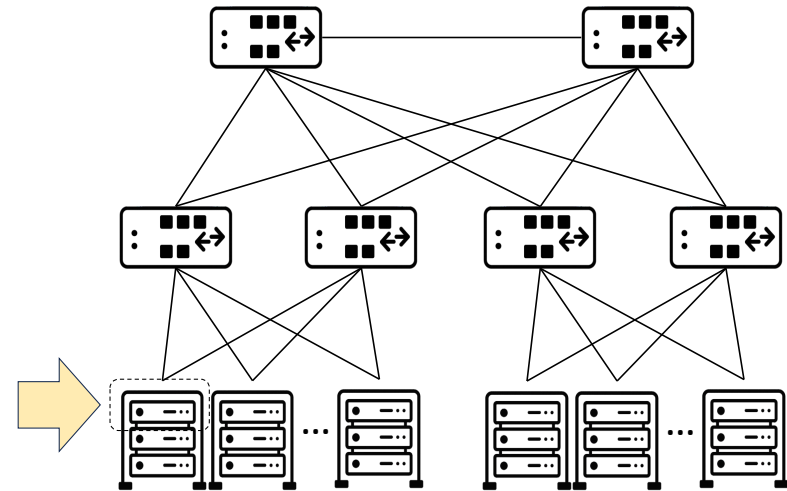
End-Host Networking

Content

- Life of a packet inside a server
- Kernel Bypass
- Programmable NICs
- Kernel programming (eBPF)

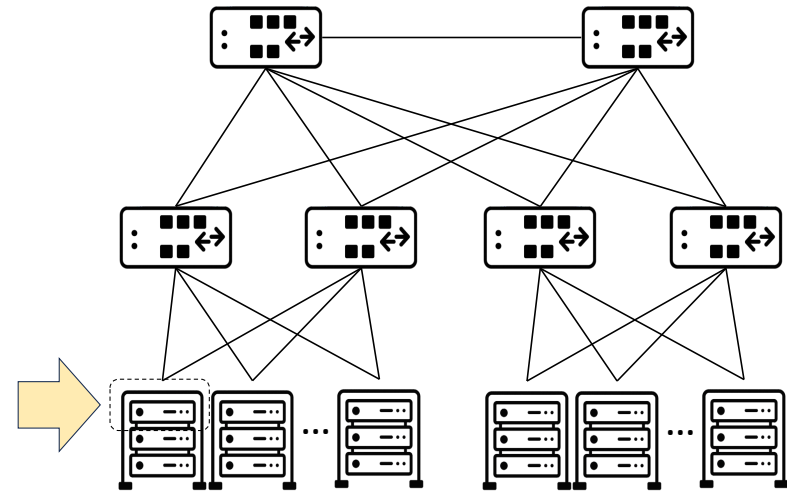
Today's lecture is about end-host!

- End-hosts (or servers) are located in racks
- This is where applications run and traffic is generated/received



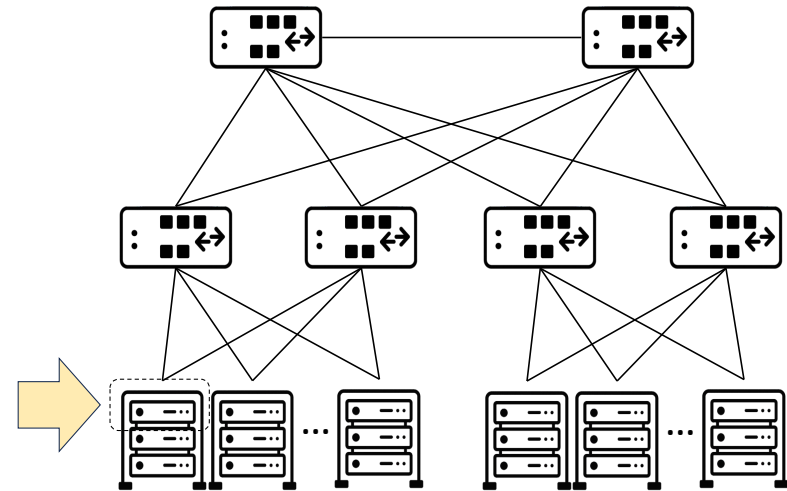
Today's lecture is about end-host!

- Lots of efforts in improving end-host stacks
- Deploy *#fancy_new_hardware* in-network is not always the best idea (from an operational standpoint)
 - New hardware == new problems
 - Deployments take time
 - Financial costs
 - Cabling
 - Many applications: who do you deploy the hardware for?



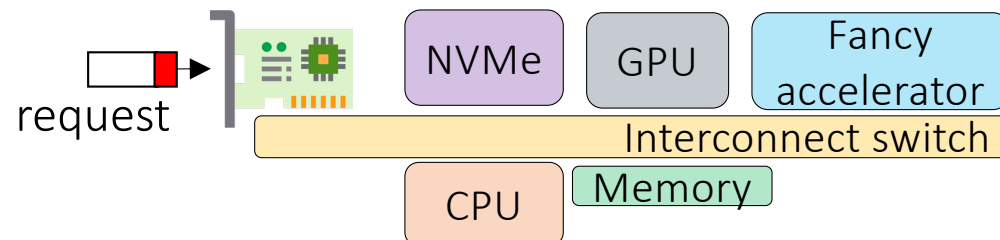
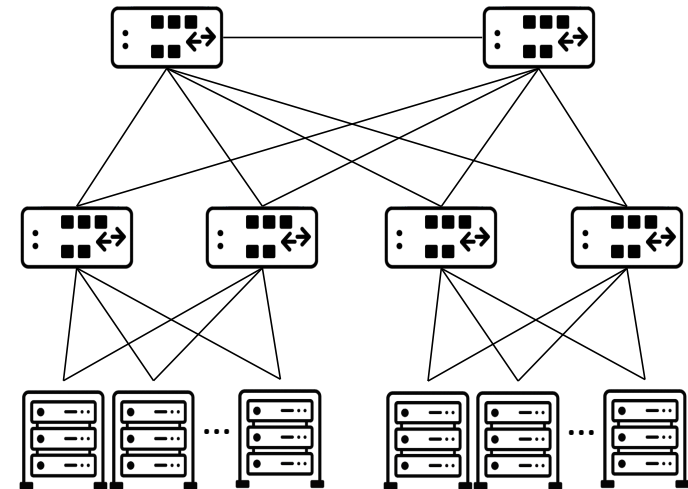
Today's lecture is about end-host!

- Deploy *#fancy_new_funcionality* at the end-host is easier
 - You are also closer to where applications are running
 - Less disruption on the network
 - More possibility to test new solutions



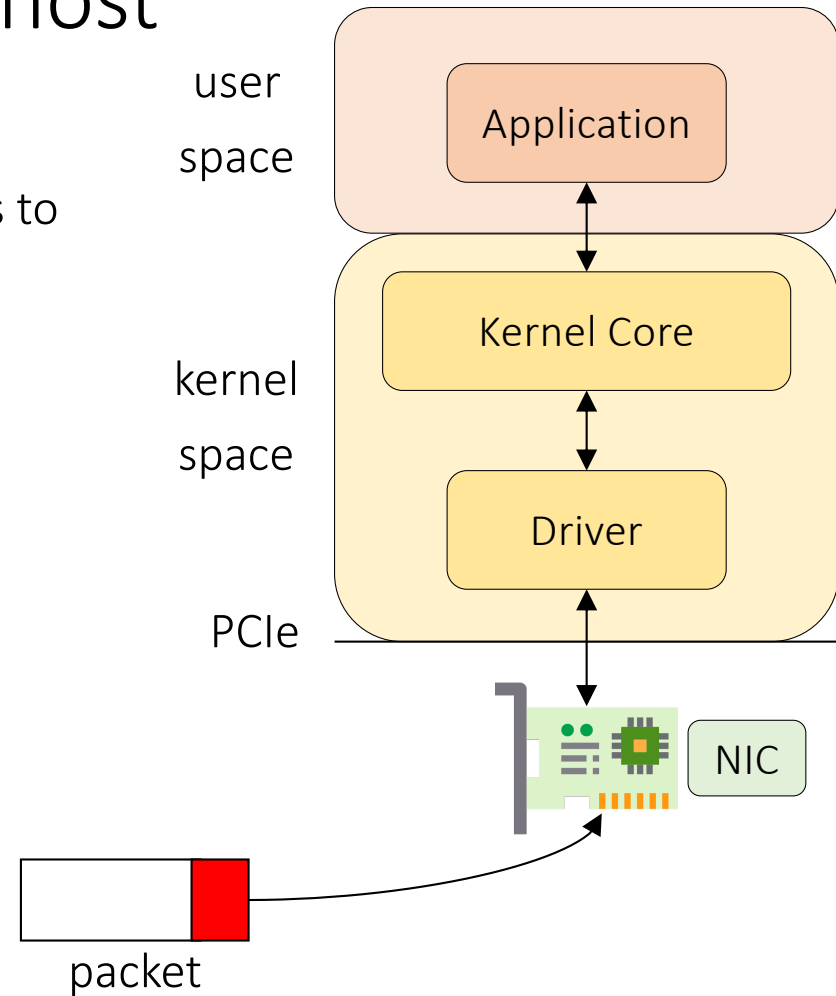
Today's lecture is about end-host!

- Deploy *#fancy_new_functionality* at the end-host is easier
 - You are also closer to where applications are running
 - Less disruption on the network
 - More possibility to test new solutions



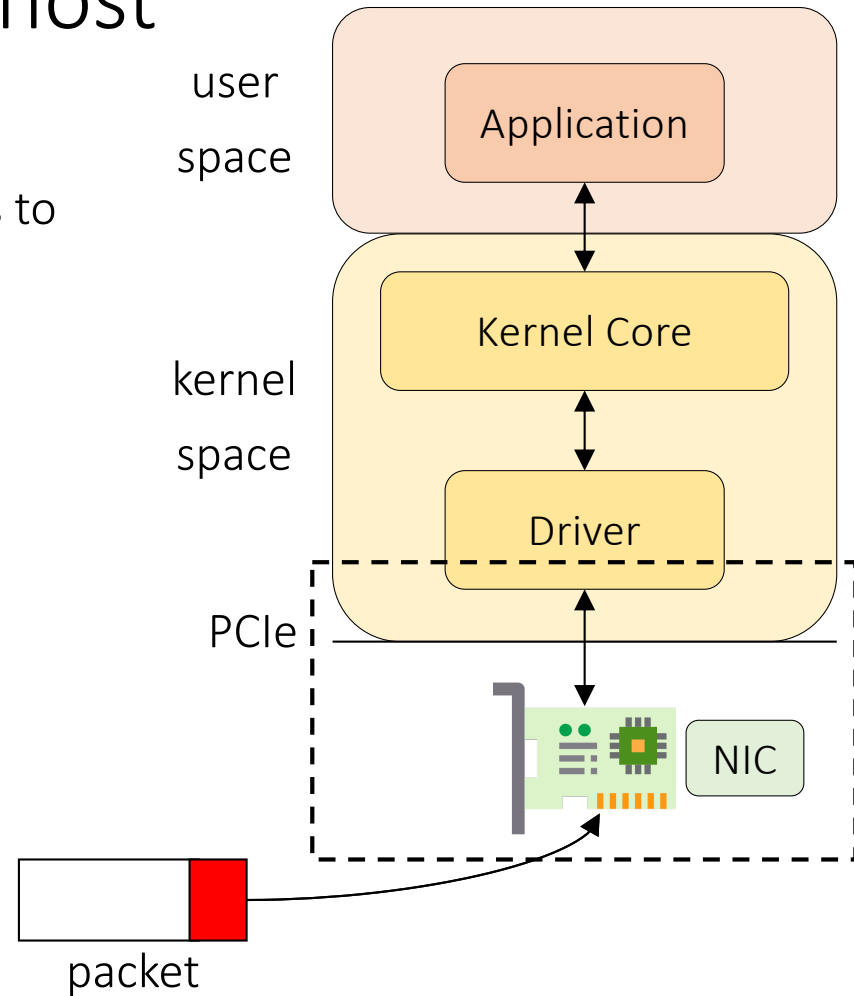
Life of a packet at the end-host

- From an high level perspective every packet has to cross:
 1. Network Interface Card (NIC)
 2. PCIe (interconnect between NIC and server)
 3. NIC driver
 4. Kernel
- ...to finally reach the application in user space



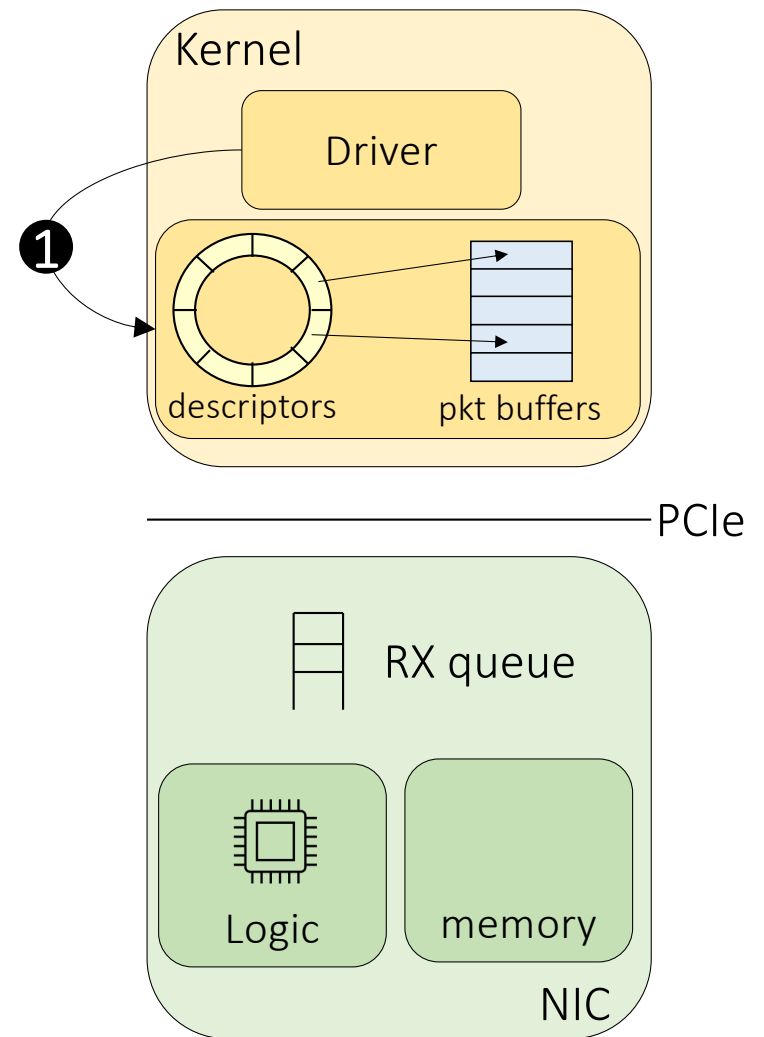
Life of a packet at the end-host

- From an high level perspective every packet has to cross:
 1. Network Interface Card (NIC)
 2. PCIe (interconnect between NIC and server)
 3. NIC driver
 4. Kernel
- ...to finally reach the application in user space



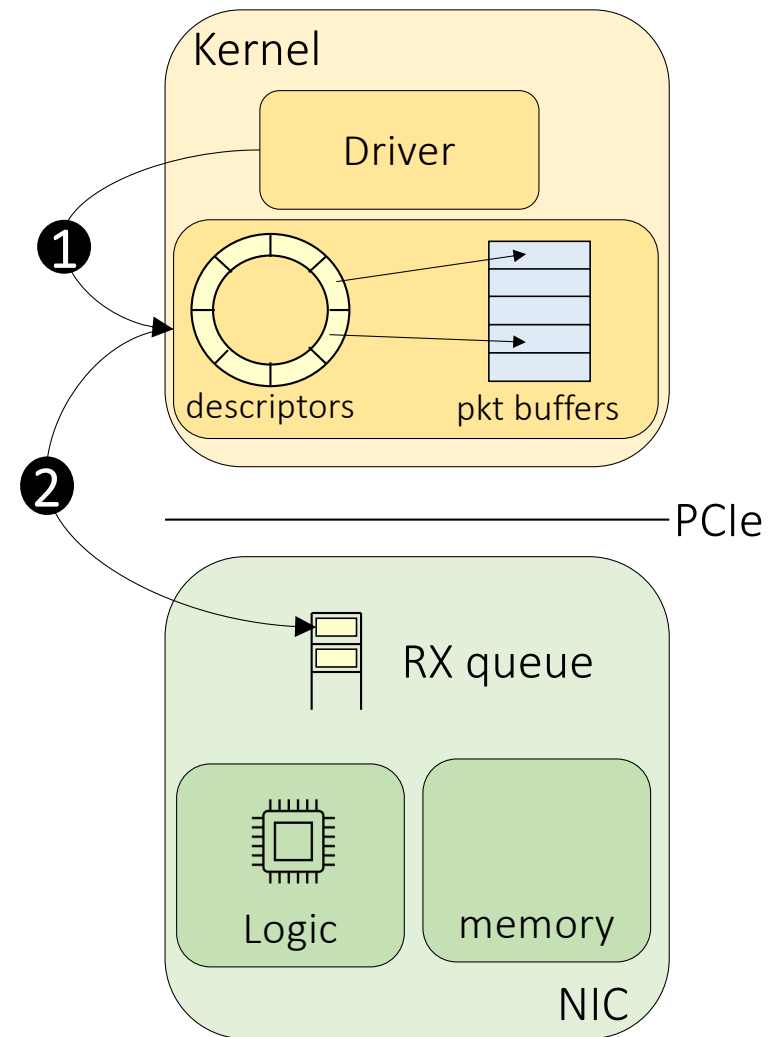
Life of a packet at the end-host

- The kernel allocates memory for storing the received packets from the NIC (*pkt buffers*)
- A descriptors ring is kept to store pointers to those buffers



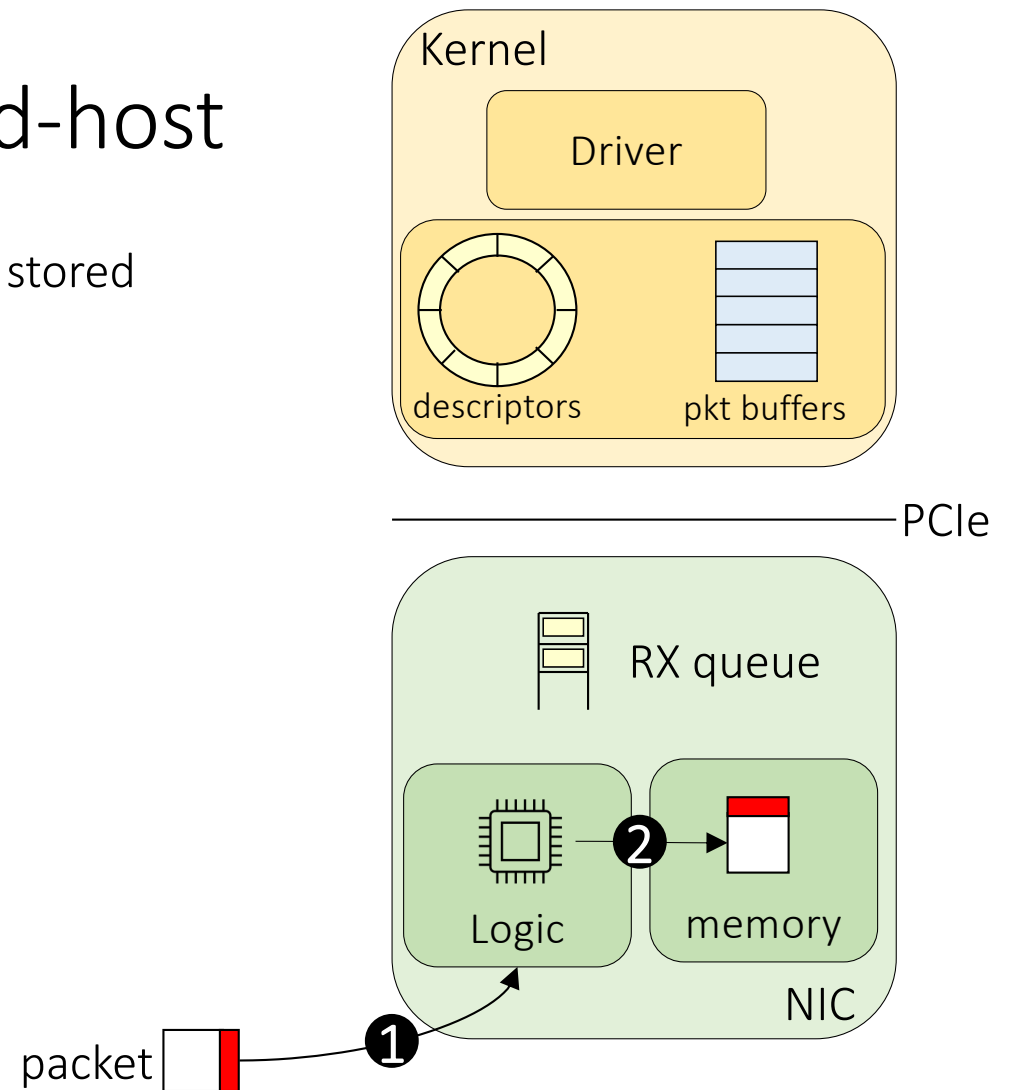
Life of a packet at the end-host

- The kernel allocates memory for storing the received packets from the NIC (*pkt buffers*)
- A descriptors ring is kept to store pointers to those buffers
- The driver populates the RX queue in the NIC with available descriptors (i.e., associated pkt buffers are empty)



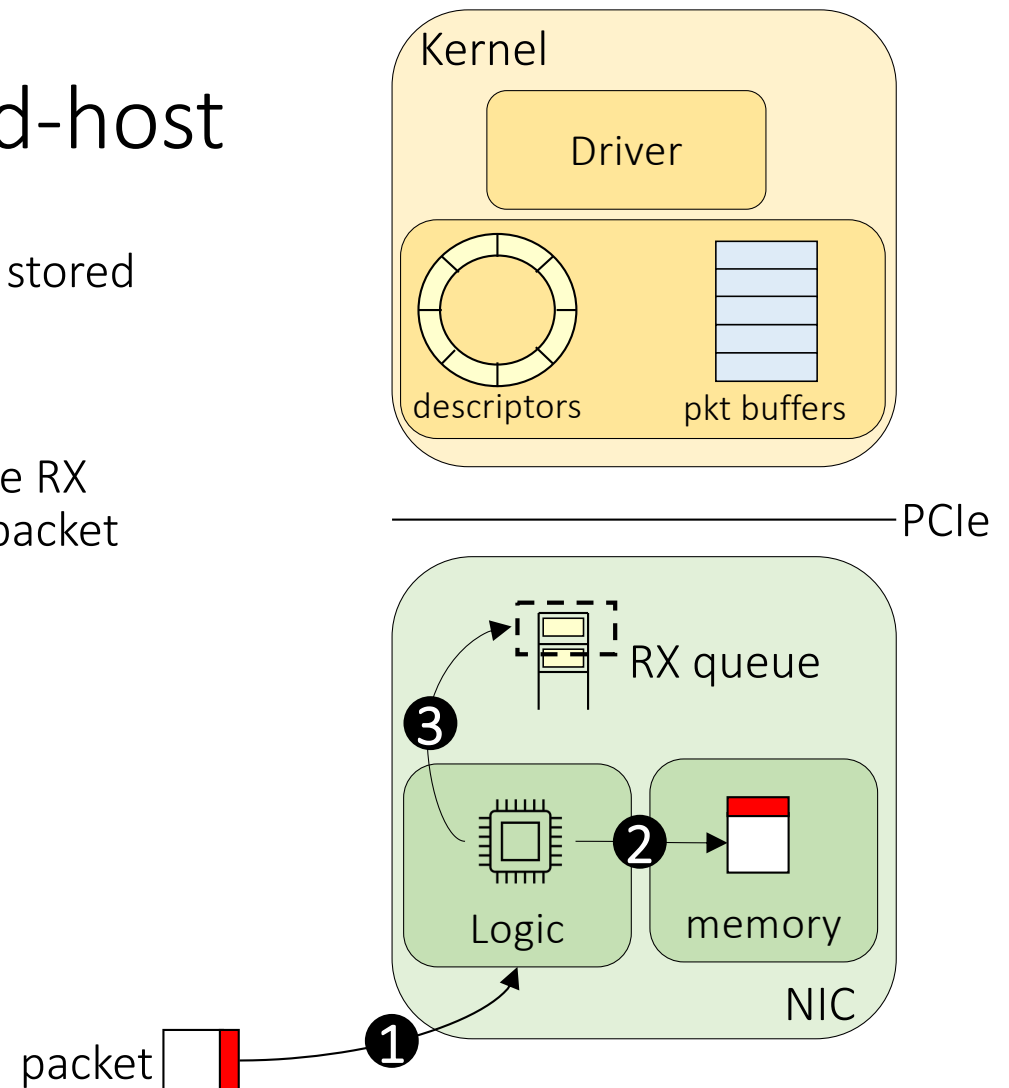
Life of a packet at the end-host

- When the packet arrives at the NIC, it is first stored in its local memory



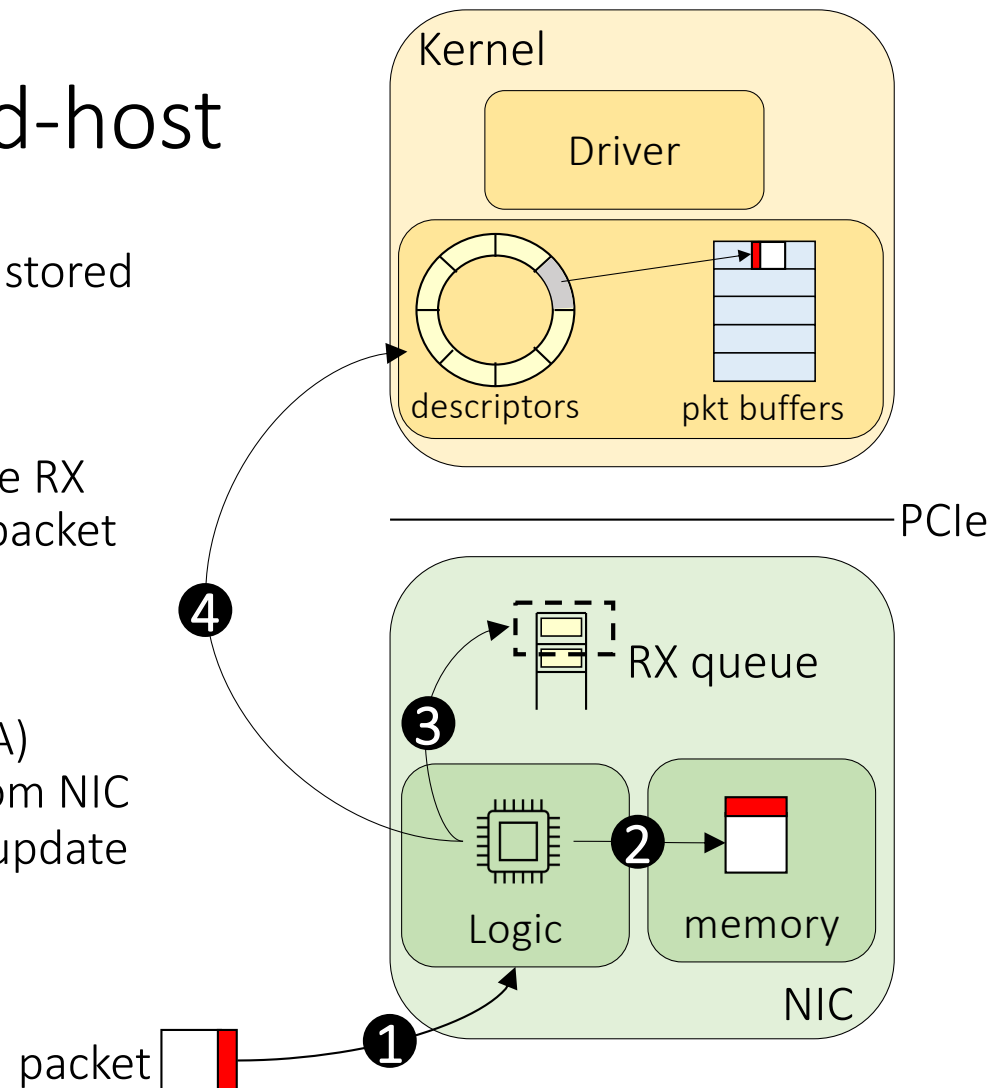
Life of a packet at the end-host

- When the packet arrives at the NIC, it is first stored in its local memory
- Then the NIC fetches one descriptor from the RX queue so it will know where to transfer the packet in the host memory



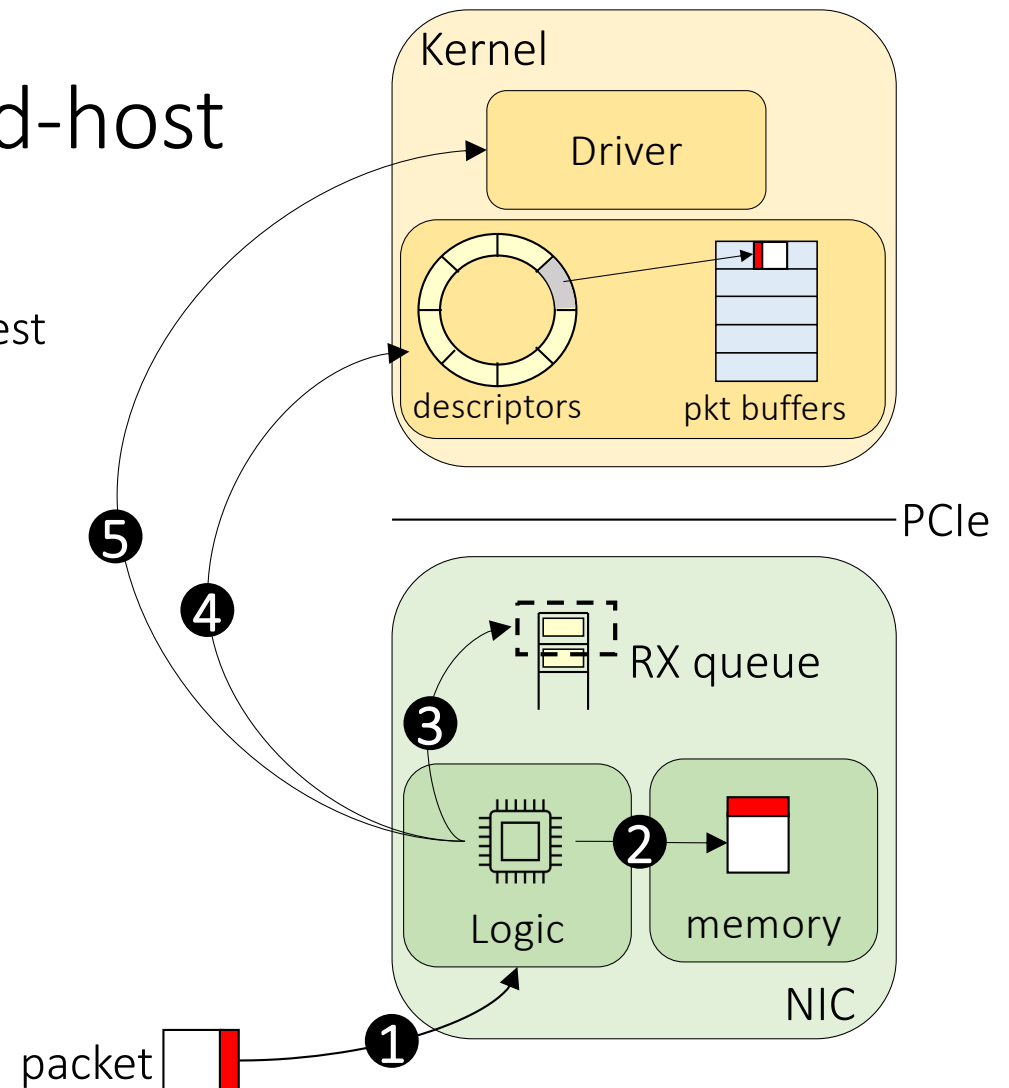
Life of a packet at the end-host

- When the packet arrives at the NIC, it is first stored in its local memory
- Then the NIC fetches one descriptor from the RX queue so it will know where to transfer the packet in the host memory
- The NIC starts a Direct Memory Access (DMA) transaction over PCIe to move the packet from NIC to host memory (NO CPU involvement) and update associated descriptor



Life of a packet at the end-host

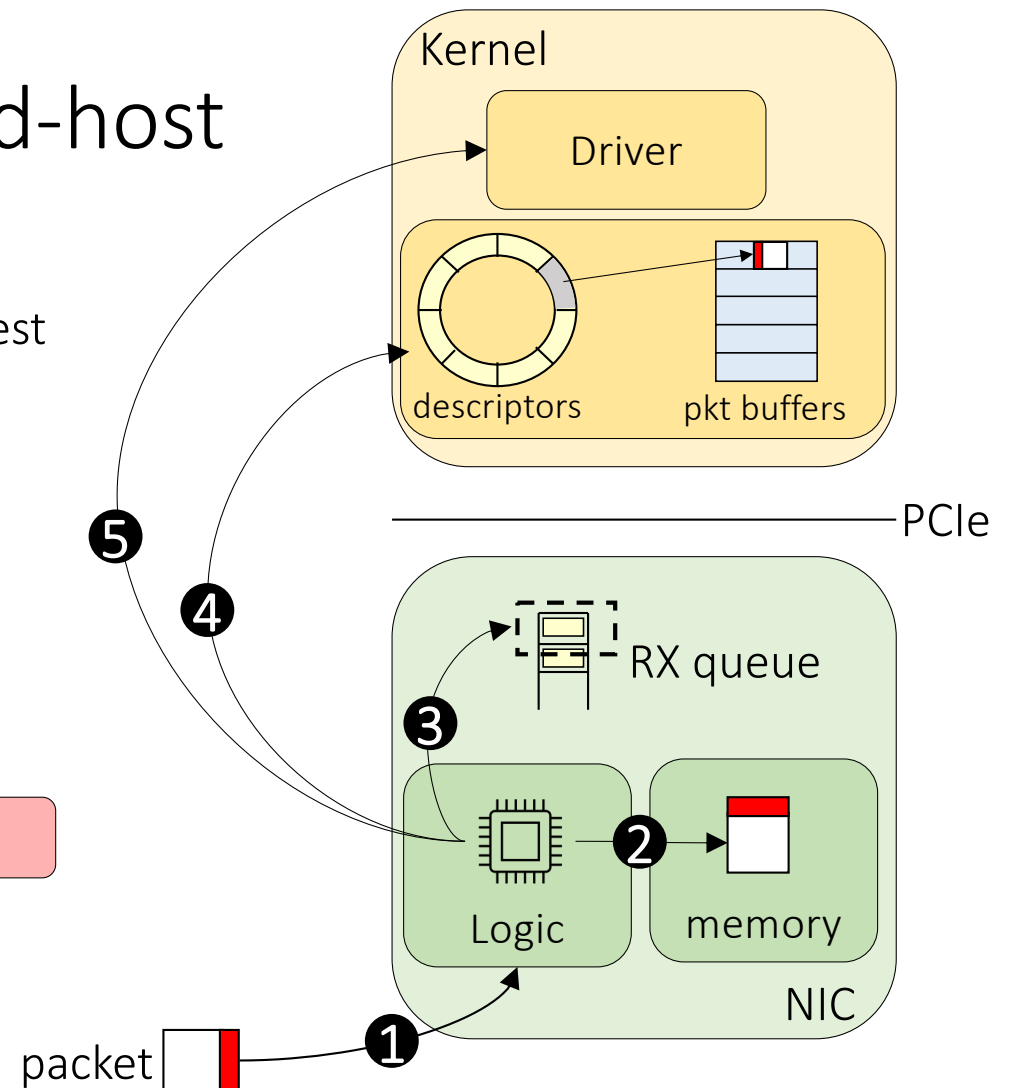
- Finally the NIC generates an Interrupt ReQuest (IRQ) to inform the driver of new data to be processed
- A CPU core will take the IRQ and the processing on the host starts



Life of a packet at the end-host

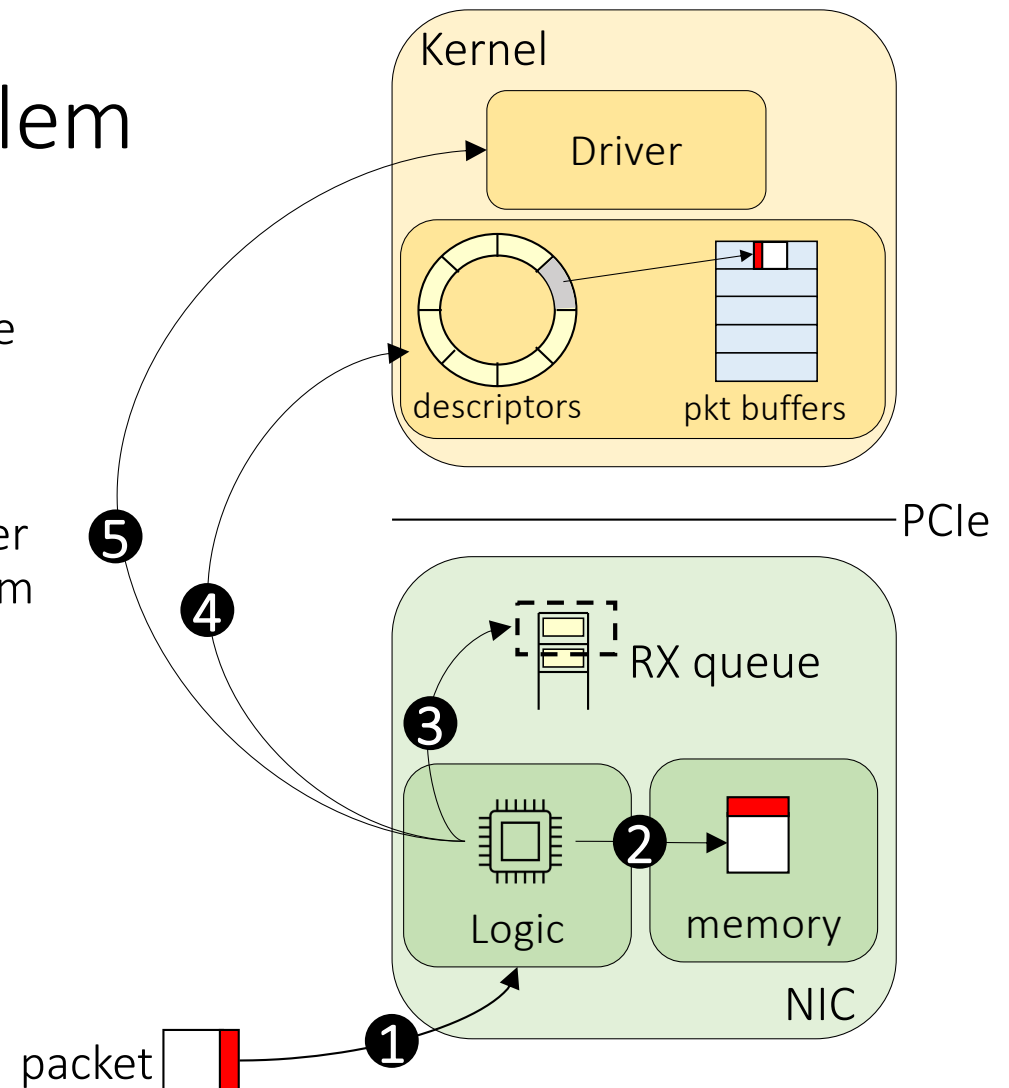
- Finally the NIC generates an Interrupt ReQuest (IRQ) to inform the driver of new data to be processed
- A CPU core will take the IRQ and the processing on the host starts

This is a very simplified view 😊



The receive livelock problem

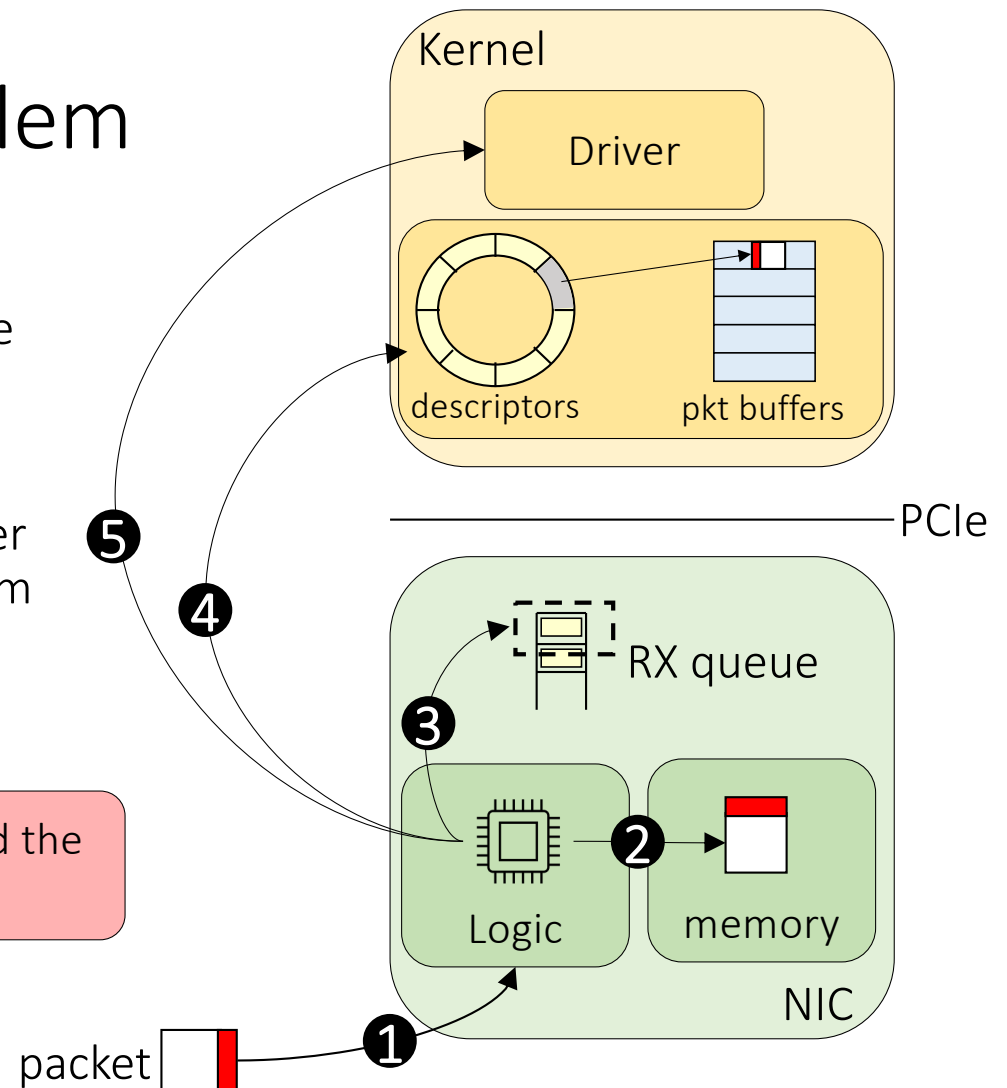
- Per-packet interrupts can cause performance degradation at high-rates
- Receiver interrupts take priority over all other activities. If packets arrive too fast, the system will spend all of its time processing receiver interrupts



The receive livelock problem

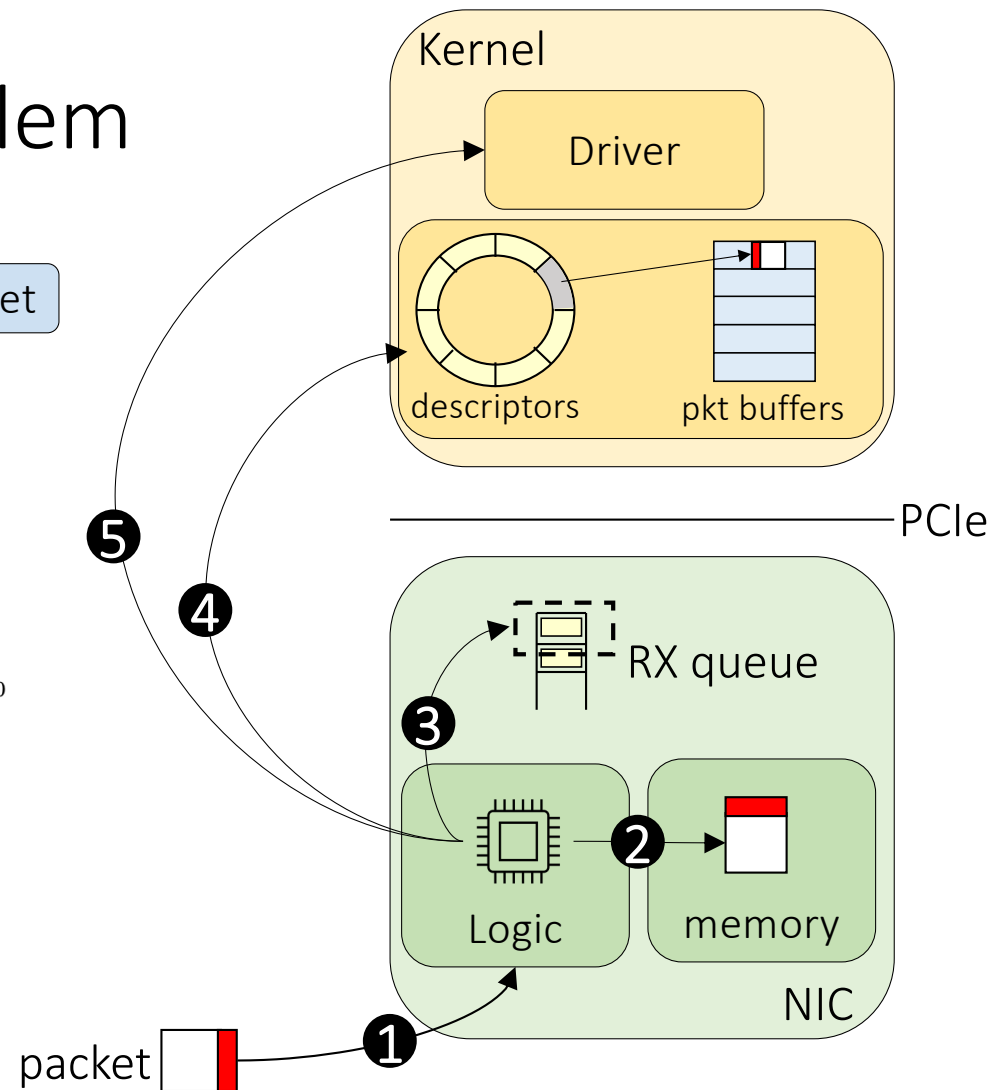
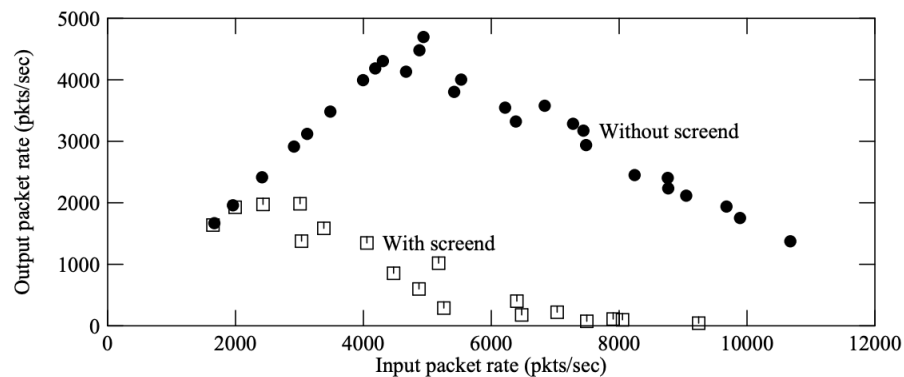
- Per-packet interrupts can cause performance degradation at high-rates
- Receiver interrupts take priority over all other activities. If packets arrive too fast, the system will spend all of its time processing receiver interrupts

Consequence: no resources left for processing and the system throughput will drop to zero



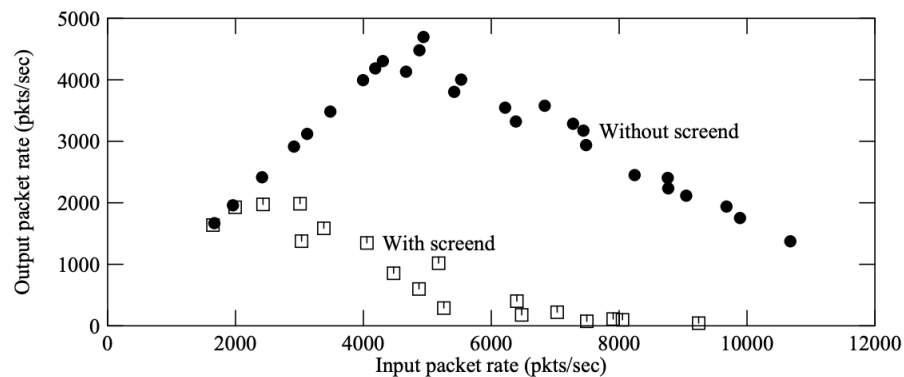
The receive livelock problem

kernel-based forwarding on a 10Mbps ethernet

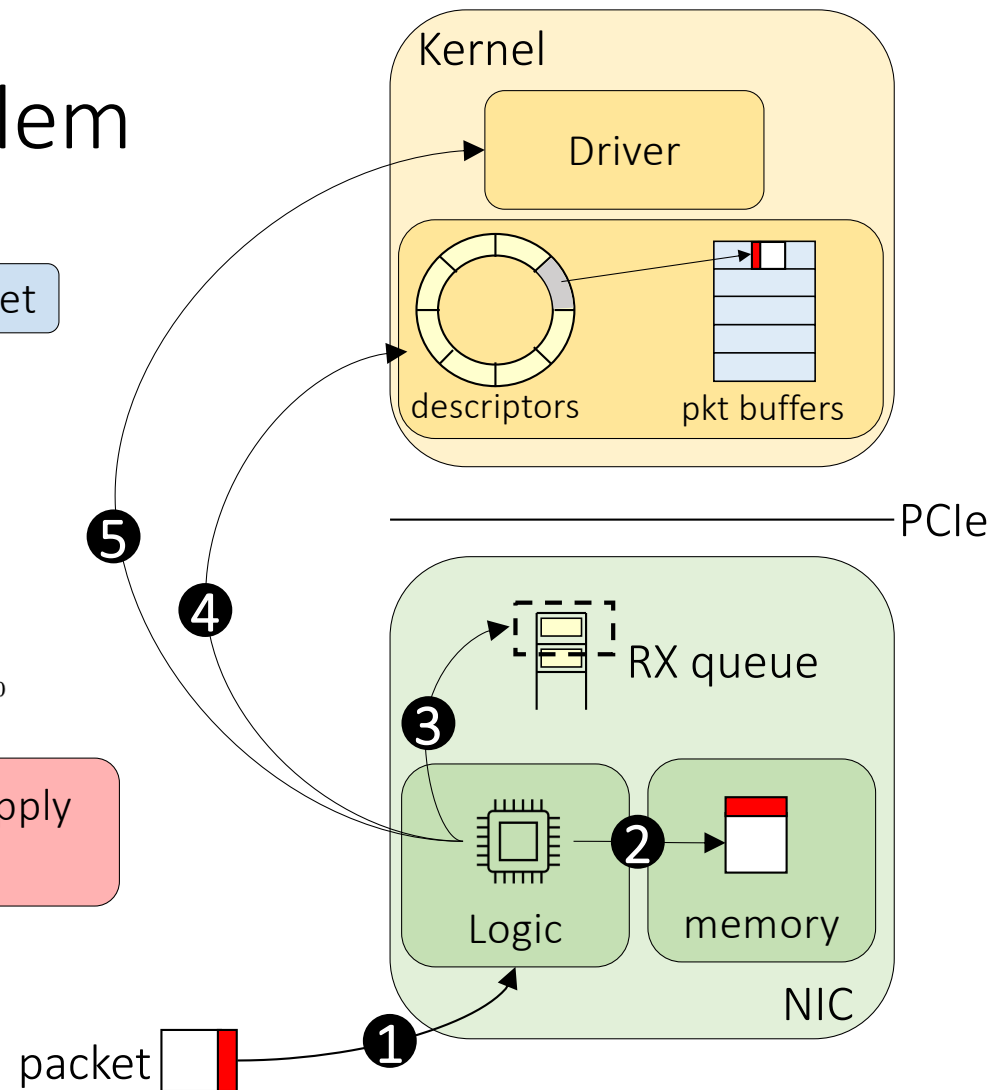


The receive livelock problem

kernel-based forwarding on a 10Mbps ethernet

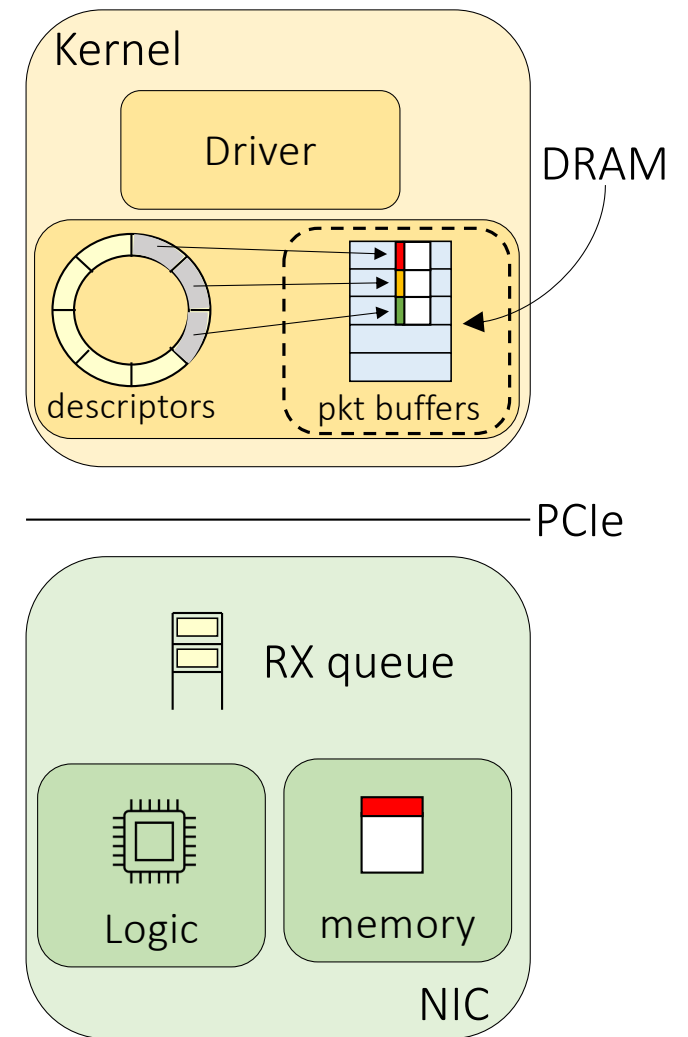
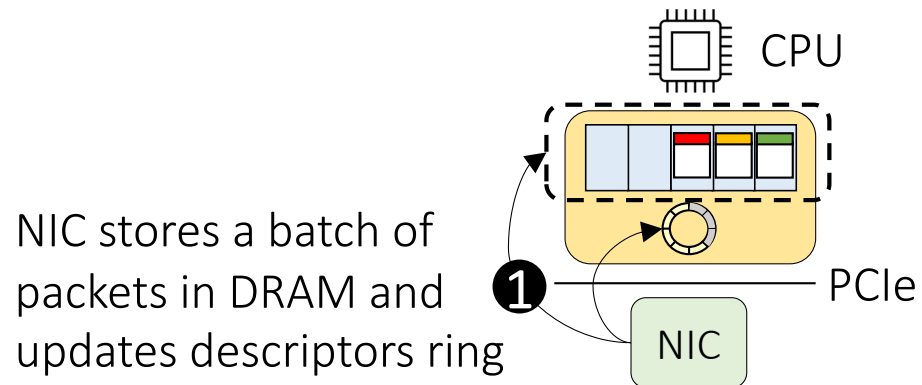


This is an old system (1996), but same concepts apply to newer CPUs as line rate increases



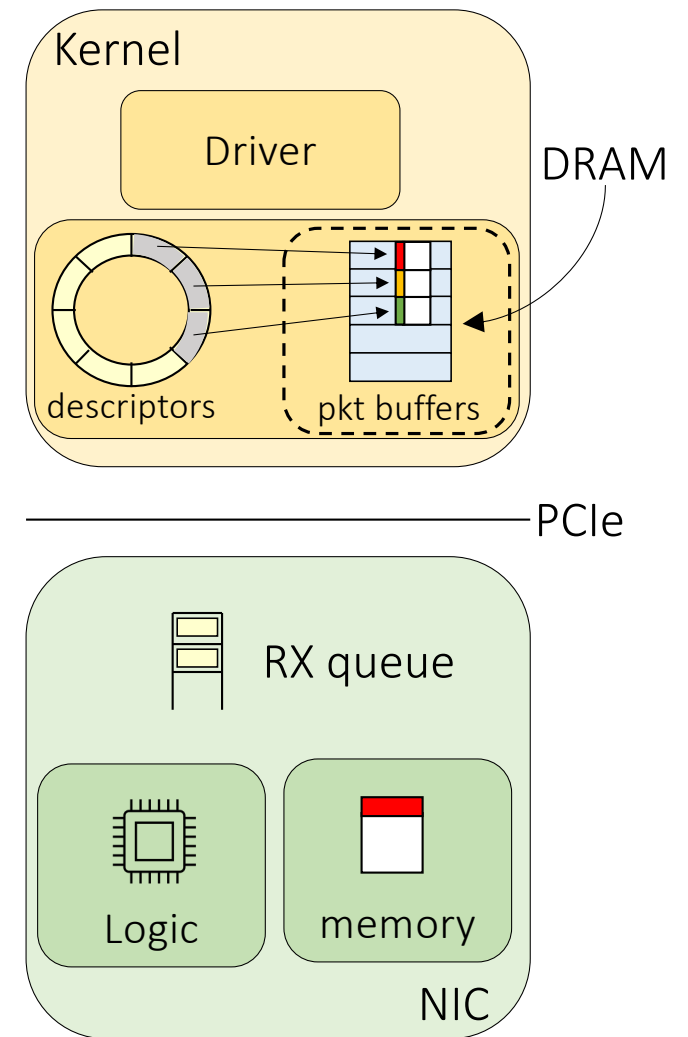
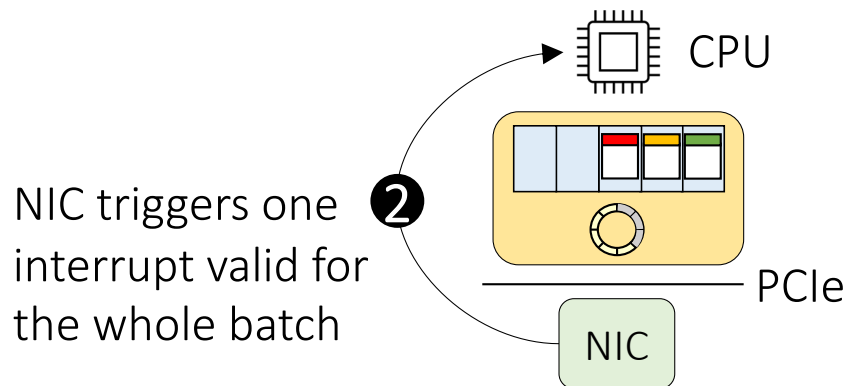
Interrupt mitigation strategies

- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - Interrupt coalescing** delay interrupts and process multiple events at once with a compromise between reaction time and overhead



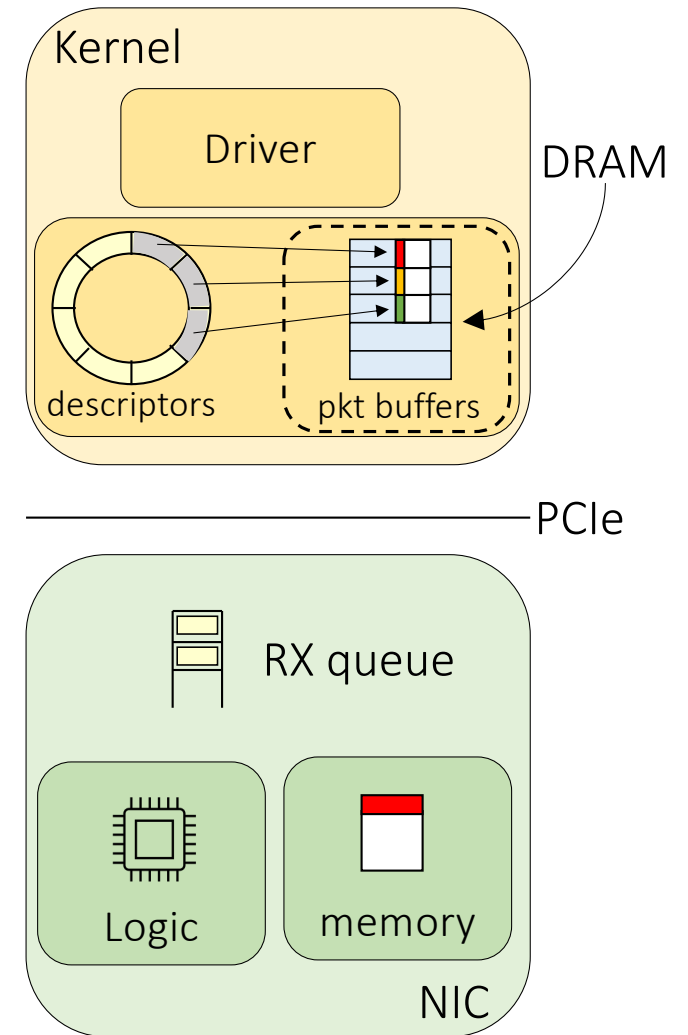
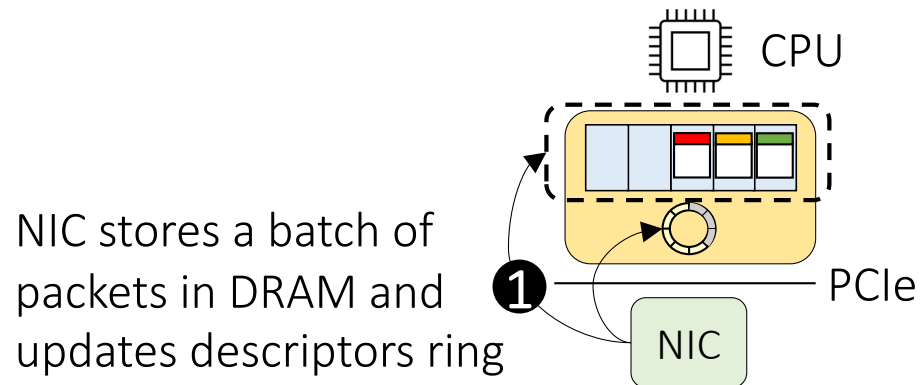
Interrupt mitigation strategies

- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - Interrupt coalescing** delay interrupts and process multiple events at once with a compromise between reaction time and overhead



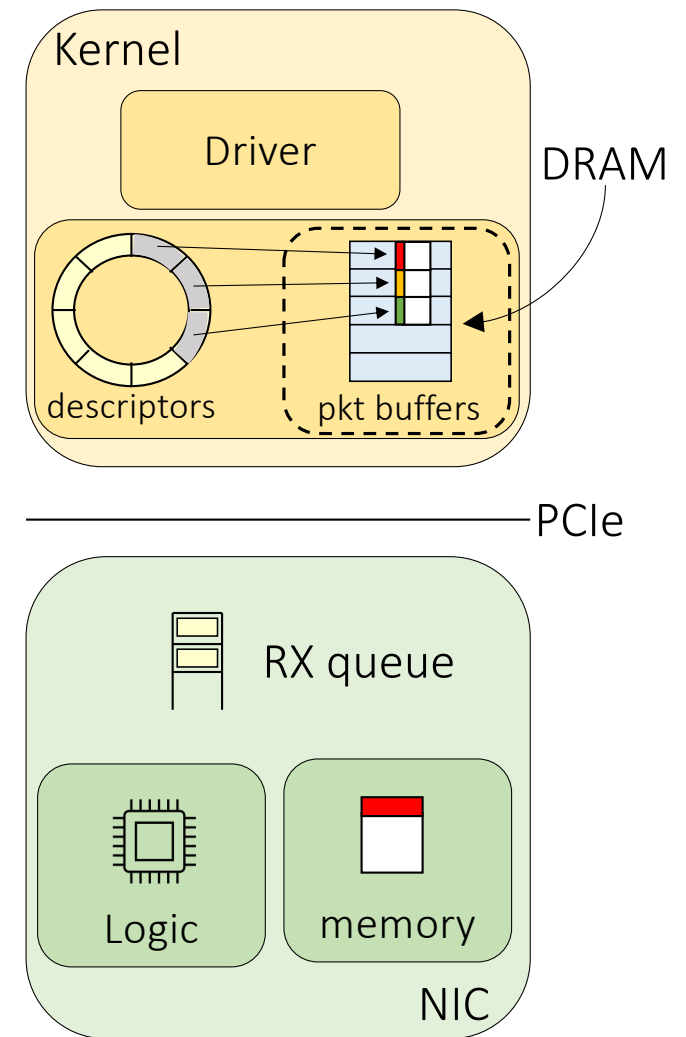
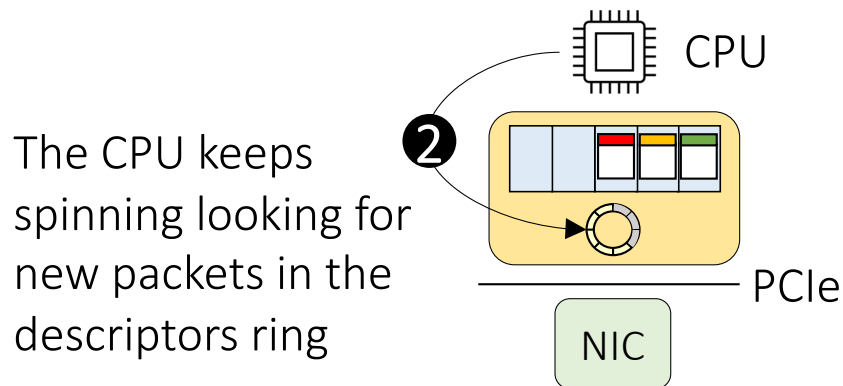
Interrupt mitigation strategies

- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - Polling** the CPU continuously check if the device has anything to send. It relies on busy-polling loop potentially wasting resources



Interrupt mitigation strategies

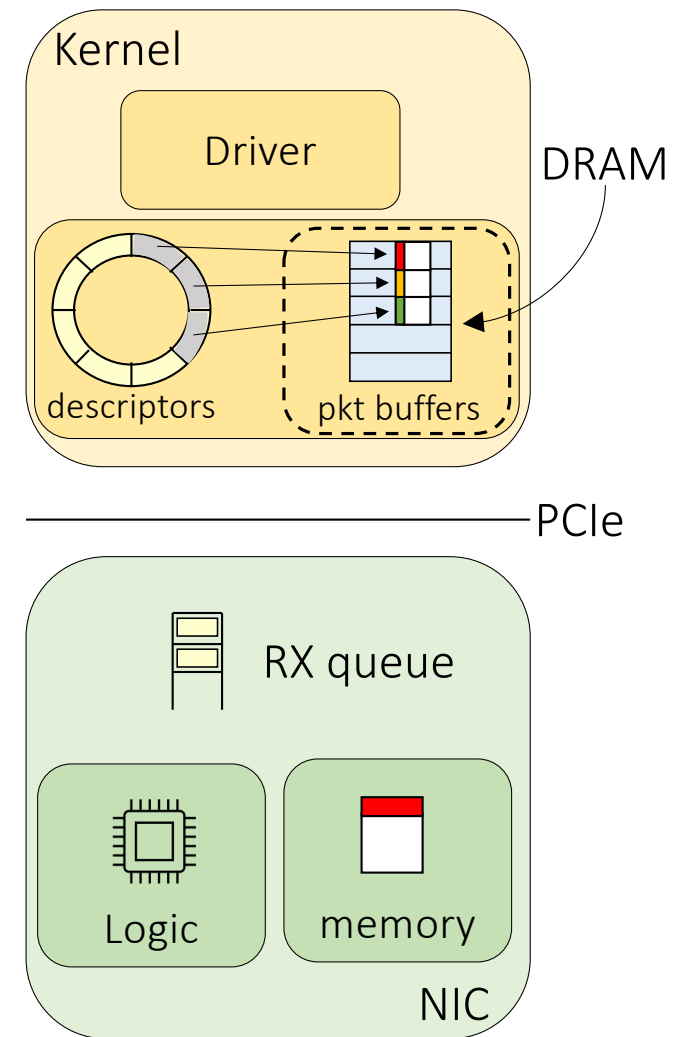
- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - Polling** the CPU continuously check if the device has anything to send. It relies on busy-polling loop potentially wasting resources



Interrupt mitigation strategies

- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - **NAPI** adaptively switching between interrupt and polling according to the current workload
 - Low load: interrupt-based
 - High load: CPU-polling

Why?

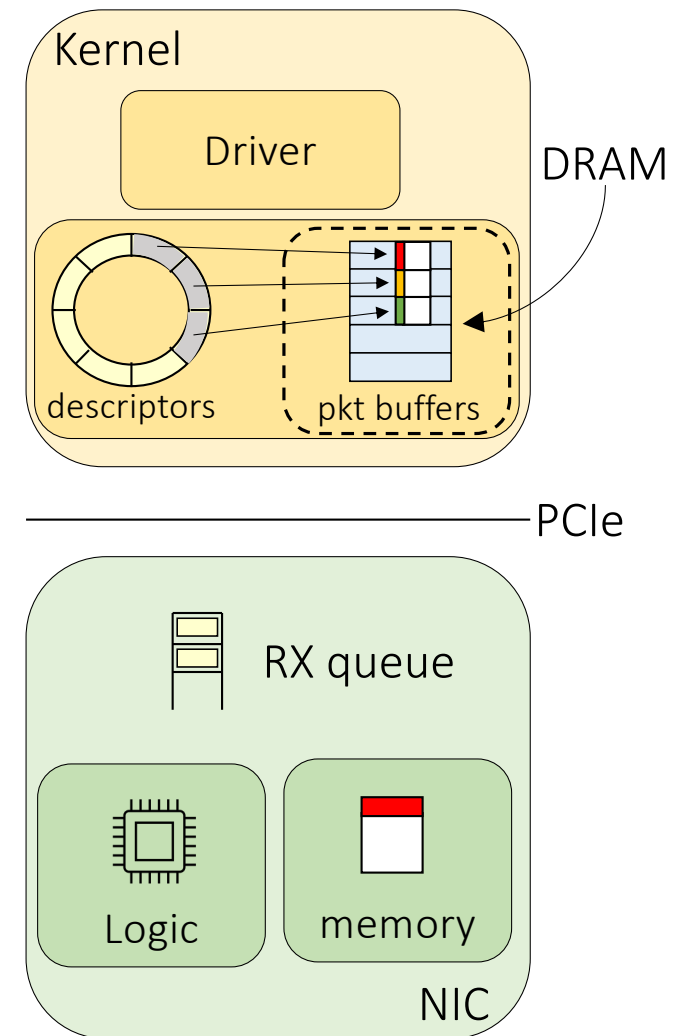


Interrupt mitigation strategies

- Modern NICs provide ways to mitigate the number of interrupts sent from HW to the CPU
 - **NAPI** adaptively switching between interrupt and polling according to the current workload
 - Low load: interrupt-based
 - High load: CPU-polling

Why?

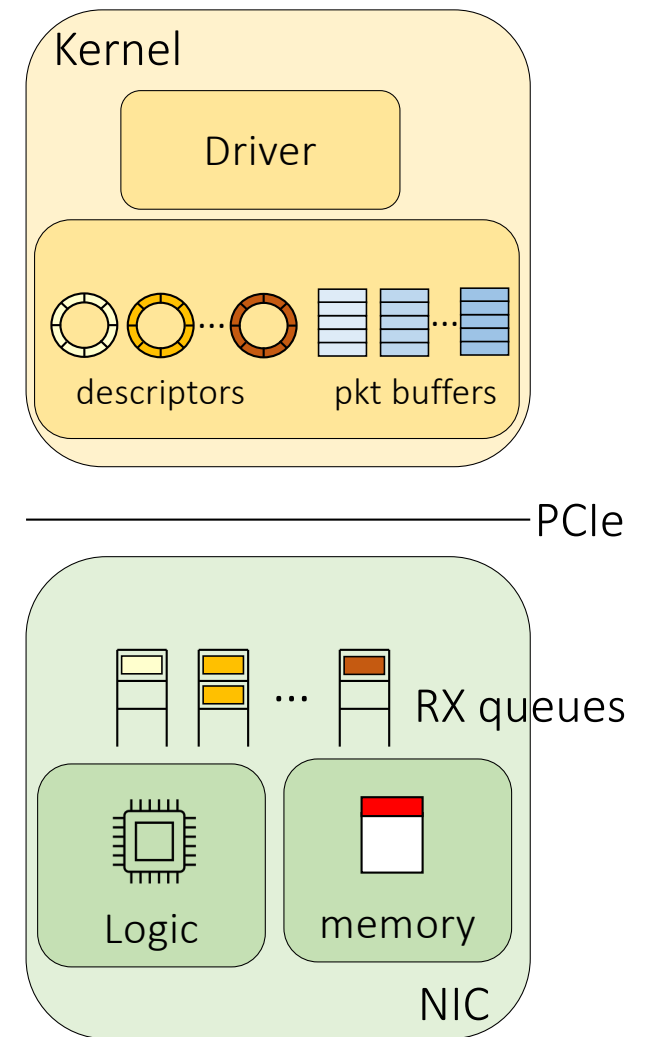
Interrupt-based guarantee low-latency processing and if the load is low, not many interrupts generated



Multi queue

- Modern NICs provide multiple hardware queues

Why?

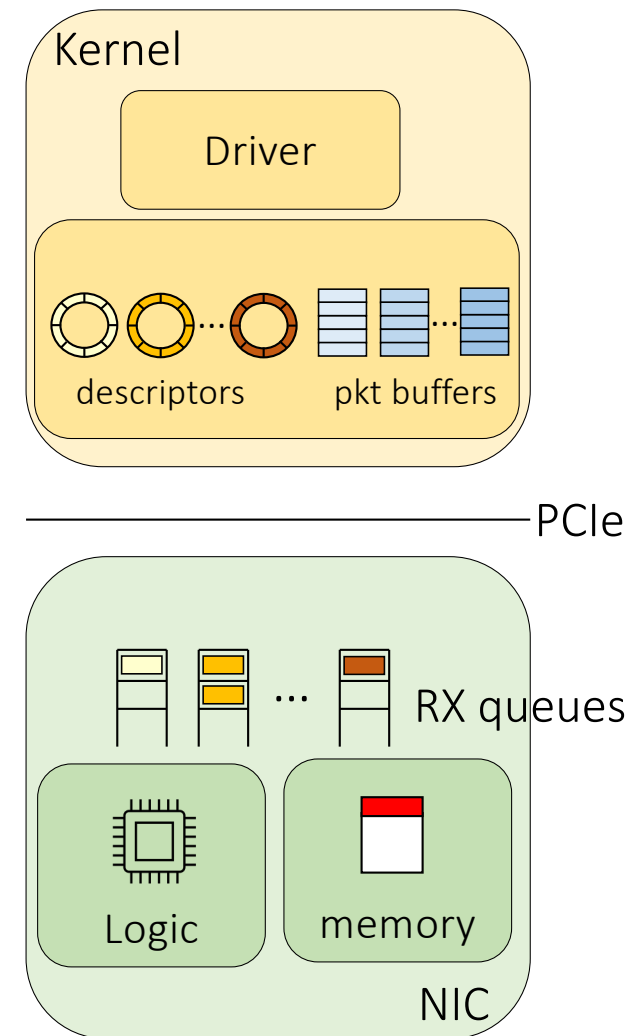


Multi queue

- Modern NICs provide multiple hardware queues

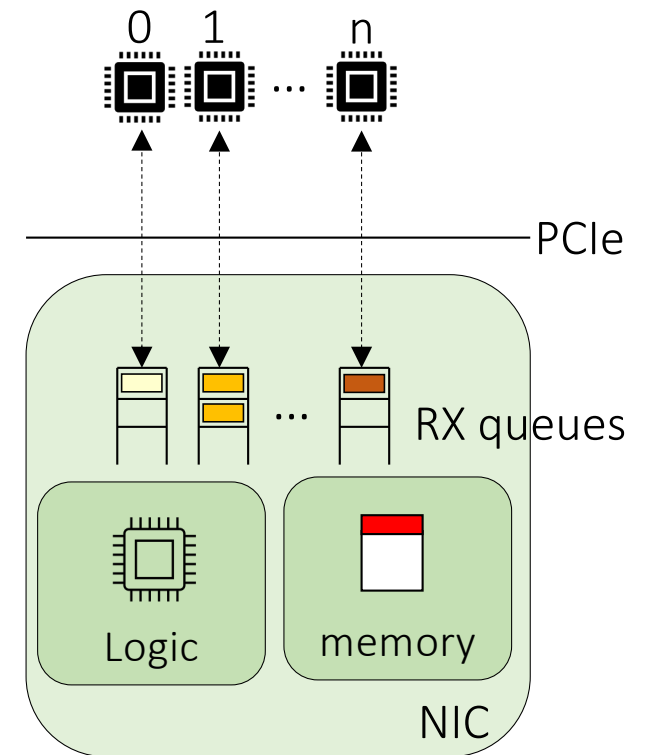
Why?

mainly to handle concurrency between cores and performance 😊



Receive-Side Scaling (RSS)

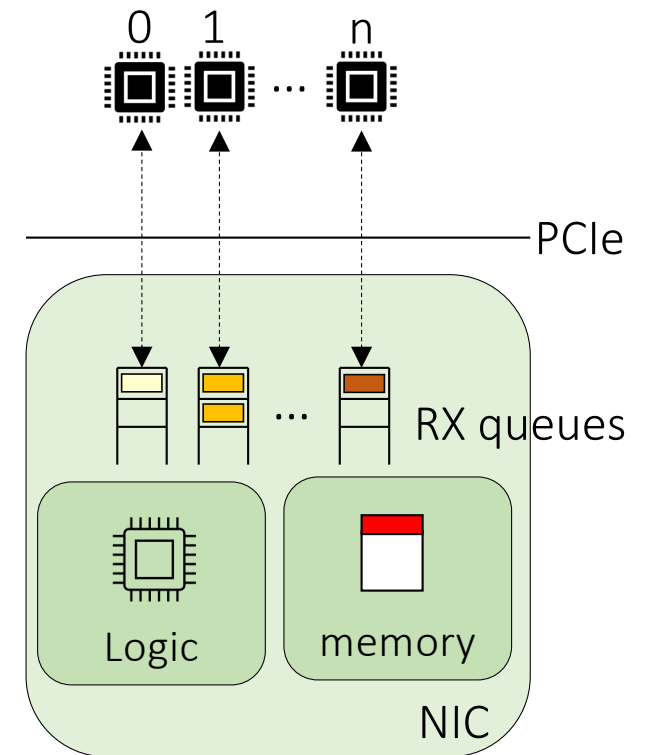
- Modern NICs provide multiple hardware queues
- You can assign each queue to a CPU core and load balance the processing
- This is called *Receive Side Scaling* which assigns flows-to-queues based on hash (IP addresses and L4 ports are common)



Receive-Side Scaling (RSS)

- Modern NICs provide multiple hardware queues
- You can assign each queue to a CPU core and load balance the processing
- This is called *Receive Side Scaling* which assigns flows-to-queues based on hash (IP addresses and L4 ports are common)

Problems with this?

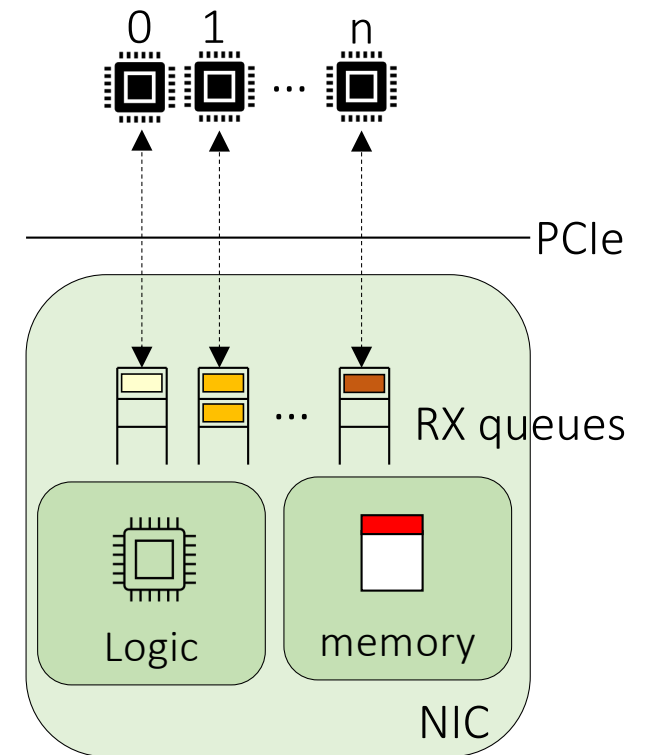


Receive-Side Scaling (RSS)

- Modern NICs provide multiple hardware queues
- You can assign each queue to a CPU core and load balance the processing
- This is called *Receive Side Scaling* which assigns flows-to-queues based on hash (IP addresses and L4 ports are common)

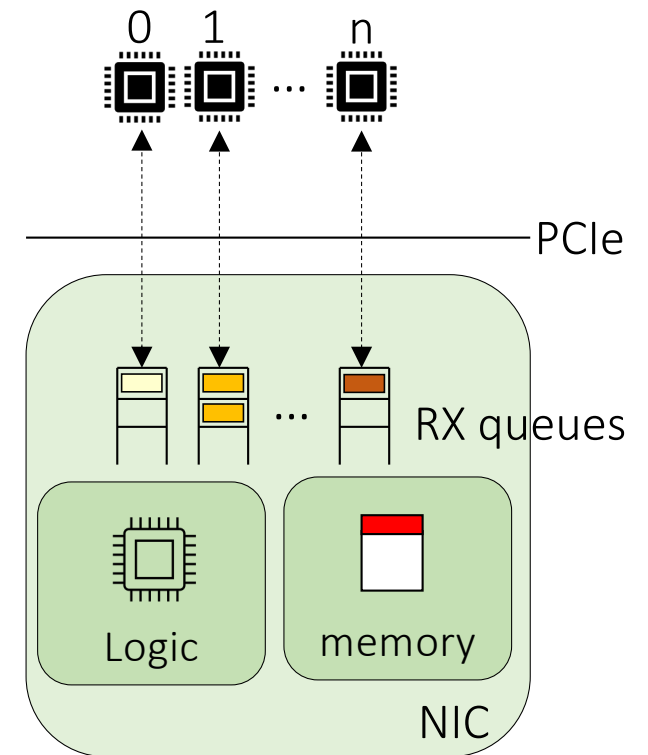
Problems with this?

Hash imbalance



Receive-Side Scaling (RSS)

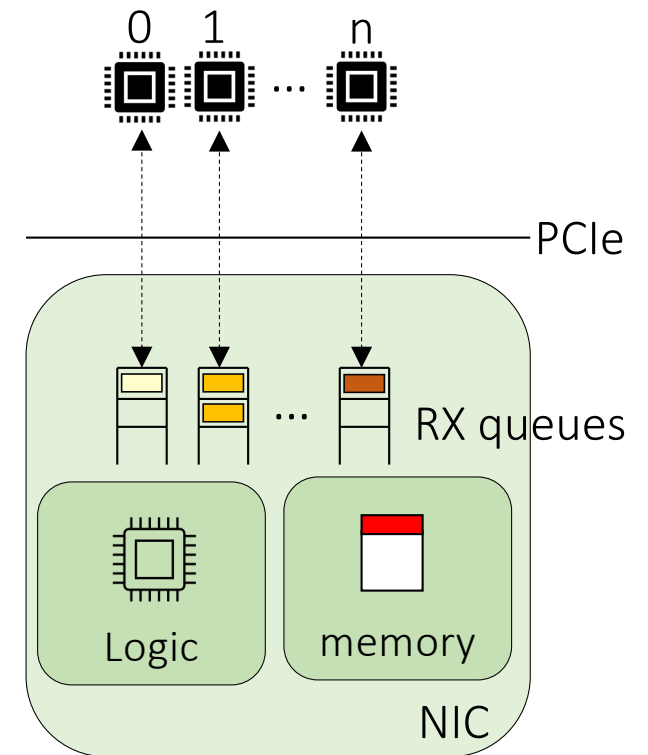
- RSS tries to solve the problem of the overloaded core receiving all network traffic
- When using RSS, packets are sent to different cores for interrupt processing



Receive-Side Scaling (RSS)

- RSS tries to solve the problem of the overloaded core receiving all network traffic
- When using RSS, packets are sent to different cores for interrupt processing

Note: application consuming the packet might be on a different core with respect to the one selected by RSS

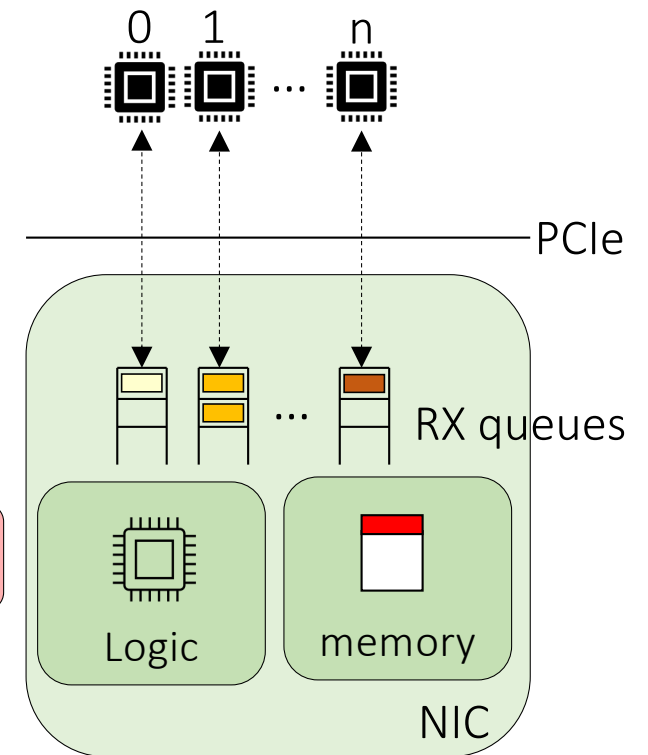


Receive-Side Scaling (RSS)

- RSS tries to solve the problem of the overloaded core receiving all network traffic
- When using RSS, packets are sent to different cores for interrupt processing

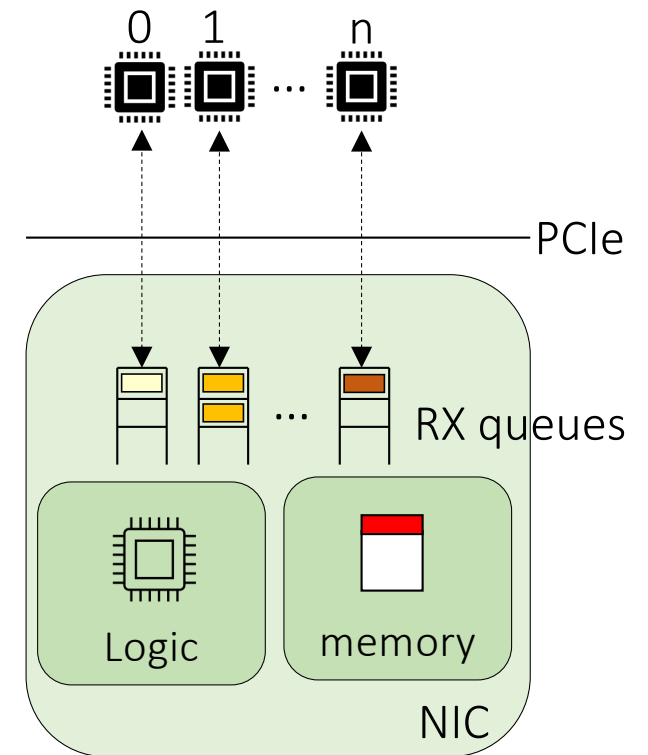
Note: application consuming the packet might be on a different core with respect to the one selected by RSS

This is bad for performance ☹️



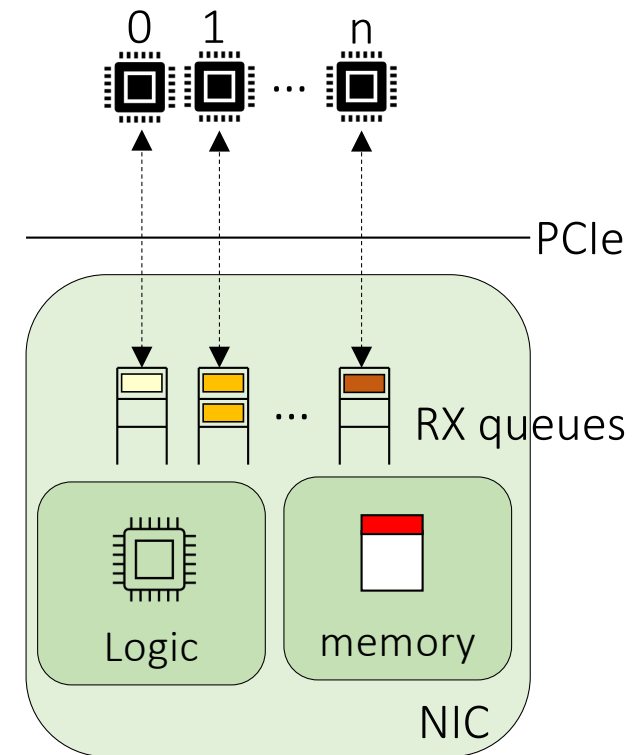
Accelerated Receive-Flow Steering (aRFS)

- aRFS allows you to direct packets to the same core where an application is running



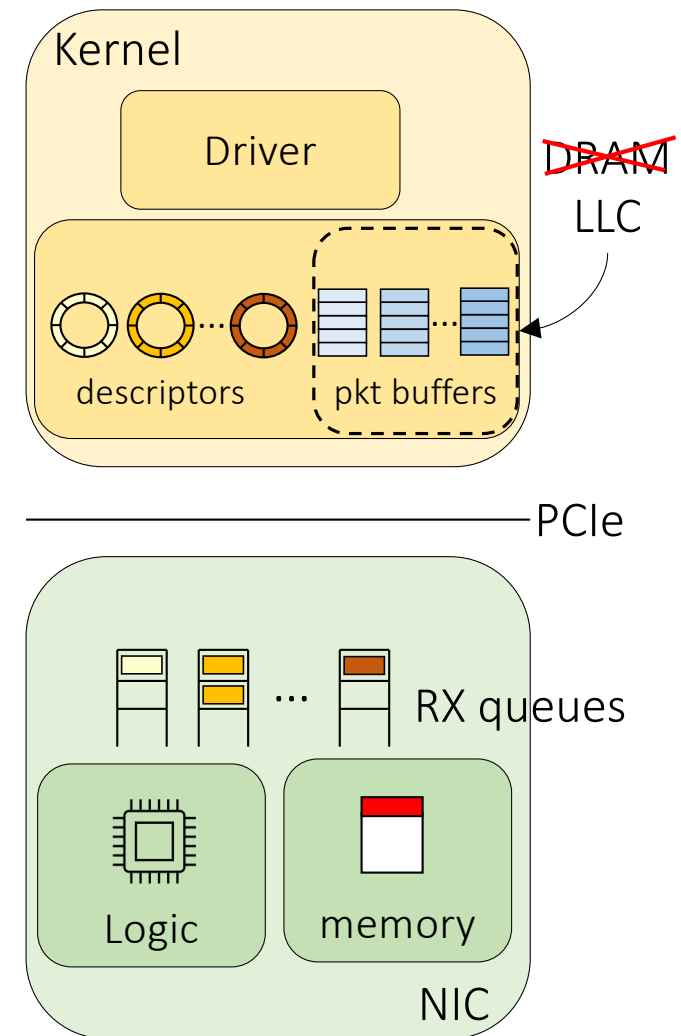
Accelerated Receive-Flow Steering (aRFS)

- aRFS allows you to direct packets to the same core where an application is running
- Example: Intel Flow-Director supports advanced filters that *direct* received packets to different queues and enables tight control on flow in the platform.
- Flow-Director matches flows and the CPU core where the application is running, and provide multiple parameters for flow classification and load balancing



Direct Cache Access (DCA)

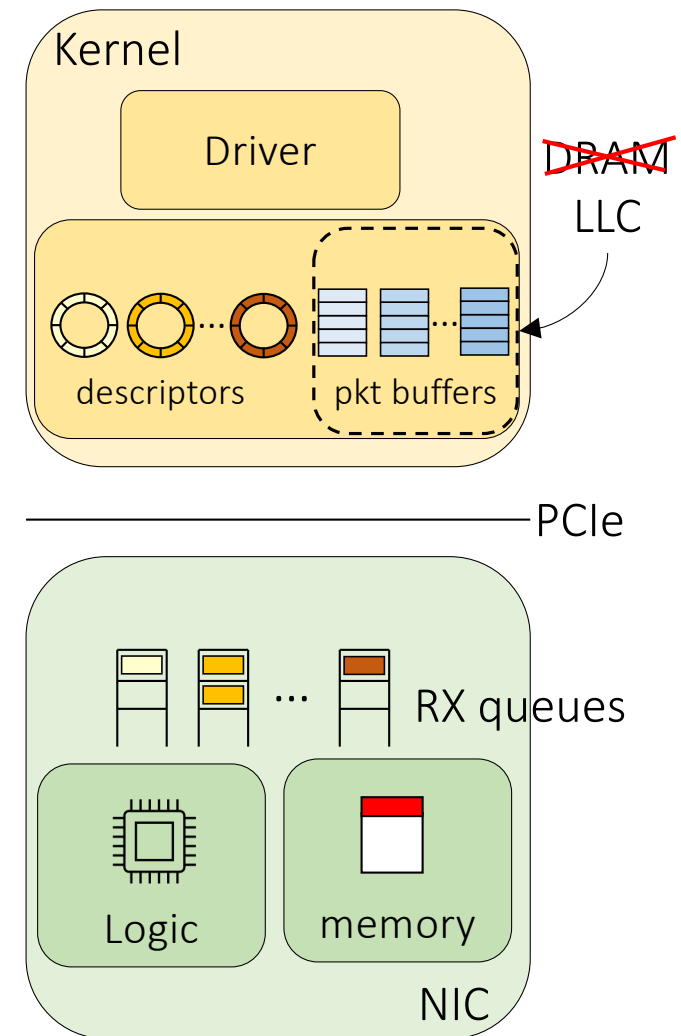
- Modern CPUs provide the possibility to directly store the packet in L3 cache (LLC)
- The CPU core does not have anymore to move data from DRAM to cache when processing the packet
- Data Direct I/O (DDIO) is the Intel's specific implementation of DCA



Direct Cache Access (DCA)

- Modern CPUs provide the possibility to directly store the packet in L3 cache (LLC)
- The CPU core does not have anymore to move data from DRAM to cache when processing the packet
- Data Direct I/O (DDIO) is the Intel's specific implementation of DCA

Problems with this?

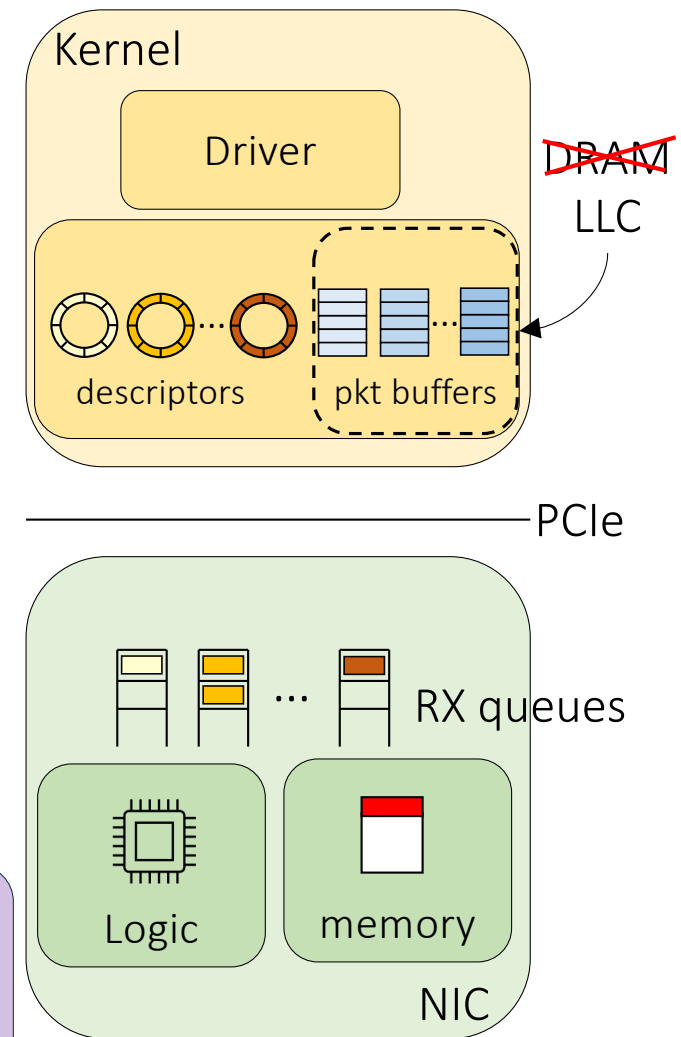


Direct Cache Access (DCA)

- Modern CPUs provide the possibility to directly store the packet in L3 cache (LLC)
- The CPU core does not have anymore to move data from DRAM to cache when processing the packet
- Data Direct I/O (DDIO) is the Intel's specific implementation of DCA

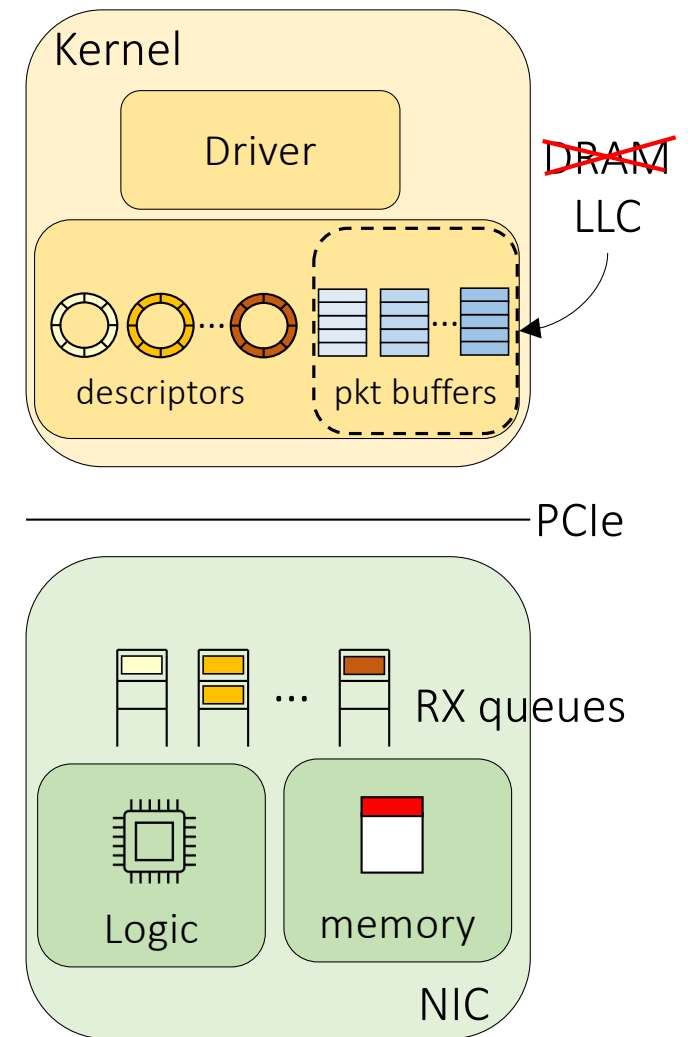
Problems with this?

new incoming packets repeatedly evict not-yet-processed packets from the LLC: this is called the *leaky DMA problem*



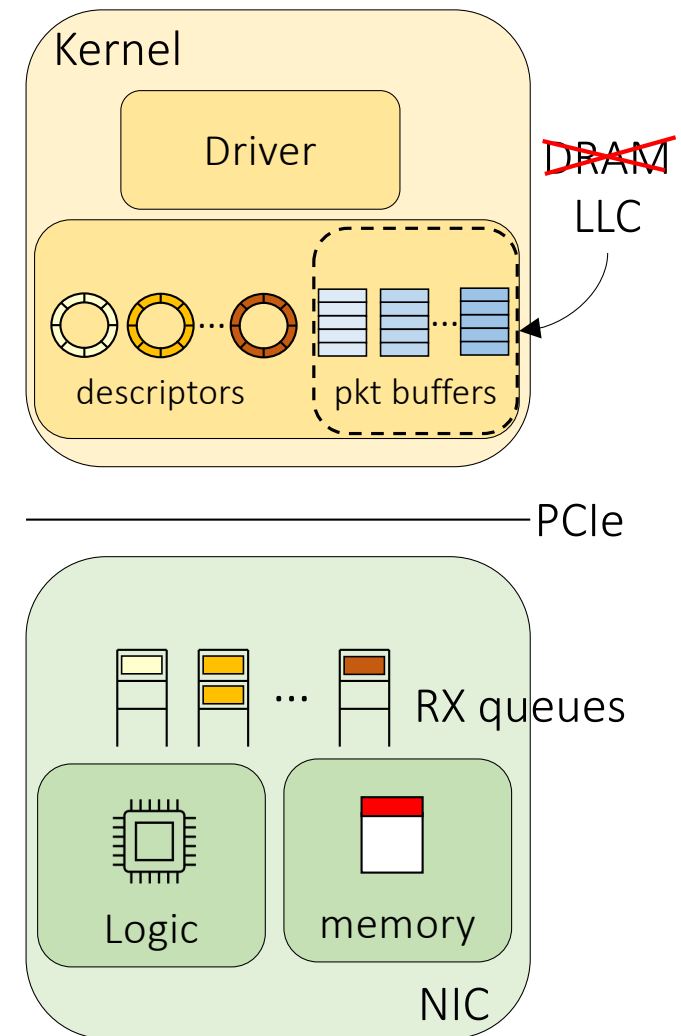
The leaky DMA problem

- Only a small portion of the LLC is allocated to DDIO



The leaky DMA problem

- Only a small portion of the LLC is allocated to DDIO
- When the NIC injects many packets into the LLC, contention causes DMA buffers to be evicted before CPU consumption, forcing data to traverse DRAM and increasing memory bandwidth usage, latency, and LLC miss rate
- Mismatch
 - Network stack processes packets in the arrival order
 - Standard cache policies prefer to store the most recently used data



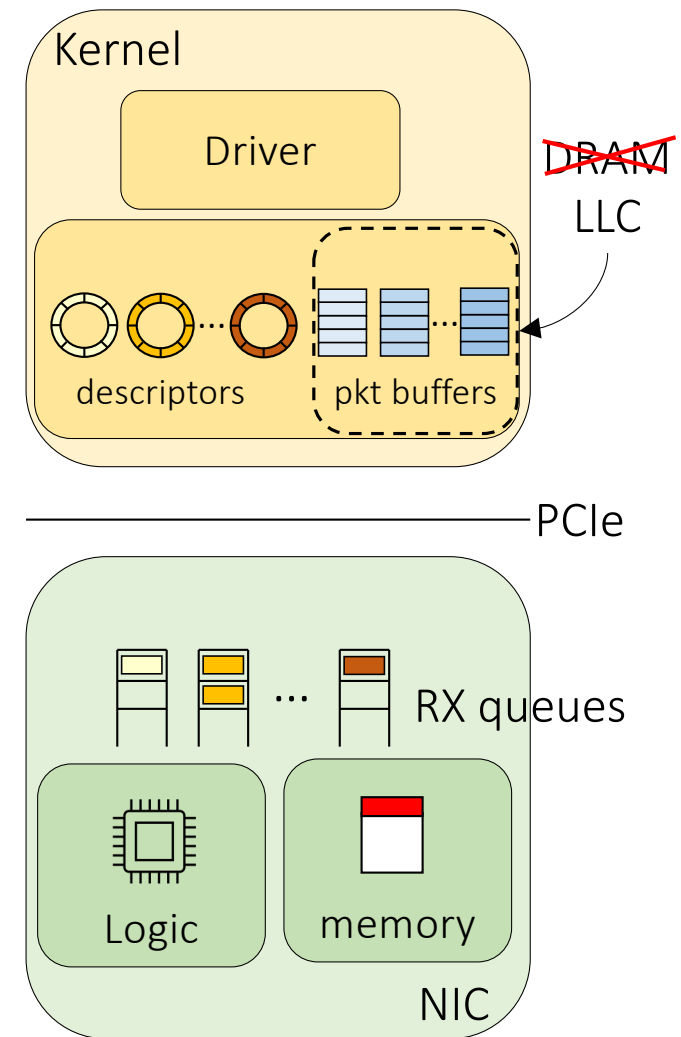
The leaky DMA problem

- Only a small portion of the LLC is allocated to DDIO
- When the NIC injects many packets into the LLC, contention causes DMA buffers to be evicted before CPU consumption, forcing data to traverse DRAM and increasing memory bandwidth usage, latency, and LLC miss rate

- Mismatch

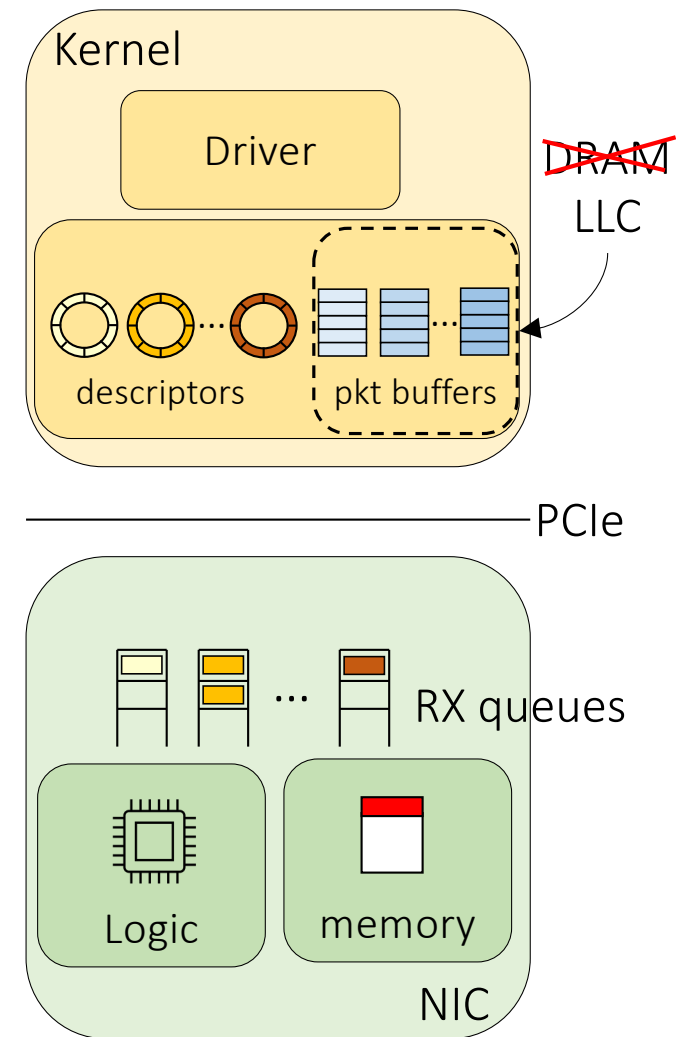
- Network stack processes packets in the arrival order
- Standard cache policies prefer to store the most recently used data

How to mitigate this?



The leaky DMA problem

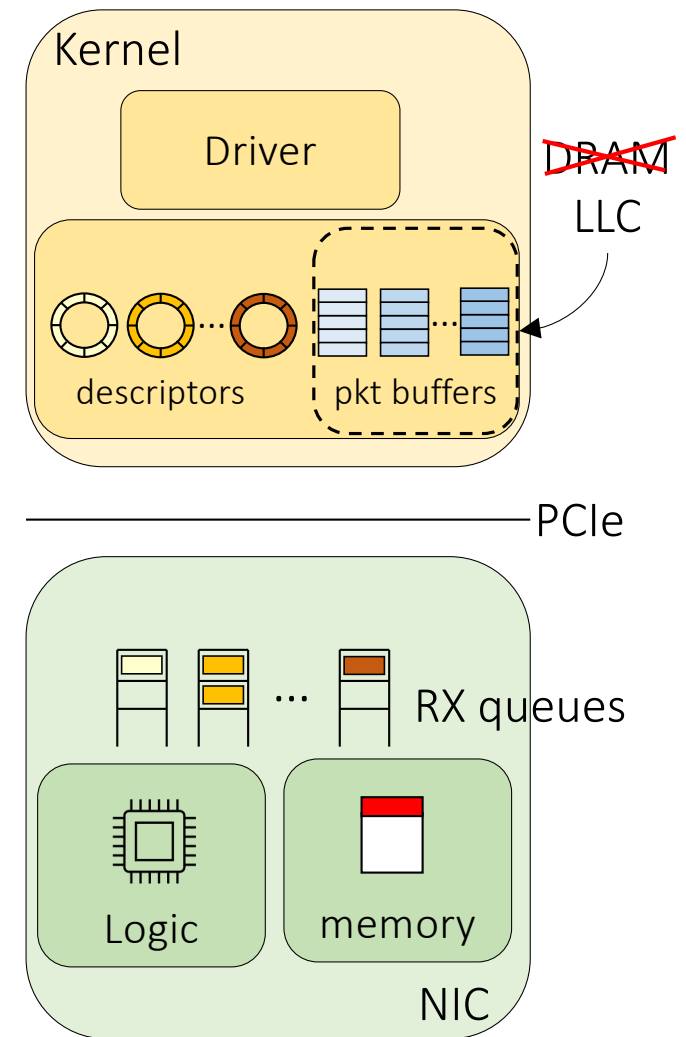
- This issue can be mitigated by limiting in-flight packets to the LLC capacity reserved for DDIO, avoiding packet leakage from the cache
- This number corresponds to the overall number of descriptors allocated to the NIC



The leaky DMA problem

- This issue can be mitigated by limiting in-flight packets to the LLC capacity reserved for DDIO, avoiding packet leakage from the cache
- This number corresponds to the overall number of descriptors allocated to the NIC

Problems with this?

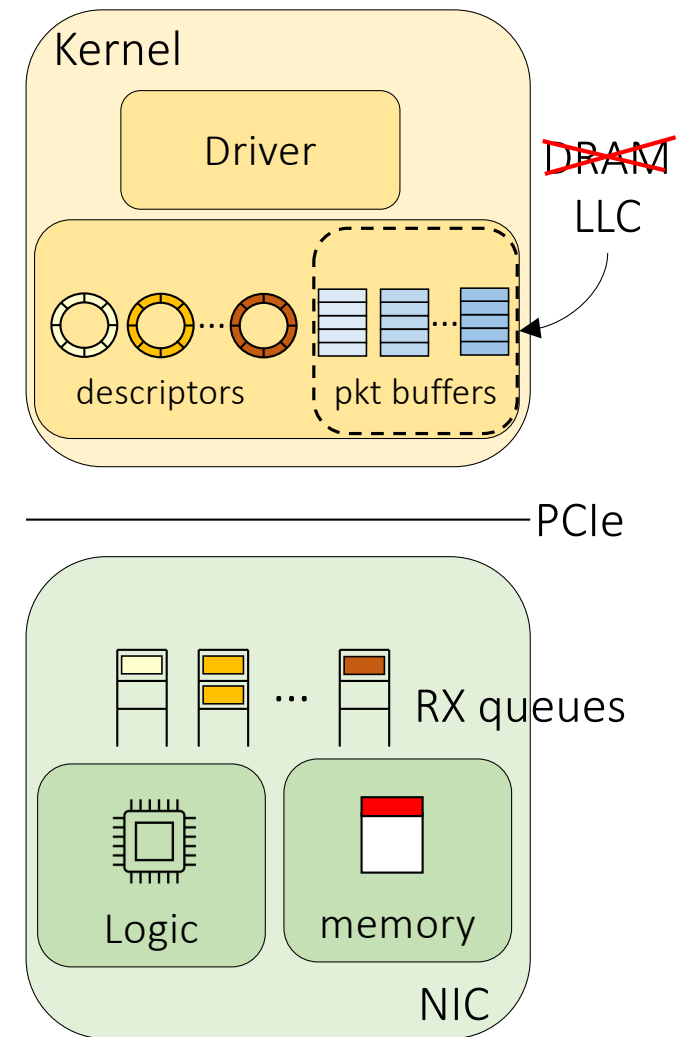


The leaky DMA problem

- This issue can be mitigated by limiting in-flight packets to the LLC capacity reserved for DDIO, avoiding packet leakage from the cache
- This number corresponds to the overall number of descriptors allocated to the NIC

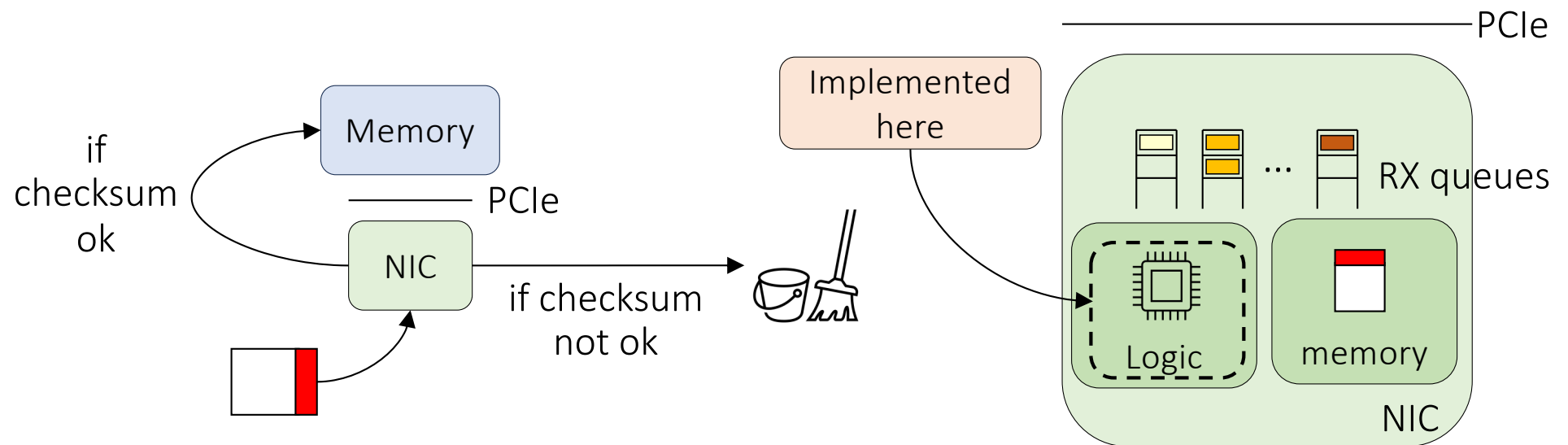
Problems with this?

- When a queue runs out of descriptors, newly arriving packets are dropped
 - limited descriptor availability prevents the system from absorbing traffic bursts or processing variability, degrading throughput



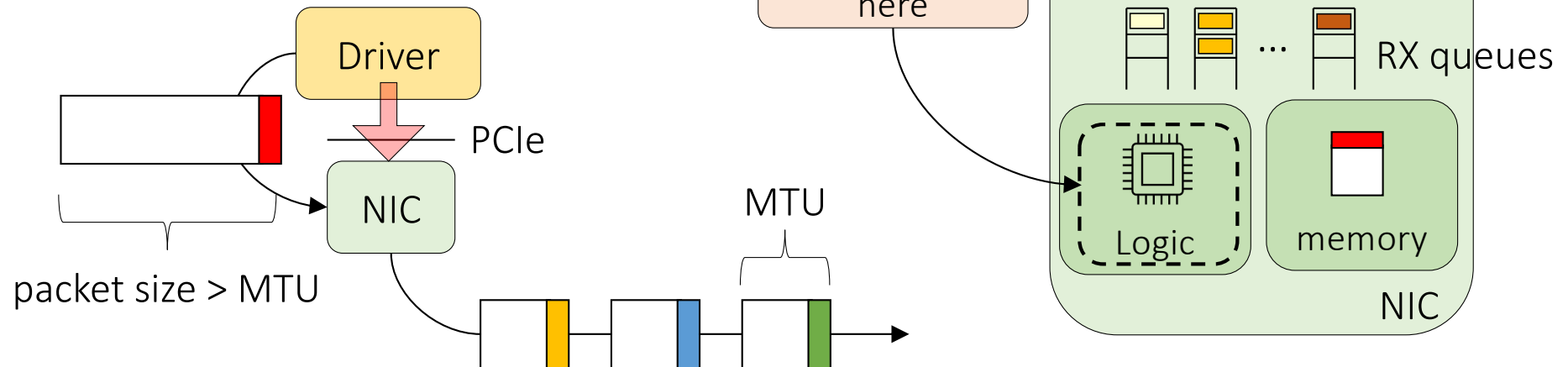
Standard offloads

- Modern NICs provide some offload capabilities in the form of:
 - Checksum calculation



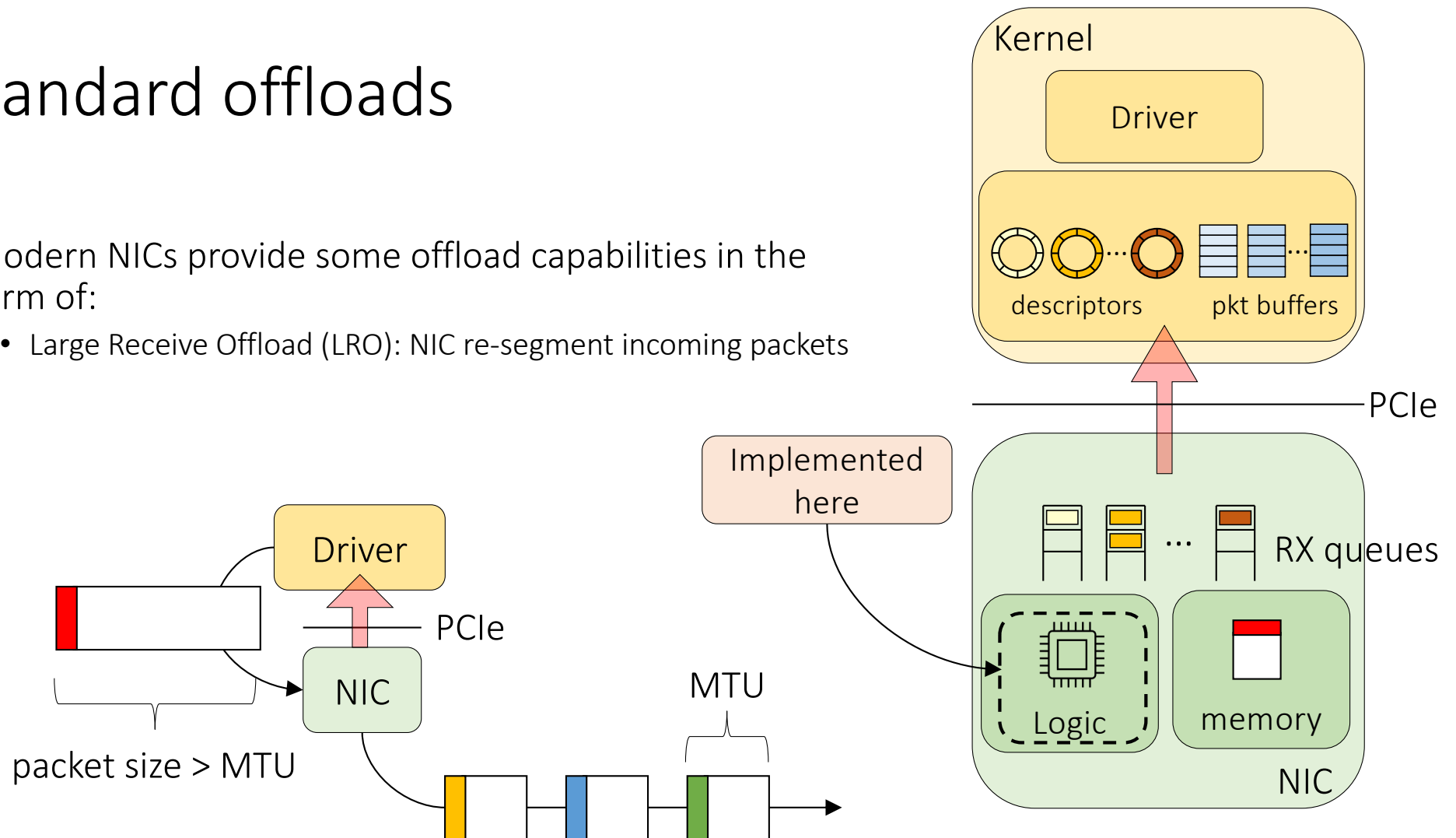
Standard offloads

- Modern NICs provide some offload capabilities in the form of:
 - TCP Segmentation Offload (TSO) and UDP Fragmentation Offload (UFO): NIC handles segmentation/fragmentation



Standard offloads

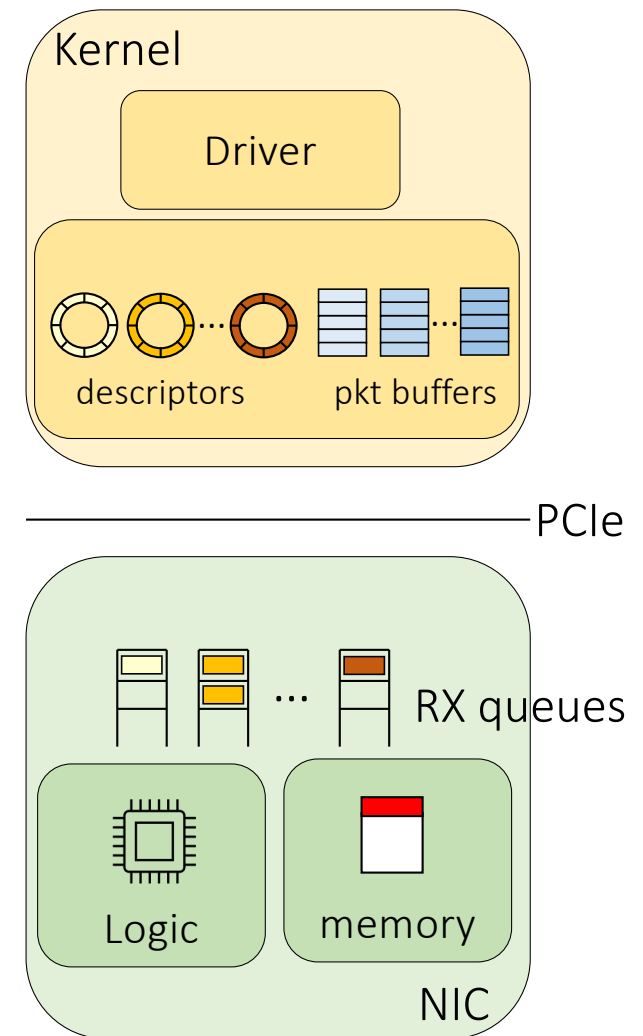
- Modern NICs provide some offload capabilities in the form of:
 - Large Receive Offload (LRO): NIC re-segment incoming packets



Standard offloads

- Modern NICs provide some offload capabilities in the form of:
 - TCP Segmentation Offload (TSO) and UDP Fragmentation Offload (UFO): NIC handles segmentation/fragmentation
 - Large Receive Offload (LRO): NIC re-segment incoming packets

Why TSO, UFO, LRO are helpful?

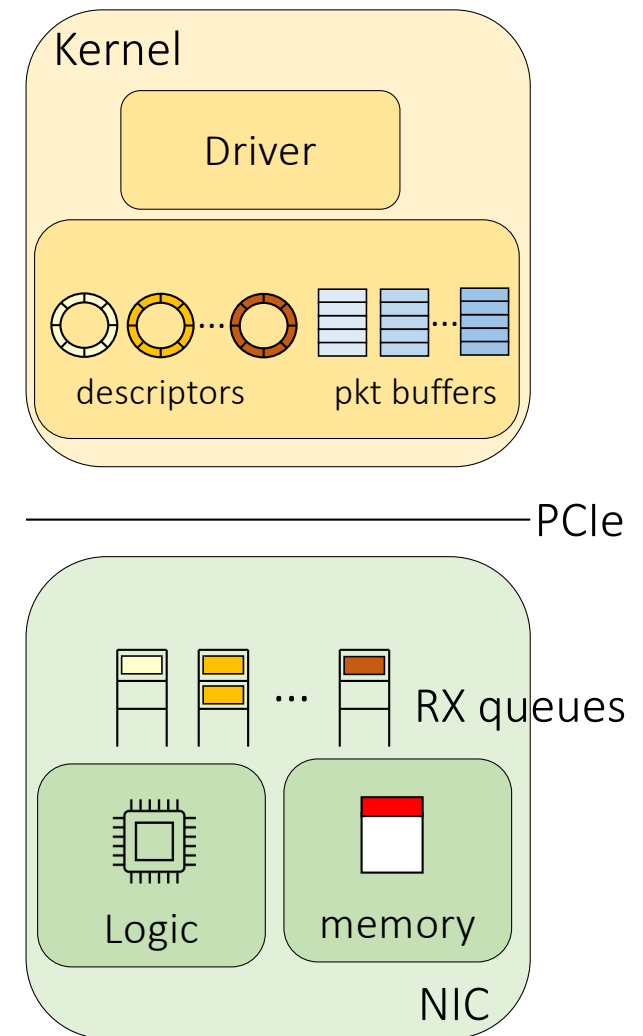


Standard offloads

- Modern NICs provide some offload capabilities in the form of:
 - TCP Segmentation Offload (TSO) and UDP Fragmentation Offload (UFO): NIC handles segmentation/fragmentation
 - Large Receive Offload (LRO): NIC re-segment incoming packets

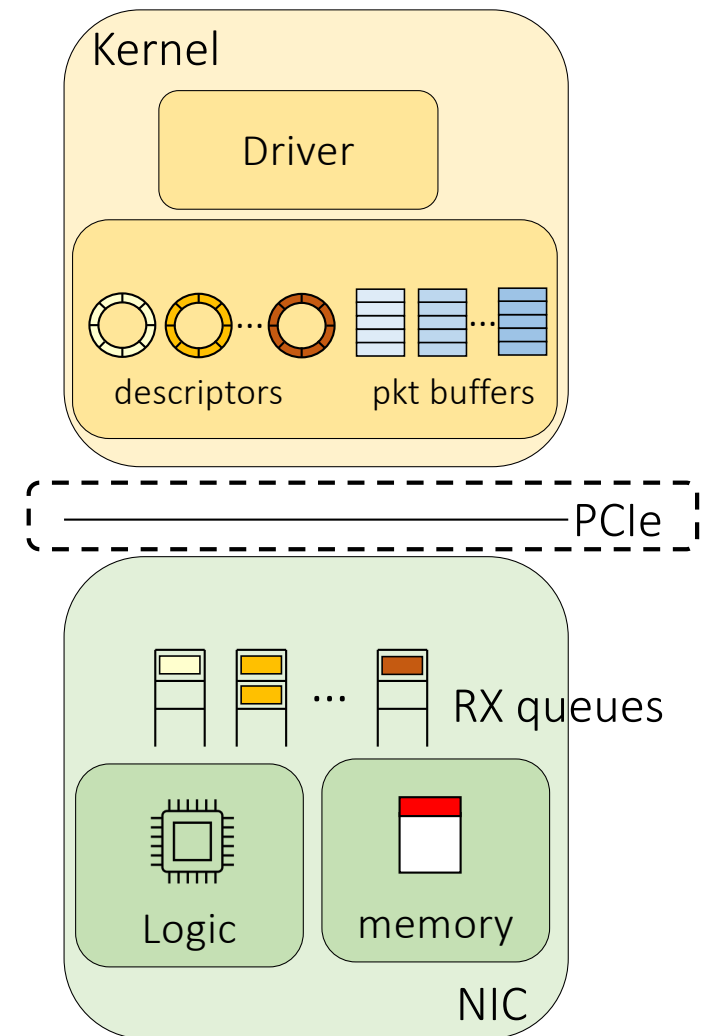
Why TSO, UFO, LRO are helpful?

We need first to understand how PCIe works



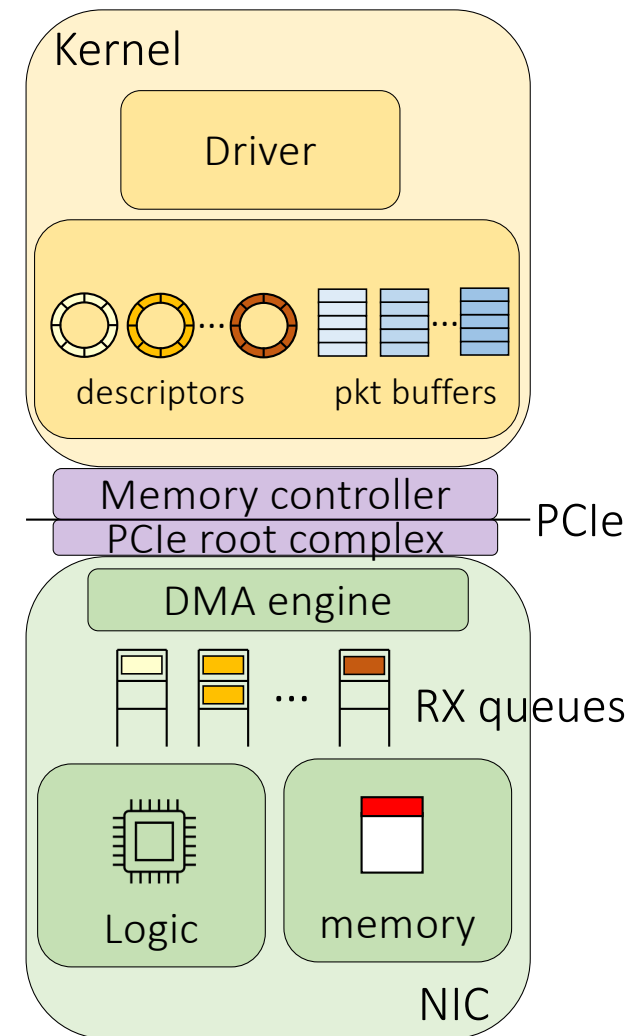
PCIe

- All the operations discussed so far require PCIe transactions
 - Packet data from NIC to host memory
 - Descriptors
 - Interrupt ReQuests (IRQs)
- PCIe is the de-facto standard to connect high performance devices to the rest of the system



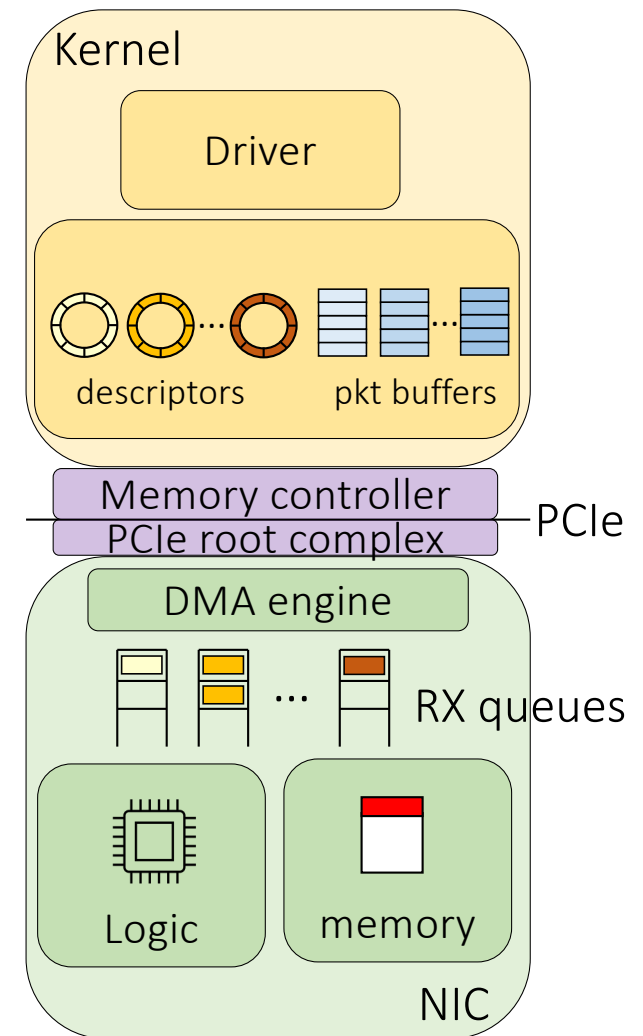
PCIe

- All the operations discussed so far require PCIe transactions
 - Packet data from NIC to host memory
 - Descriptors
 - Interrupt ReQuests (IRQs)
- PCIe is the de-facto standard to connect high performance devices to the rest of the system



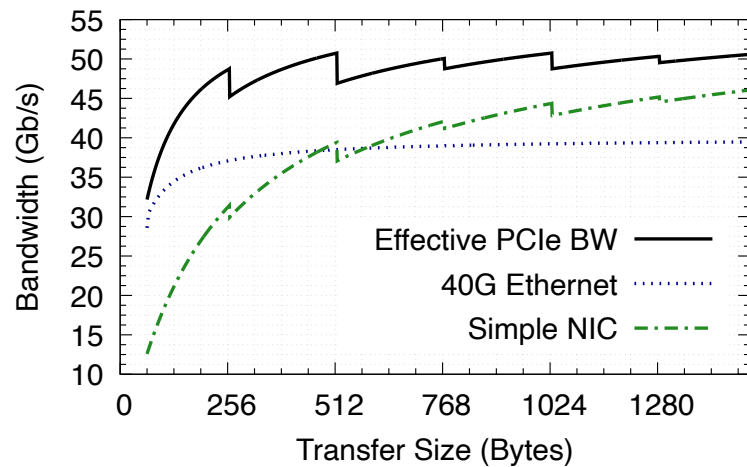
PCIe

- When you DMA something (e.g., write X to host at address Y), the DMA engine breaks the request in multiple PCIe Memory Write packets (*Transaction Layer Packets*)
- PCIe is almost like a network protocol with packets (TLPs), headers, MTU, flow control, addressing etc..

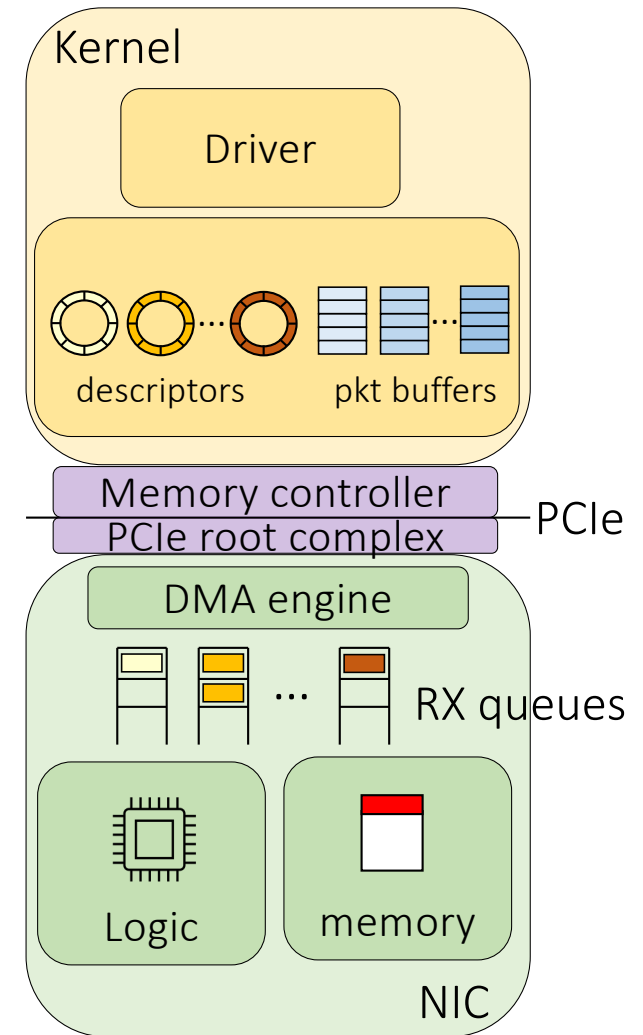


The impact of PCIe

PCIe Gen 3 x8 (typically used by 40Gbps NICs)

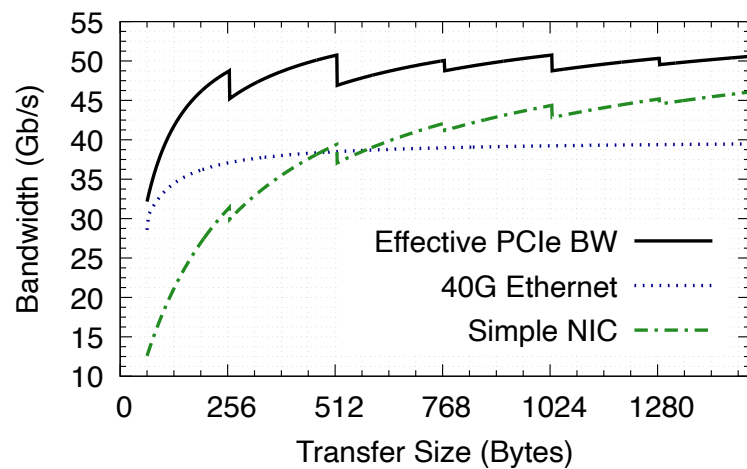


At the physical layer it would have 62.96Gbps of throughput but the PCIe protocol reduces the usable bandwidth to approx 50Gbps

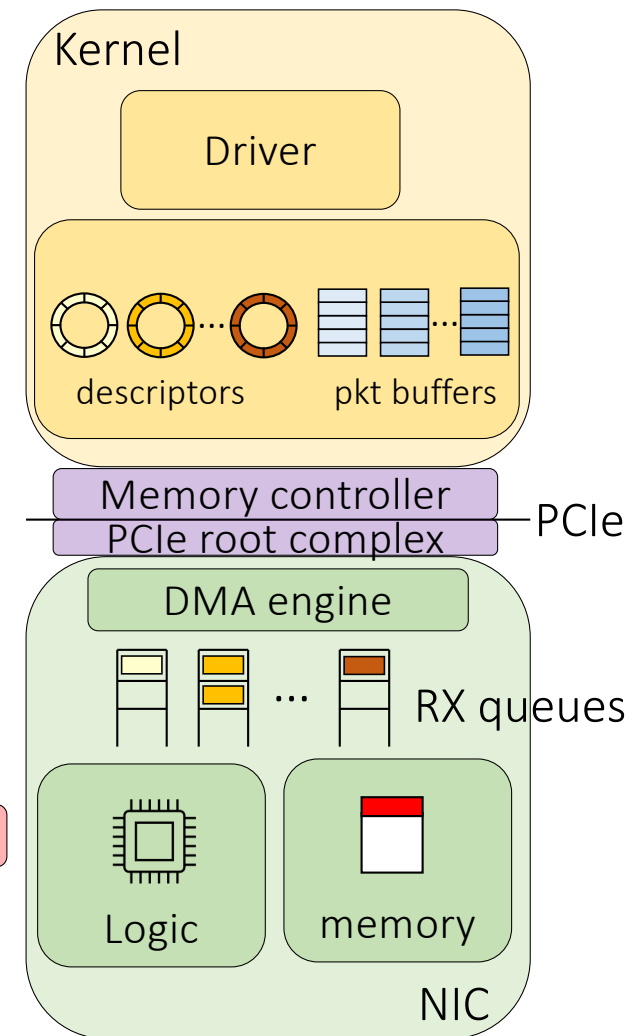


The impact of PCIe

PCIe Gen 3 x8 (typically used by 40Gbps NICs)

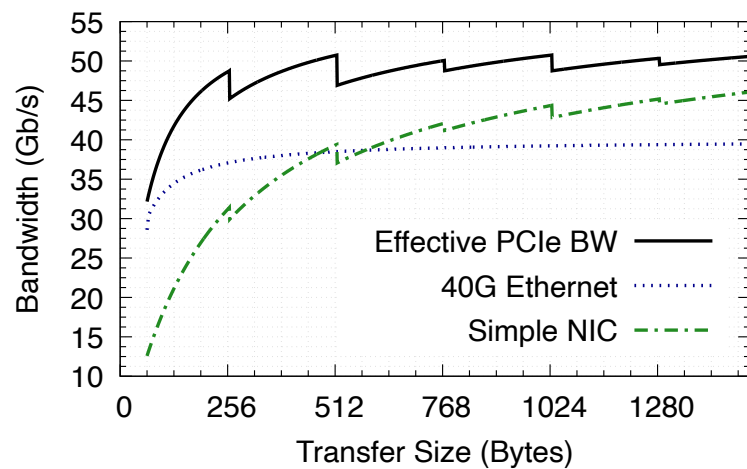


Why the sawtooth pattern?



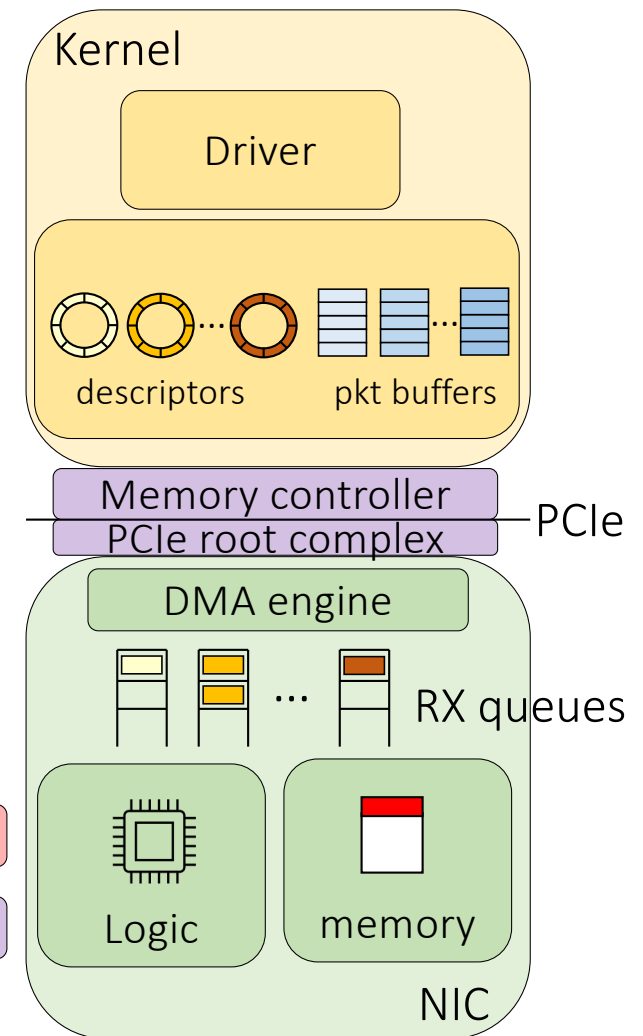
The impact of PCIe

PCIe Gen 3 x8 (typically used by 40Gbps NICs)



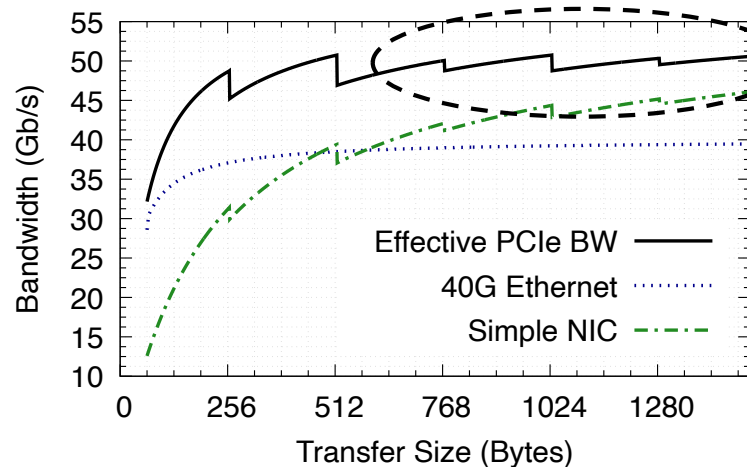
Why the sawtooth pattern?

Because of the “packetized” nature of PCIe protocol



The impact of PCIe

PCIe Gen 3 x8 (typically used by 40Gbps NICs)

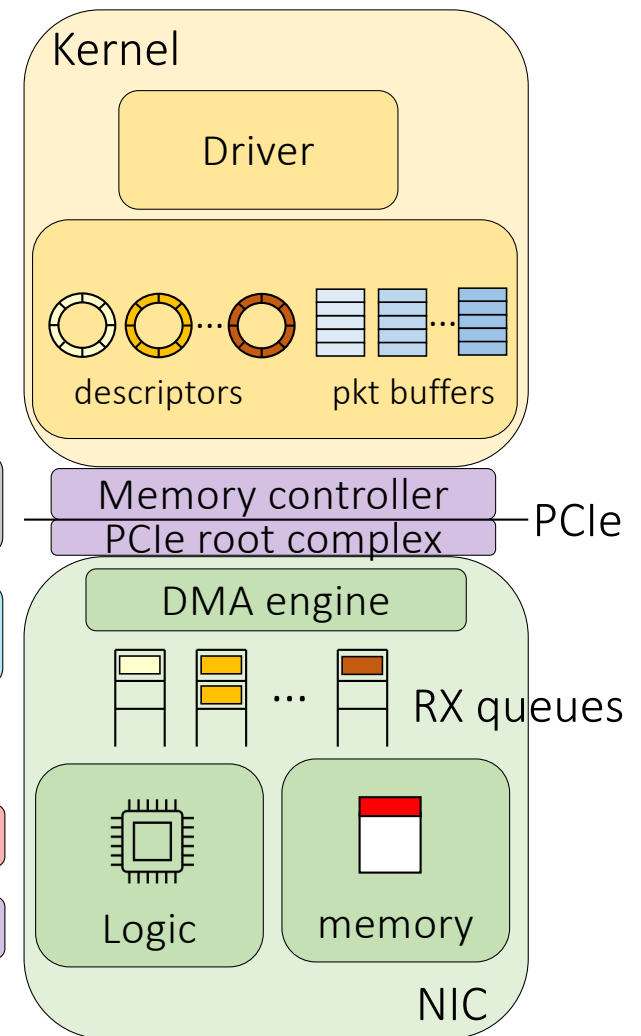


This why TSO/UFO/LRO are useful 😊

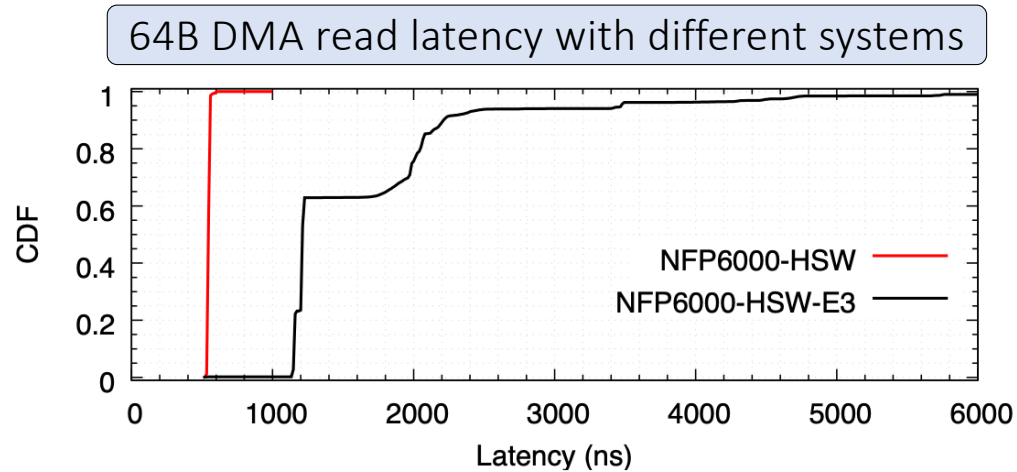
With TSO/UFO/LRO you increase transfer size

Why the sawtooth pattern?

Because of the “packetized” nature of PCIe protocol



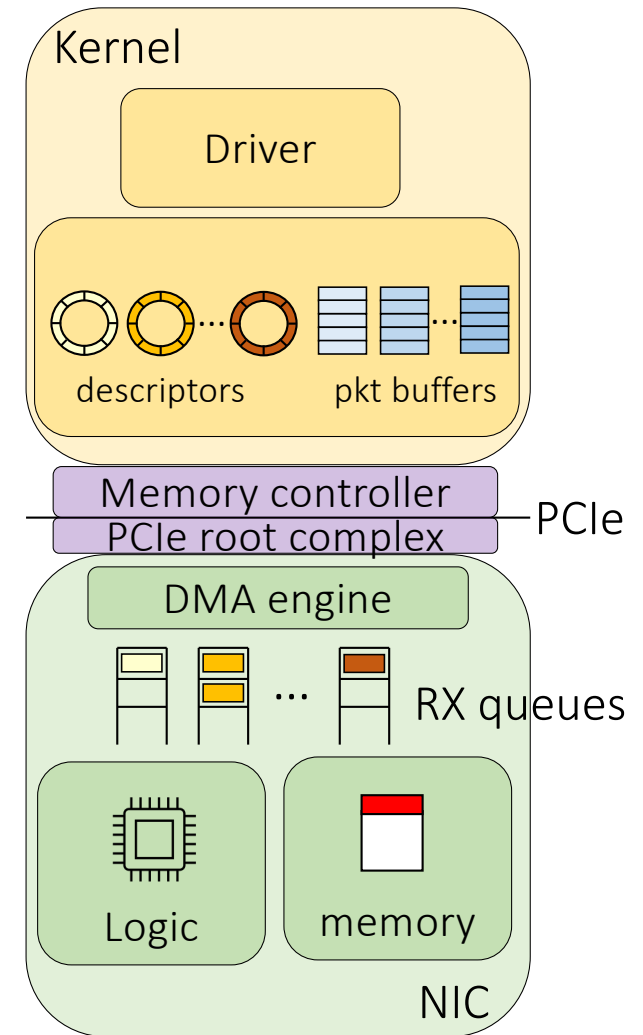
The impact of PCIe



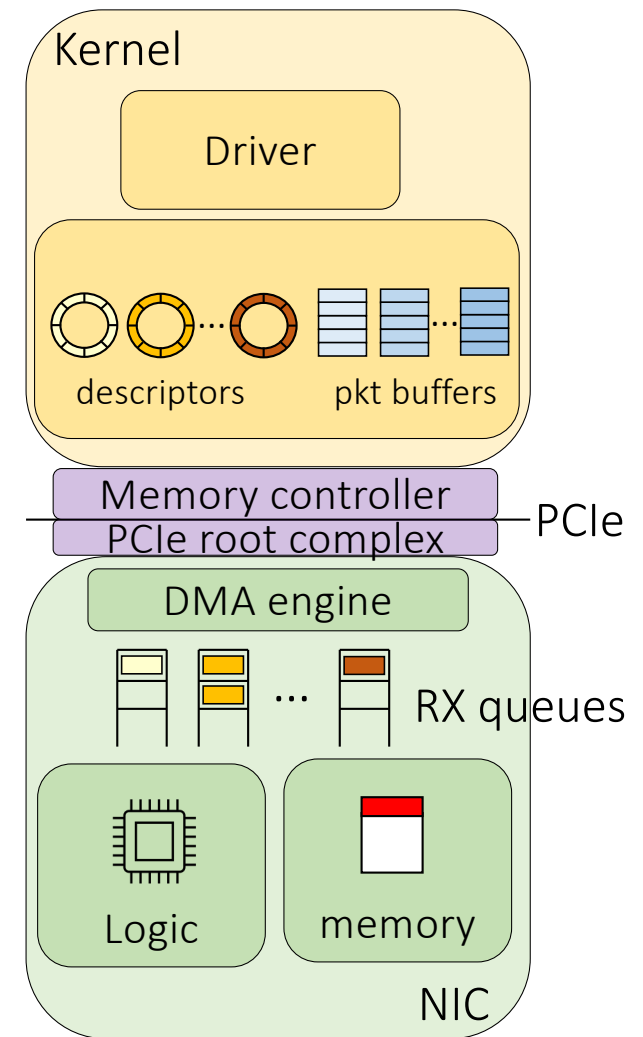
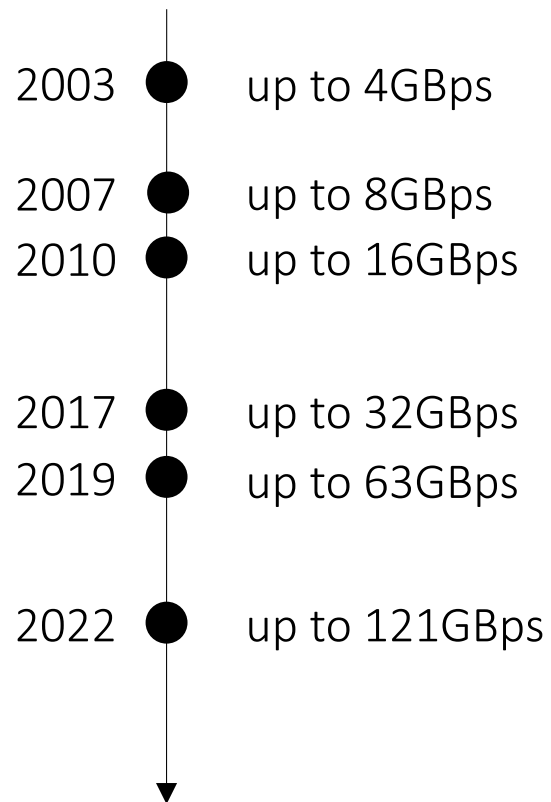
Xeon E5 has 547ns median latency and 1136ns max

Xeon E3 has 1213ns median latency and 5.38ms max

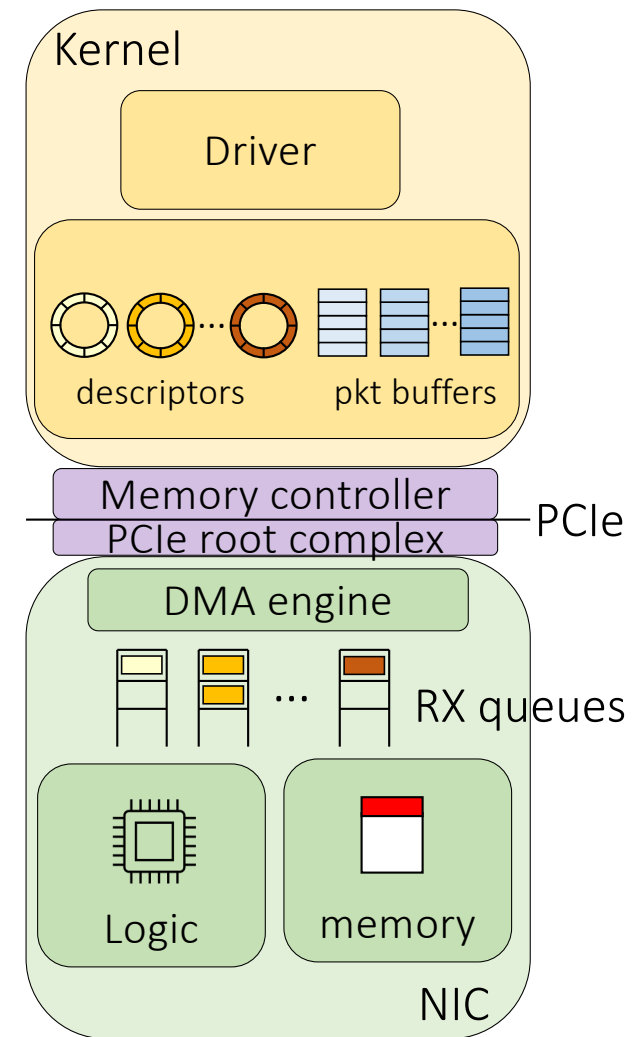
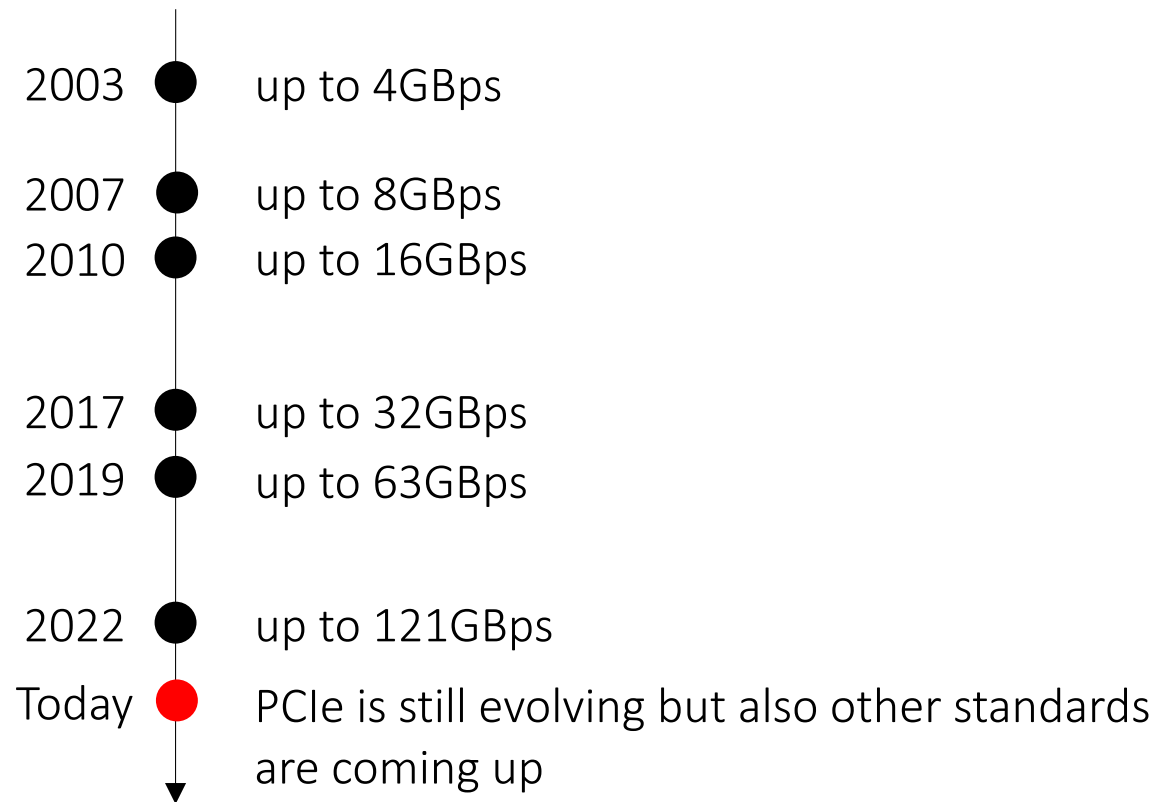
This is dependent on PCIe root complex implementation!



PCIe is continuously evolving



PCIe is continuously evolving





Compute eXpress Link

- Replacement for PCIe
 - Same physical layer, signalling and form factor
 - You can plug in a CXL or PCIe card, either/both will work
- Lower latency
 - Simplifies PCIe protocol to bring minimum latency down to around 200ns (PCIe can cost at least around 400ns)

NIC speeds: 400G today, 800G soon
At 400G, a 4kB packet is 80ns
At 400G, a 64B packet is 1.25ns

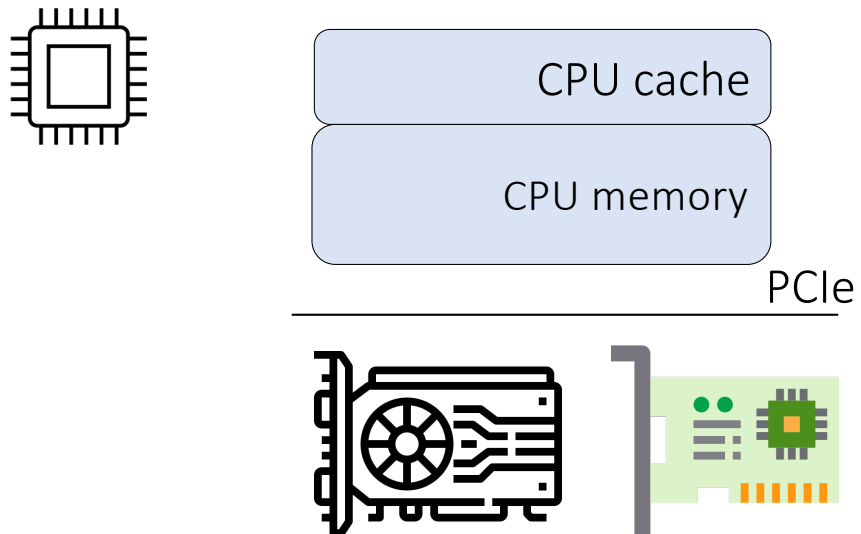


Compute eXpress Link

- Replacement for PCIe
 - Same physical layer, signalling and form factor
 - You can plug in a CXL or PCIe card, either/both will work
- Lower latency
 - Simplifies PCIe protocol to bring minimum latency down to around 200ns (PCIe can cost at least around 400ns)
- Cache coherent
 - A read from one device can be put in cache and accessed later without pain
 - This is because the device can invalidate the cache line

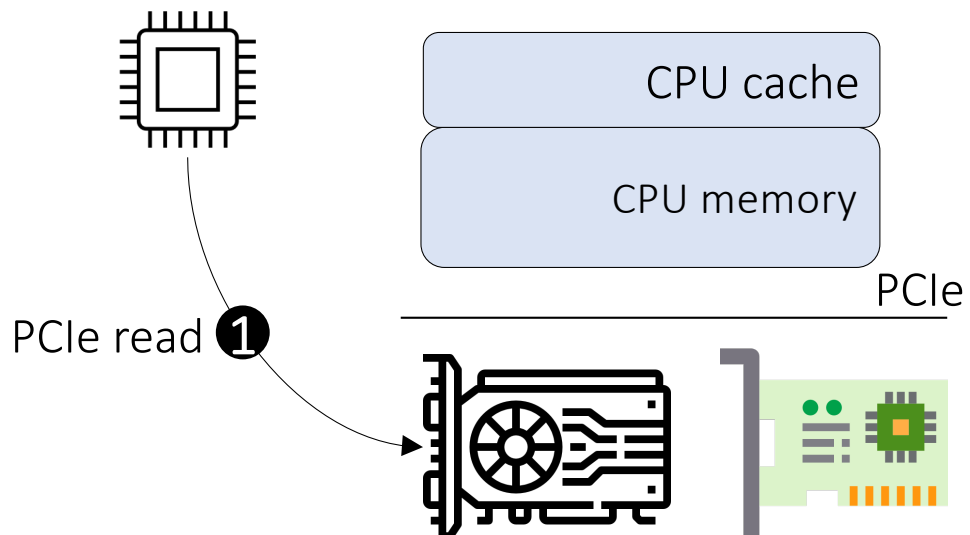
How PCIe works

- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



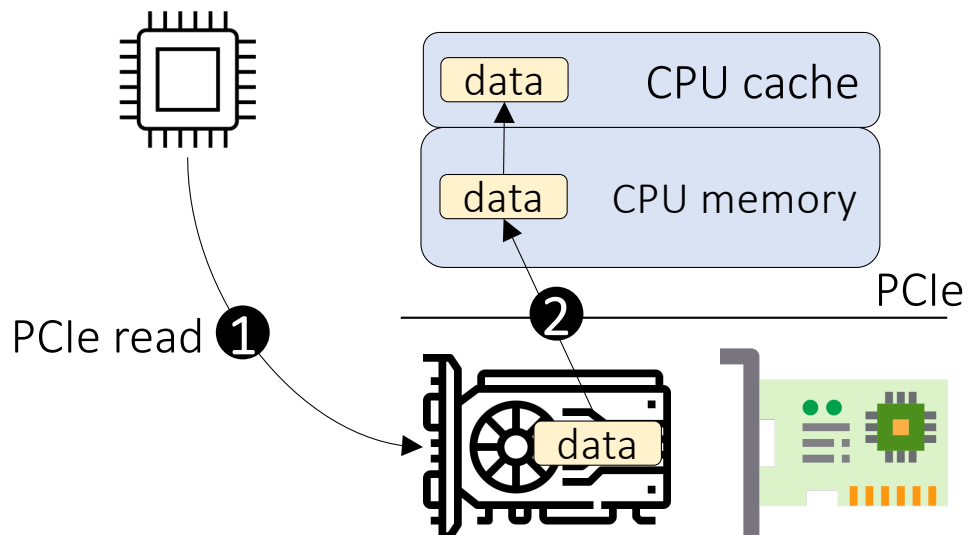
How PCIe works

- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



How PCIe works

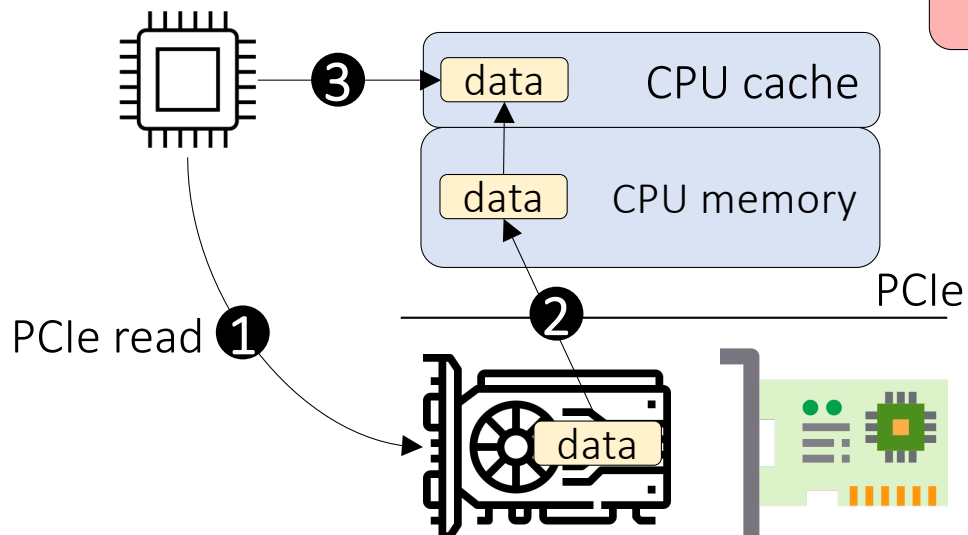
- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



How PCIe works

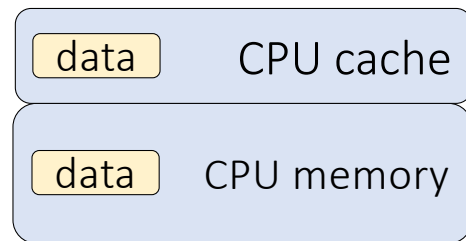
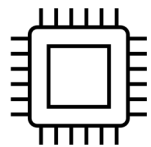
- Just a data bus
 - Can read/write but memories on two sides of the bus are independent

CPU can now read from cache, but cannot know if, in the meantime, *data* on the accelerator has changed



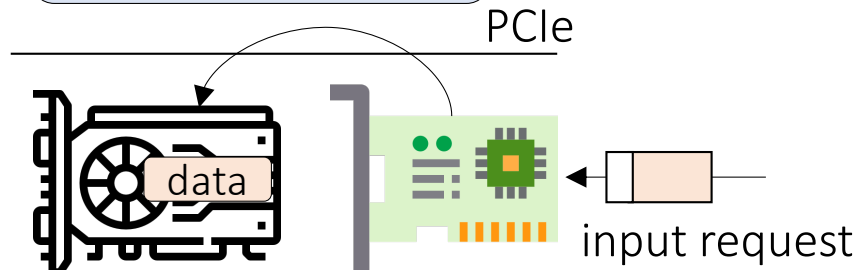
How PCIe works

- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



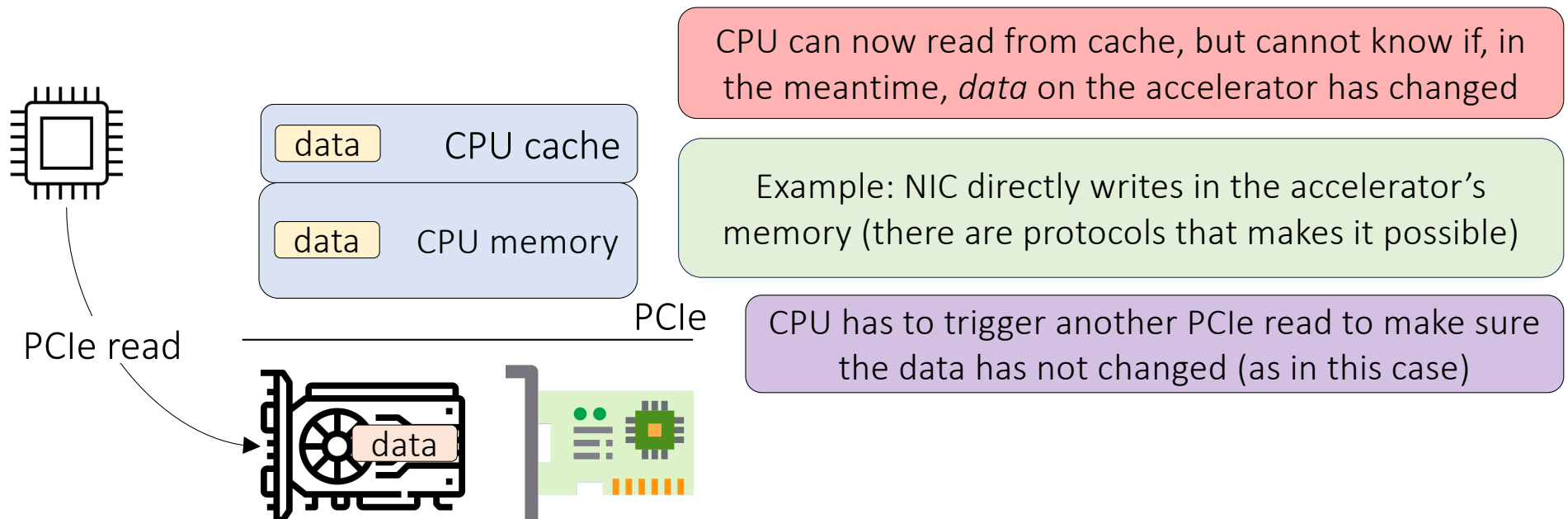
CPU can now read from cache, but cannot know if, in the meantime, *data* on the accelerator has changed

Example: NIC directly writes in the accelerator's memory (there are protocols that makes it possible)



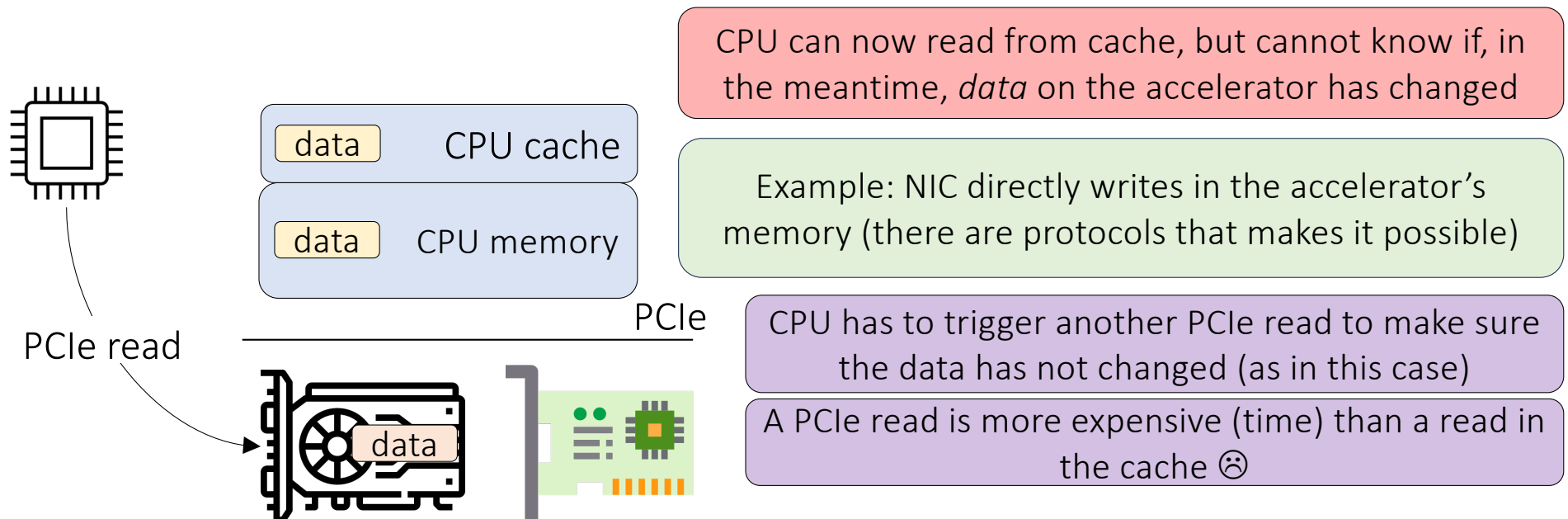
How PCIe works

- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



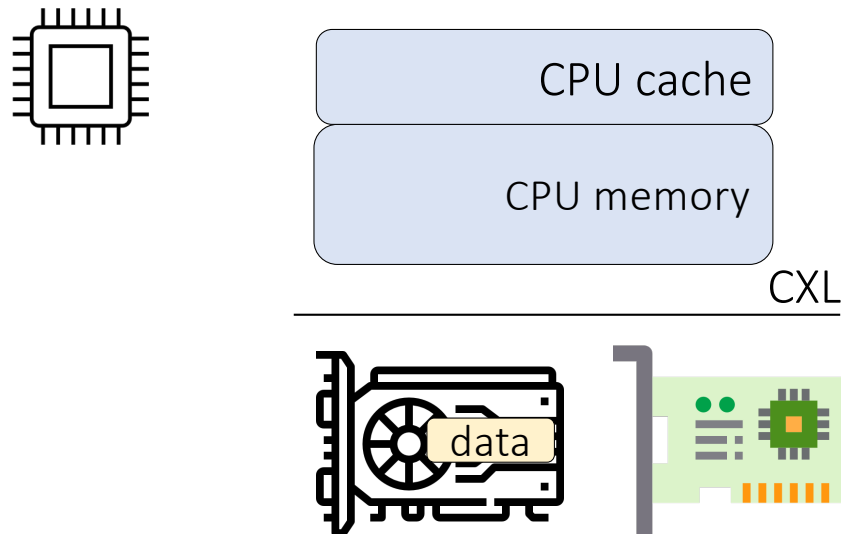
How PCIe works

- Just a data bus
 - Can read/write but memories on two sides of the bus are independent



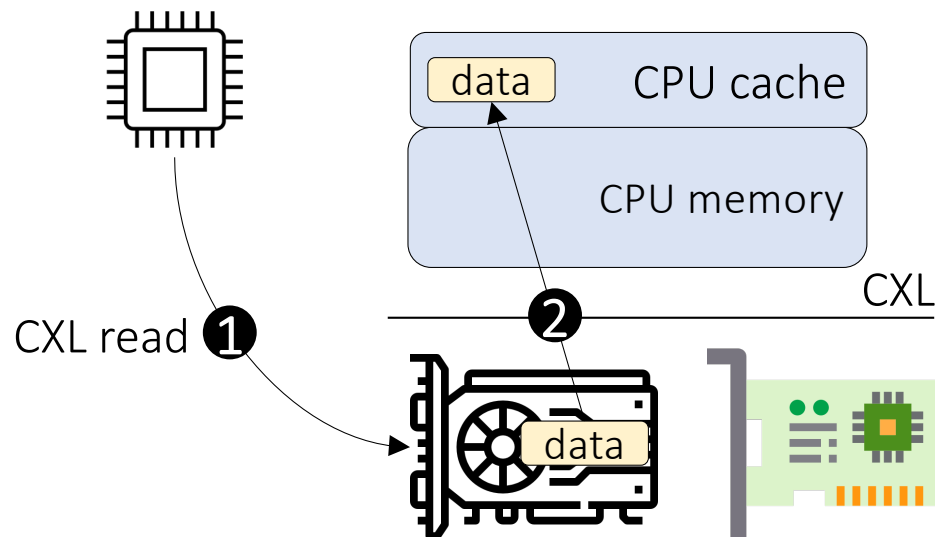
The beauty of CXL

- CXL can guarantee a cache coherent domain



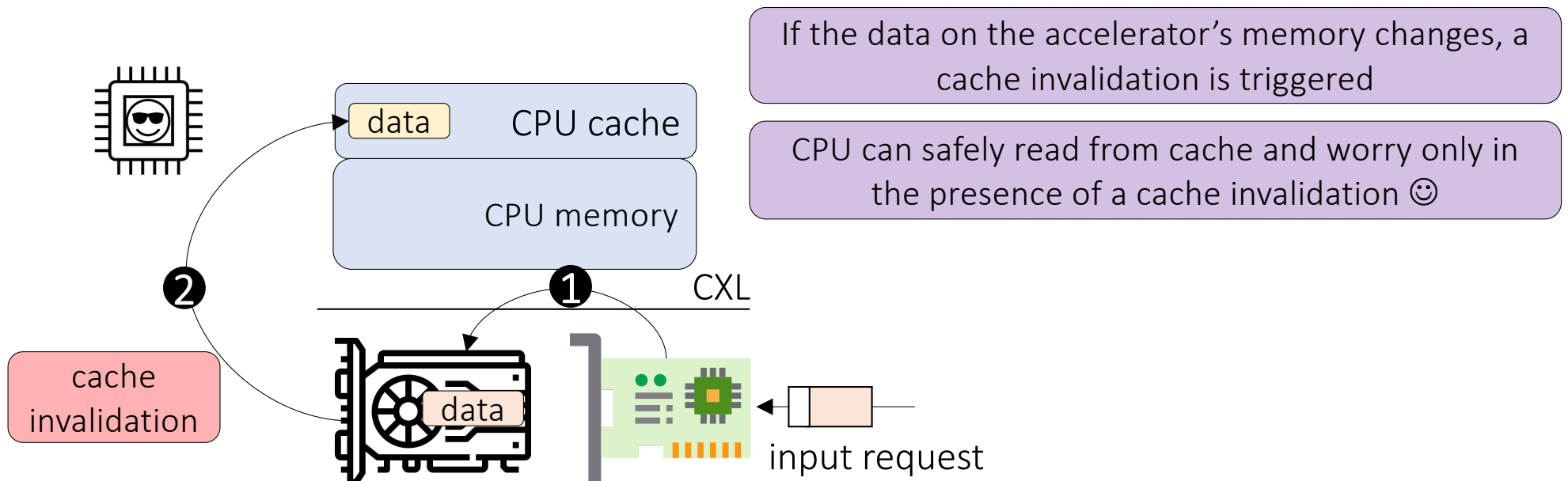
The beauty of CXL

- CXL can guarantee a cache coherent domain



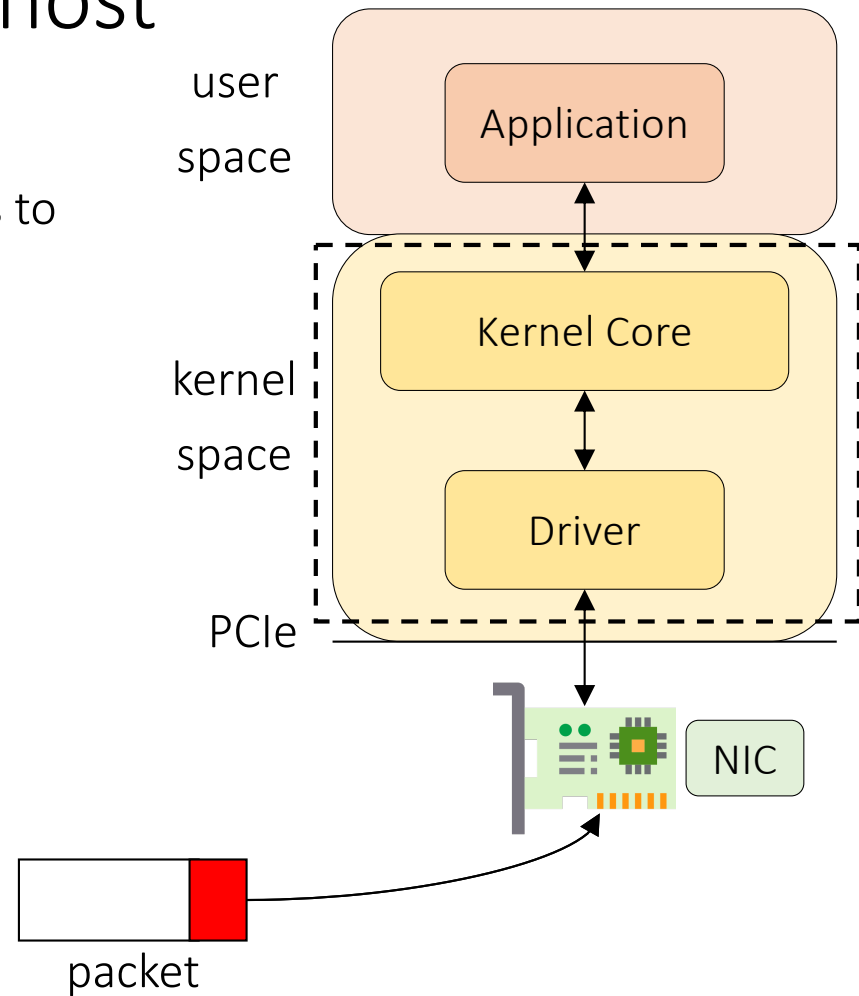
The beauty of CXL

- CXL can guarantee a cache coherent domain



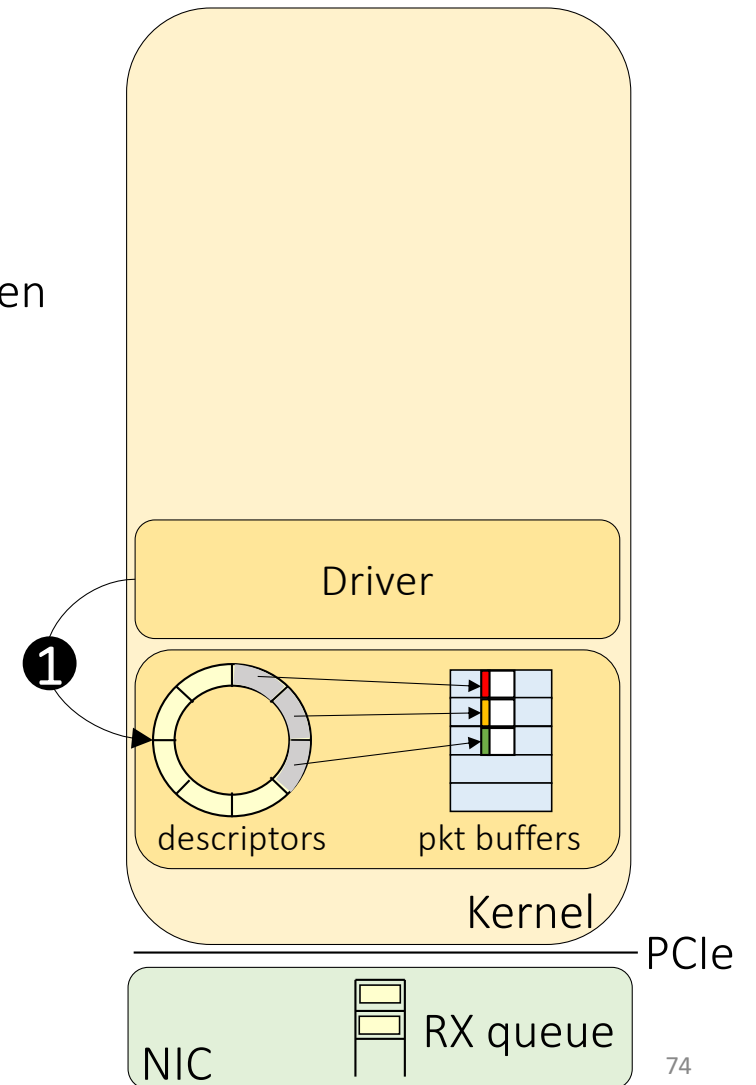
Life of a packet at the end-host

- From an high level perspective every packet has to cross:
 1. Network Interface Card (NIC)
 2. PCIe (interconnect between NIC and server)
 3. NIC driver
 4. Kernel
- ...to finally reach the application in user space



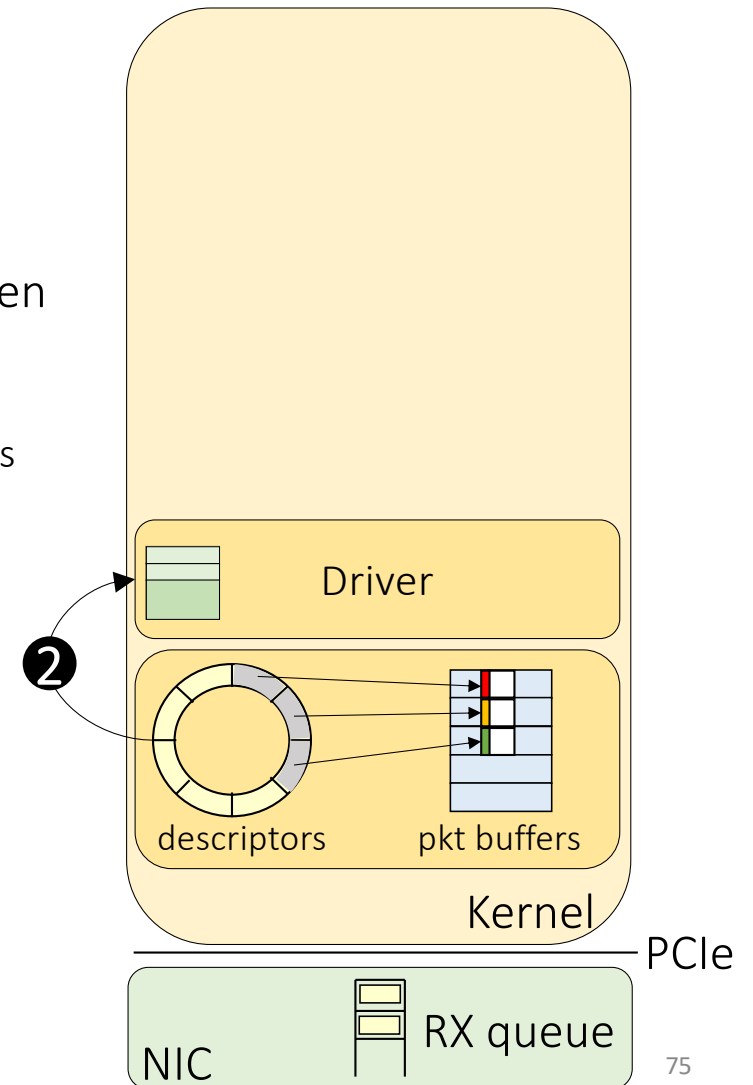
The NIC driver

- Regardless the NIC works in polling mode or interrupt-driven
 - The driver reads the descriptors ring to get a new packet



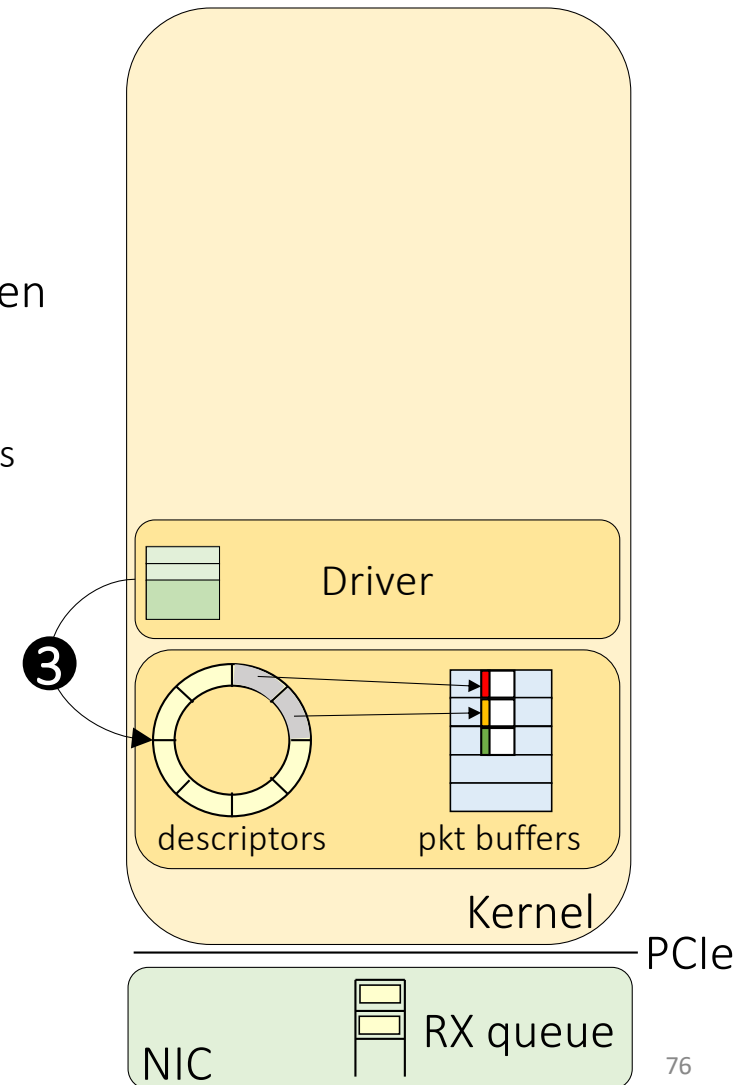
The NIC driver

- Regardless the NIC works in polling mode or interrupt-driven
 - The driver reads the descriptors ring to get a new packet
 - Fetches the packet from memory and allocates a specific data structure to handle it during its journey towards user-space: this is called *Socket Buffer* or *sk_buff* or *skb*



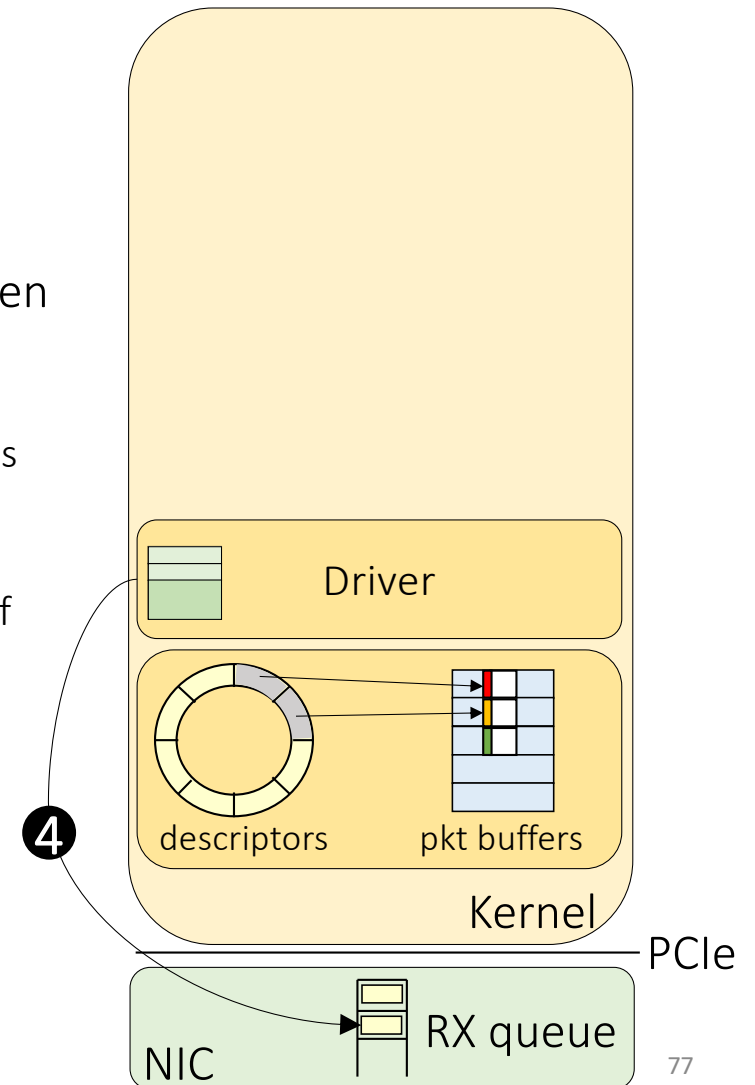
The NIC driver

- Regardless the NIC works in polling mode or interrupt-driven
 - The driver reads the descriptors ring to get a new packet
 - Fetches the packet from memory and allocates a specific data structure to handle it during its journey towards user-space: this is called *Socket Buffer* or *sk_buff* or *skb*
 - Clears the entry in the descriptor ring



The NIC driver

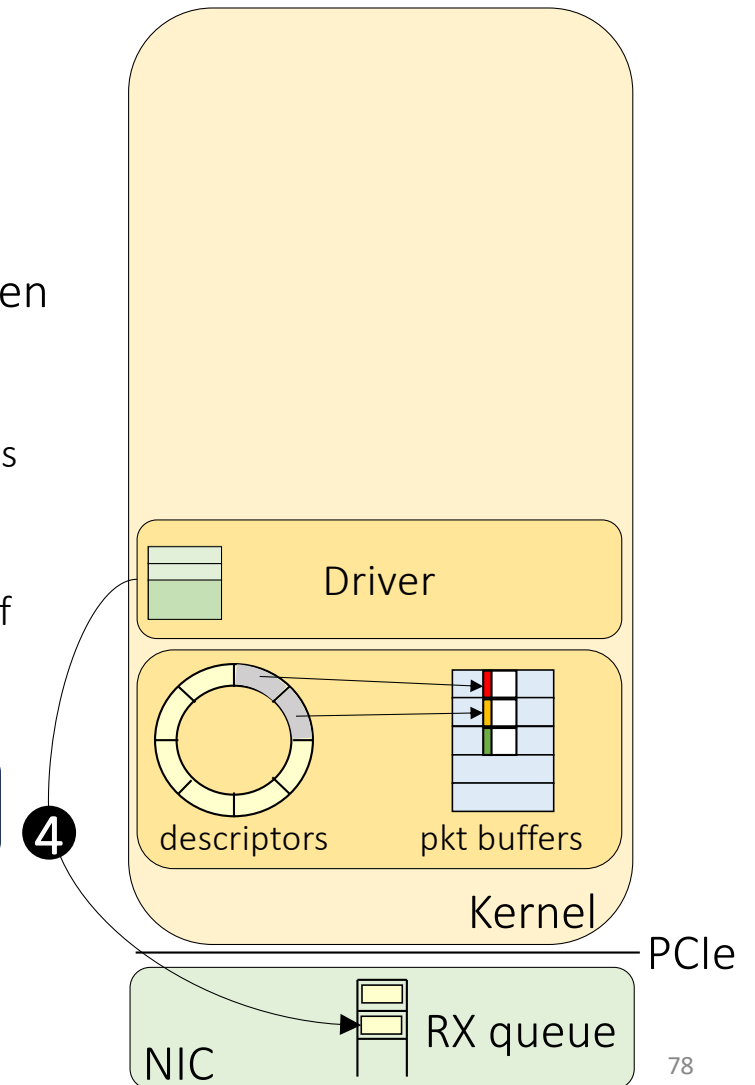
- Regardless the NIC works in polling mode or interrupt-driven
 - The driver reads the descriptors ring to get a new packet
 - Fetches the packet from memory and allocates a specific data structure to handle it during its journey towards user-space: this is called *Socket Buffer* or *sk_buff* or *skb*
 - Clears the entry in the descriptor ring
 - Writes back a new descriptor in the NIC's RX queue from a pool of free descriptors



The NIC driver

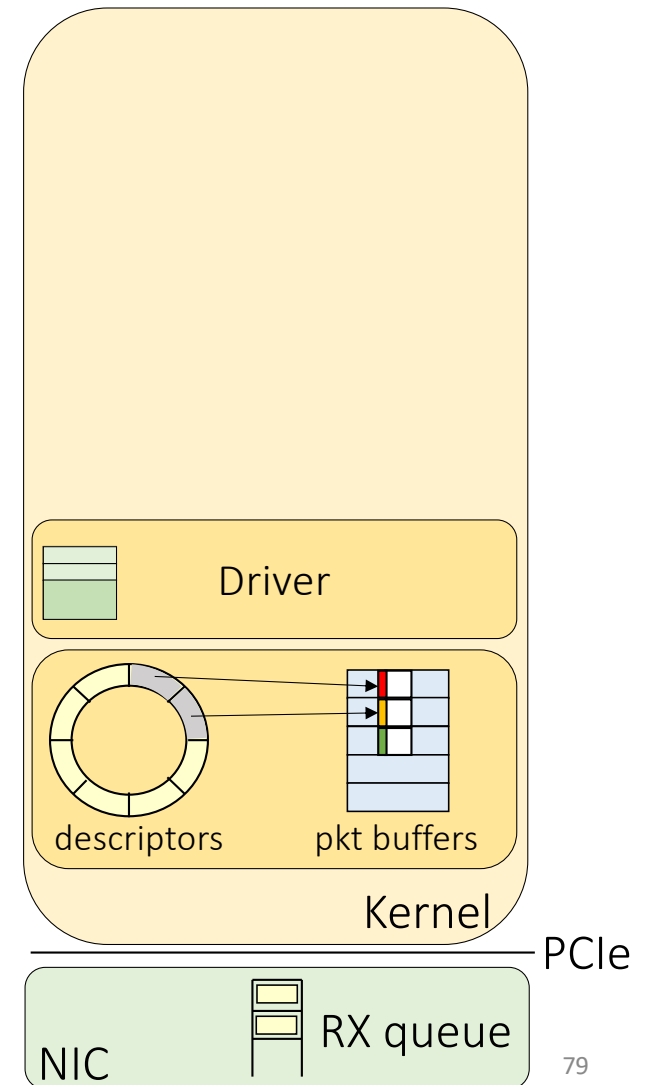
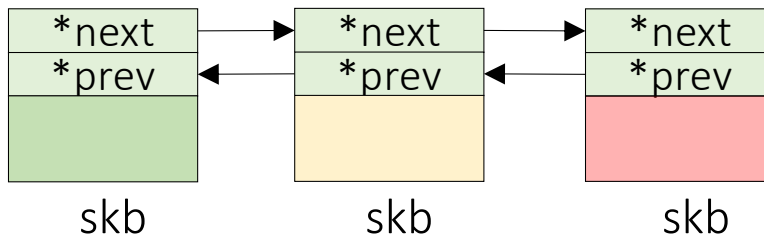
- Regardless the NIC works in polling mode or interrupt-driven
 - The driver reads the descriptors ring to get a new packet
 - Fetches the packet from memory and allocates a specific data structure to handle it during its journey towards user-space: this is called *Socket Buffer* or *sk_buff* or *skb*
 - Clears the entry in the descriptor ring
 - Writes back a new descriptor in the NIC's RX queue from a pool of free descriptors

The packet is not yet removed from the pkt buffers, since it is still pending kernel-side processing



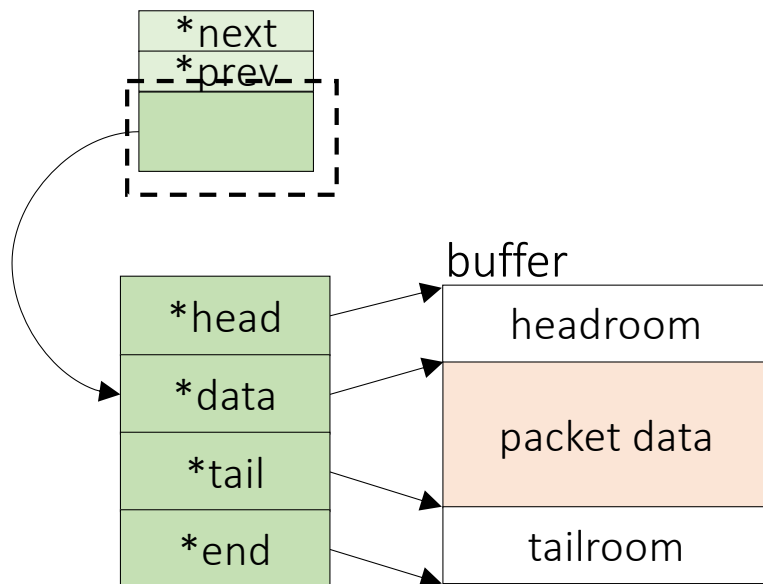
The Socket Buffer

- Socket buffers are usually organised in circular lists to speed up certain operations (as we will see later)



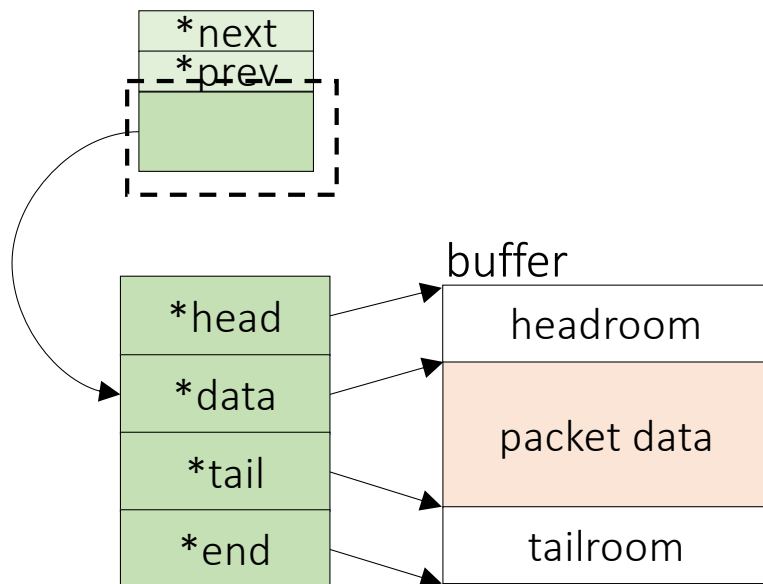
The Socket Buffer

- The skb data structure keeps reference to where the packet is stored in memory



The Socket Buffer

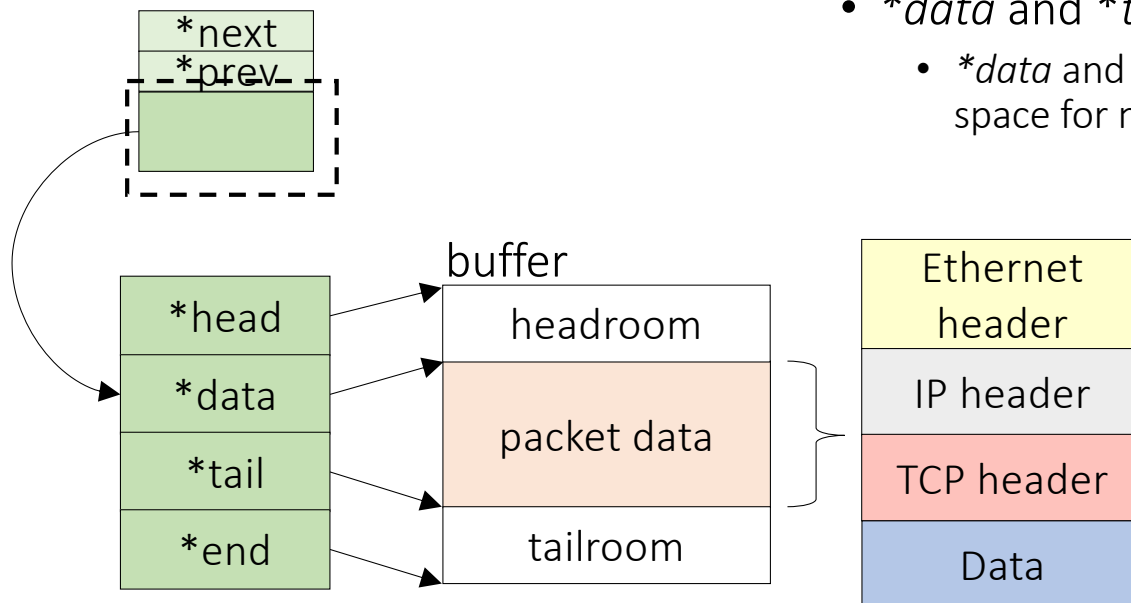
- The skb data structure keeps reference to where the packet is stored in memory



- **head* and **end* points to the limits of the buffer

The Socket Buffer

- The skb data structure keeps reference to where the packet is stored in memory

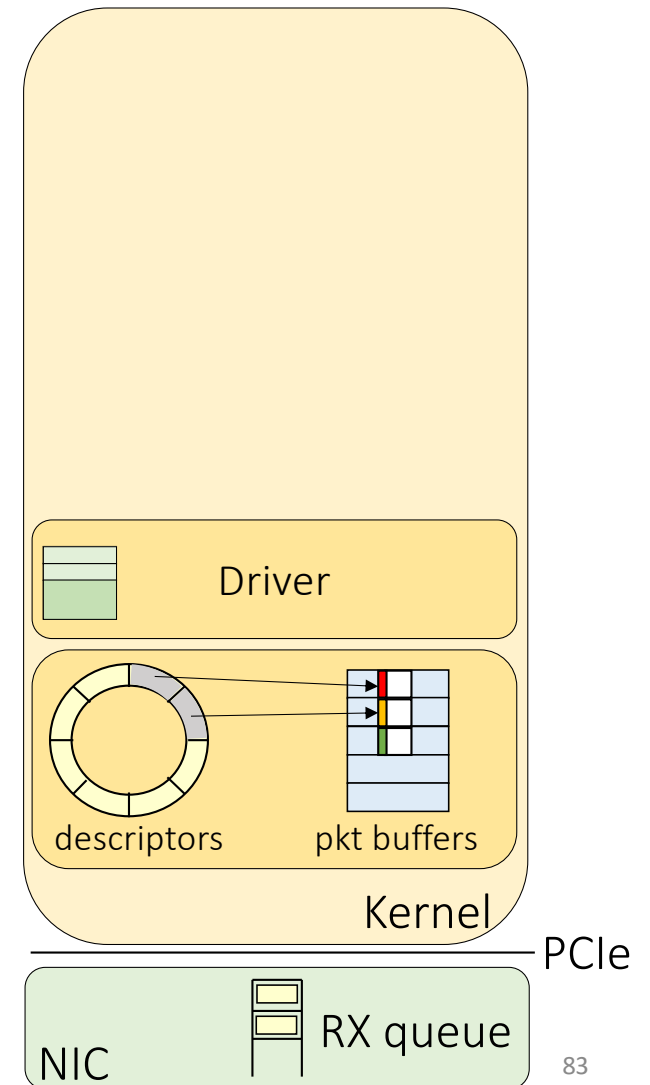


- `*head` and `*end` points to the limits of the buffer
- `*data` and `*tail` delimit the packet data
 - `*data` and `*tail` can be moved (inside buffer) to create space for new headers without allocating a new structure

The Socket Buffer

- skb is more complex than that and can also store
 - Metadata (e.g., input interface, timestamp)
 - Kernel internal information
 - Value of the most useful fields to speed up processing

```
struct sk_buff {  
    ...  
    struct net_device * dev; // The input or output device  
    ktime_t tstamp; // The timestamp of reception  
    __be16 protocol; //The L3 protocol of the packet  
    __u16 transport_header; // Pointers to the various headers in the buffer  
    __u16 network_header;  
    ...  
}
```

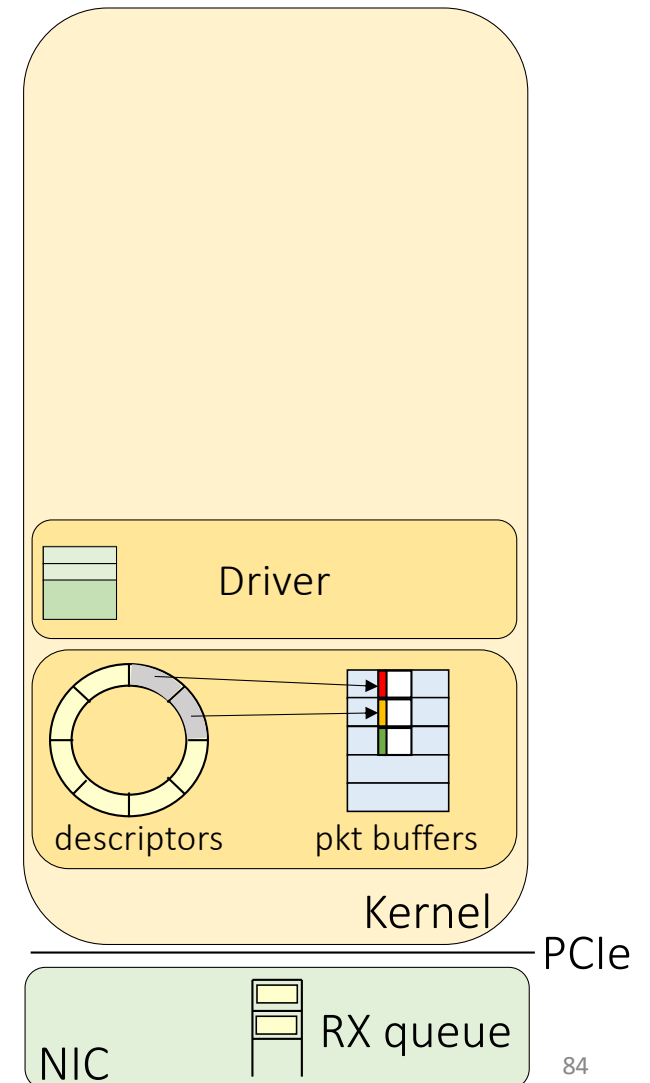


The Socket Buffer

- skb is more complex than that and can also store
 - Metadata (e.g., input interface, timestamp)
 - Kernel internal information
 - Value of the most useful fields to speed up processing

```
struct sk_buff {  
    ...  
    struct net_device * dev; // The input or output device  
    ktime_t tstamp; // The timestamp of reception  
    __be16 protocol; //The L3 protocol of the packet  
    __u16 transport_header; // Pointers to the various headers in the buffer  
    __u16 network_header;  
    ...  
}
```

..and many more!



The Socket Buffer



```
/**
 * struct sk_buff - socket buffer
 *
 * @next: Next buffer in list
 * @prev: Previous buffer in list
 * @timestamp: Time we arrived/left
 * @skb_mstamp_ns: (aka @tstamp) earliest departure time; start
 * point for retransmit timer
 * @rbnode: RB tree node, alternative to next/prev for netem/tcp
 * @list: queue head
 * @ll_node: anchor in an llist (eg socket defer_list)
 * @sk: Socket we are owned by
 * @ip_defrag_offset: (aka @sk) alternate use of @sk, used in
 * fragmentation management
 * @dev: Device we arrived on/are leaving by
 * @dev_scratch: (aka @dev) alternate use of @dev when @dev would be
 %NULL
 * @cb: Control buffer. Free for use by every layer. Put private vars here
 * @skb_refdst: destination entry (with norefcount bit)
 * @sp: the security path, used for xfrm
 * @len: Length of actual data
 * @data_len: Data length
 * @mac_len: Length of link layer header
 * @hdr_len: writable header length of cloned skb
 * @csum: Checksum (must include start/offset pair)
 * @csum_start: Offset from skb->head where checksumming should start
 * @csum_offset: Offset from csum_start where checksum should be stored
 * @priority: Packet queueing priority
 * @ignore_df: allow local fragmentation
 * @cloned: Head may be cloned (check refcnt to be sure)
 * @ip_summed: Driver fed us an IP checksum
 * @nohdr: Payload reference only, must not modify header
 * @pkt_type: Packet class
 * @fclone: skbuff clone status
 * @ipvs_property: skbuff is owned by ipvs
 * @inner_protocol_type: whether the inner protocol is
 * ENCAP_TYPE_ETHER or ENCAP_TYPE_IPPROTO
 * @remcsum_offload: remote checksum offload is enabled
 * @offload_fwd_mark: Packet was L2-forwarded in hardware
 * @offload_l3_fwd_mark: Packet was L3-forwarded in hardware
```

```
 * @tc_skip_classify: do not classify packet. set by IFB device
 * @tc_at_ingress: used within tc_classify to distinguish in/egress
 * @redirected: packet was redirected by packet classifier
 * @from_ingress: packet was redirected from the ingress path
 * @nf_skip_egress: packet shall skip nf egress - see netfilter_netdev.h
 * @peeked: this packet has been seen already, so stats have been
 * done for it, don't do them again
 * @nf_trace: netfilter packet trace flag
 * @protocol: Packet protocol from driver
 * @destructor: Destruct function
 * @tcp_tsorted_anchor: list structure for TCP (tp->tsorted_sent_queue)
 * @sk_redir: socket redirection information for skmsg
 * @_nfct: Associated connection, if any (with nfctinfo bits)
 * @nf_bridge: Saved data about a bridged frame - see br_netfilter.c
 * @skb_iif: ifindex of device we arrived on
 * @tc_index: Traffic control index
 * @hash: the packet hash
 * @queue_mapping: Queue mapping for multiqueue devices
 * @head_frag: skb was allocated from page fragments,
 * not allocated by kmalloc() or vmalloc().
 * @pfmemalloc: skbuff was allocated from PFMEMALLOC reserves
 * @pp_recycle: mark the packet for recycling instead of freeing
 * (implies page_pool support on driver)
 * @active_extensions: active extensions (skb_ext_id types)
 * @ndisc_nodetype: router type (from link layer)
 * @ooo_okay: allow the mapping of a socket to a queue to be changed
 * @l4_hash: indicate hash is a canonical 4-tuple hash over transport
 * ports.
 * @sw_hash: indicates hash was computed in software stack
 * @wifi_acked_valid: wifi_acked was set
 * @wifi_acked: whether frame was acked on wifi or not
 * @no_fcs: Request NIC to treat last 4 bytes as Ethernet FCS
 * @encapsulation: indicates the inner headers in the skbuff are valid
 * @encap_hdr_csum: software checksum is needed
 * @csum_valid: checksum is already valid
 * @csum_not_inet: use CRC32c to resolve CHECKSUM_PARTIAL
 * @csum_complete_sw: checksum was completed by software
 * @csum_level: indicates the number of consecutive checksums
 * found in the packet minus one that have been verified as
```

```
 * CHECKSUM_UNNECESSARY (max 3)
 * @dst_pending_confirm: need to confirm neighbour
 * @decrypted: Decrypted SKB
 * @slow_gro: state present at GRO time, slower prepare step required
 * @mono_delivery_time: When set, skb->tstamp has the
 * delivery_time in mono clock base (i.e. EDT). Otherwise, the
 * skb->tstamp has the (rcv) timestamp at ingress and
 * delivery_time at egress.
 * @napi_id: id of the NAPI struct this skb came from
 * @sender_cpu: (aka @napi_id) source CPU in XPS
 * @alloc_cpu: CPU which did the skb allocation.
 * @secmark: security marking
 * @mark: Generic packet mark
 * @reserved_tailroom: (aka @mark) number of bytes of free space
 * available at the tail of an sk_buff
 * @vlan_present: VLAN tag is present
 * @vlan_proto: vlan encapsulation protocol
 * @vlan_tci: vlan tag control information
 * @inner_protocol: Protocol (encapsulation)
 * @inner_ipproto: (aka @inner_protocol) stores ipproto when
 * skb->inner_protocol_type == ENCAP_TYPE_IPPROTO;
 * @inner_transport_header: Inner transport layer header (encapsulation)
 * @inner_network_header: Network layer header (encapsulation)
 * @inner_mac_header: Link layer header (encapsulation)
 * @transport_header: Transport layer header
 * @network_header: Network layer header
 * @mac_header: Link layer header
 * @kcv_handle: KCOV remote handle for remote coverage collection
 * @tail: Tail pointer
 * @end: End pointer
 * @head: Head of buffer
 * @data: Data head pointer
 * @truesize: Buffer size
 * @users: User count - see {datagram,tcp}.c
 * @extensions: allocated extensions, valid if active_extensions is nonzero
 */
```

The Socket Buffer



```
/**
 * struct sk_buff - socket buffer
 *
 * @next: Next buffer in list
 * @prev: Previous buffer in list
 * @tstamp: Time we arrived/left
 * @skb_mstamp_ns: (aka @tstamp) earliest departure time; start
 * point for retransmit timer
 * @rbnode: RB tree node, alternative to next/prev for netem/tcp
 * @list: queue head
 * @ll_node: anchor in an llist (eg socket defer_list)
 * @sk: Socket we are owned by
 * @ip_defrag_offset: (aka @sk) alternate use of @sk, used in
 * fragmentation management
 * @dev: Device we arrived on/are leaving by
 * @dev_scratch: (aka @dev) alternate use of @dev when @dev would be
 %NULL
 * @cb: Control buffer. Free for use by every layer. Put private vars here
 * @skb_refdst: destination entry (with norefcount bit)
 * @sp: the security path, used for xfrm
 * @len: Length of actual data
 * @data_len: Data length
 * @mac_len: Length of link layer header
 * @hdr_len: writable header length of cloned skb
 * @csum: Checksum (must include start/offset pair)
 * @csum_start: Offset from skb->head where checksumming should start
 * @csum_offset: Offset from csum_start where checksum should be stored
 * @priority: Packet queueing priority
 * @ignore_df: allow local fragmentation
 * @cloned: Head may be cloned (check refcnt to be sure)
 * @ip_summed: Driver fed us an IP checksum
 * @nohdr: Payload reference only, must not modify header
 * @pkt_type: Packet class
 * @fclone: skbuff clone status
 * @ipvs_property: skbuff is owned by ipvs
 * @inner_protocol_type: whether the inner protocol is
 * ENCAP_TYPE_ETHER or ENCAP_TYPE_IPPROTO
 * @remcsum_offload: remote checksum offload is enabled
 * @offload_fwd_mark: Packet was L2-forwarded in hardware
 * @offload_l3_fwd_mark: Packet was L3-forwarded in hardware
```

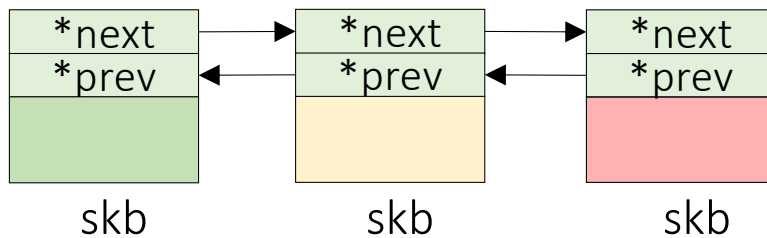
```
 * @tc_skip_classify: do not classify packet. set by IFB device
 * @tc_at_ingress: used within tc_classify to distinguish in/egress
 * @redirected: packet was redirected by packet classifier
 * @from_ingress: packet was redirected from the ingress path
 * @nf_skip_egress: packet shall skip nf egress - see netfilter_netdev.h
 * @peeked: this packet has been seen already, so stats have been
 * done for it, don't do them again
 * @nf_trace: netfilter packet trace flag
 * @protocol: Packet protocol from driver
 * @destructor: Destruct function
 * @tcp_tsorted_anchor: list structure for TCP (tp->tsorted_sent_queue)
 * @sk_redir: socket redirection information for skmsg
 * @_nfct: Associated connection, if any (with nfctinfo bits)
 * @nf_bridge: Saved data about a bridged frame - see br_netfilter.c
 * @skb_iif: ifindex of device we arrived on
 * @tc_index: Traffic control index
 * @hash: the packet hash
 * @queue_mapping: Queue mapping for multiqueue devices
 * @head_frag: skb was allocated from page fragments,
 * not allocated by kmalloc() or vmalloc().
 * @pfmemalloc: skbuff was allocated from PFMEMALLOC reserves
 * @pp_recycle: mark the packet for recycling instead of freeing
 * (implies page_pool support on driver)
 * @active_extensions: active extensions (skb_ext_id types)
 * @ndisc_nodetype: router type (from link layer)
 * @ooo_okay: allow the mapping of a socket to a queue to be changed
 * @l4_hash: indicate hash is a canonical 4-tuple hash over transport
 * ports.
 * @sw_hash: indicates hash was computed in software stack
 * @wifi_acked_valid: wifi_acked was set
 * @wifi_acked: whether frame was acked on wifi or not
 * @no_fcs: Request NIC to treat last 4 bytes as Ethernet FCS
 * @encapsulation: indicates the inner headers in the skbuff are valid
 * @encap_hdr_csum: software checksum is needed
 * @csum_valid: checksum is already valid
 * @csum_not_inet: use CRC32c to resolve CHECKSUM_PARTIAL
 * @csum_complete_sw: checksum was completed by software
 * @csum_level: indicates the number of consecutive checksums
 * found in the packet minus one that have been verified as
```

```
 * CHECKSUM_UNNECESSARY (max 3)
 * @dst_pending_confirm: need to confirm neighbour
 * @decrypted: Decrypted SKB
 * @slow_gro: state present at GRO time, slower prepare step required
 * @mono_delivery_time: When set, skb->tstamp has the
 * delivery_time in mono clock base (i.e. EDT). Otherwise, the
 * skb->tstamp has the (rcv) timestamp at ingress and
 * delivery_time at egress.
 * @napi_id: id of the NAPI struct this skb came from
 * @sender_cpu: (aka @napi_id) source CPU in XPS
 * @alloc_cpu: CPU which did the skb allocation.
 * @secmark: security marking
 * @mark: Generic packet mark
 * @reserved_tailroom: (aka @mark) number of bytes of free space
 * available at the tail of an sk_buff
 * @vlan_present: VLAN tag is present
 * @vlan_proto: vlan encapsulation protocol
 * @vlan_tci: vlan tag control information
 * @inner_protocol: Protocol (encapsulation)
 * @inner_ipproto: (aka @inner_protocol) stores ipproto when
 * skb->inner_protocol_type == ENCAP_TYPE_IPPROTO;
 * @inner_transport_header: Inner transport layer header (encapsulation)
 * @inner_network_header: Network layer header (encapsulation)
 * @inner_mac_header: Link layer header (encapsulation)
 * @transport_header: Transport layer header
 * @network_header: Network layer header
 * @mac_header: Link layer header
 * @kcv_handle: KCOV remote handle for remote coverage collection
 * @tail: Tail pointer
 * @end: End pointer
 * @head: Head of buffer
 * @data: Data head pointer
 * @truesize: Buffer size
 * @users: User count - see {datagram,tcp}.c
 * @extensions: allocated extensions, valid if active_extensions is nonzero
 */
```

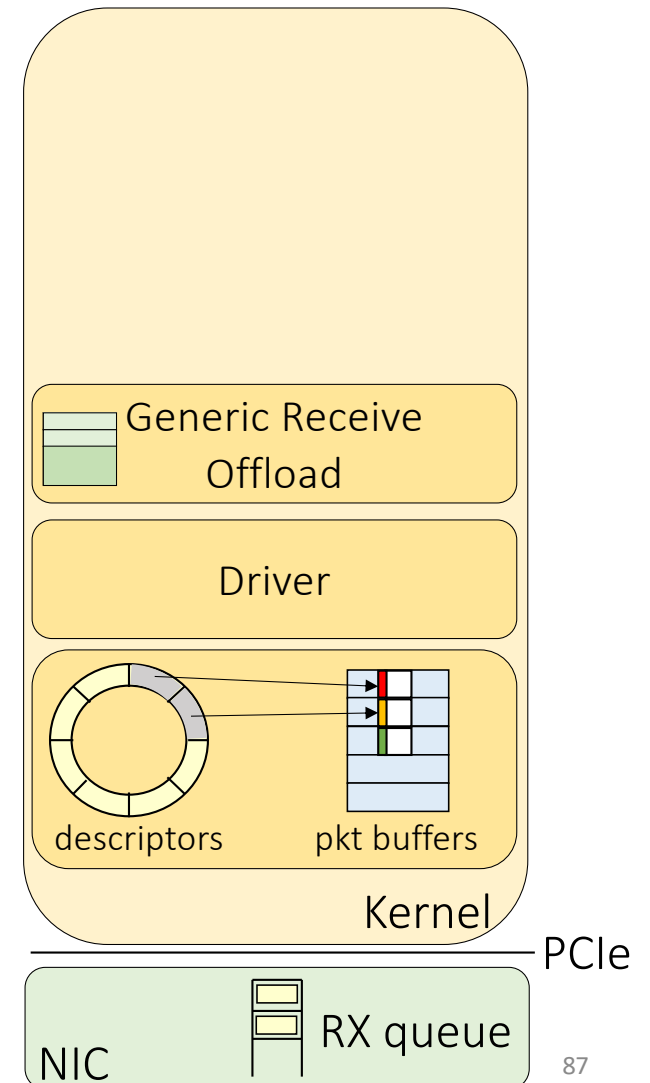
`sizeof(sk_buff) ≈ 400 B`

The network stack: GRO

- The Generic Receive Offload (GRO) module attempts to reduce the number of skbs by merging them (having them organized circularly helps!)

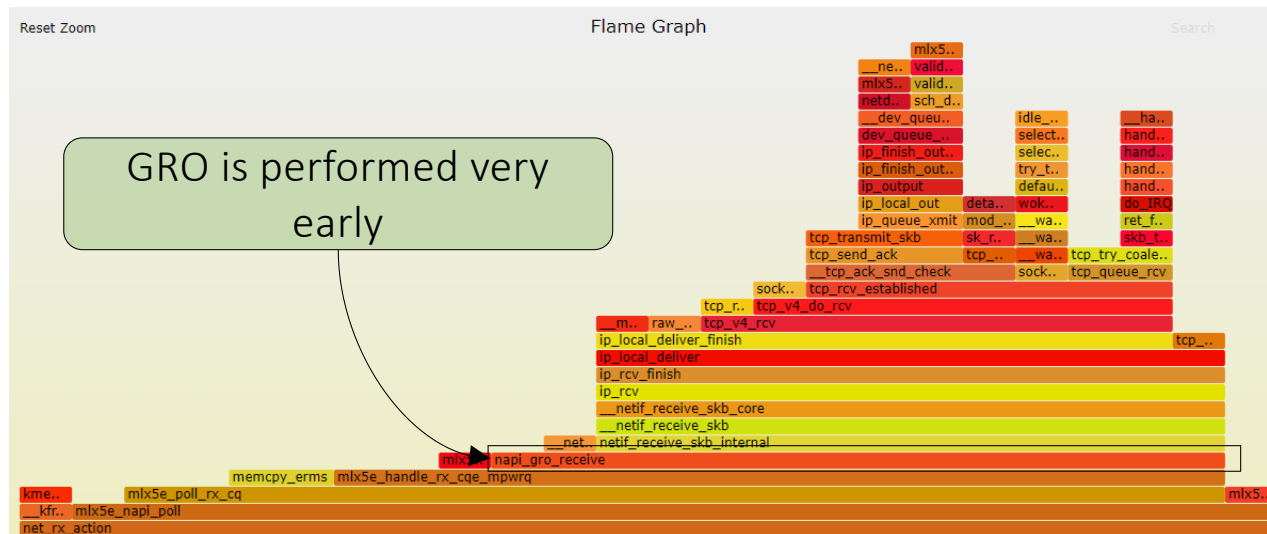


- This operation is the software counterpart of the Large Received Offload (LRO), which is performed by the NIC



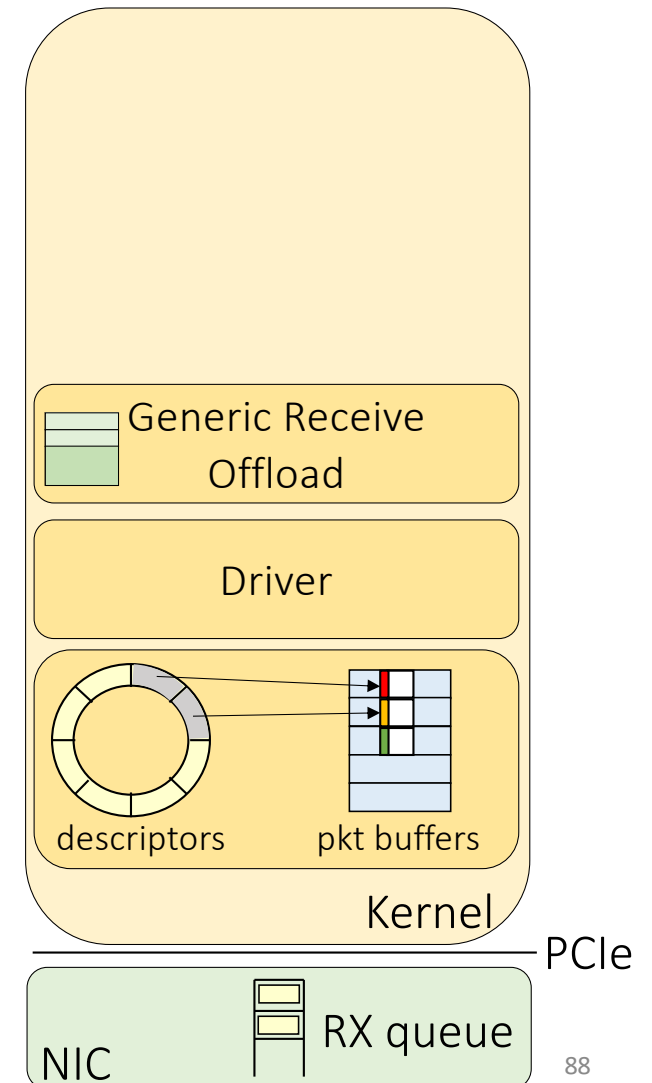
The network stack: GRO

Mellanox ConnectX-4 LX



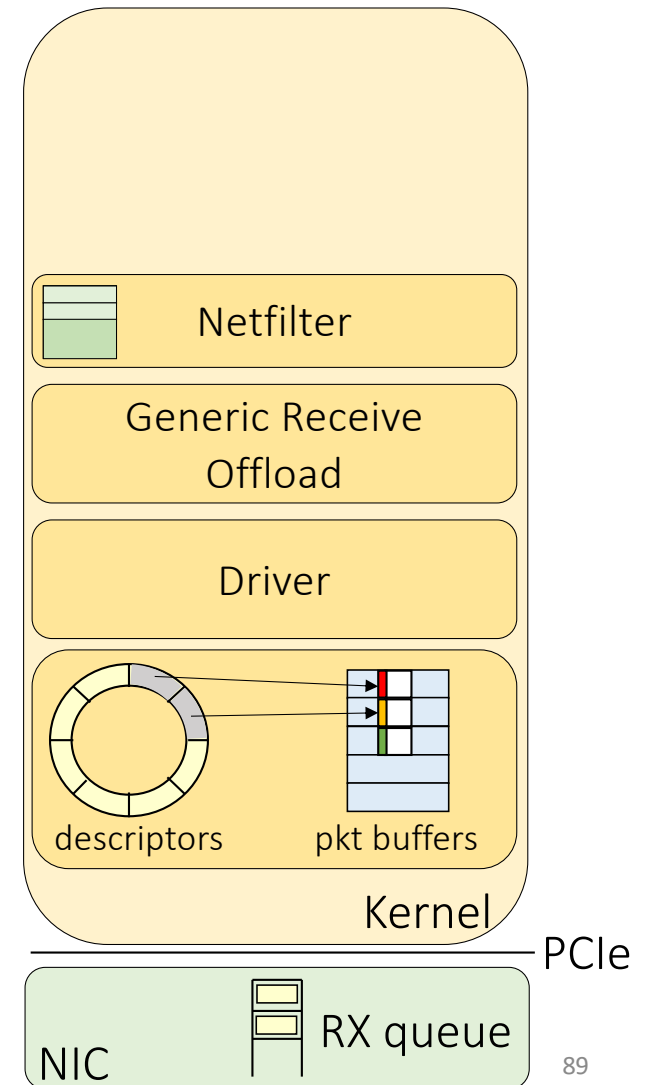
This means that all the next functions in the receive stack do much less processing (you act on a single skb for multiple packets)

<https://stackoverflow.com/questions/47332232/why-is-gro-more-efficient>



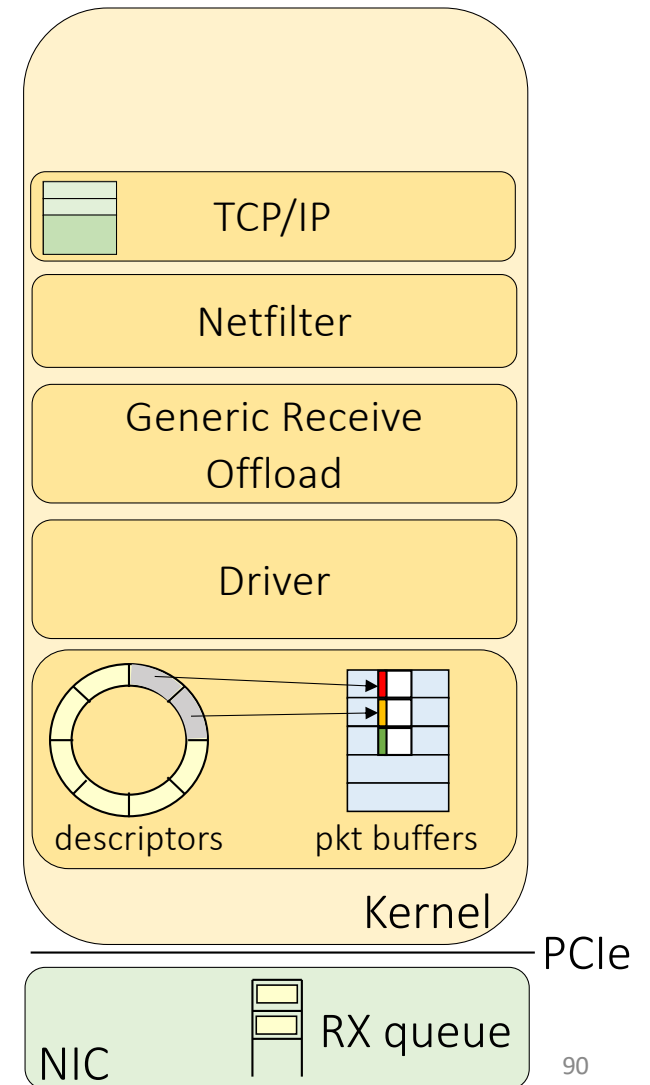
The network stack: Netfilter

- Netfilter is a framework that offers various functions and operations for packet filtering, network address translation (NAT), port translation and connection tracking
- The IPtables Linux firewall leverages Netfilter to implement its policies



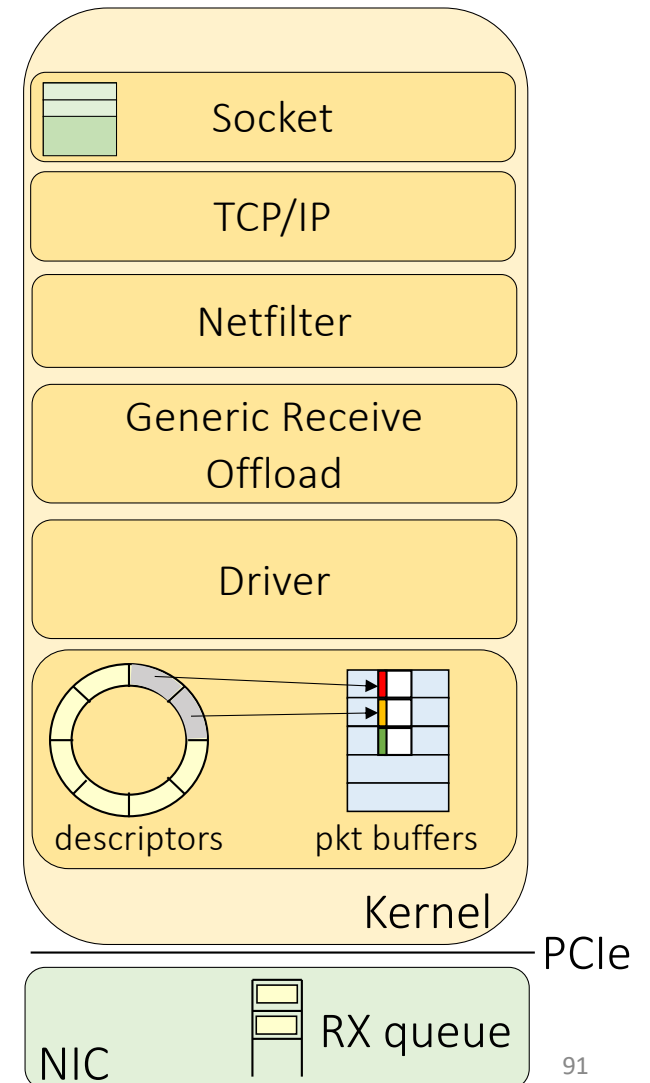
The network stack: TCP/IP

- Here all the processing related to the transport layer are performed
- For TCP packets, this means, for example:
 - Congestion control
 - Flow control
 - Connection state handling



The network stack: Socket

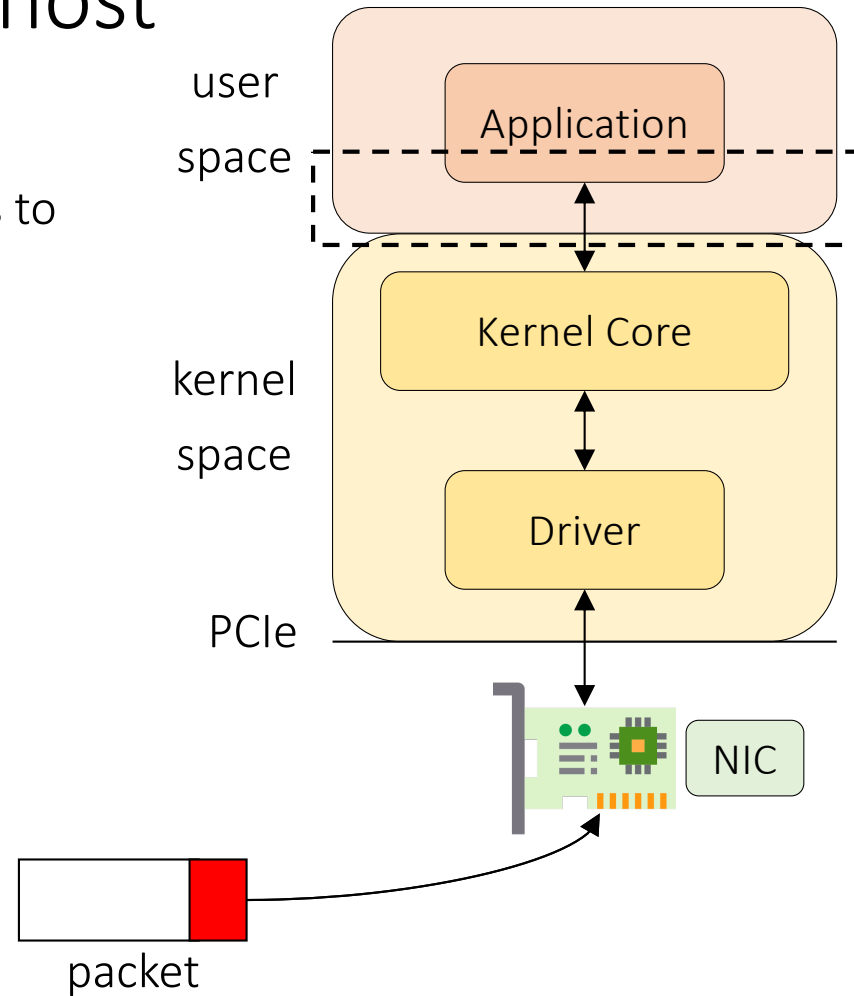
- The socket layer maintains per-connection state and buffering. It is also in charge of transferring data between kernel and user space via system calls
- Here, per-connection state refers to what it is needed by the OS and application, such as:
 - File descriptor association
 - Receive/send buffer sizes and limits
 - Ownership and lifecycle management



Life of a packet at the end-host

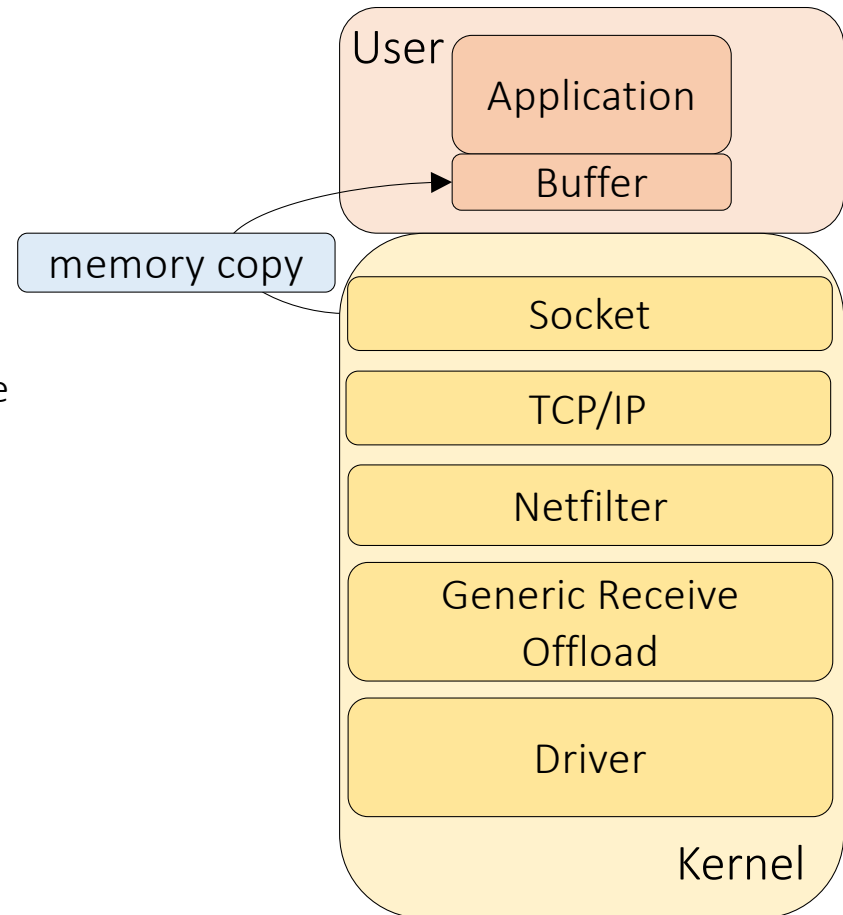
- From an high level perspective every packet has to cross:
 1. Network Interface Card (NIC)
 2. PCIe (interconnect between NIC and server)
 3. NIC driver
 4. Kernel

• ...to finally reach the application in user space



Kernel to User Space

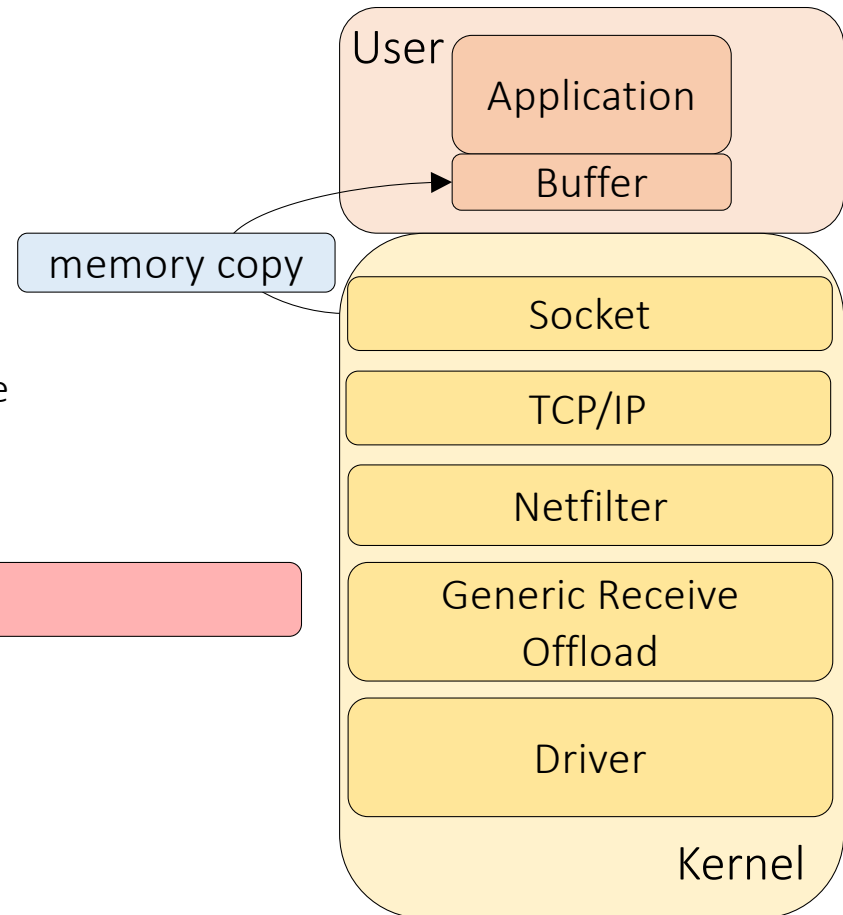
- When a user application calls `recv()`, `read()`, or similar, the socket layer:
 - Dequeues `sk_buffs` from the receive queue
 - Copies the payload data from kernel memory into the user-provided buffer using `copy_to_user()`
 - Updates socket state and wakeups



Kernel to User Space

- When a user application calls `recv()`, `read()`, or similar, the socket layer:
 - Dequeues `sk_buffs` from the receive queue
 - Copies the payload data from kernel memory into the user-provided buffer using `copy_to_user()`
 - Updates socket state and wakeups

Why a data copy?

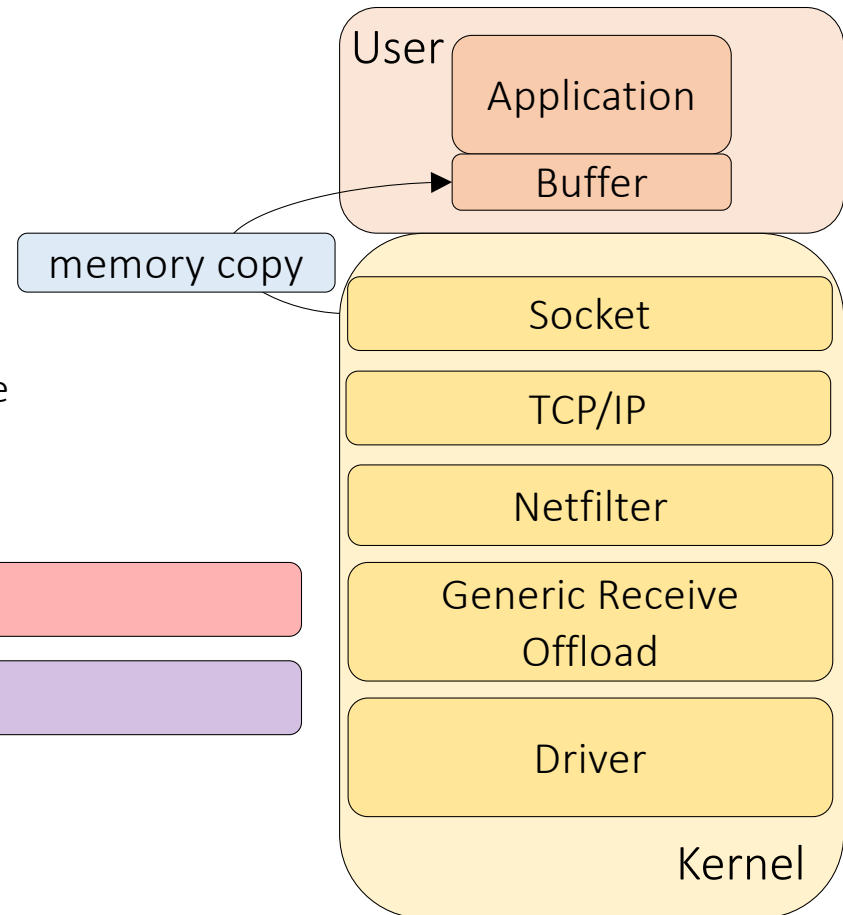


Kernel to User Space

- When a user application calls `recv()`, `read()`, or similar, the socket layer:
 - Dequeues `sk_buffs` from the receive queue
 - Copies the payload data from kernel memory into the user-provided buffer using `copy_to_user()`
 - Updates socket state and wakeups

Why a data copy?

We need to protect kernel memory!



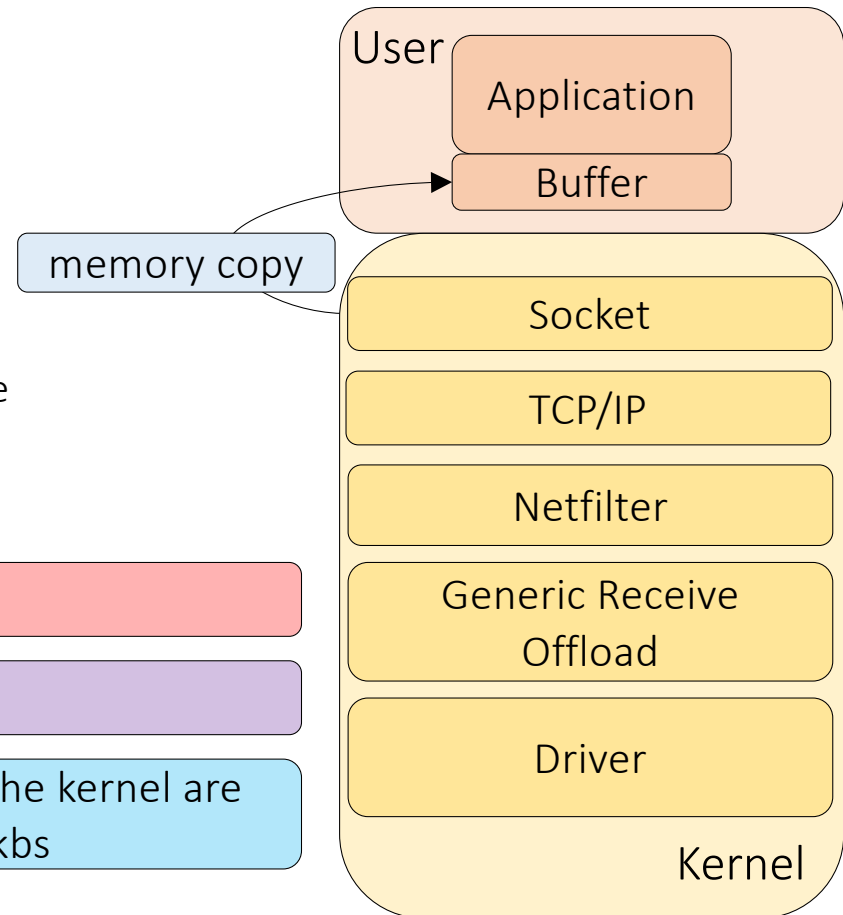
Kernel to User Space

- When a user application calls `recv()`, `read()`, or similar, the socket layer:
 - Dequeues `sk_buffs` from the receive queue
 - Copies the payload data from kernel memory into the user-provided buffer using `copy_to_user()`
 - Updates socket state and wakeups

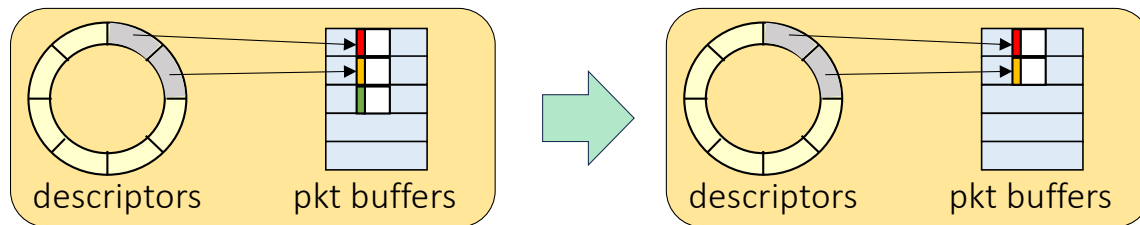
Why a data copy?

We need to protect kernel memory!

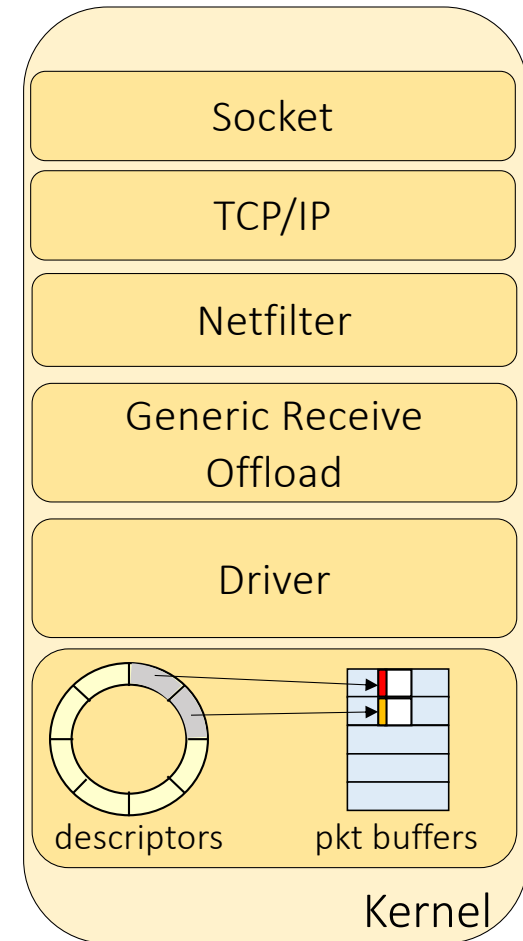
Data copy is performed only now. Operations within the kernel are done using metadata and pointer manipulations on `skbs`



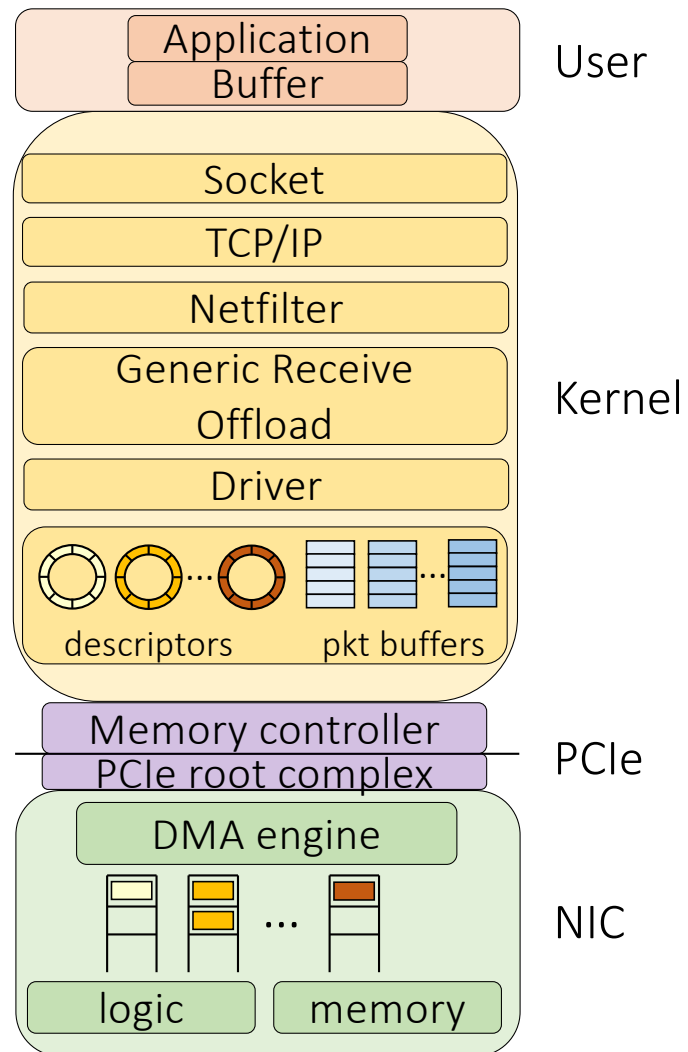
Kernel to User Space



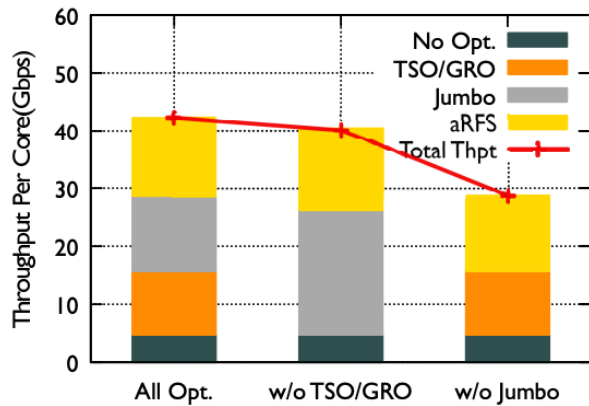
The kernel packet buffer is only now given back to the descriptor pool



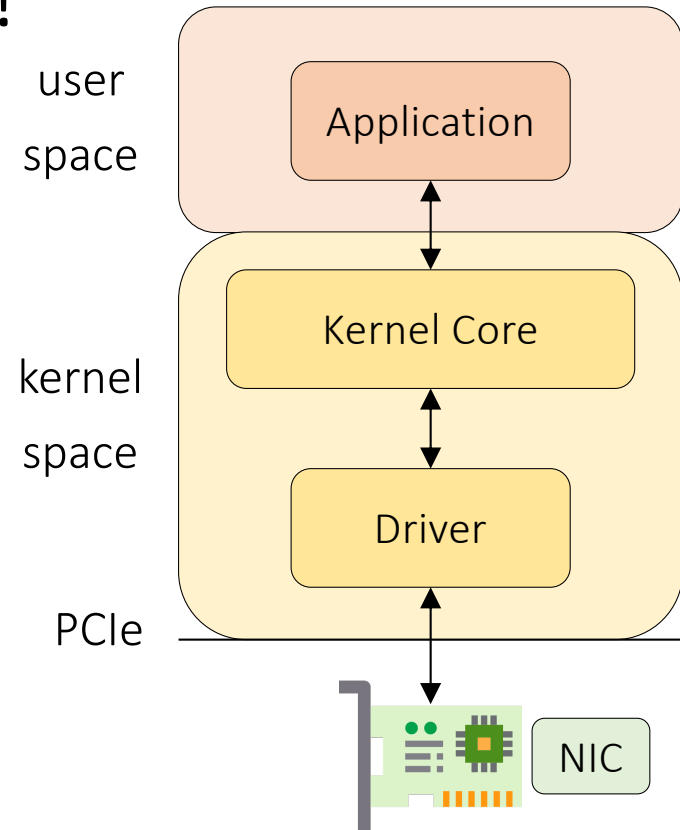
putting this altogether



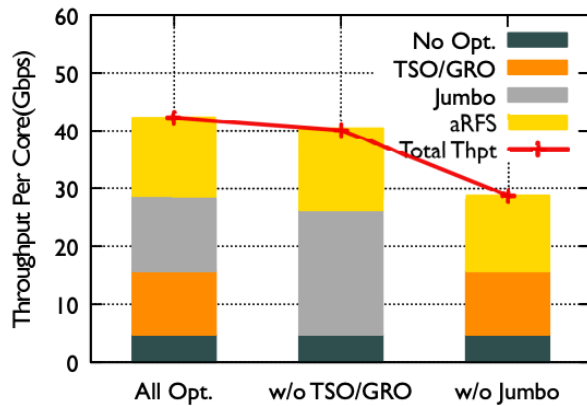
Is the network stack expensive?



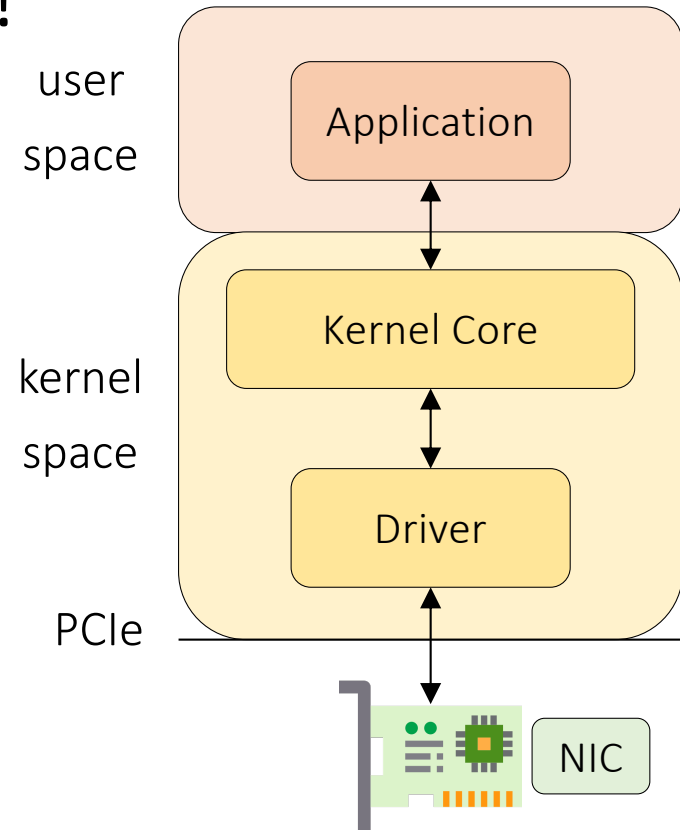
- A single core process up to 42Gbps
- TSO/GRO and Jumbo helps because they reduce the number of skbs



Is the network stack expensive?

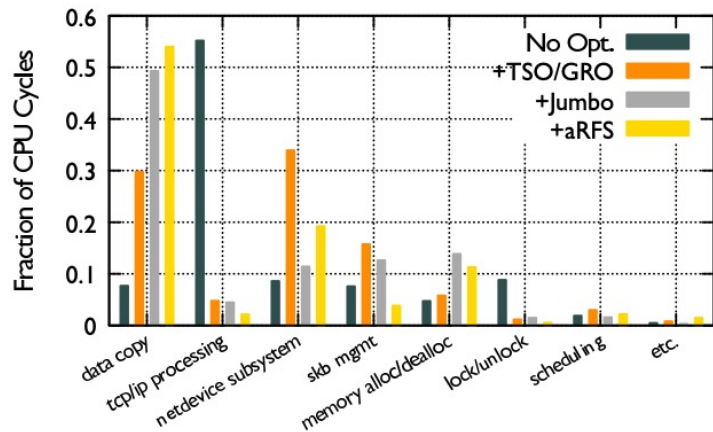


skbs allocation
and processing is
expensive!

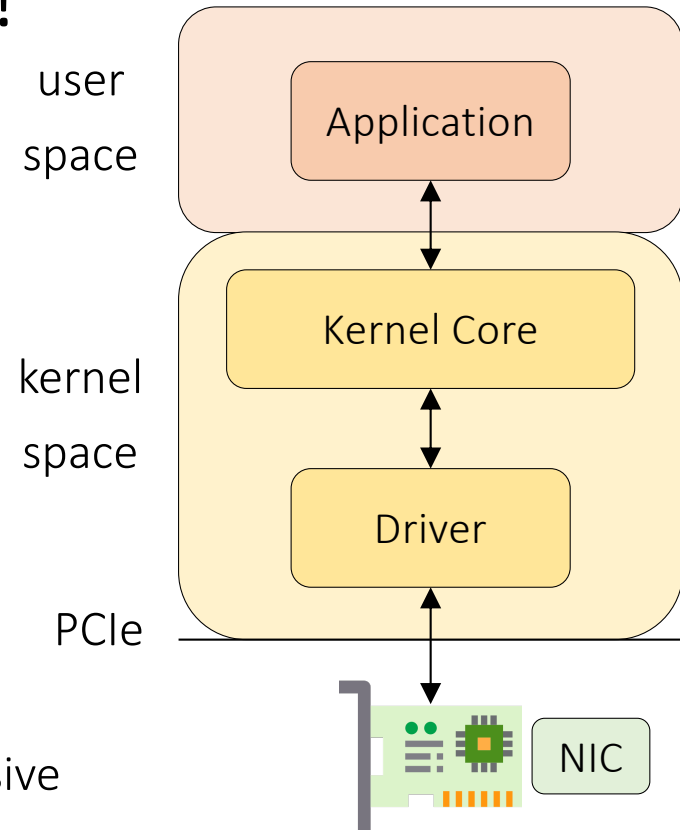


- A single core process up to 42Gbps
- TSO/GRO and Jumbo helps because they reduce the number of skbs

Is the network stack expensive?



- No optimizations: TCP/IP processing is the most expensive
- All optimization: the data copy is the most expensive



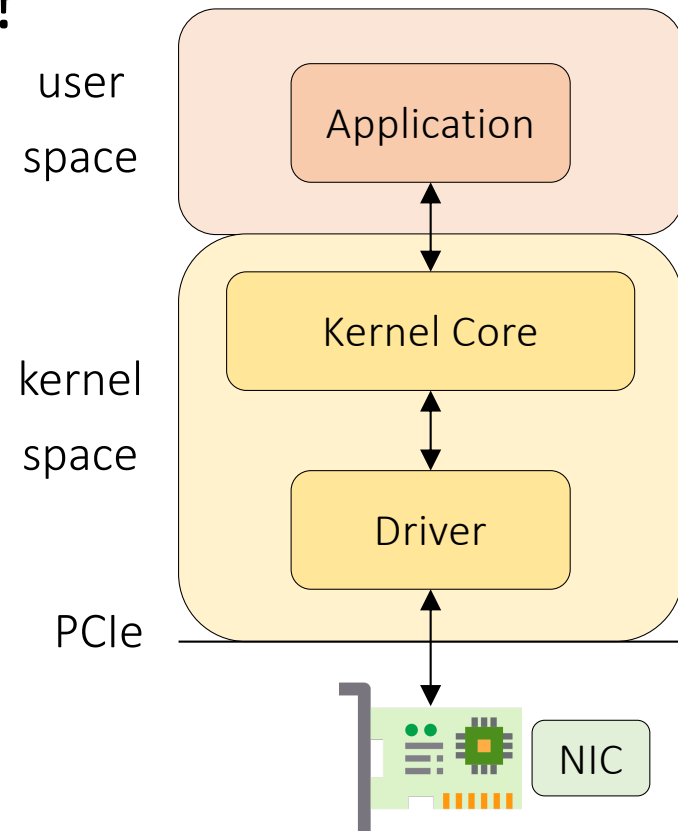
Is the network stack expensive?

It is also super complex!

From: Coco Li <lixiaoyan@google.com>
To: Jakub Kicinski <kuba@kernel.org>, Eric Dumazet <edumazet@google.com>, Neal Cardwell <ncardwell@google.com>, Mubashir Adnan Qureshi <mubashirq@google.com>, Paolo Abeni <pabeni@redhat.com>, Andrew Lunn <andrew@lunn.ch>, Jonathan Corbet <corbet@lwn.net>, David Ahern <dsahern@kernel.org>, Daniel Borkmann <daniel@iogearbox.net>
Cc: netdev@vger.kernel.org, Chao Wu <wwchao@google.com>, Wei Wang <weiwan@google.com>, Pradeep Nemavat <pnemavat@google.com>, Coco Li <lixiaoyan@google.com>
Subject: [PATCH v8 net-next 0/5] Analyze and Reorganize core Networking Strcuts to optimize cacheline consumption
Date: Wed, 29 Nov 2023 07:27:51 +0000 [thread overview]
Message-ID: <20231129072756.3684495-1-lixiaoyan@google.com> (raw)

Currently, variable-heavy structs in the networking stack is organized chronologically, logically and sometimes by cacheline access.

This patch series attempts to reorganize the core networking stack variables to minimize cacheline consumption during the phase of data transfer. Specifically, we looked at the TCP/IP stack and the fast path definition in TCP.



Is the network stack expensive?

It is also super complex!

From: Coco Li <lixiaoyan@google.com>
To: Jakub Kicinski <kuba@kernel.org>,
Eric Dumazet <edumazet@google.com>,
Neal Cardwell <ncardwell@google.com>,
Mubashir Adnan Qureshi <mubashirq@google.com>,
Paolo Abeni <pabeni@redhat.com>, Andrew Lunn <andrew@lunn.ch>,
Jonathan Corbet <corbet@lwn.net>,
David Ahern <dsahern@kernel.org>,
Daniel Borkmann <daniel@iogearbox.net>
Cc: netdev@vger.kernel.org, Chao Wu <wwchao@google.com>,
Wei Wang <weiwan@google.com>,
Pradeep Nemavat <pnemavat@google.com>,
Coco Li <lixiaoyan@google.com>
Subject: [\[PATCH v8 net-next 0/5\] Analyze and Reorganize core Networking Struts to optimize cacheline consumption](#)
Date: Wed, 29 Nov 2023 07:27:51 +0000 [\[thread overview\]](#)
Message-ID: <20231129072756.3684495-1-lixiaoyan@google.com> (raw)

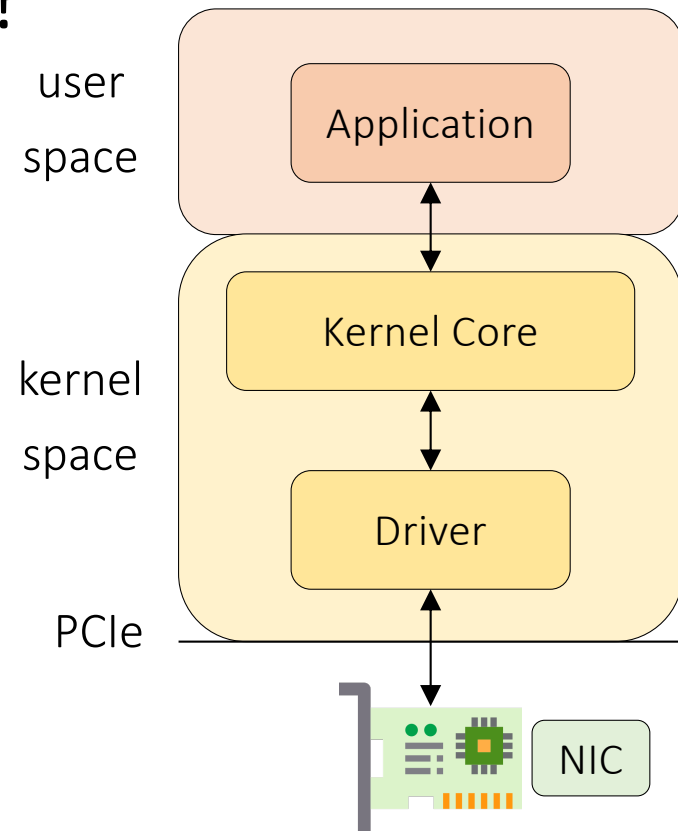
Currently, variable-heavy structs in the networking stack is organized chronologically, logically and sometimes by cacheline access.

This patch series attempts to reorganize the core networking stack variables to minimize cacheline consumption during the phase of data transfer. Specifically, we looked at the TCP/IP stack and the fast path definition in TCP.



On AMD platforms with 100Gb/s NIC and 256Mb L3 cache:

IPv4			
Flows	with patches	clean kernel	Percent reduction
30k	0.0001736538065	0.0002741191042	-36.65%
20k	0.0001583661752	0.0002712559158	-41.62%
10k	0.0001639148817	0.0002951800751	-44.47%
5k	0.0001859683866	0.0003320642536	-44.00%
1k	0.0002035190546	0.0003152056382	-35.43%



Is the network stack expensive?

It is also super complex!

From: Coco Li <lixiaoyan@google.com>
To: Jakub Kicinski <kuba@kernel.org>, Eric Dumazet <edumazet@google.com>, Neal Cardwell <ncardwell@google.com>, Mubashir Adnan Qureshi <mubashirq@google.com>, Paolo Abeni <pabeni@redhat.com>, Andrew Lunn <andrew@lunn.ch>, Jonathan Corbet <corbet@lwn.net>, David Ahern <dsahern@kernel.org>, Daniel Borkmann <daniel@iogearbox.net>
Cc: netdev@vger.kernel.org, Chao Wu <wwchao@google.com>, Wei Wang <weiwan@google.com>, Pradeep Nemavat <pnemavat@google.com>, Coco Li <lixiaoyan@google.com>
Subject: [PATCH v8 net-next 0/5] Analyze and Reorganize core Networking Strcuts to optimize cacheline consumption
Date: Wed, 29 Nov 2023 07:27:51 +0000 [\[thread overview\]](#)
Message-ID: <20231129072756.3684495-1-lixiaoyan@google.com> (raw)

Currently, variable-heavy structs in the networking stack is organized chronologically, logically and sometimes by cacheline access.

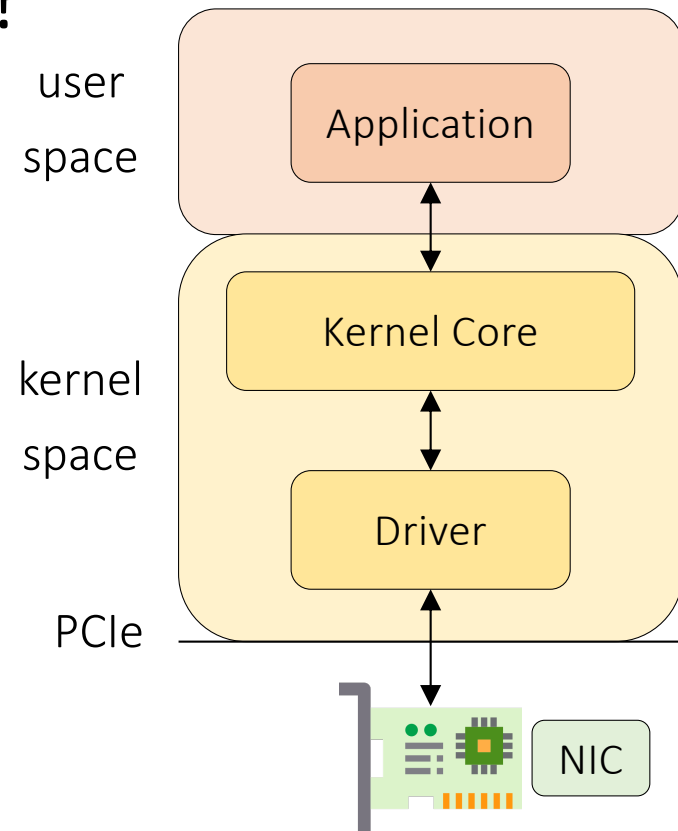
This patch series attempts to reorganize the core networking stack variables to minimize cacheline consumption during the phase of data transfer. Specifically, we looked at the TCP/IP stack and the fast path definition in TCP.



On AMD platforms with 100Gb/s NIC and 256Mb L3 cache:

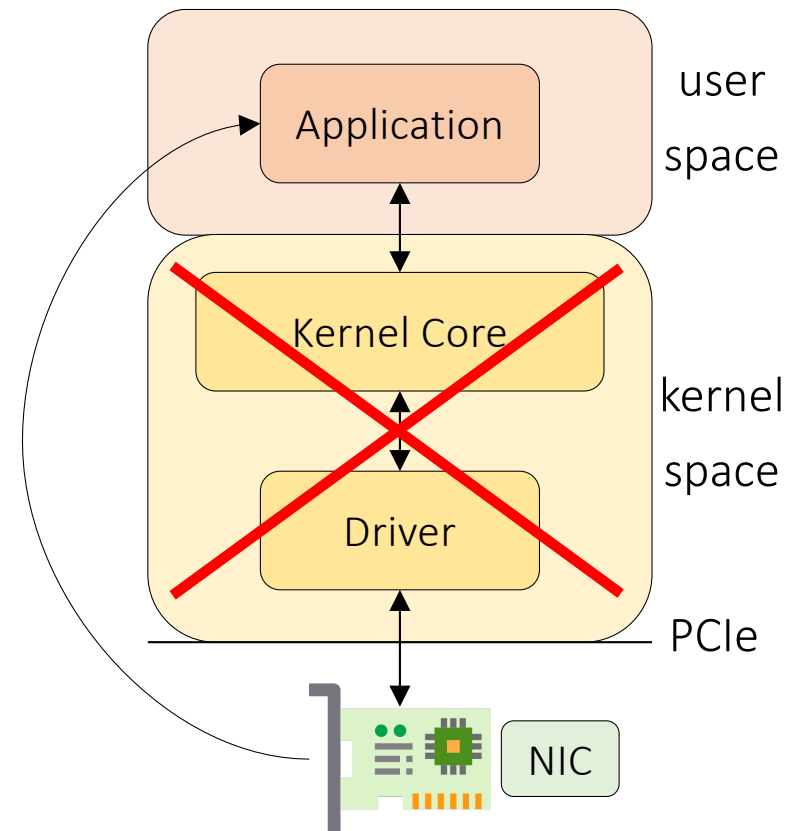
IPv4			
Flows	with patches	clean kernel	Percent reduction
30k	0.0001736538065	0.0002741191042	-36.65%
20k	0.0001583661752	0.0002712559158	-41.62%
10k	0.0001639148817	0.0002951800751	-44.47%
5k	0.0001859683866	0.0003320642536	-44.00%
1k	0.0002035190546	0.0003152056382	-35.43%

can we do better?



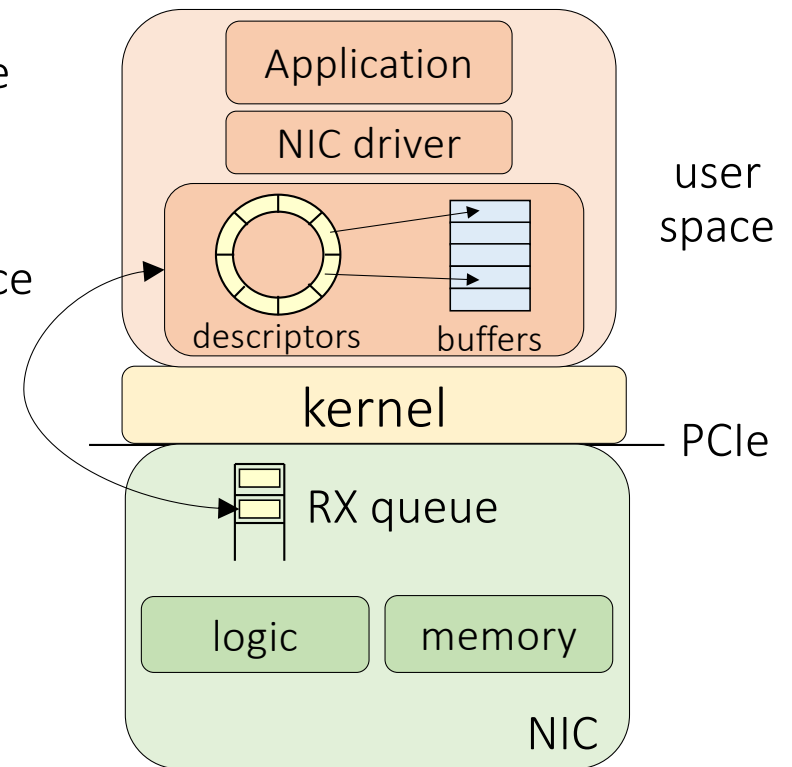
Option #1: kernel bypass with DPDK

- Bypass the Kernel!
- Example: Data Plane Development Kit (DPDK)
- DPDK is a set of libraries and drivers that allows to bypass the Linux kernel stack and pass the packet directly at user-space



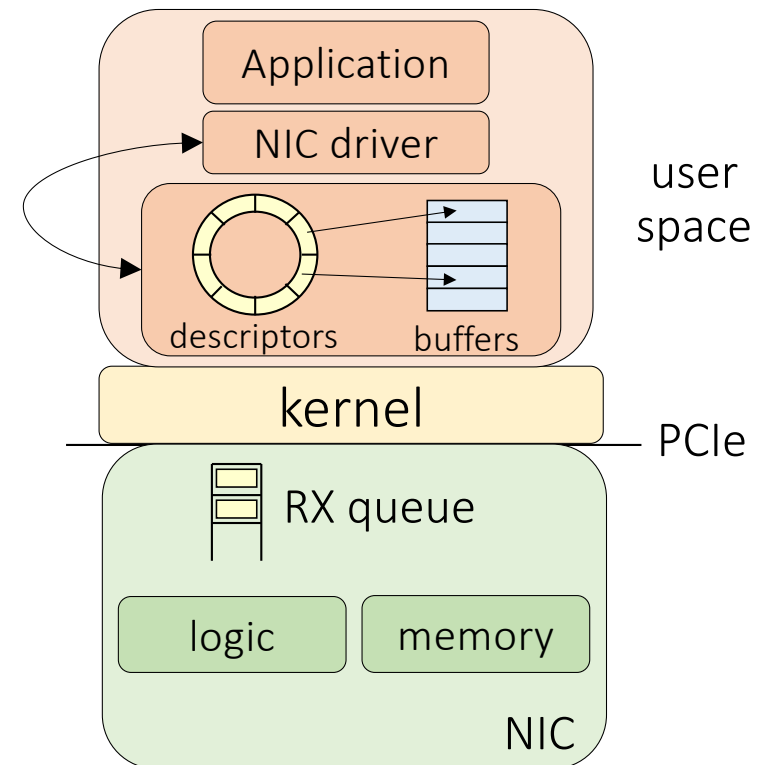
Option #1: kernel bypass with DPDK

- DPDK enables NICs to DMA data directly in user-space memory
- The NIC driver is a user-space application that interface directly with the NIC updating its RX queue with available descriptors

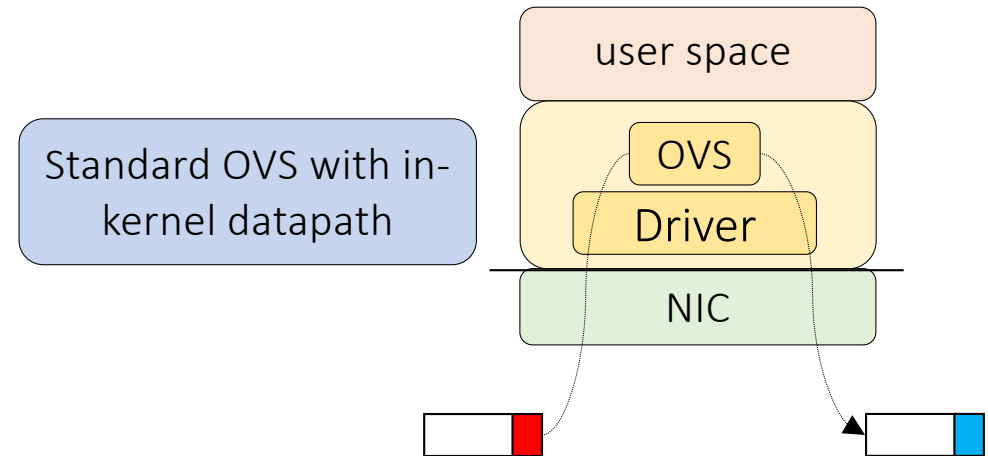


Option #1: kernel bypass with DPDK

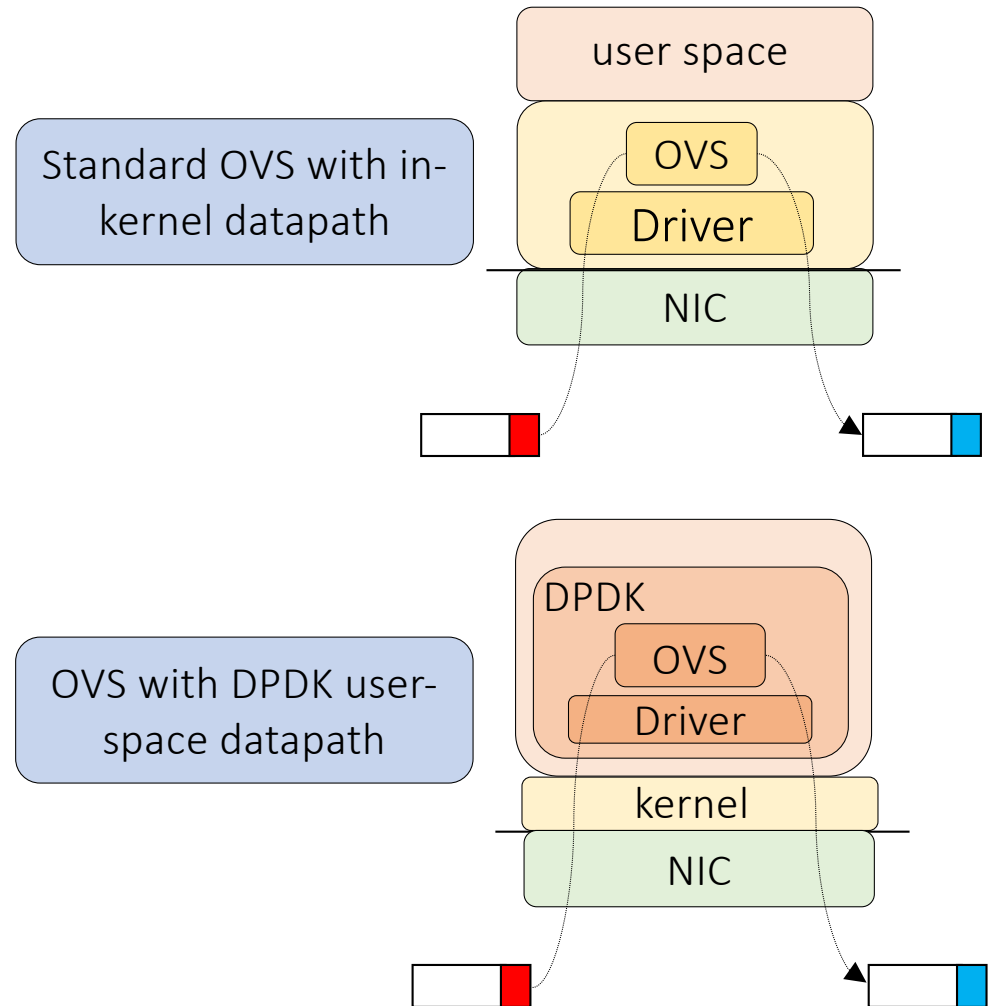
- DPDK drivers work in Poll mode
- Rather than relying on NIC-generated interrupts, the CPU employs a poll-mode driver (PMD) that continuously polls the NIC for newly received packets
- As a consequence, a CPU core must be dedicated and assigned to running PMD



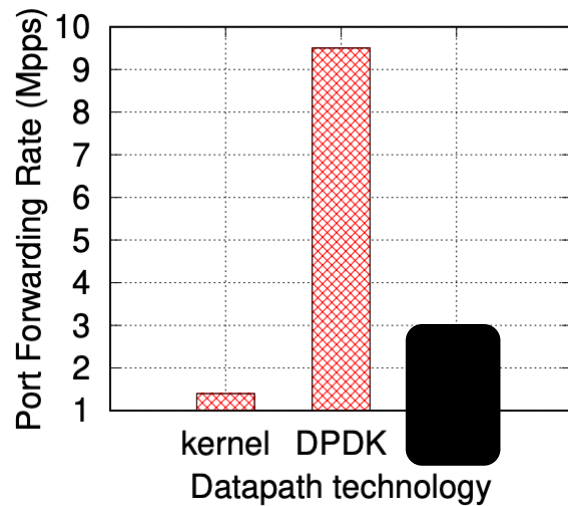
DPDK vs Kernel



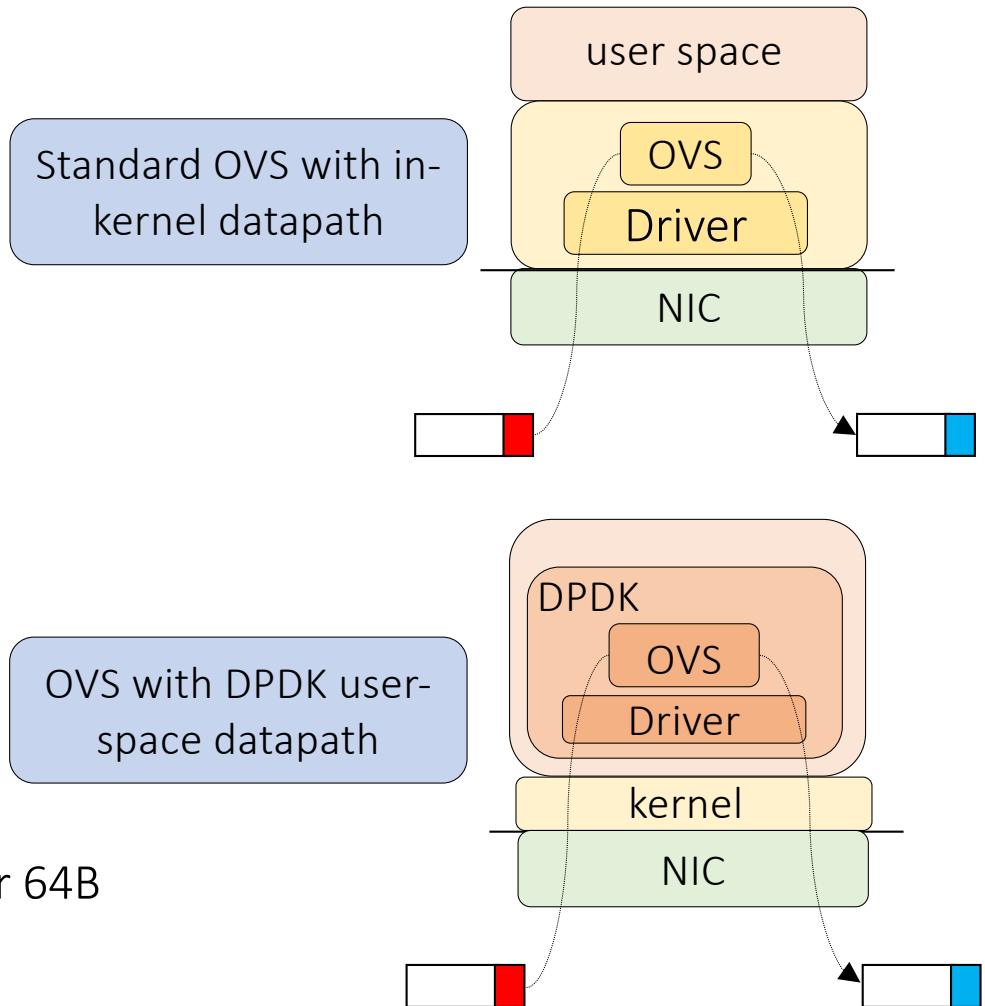
DPDK vs Kernel



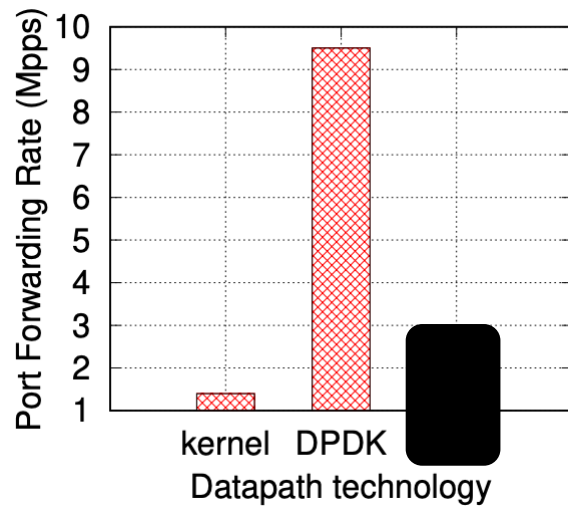
DPDK vs Kernel



- This is OVS (single core) forwarding rate for 64B packets



DPDK vs Kernel

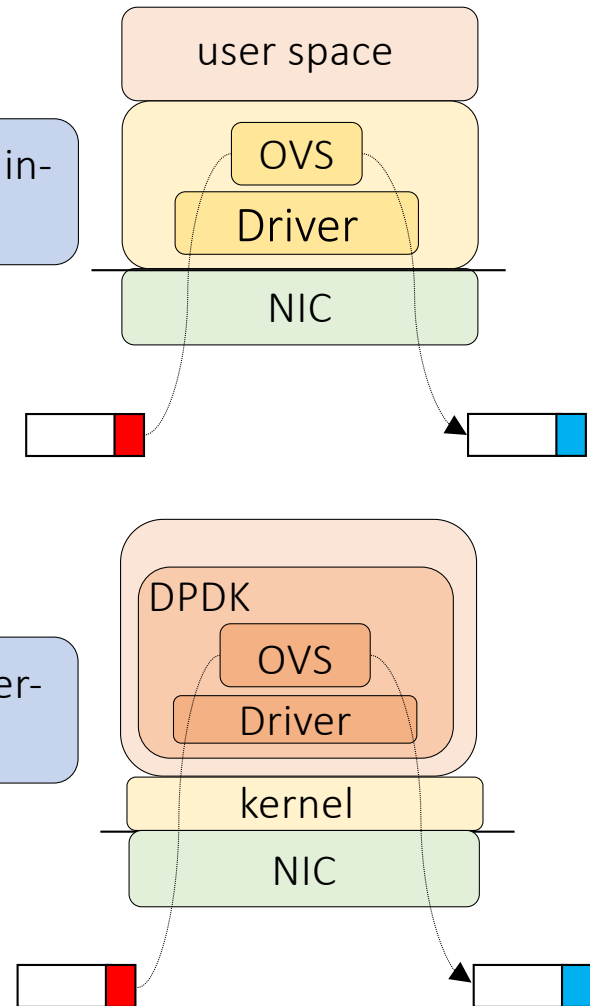


DPDK is fast 😊

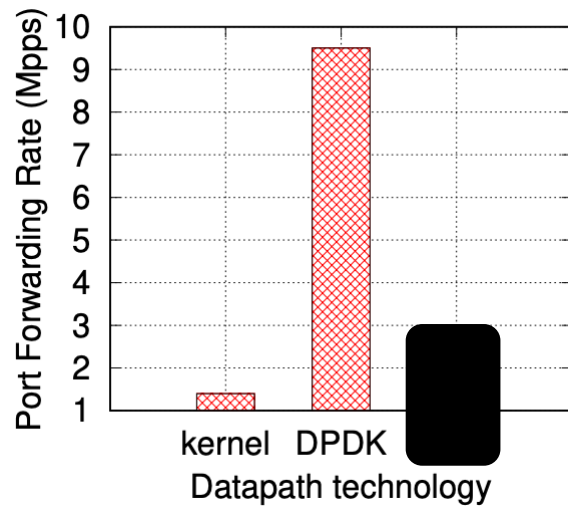
- This is OVS (single core) forwarding rate for 64B packets

Standard OVS with in-kernel datapath

OVS with DPDK user-space datapath



DPDK vs Kernel



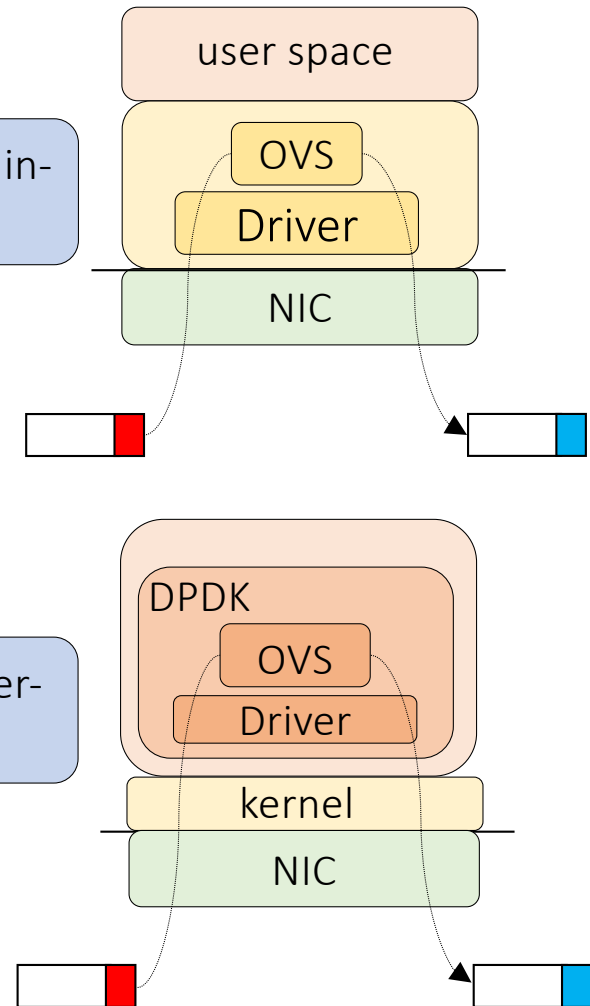
- This is OVS (single core) forwarding rate for 64B packets

Standard OVS with in-kernel datapath

DPDK is fast 😊

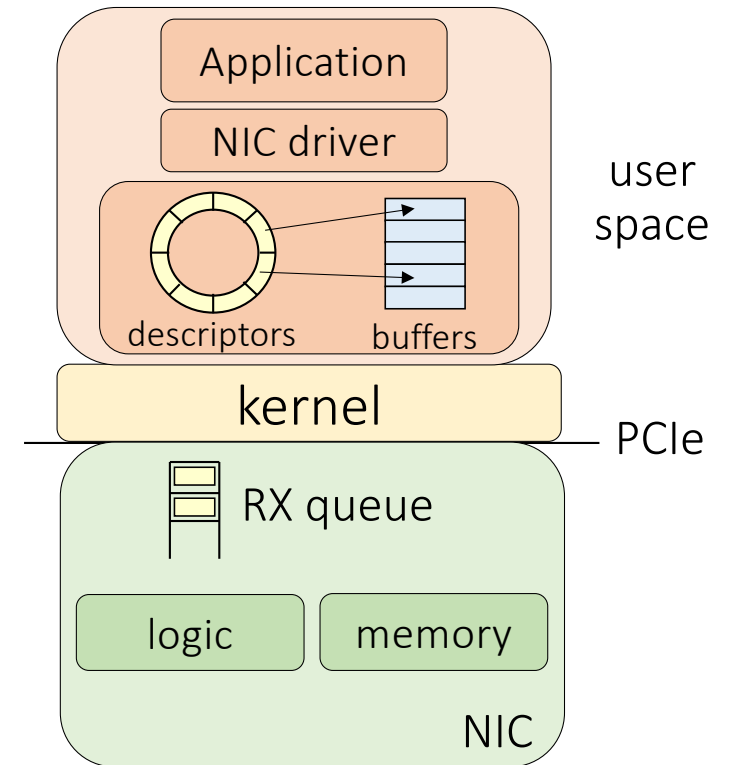
What's the catch then?

OVS with DPDK user-space datapath



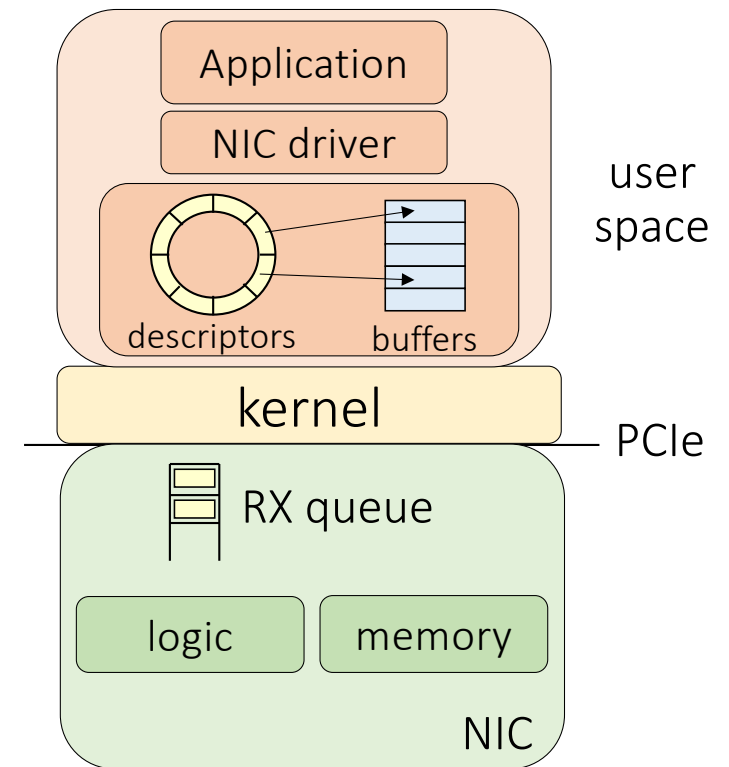
Problems with DPDK

- If you bypass the kernel, you give up all the features the kernel provides to you
 - TCP/IP processing
 - NATting/Firewalling
- *ping* does not work anymore ☹️
- *tcpdump* does not work ☹️
- *ip link* does not work ☹️

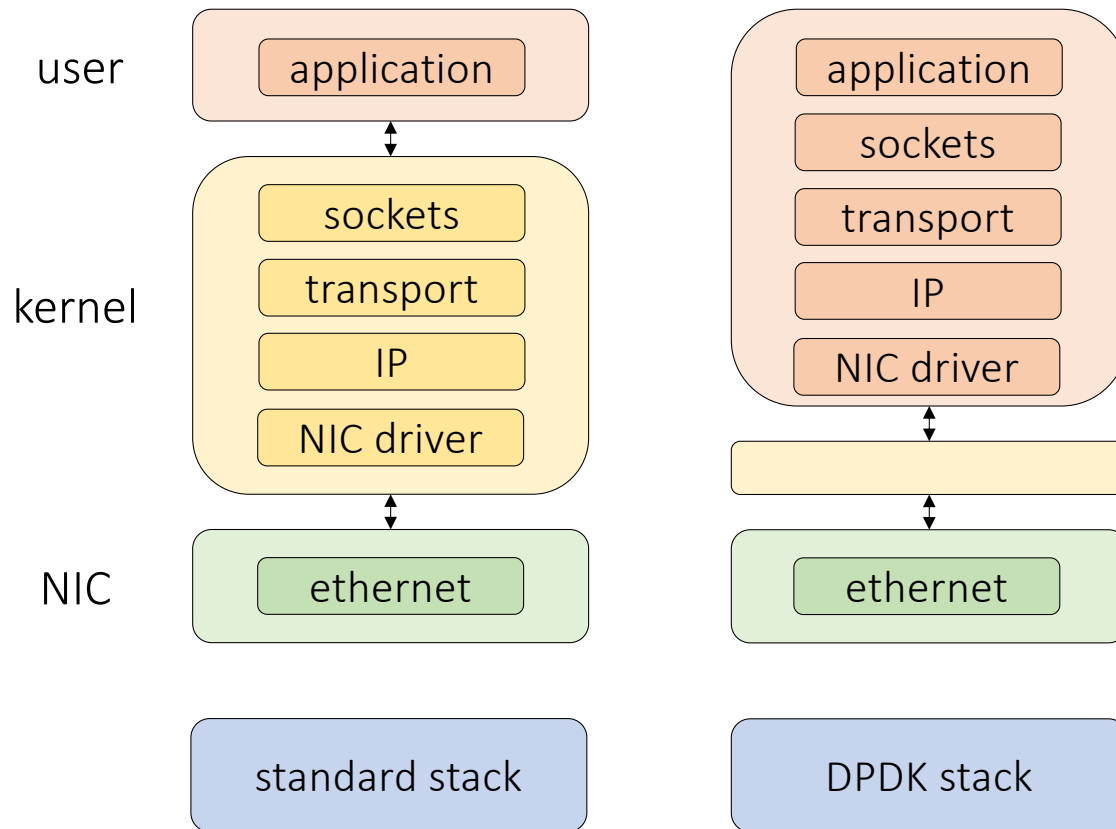


Problems with DPDK

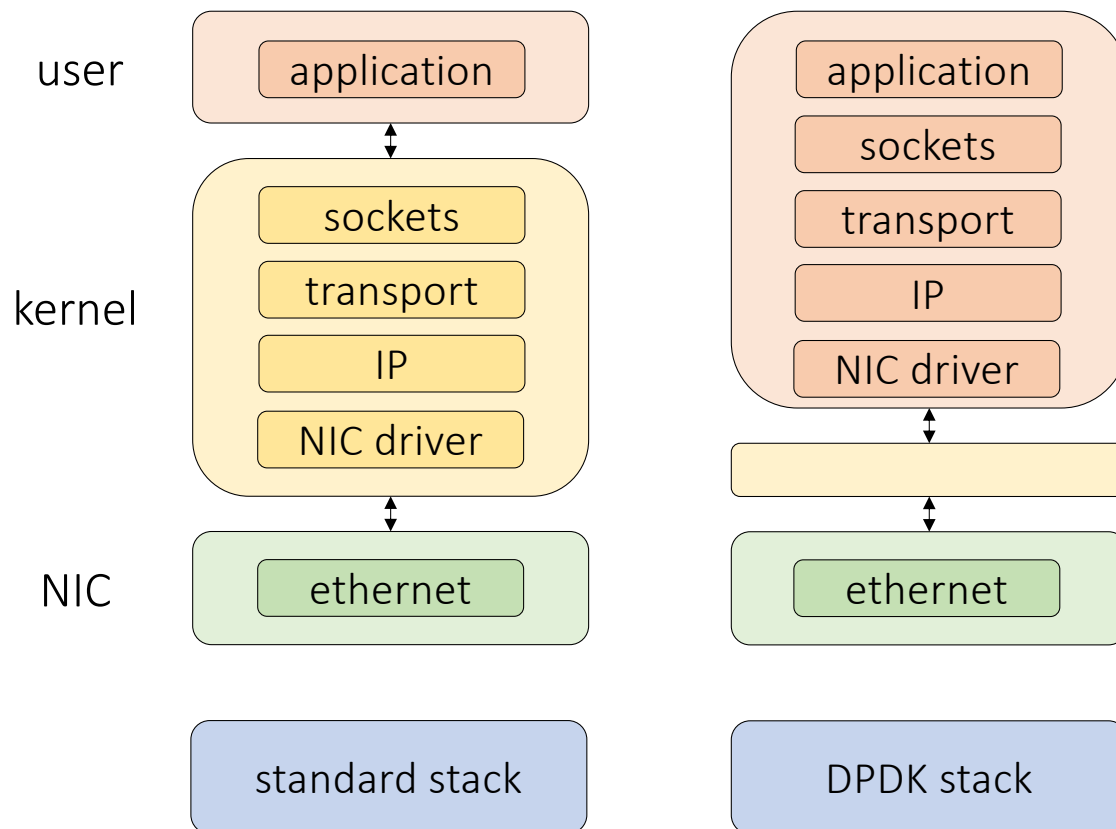
- No sharing of network resources
 - A single application take entire ownership of the DPDK port
- You can have different VMs sharing with Single Root Input/Output Virtualization (SR-IOV)
- SR-IOV is a specification that allows the isolation of PCIe addresses



Option #2: kernel bypass with a twist

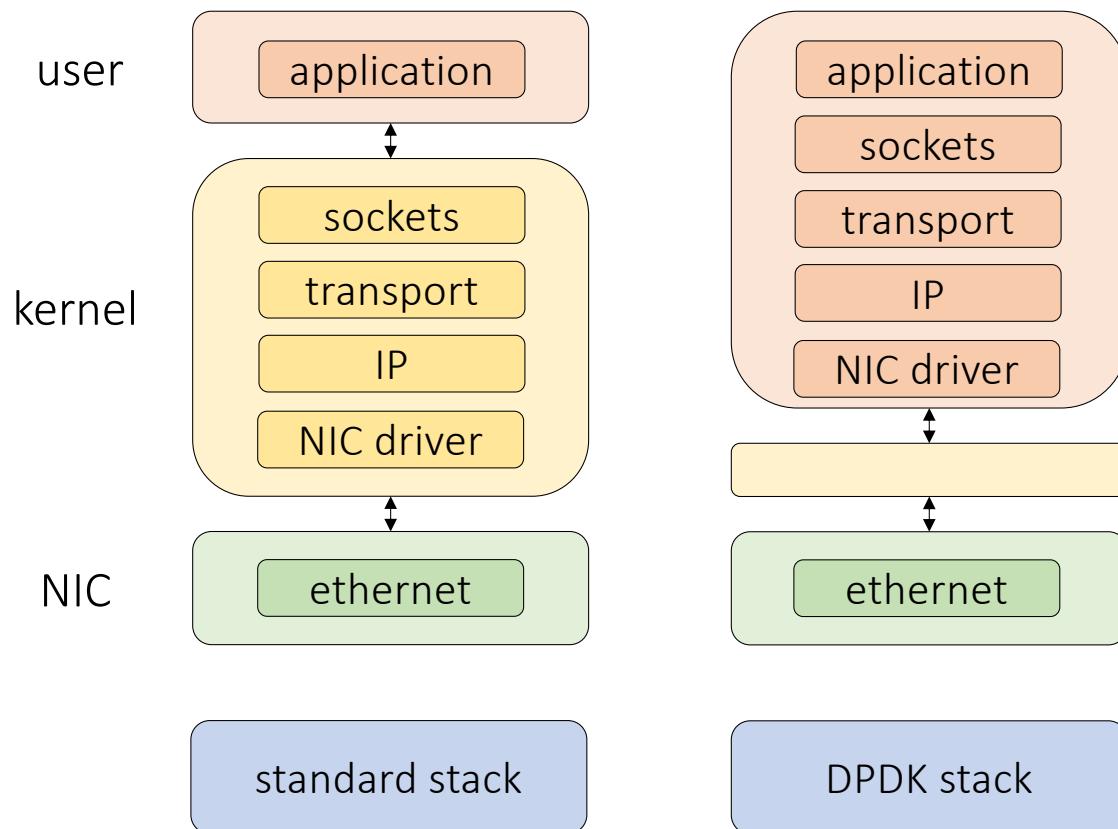


Option #2: kernel bypass with a twist



- **The problem:** we want to sustain high throughput, but the end of Moore's law and Dennard scaling imposes limits on computational capabilities of CPU cores
- More packets, more CPU cores spent in processing them, less resources for applications ☹ (CPU = \$\$\$)

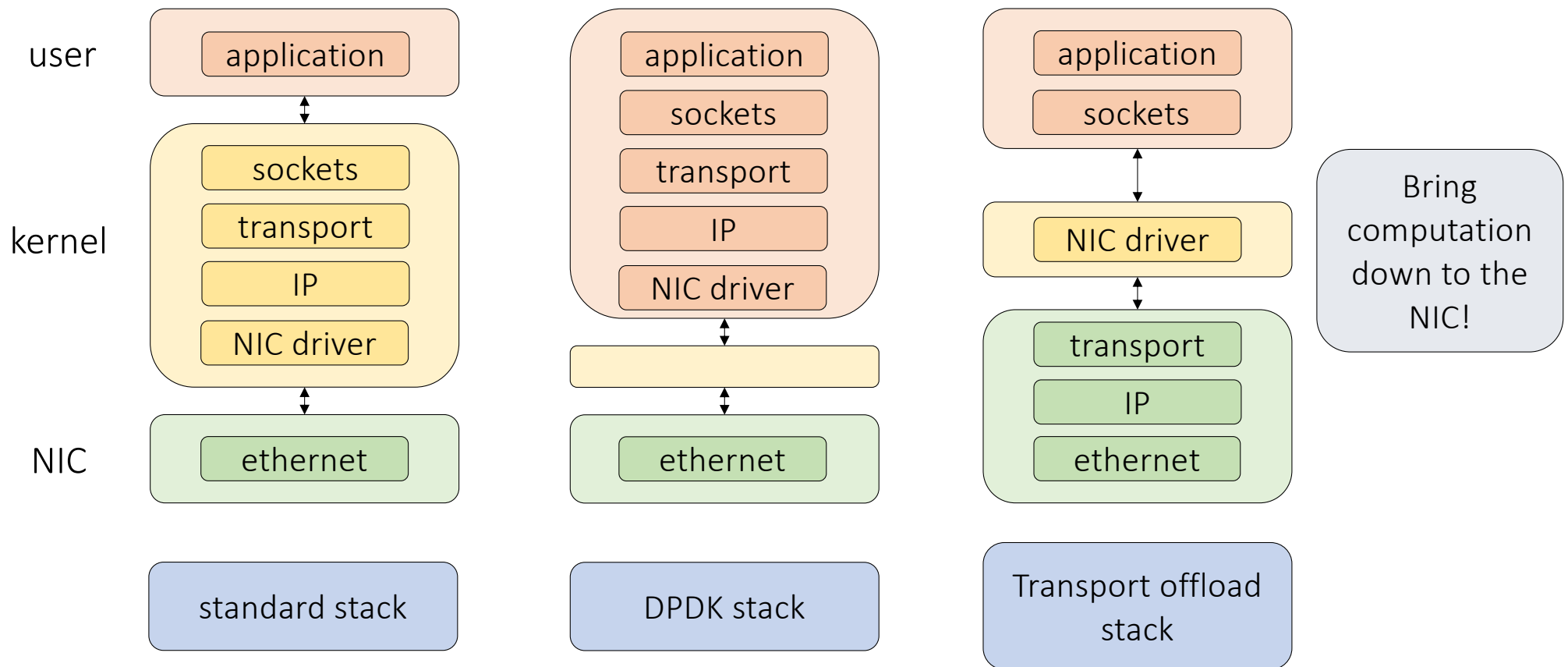
Option #2: kernel bypass with a twist



- **The problem:** we want to sustain high throughput, but the end of Moore's law and Dennard scaling imposes limits on computational capabilities of CPU cores
- More packets, more CPU cores spent in processing them, less resources for applications ☹ (CPU = \$\$\$)

What can we do?

Option #2: kernel bypass with a twist



Option #2: kernel bypass with TCP offload (TOE)?

TCP offload is a dumb idea whose time has come

Jeffrey C. Mogul
Hewlett-Packard Laboratories
Palo Alto, CA, 94304
JeffMogul@acm.org

...back in 2003!!

Abstract

Network interface implementors have repeatedly attempted to offload TCP processing from the host CPU. These efforts met with little success, because they were based on faulty premises. TCP offload *per se* is neither of much overall benefit nor free from significant costs and risks. But TCP offload in the service of very specific goals might actually be useful. In the context of the replacement of storage-specific interconnect via commoditized network hardware, TCP offload (and more generally, offloading the transport protocol) appropriately solves an important problem.

To this day, TCP offload has never firmly caught on in the commercial world (except sometimes as a stopgap to add TCP support to immature systems [16]), and has been scorned by the academic community and Internet purists. This paper starts by analyzing why TCP offload has repeatedly failed.

The lack of prior success with TCP offload does not, however, necessarily imply that this approach is categorically without merit. Indeed, the analysis of past failures points out that novel applications of TCP might benefit from TCP offload, but for reasons not clearly anticipated by early proponents. TCP offload does appear to

Option #2: kernel bypass with TCP offload (TOE)?

CHELSIO COMMUNICATIONS ANNOUNCES OUTSTANDING NFSv4 PERFORMANCE WITH T5 AND T6 ADAPTERS

Record-Breaking Results with T6 100GbE and T5 40GbE Adapters in Network File System Version 4 Testing

SUNNYVALE, CA – December 22, 2023 – Chelsio Communications, Inc., a leading provider of high-performance (10Gb/25Gb/40Gb/50Gb/100Gb) Ethernet adapters for storage, networking, virtualized enterprise data centers, cloud service installations, and cluster computing environments today proudly unveiled exceptional performance results for its T6 2x100GbE and T5 4x10GbE series adapters in NFSv4 environments, leveraging its innovative TCP/IP Offload Engine (TOE) technology.

Option #2: kernel bypass with TCP offload (TOE)?

Linux and TCP offload engines

[Posted August 22, 2005 by corbet]

The TCP/IP protocol suite takes a certain amount of CPU power to implement. So it is not surprising that network adapter manufacturers have long been adding protocol support to their cards. This support can vary from the simple (checksumming of packets, for example) through to full TCP/IP implementations. An adapter with full protocol support is often called a TCP offload engine or TOE.

Linux has never supported the TOE features of any network cards. For some time, there had not even been much discussion of TOE support. The topic has returned, however, with [this patch adding TOE support](#) which was posted by Scott Bardone of Chelsio Communications. This TOE patch is clearly intended to support Chelsio's line of network adapters, but it has been coded as a more generic "open TOE" framework. The Chelsio folks would very much like to see this patch merged for the 2.6.14 kernel release.

Those who are curious about the TOE patch can go in and look at the code; it is relatively straightforward. At its core, it creates a new type of extended network device (`struct toedev`) with an additional set of methods:

```
int (*open)(struct toedev *dev);
int (*close)(struct toedev *dev);
int (*can_offload)(struct toedev *dev, struct sock *sk);
int (*connect)(struct toedev *dev, struct sock *sk);
int (*send)(struct toedev *dev, struct sk_buff *skb);
int (*recv)(struct toedev *dev, struct sk_buff **skb, int n);
int (*ctl)(struct toedev *dev, unsigned int req, void *data);
void (*neigh_update)(struct net_device *lldev,
                    struct toedev *dev,
                    struct neighbour *neigh, int fl);
```

Option #2: kernel bypass with TCP offload (TOE)?

- **Complexity:** TCP is stateful and complex. Implementing in hardware requires a significant engineering effort

Option #2: kernel bypass with TCP offload (TOE)?

- **Complexity:** TCP is stateful and complex. Implementing in hardware requires a significant engineering effort
- **Vendor lock-in:** each vendor might implement TOE differently making different to get consistent results across different hardware

Option #2: kernel bypass with TCP offload (TOE)?

- **Complexity:** TCP is stateful and complex. Implementing in hardware requires a significant engineering effort
- **Vendor lock-in:** each vendor might implement TOE differently making different to get consistent results across different hardware
- **Security challenges:** hardware implementations are harder to patch than software

Option #2: kernel bypass with TCP offload (TOE)?

- **Complexity:** TCP is stateful and complex. Implementing in hardware requires a significant engineering effort
- **Vendor lock-in:** each vendor might implement TOE differently making different to get consistent results across different hardware
- **Security challenges:** hardware implementations are harder to patch than software
- **Hardware limitations:** TOEs have fixed resources which can lead to resource starvation under heavy load

Option #2: kernel bypass with TCP offload (TOE)?

T E S L A

...even Tesla, that arguably does not have problems with money, runs away from TCP!

**Tesla Transport Protocol
over Ethernet (TTPoE)**

Option #2: kernel bypass with TCP offload (TOE)?

T E S L A

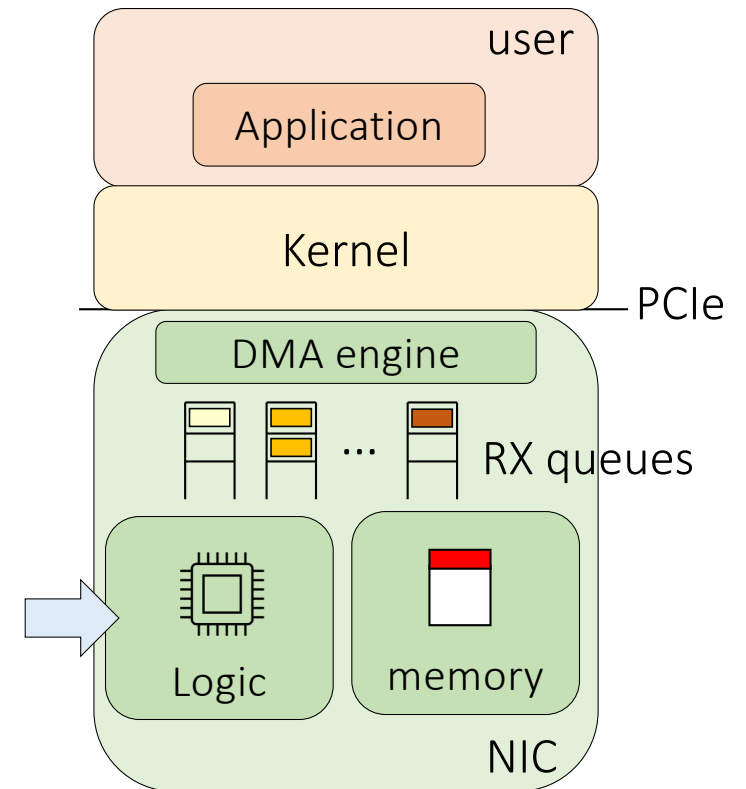
Tesla Transport Protocol over Ethernet (TTPoE)

...even Tesla, that arguably does not have problems with money, runs away from TCP!

In one of the next lectures, we will cover RDMA, a way to delegate transport-level processing to the NIC

Option #3: offloading other compute to the NIC

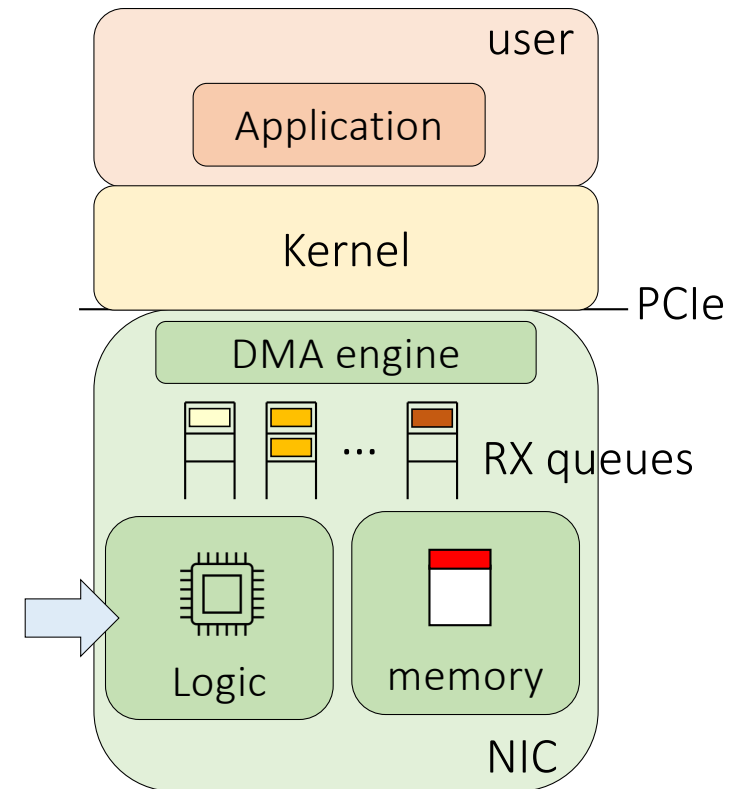
- Is it only about delegating transport-level processing to the NIC?



Option #3: offloading other compute to the NIC

- Is it only about delegating transport-level processing to the NIC?

What if we could do more?

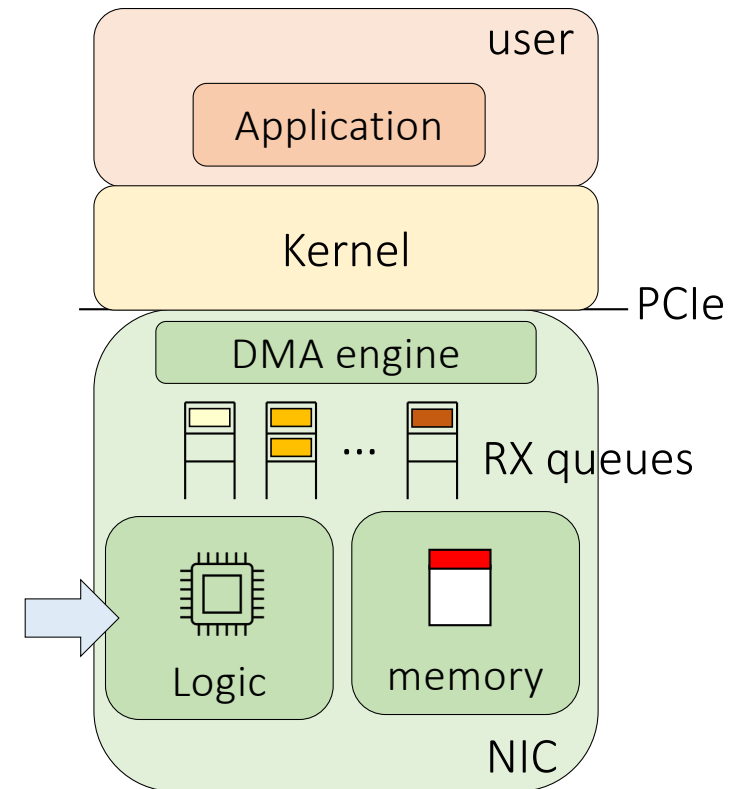


Option #3: offloading other compute to the NIC

- Is it only about delegating transport-level processing to the NIC?

What if we could do more?

What if this more can be achieved programmatically?



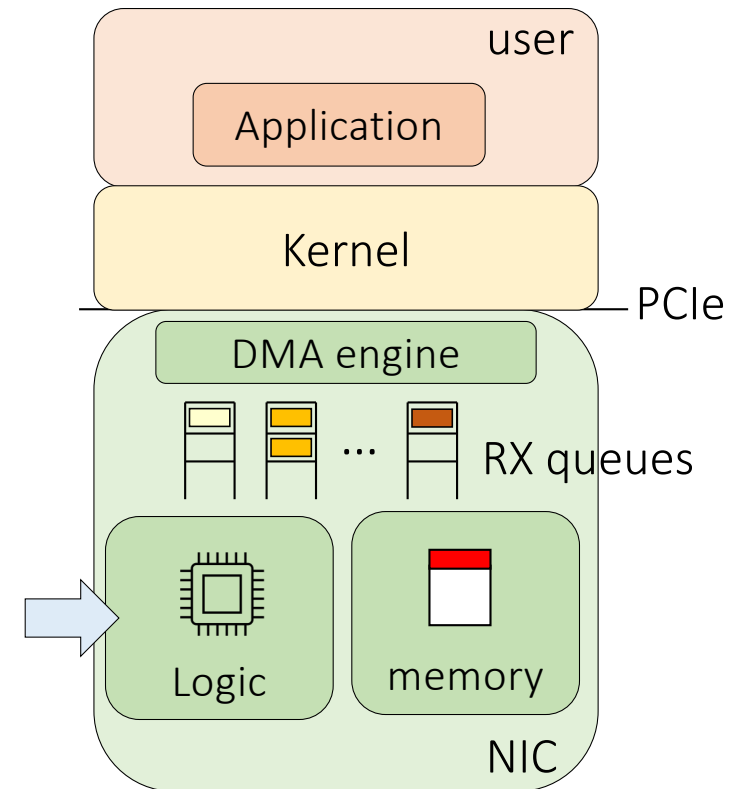
Option #3: offloading other compute to the NIC

- Is it only about delegating transport-level processing to the NIC?

What if we could do more?

What if this more can be achieved programmatically?

- This is possible now with what are called Programmable NICs or SmartNICs
- Many manufacturers are building those devices



Programmable NICs



SMART ADAPTER CARDS
PRODUCT BRIEF




**NVIDIA® Mellanox® BlueField®
SmartNIC for Ethernet**
High Performance Ethernet Network Adapter Cards

Combining Arm® processing power with advanced network and storage offloads to accelerate a multitude of security, networking and storage applications with world-leading performance, flexibility and efficiency.

BlueField SmartNIC features the BlueField Data Processing Unit (DPU) — an innovative and high-

HIGHLIGHTS

- Intelligent programmable network adapter



Alveo™ U50 Data Center Accelerator Card

AMD-Xilinx's Alveo U50 data center accelerator card for custom solutions provide the framework for developers to bring differentiated applications to market

AMD-Xilinx's Alveo U50 data center accelerator cards provide acceleration for workloads in financial computing, machine learning, computational storage, data search, and analytics. Built on AMD-Xilinx UltraScale+ architecture and packaged in a 75 watt, small form factor and fitted with 100 Gbps networking I/O, PCIe® Gen4, and HBM, Alveo U50 is designed for deployment in any server.

Enabling Alveo accelerator cards is an ecosystem of AMD-Xilinx and partner applications for common data center workloads. For custom solutions, AMD-Xilinx's application developer tool suite (Vitis™ environment) and machine learning suite provide the frameworks for developers to bring differentiated applications to market.

Features

- Performance and efficiency
 - 8 GB HBM memory and PCIe for faster application performance
 - 100G networking with support for 4 x 10 GbE, 4 x 15 GbE, or 1 x 40 GbE/1 x 100 GbE for low latency network capability
- Adaptable for acceleration of any workload
 - Accelerates computing, network, and storage workloads
 - Maximized application performance as workloads and algorithms evolve through the reconfigurable fabric, unlike fixed-architecture alternatives
- Accessible on cloud or on-premises mobility
 - Built for scale-out architectures for deployment solutions on the cloud or on-premises interchangeably

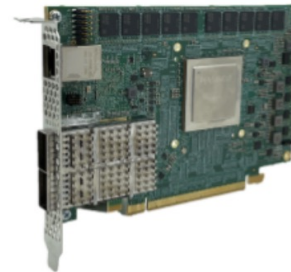


Intel® IPU Platform F2000X-PL



- 2 x 100 GbE connectivity
- Intel® Agilex-F FPGA
- Intel® Xeon D-1736 Processor
- 32GB DRAM

- Packet processing
- OVS
- NVMe-oF
- Security/Isolation
- Crypto
- RDMA/RoCEv2



Pensando DSC-200 PCIe Card
(shown with heatsink removed)

HIGHLIGHTS

FEATURES

- Integrated security, networking, observability and storage services in a single card
- Incorporates both software-defined data plane and control plane, eliminating host agents
- Customizable control and data plane; customers can develop their own software, and integrate with existing management applications
- Pre-built services packages for a variety of functions, tested for scale
- Supports cloud-scale networks with 100k+ flow table entries and 1M+ routes

Azure Accelerated Networking: SmartNICs in the Public Cloud

Daniel Firestone Andrew Putnam Sambhrama Mundkur Derek Chiou Alireza Dabagh
Mike Andrewartha Hari Angepat Vivek Bhanu Adrian Caulfield Eric Chung
Harish Kumar Chandrappa Somesh Chaturmohta Matt Humphrey Jack Lavier Norman Lam
Fengfen Liu Kalin Ovtcharov Jitu Padhye Gautham Popuri Shachar Raindel Tejas Sapre
Mark Shaw Gabriel Silva Madhan Sivakumar Nisheeth Srivastava Anshuman Verma Qasim Zuhair
Deepak Bansal Doug Burger Kushagra Vaid David A. Maltz Albert Greenberg

Microsoft

Abstract

Modern cloud architectures rely on each server running its own networking stack to implement policies such as tunneling for virtual networks, security, and load balancing. However, these networking stacks are becoming increasingly complex as features are added and as network speeds increase. Running these stacks on CPU cores takes away processing power from VMs, increasing the cost of running cloud services, and adding latency and variability to network performance.

We present Azure Accelerated Networking (AccelNet), our solution for offloading host networking to hardware, using custom Azure SmartNICs based on FPGAs. We define the goals of AccelNet, including programmability comparable to software, and performance and efficiency comparable to hardware. We show that FPGAs are the best current platform for offloading our networking stack as ASICs do not provide sufficient programmability, and embedded CPU cores do not provide scalable performance, especially on single network flows.

Azure SmartNICs implementing AccelNet have been deployed on all new Azure servers since late 2015 in a fleet of >1M hosts. The AccelNet service has been available for Azure customers since 2016, providing consistent $<15\mu\text{s}$ VM-VM TCP latencies and 32Gbps throughput, which we believe represents the fastest network available to customers in the public cloud. We present the design of AccelNet, including our hardware/software co-design model, performance results on key workloads, and experiences and lessons learned from developing and deploying AccelNet on FPGA-based Azure SmartNICs.

all virtual networking features, such as private virtual networks with customer supplied address spaces, scalable L4 load balancers, security groups and access control lists (ACLs), virtual routing tables, bandwidth metering, QoS, and more. These features are the responsibility of the host platform, which typically means software running in the hypervisor.

The cost of providing these services continues to increase. In the span of only a few years, we increased networking speeds by 40x and more, from 1GbE to 40GbE+, and added countless new features. And while we built increasingly well-tuned and efficient host SDN packet processing capabilities, running this stack in software on the host requires additional CPU cycles. Burning CPUs for these services takes away from the processing power available to customer VMs, and increases the overall cost of providing cloud services.

Single Root I/O Virtualization (SR-IOV) [4, 5] has been proposed to reduce CPU utilization by allowing direct access to NIC hardware from the VM. However, this direct access would bypass the host SDN stack, making the NIC responsible for implementing all SDN policies. Since these policies change rapidly (weeks to months), we required a solution that could provide software-like programmability while providing hardware-like performance.

In this paper we present Azure Accelerated Networking (AccelNet), our host SDN stack implemented on the FPGA-based Azure SmartNIC. AccelNet provides near-native network performance in a virtualized environment, offloading packet processing from the host CPU to the Azure SmartNIC. Building upon the software-based VFP host SDN platform [6], and the hardware and software in-

Azure Accelerated Networking: SmartNICs in the Public Cloud

Daniel Firestone Andrew Putnam Sambhrama Mundkur Derek Chiou Alireza Dabagh
Mike Andrewartha Hari Angepat Vivek Bhanu Adrian Caulfield Eric Chung
Harish Kumar Chandrappa Somesh Chaturmohta Matt Humphrey Jack Lavier Norman Lam

Accelerometer: Understanding Acceleration Opportunities for Data Center Overheads at Hyperscale

Akshitha Sriraman^{*†} Abhishek Dhanotia[†]
University of Michigan^{*}, Facebook[†]
akshitha@umich.edu, abhishek@fb.com

processing power from VMs, increasing the cost of running cloud services, and adding latency and variability to network performance.

crease. In the span of only a few years, we increased networking speeds by 40x and more, from 1GbE to 40GbE+, and added countless new features. And while we built in

...and many more!

Autonomous NIC Offloads

Boris Pismenny
Technion and NVIDIA
Haifa, Israel

Haggai Eran
Technion and NVIDIA
Haifa, Israel

Aviad Yehezkel
NVIDIA
Yokne'am Illit, Israel

Liran Liss
NVIDIA
Yokne'am Illit, Israel

Adam Morrison
Tel Aviv University
Tel Aviv, Israel

Dan Tsafir
Technion and VMware Research
Haifa, Israel

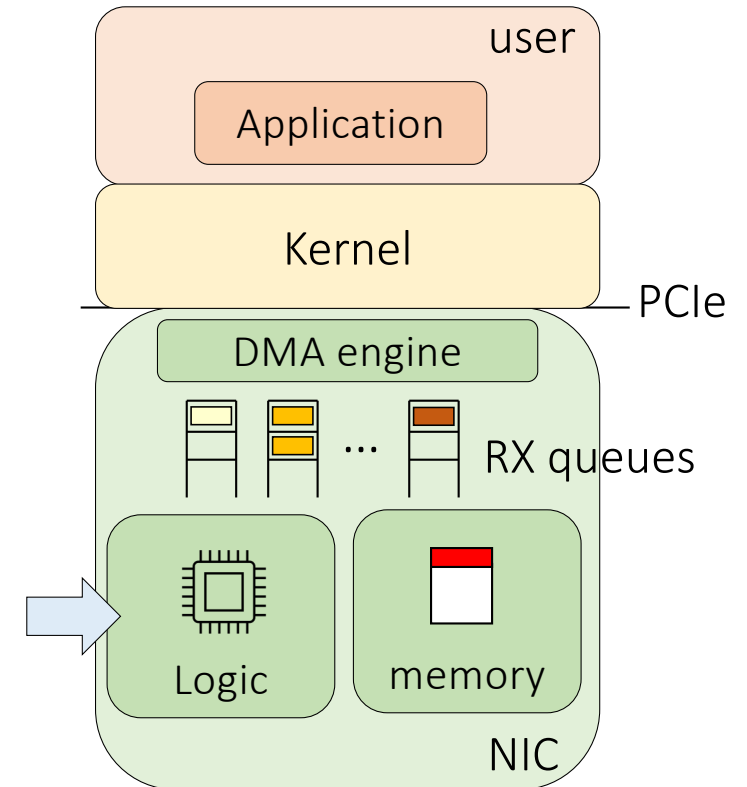
deployed on all new Azure VMs since late 2015 in a fleet of >1M hosts. The AccelNet service has been available for Azure customers since 2016, providing consistent <15 μ s VM-VM TCP latencies and 32Gbps throughput, which we believe represents the fastest network available to customers in the public cloud. We present the design of AccelNet, including our hardware/software co-design model, performance results on key workloads, and experiences and lessons learned from developing and deploying AccelNet on FPGA-based Azure SmartNICs.

Since these policies change rapidly (weeks to months), we required a solution that could provide software-like programmability while providing hardware-like performance.

In this paper we present Azure Accelerated Networking (AccelNet), our host SDN stack implemented on the FPGA-based Azure SmartNIC. AccelNet provides near-native network performance in a virtualized environment, offloading packet processing from the host CPU to the Azure SmartNIC. Building upon the software-based VFP host SDN platform [6], and the hardware and software in-

Programmable NICs

- This is *not just* for improved performance
- In the cloud era, there are really two distinct classes of function:
 - those that belong to guests (customers)
 - those that are the responsibility of the cloud service provider
- It is a way to fully separate those two classes of workloads into two separate types of computing system



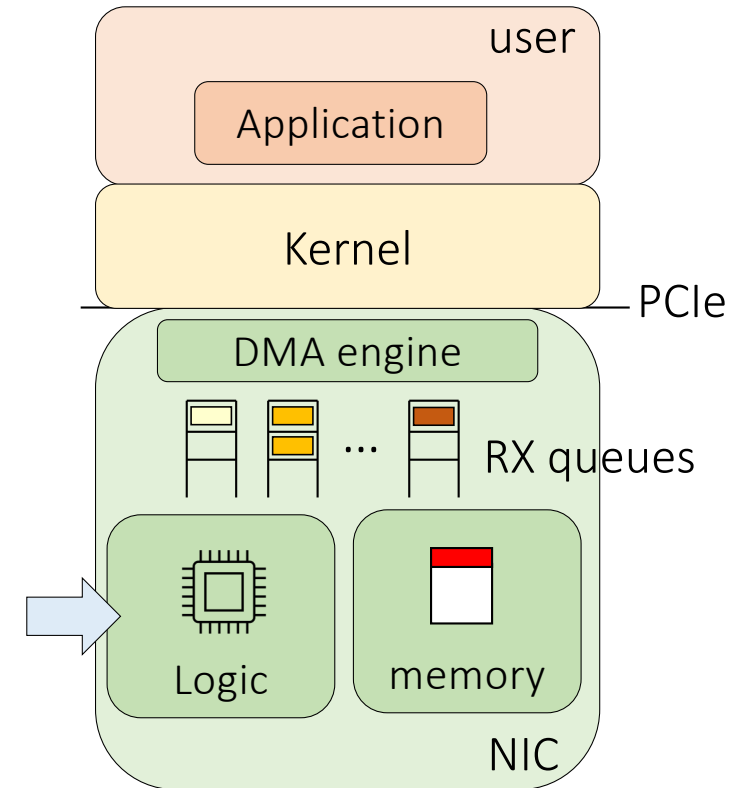
Programmable NICs

«Always in the kitchen at parties»

(cit. Bruce Davie)

analogy of a kitchen in a hotel being off limits to guests

Programmable NICs allow to meet the evolving requirements of infrastructure functions such as network virtualization, storage or encryption more efficiently



Programmable NICs

«Always in the kitchen at parties»

(cit. Bruce Davie)

analogy of a kitchen in a hotel being off limits to guests

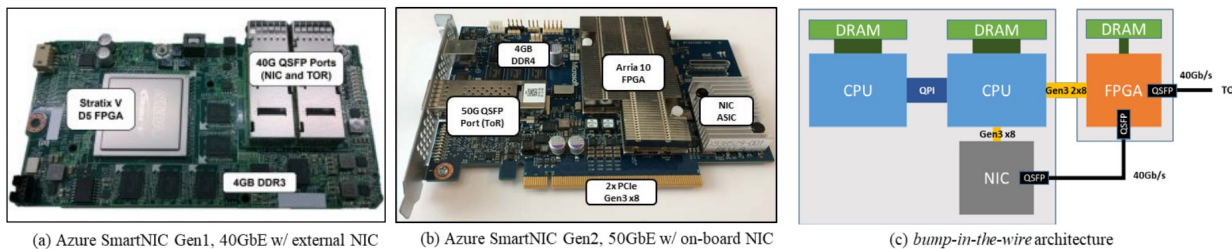
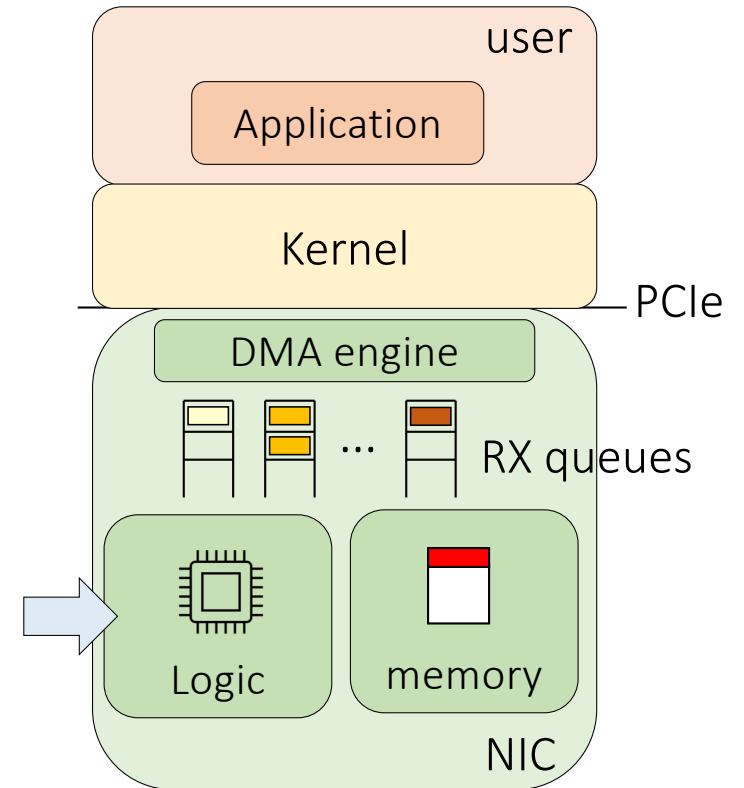
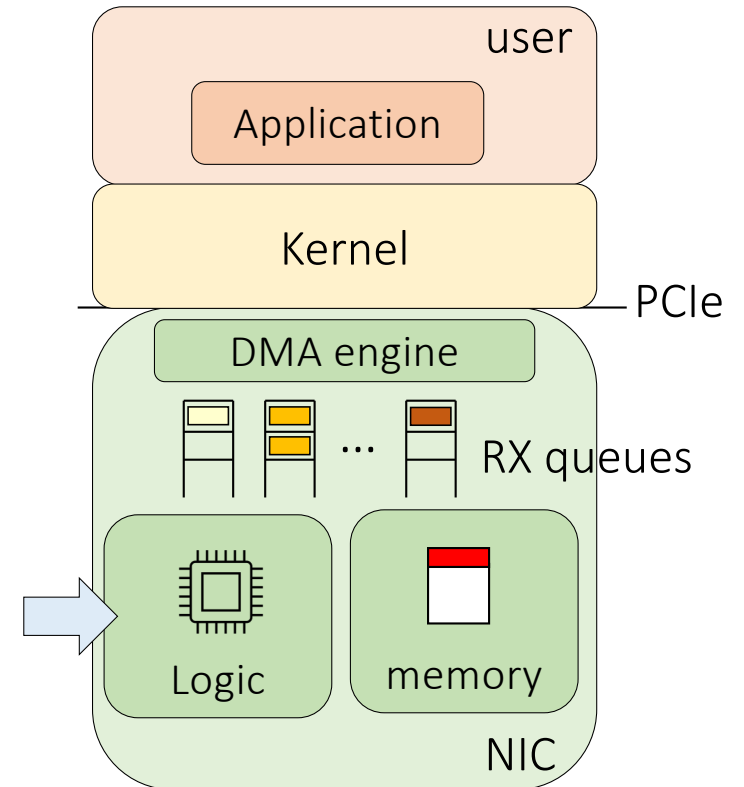


Figure 2: Azure SmartNIC boards with Bump-in-the-Wire Architecture



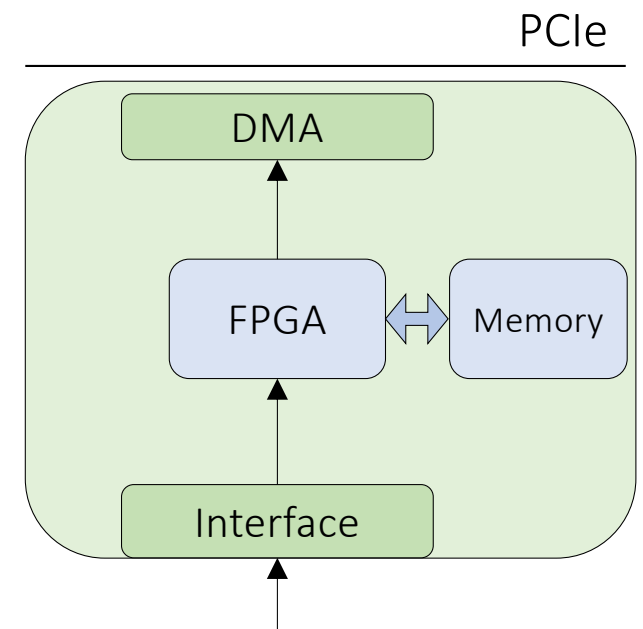
Programmable NICs

- There are many types of Programmable NICs with different
 - processing units (crypto, compression, CPU)
 - architectures (FPGA, SoC, ASIC)
 - programming models (Verilog/VHDL, C, P4, DOCA)
 - trade-offs (performance vs costs vs programmability)
- They can also be coupled with fast I/O frameworks such as DPDK

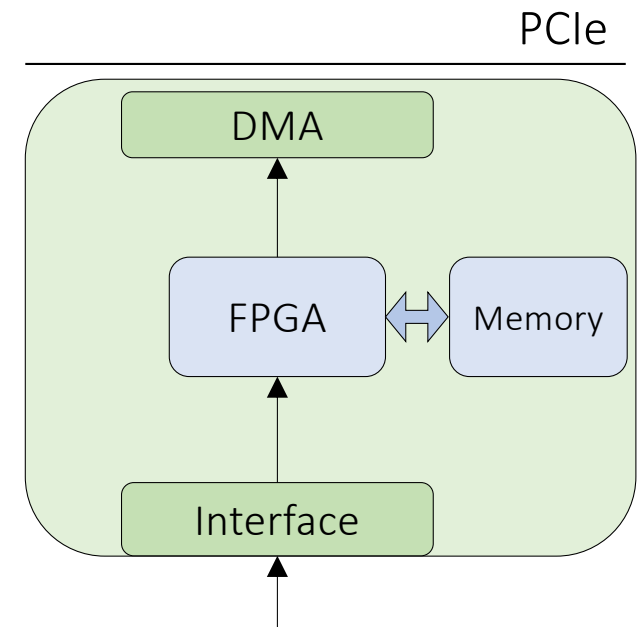
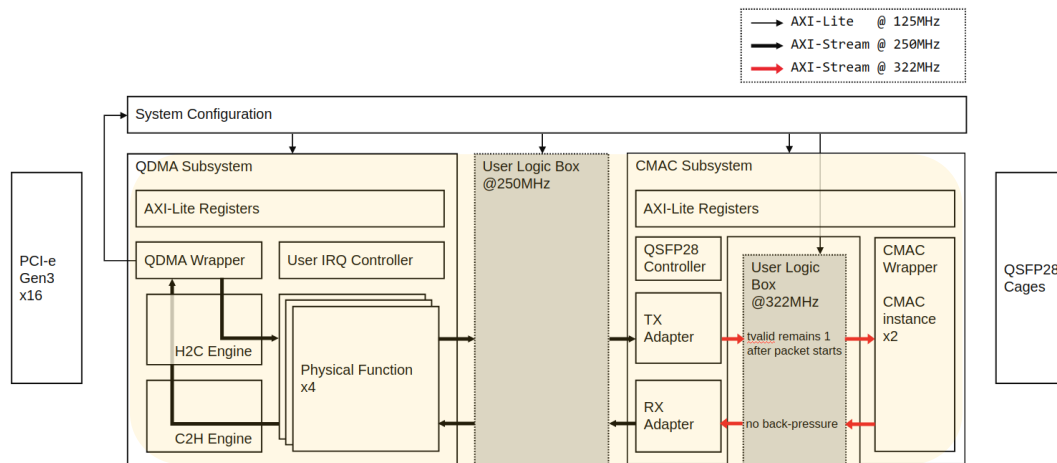


FPGA-based Programmable NICs

- Field Programmable Gate Arrays (FPGAs) are semiconductor devices can be reprogrammed to desired application or functionality requirements after manufacturing
- FPGAs provide a flexible alternative with near-ASIC performance
- They can be expensive and power-hungry, and programming them requires expert knowledge of the hardware

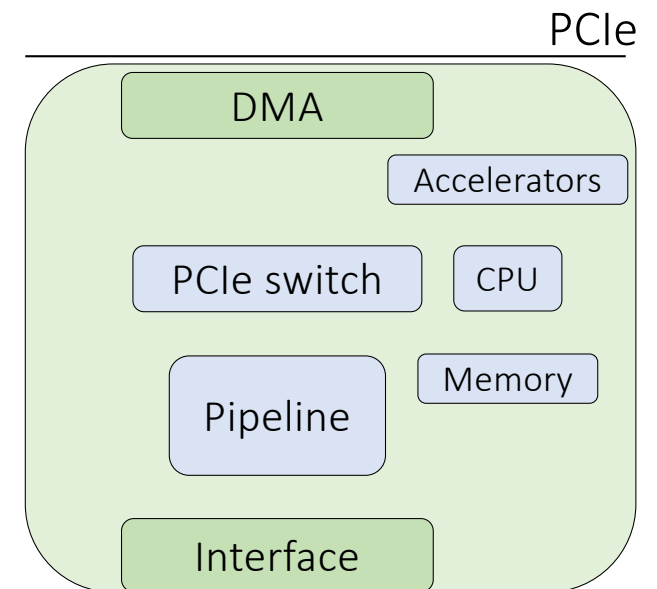


FPGA-based Programmable NICs



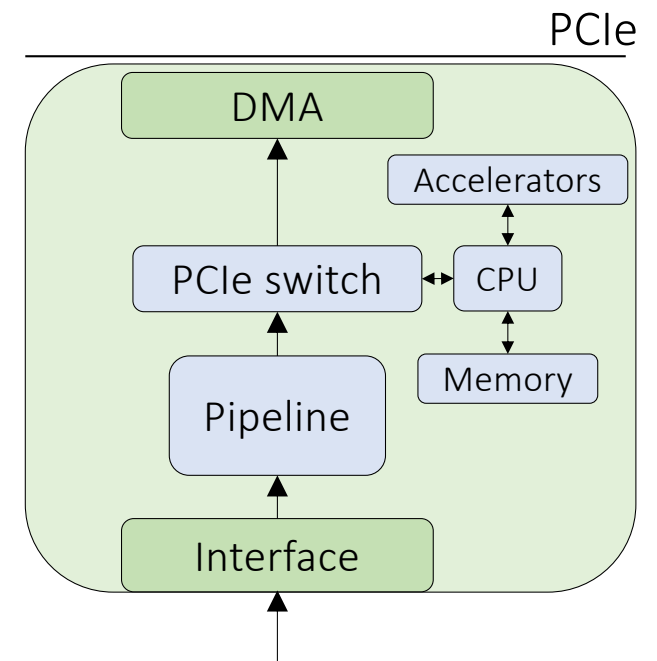
SoC based Programmable NICs

- System-on-a-chip (SoC) SmartNICs combines traditional ASICs with a modest number of general-purpose cores for much easier programming and fixed-function co-processors for custom workload acceleration

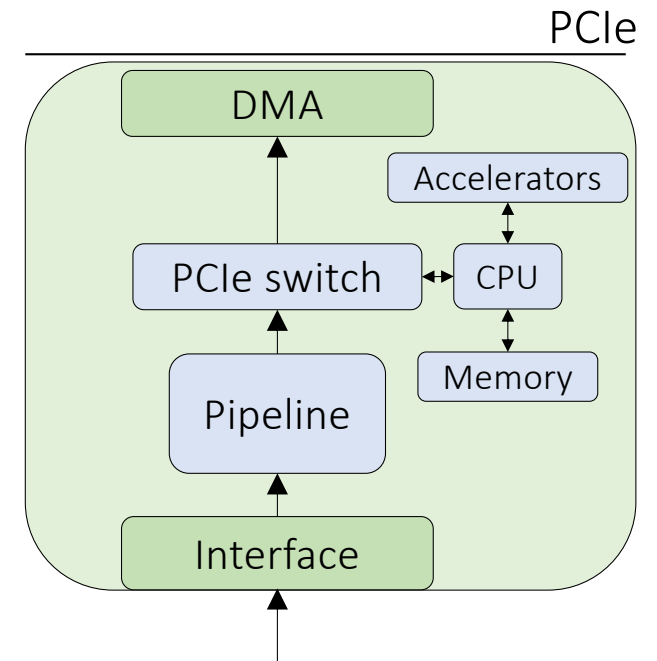
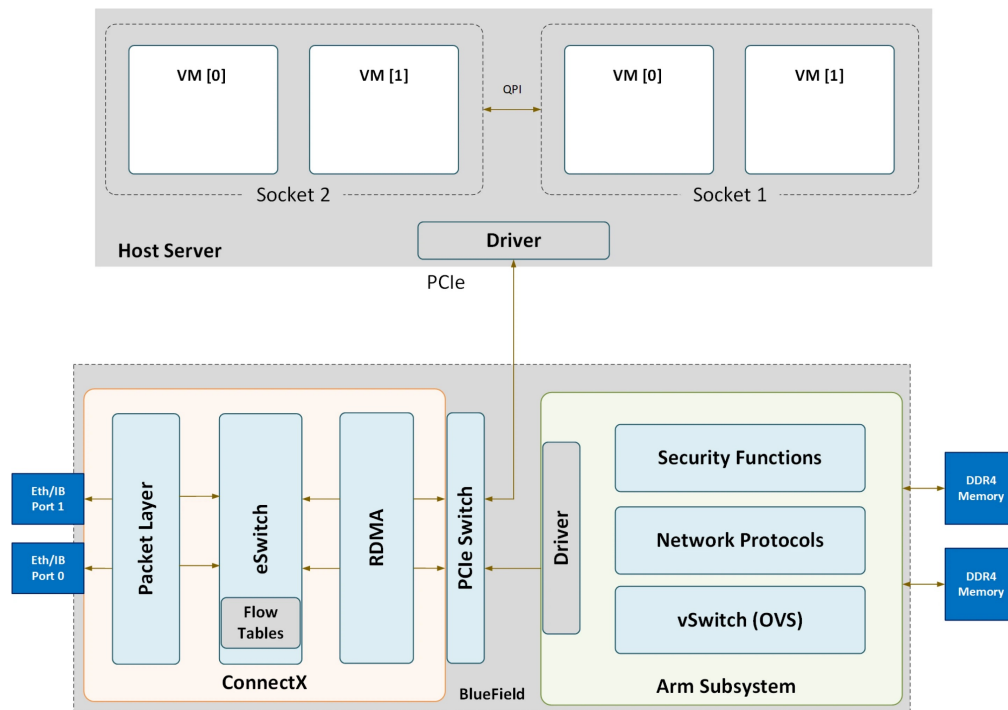


SoC based Programmable NICs

- System-on-a-chip (SoC) SmartNICs combines traditional ASICs with a modest number of general-purpose cores for much easier programming and fixed-function co-processors for custom workload acceleration
- off-path design pattern uses an on-NIC switch to route traffic between the network and NIC and host cores

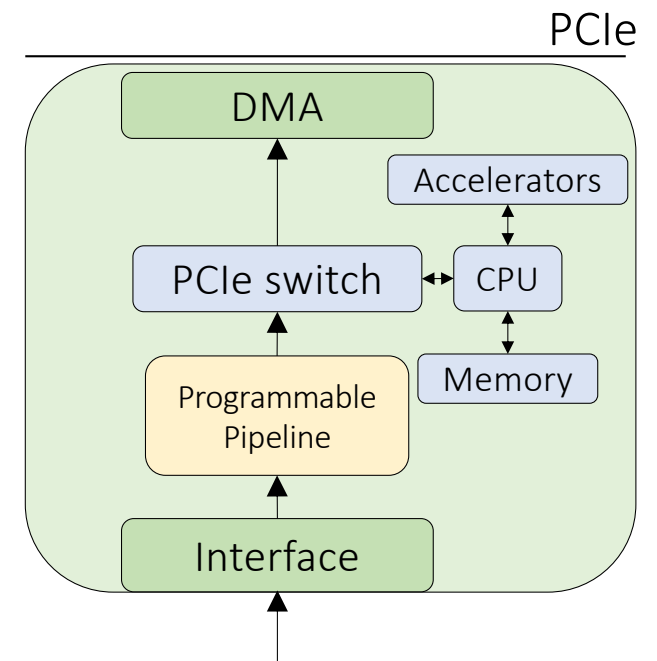


SoC based Programmable NICs

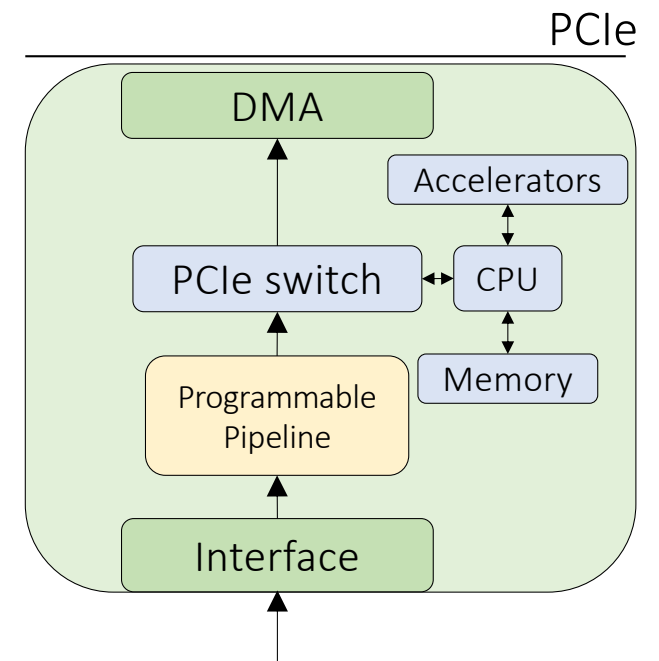
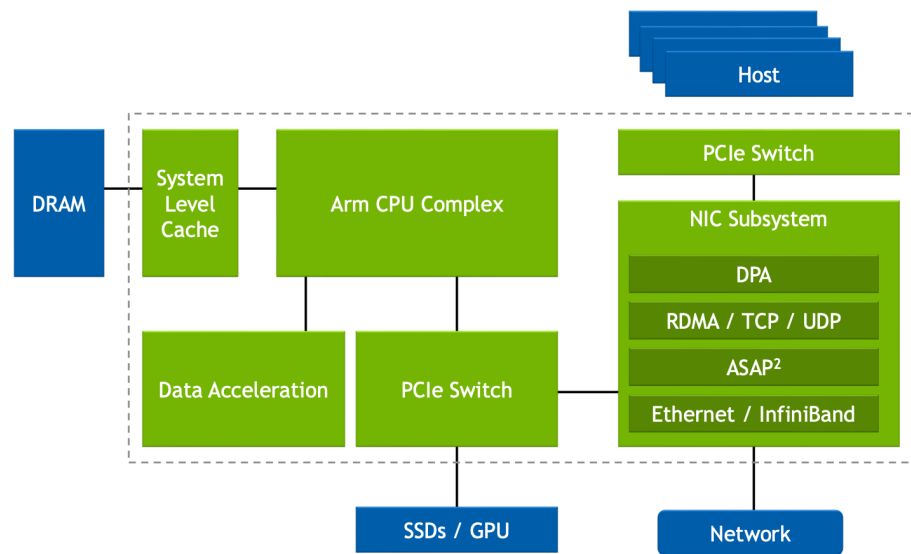


SoC based Programmable NICs

- System-on-a-chip (SoC) SmartNICs combines traditional ASICs with a modest number of general-purpose cores for much easier programming and fixed-function co-processors for custom workload acceleration
- **off-path** design pattern uses an on-NIC switch to route traffic between the network and NIC and host cores
- **on-path** approach passes all packets through a programmable pipeline on the way to or from the network

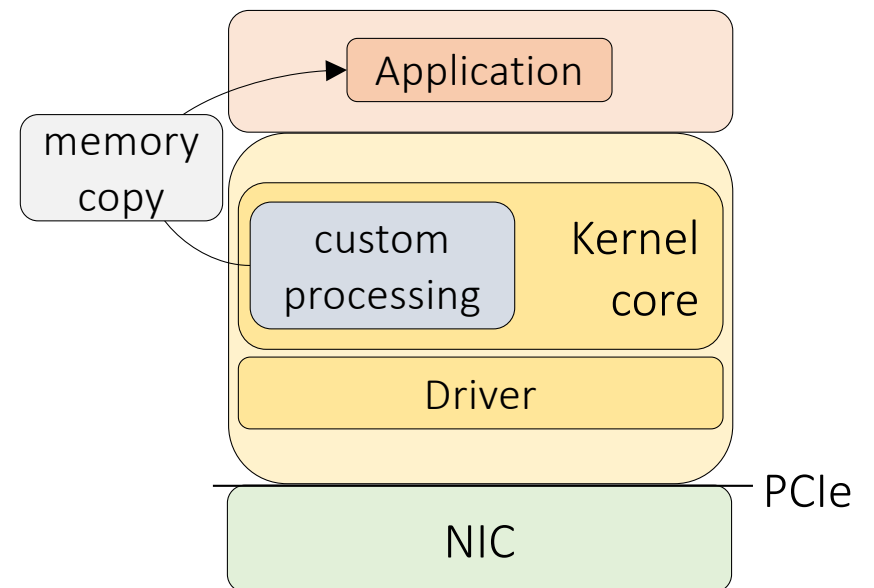


SoC based Programmable NICs



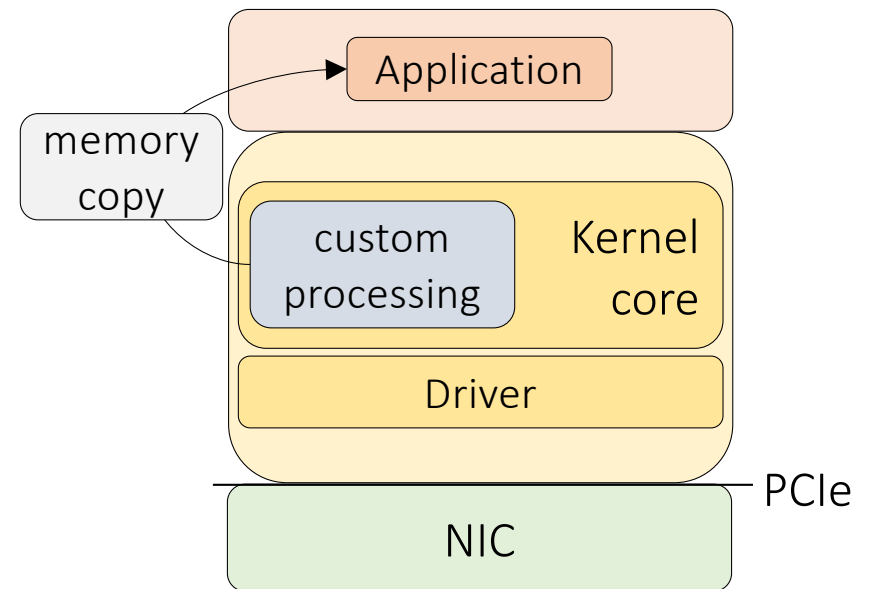
Option #4: Change kernel behavior

- You can improve system performance and reduce CPU utilization by rethinking the way kernel process packets
- You can add new functionalities that help in
 - reducing kernel to user-space data movement
 - creating fast-path in the kernel



Option #4: Change kernel behavior

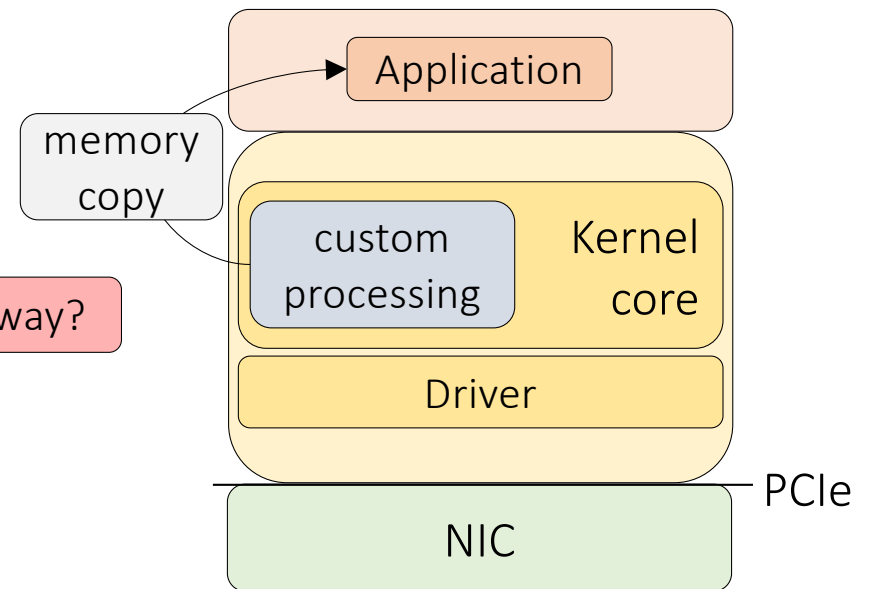
- The Linux Kernel is open source 😊
- You can change its code if you want



Option #4: Change kernel behavior

- The Linux Kernel is open source 😊
- You can change its code if you want

So, why just not change Linux kernel behavior straight away?

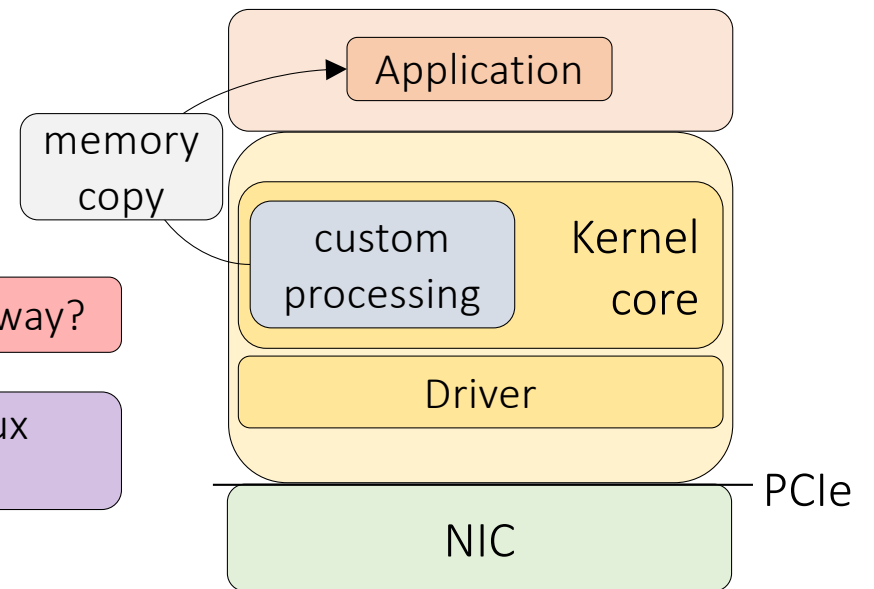


Option #4: Change kernel behavior

- The Linux Kernel is open source 😊
- You can change its code if you want

So, why just not change Linux kernel behavior straight away?

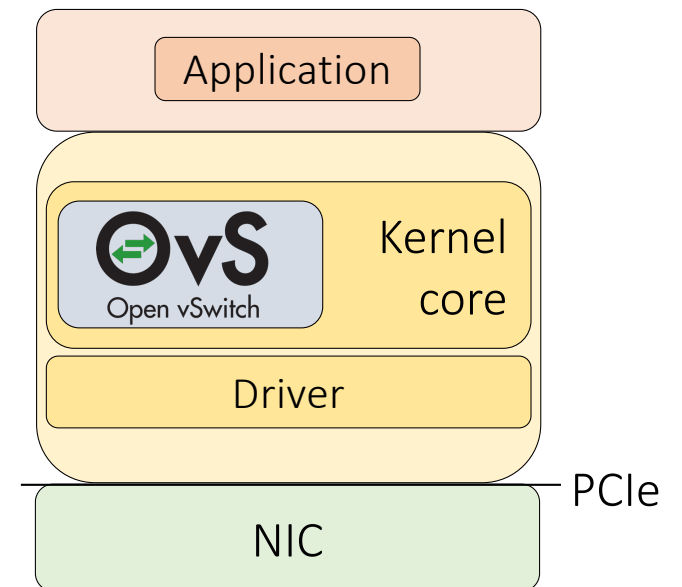
Either you keep a custom kernel or you need the Linux community to accept your changes 😞



Option #4: Change kernel behavior

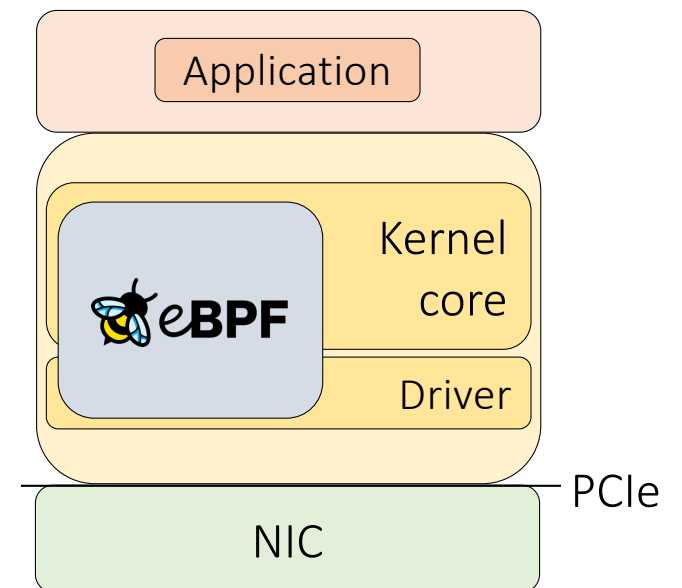
- OVS is part of the NSX product for datacenter virtualization used by thousands of VMware customers!

“The high bar to acceptance discourages developers from proposing features that would require experimentation or refinement, and Linux developers do reject some new OVS features”



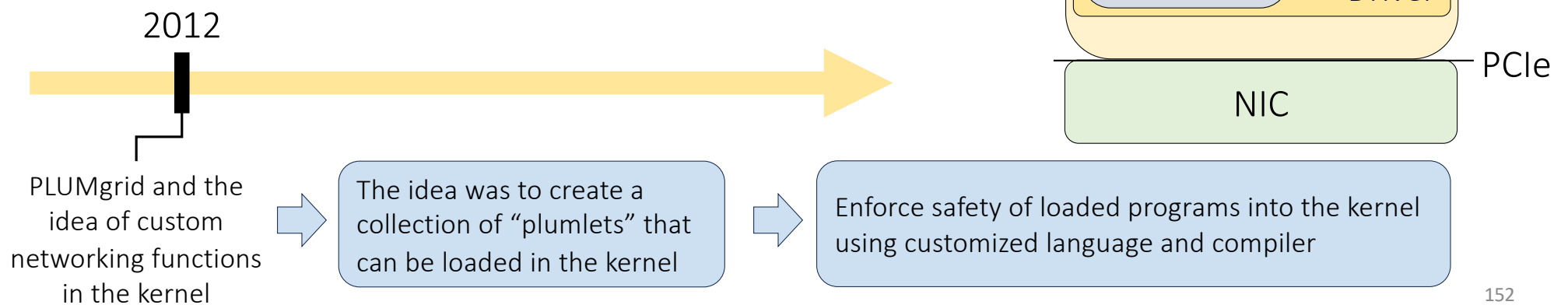
eBPF storyline

- eBPF is a programmable in-kernel execution framework that allows extending OS behavior safely at runtime



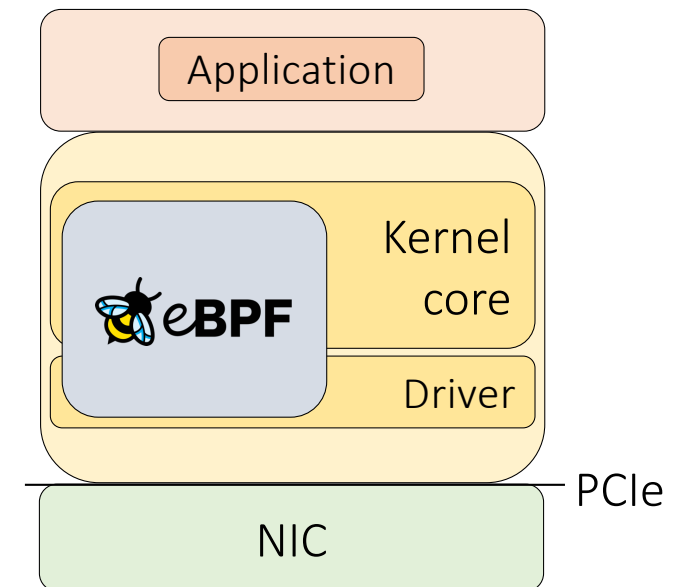
eBPF storyline

- eBPF is a programmable in-kernel execution framework that allows extending OS behavior safely at runtime



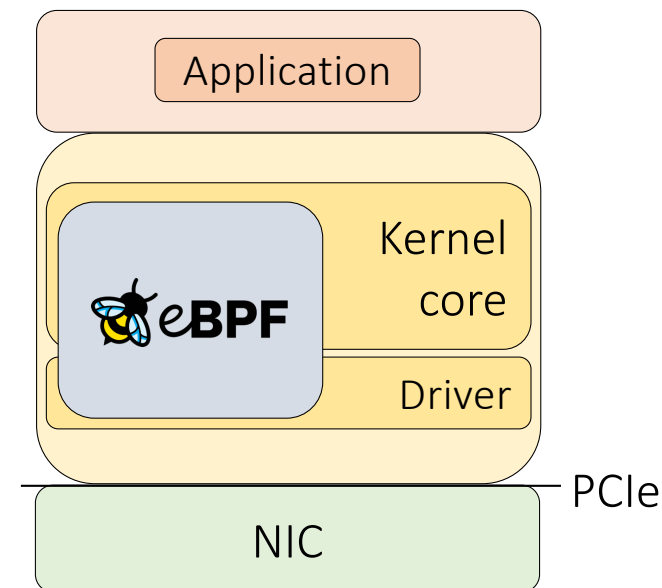
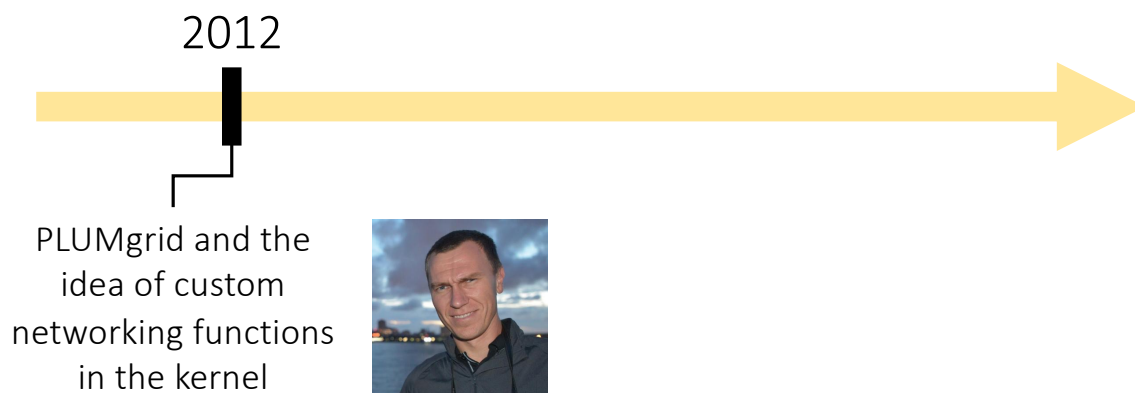
eBPF features

- **Feature 1:** Runtime bytecode injection
- eBPF programs can be dynamically created and injected in the kernel at run-time
 - Vanilla Linux kernel
 - No need for additional kernel modules
 - No need to recompile the kernel



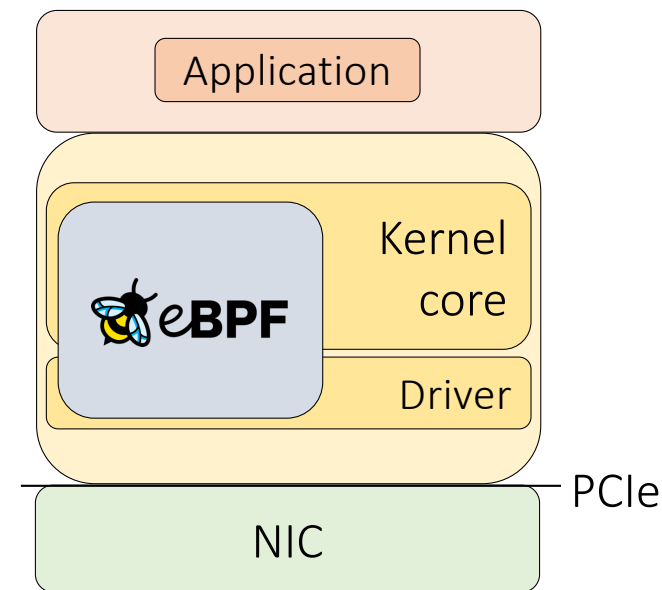
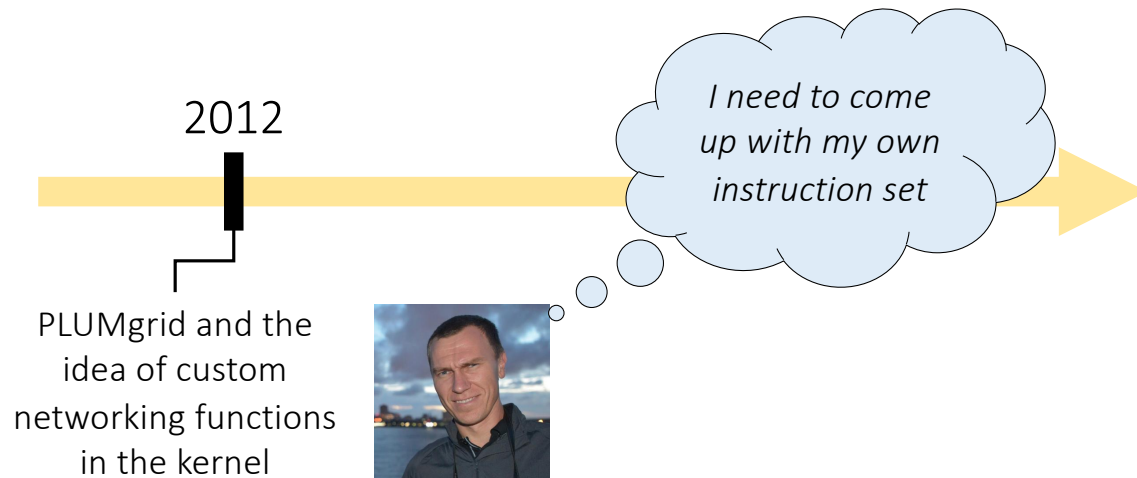
eBPF storyline

- One day, Alexei hit a bug that was causing the host to crash after hours of networking traffic
- He recognized that we cannot trust the compiler, we need something more



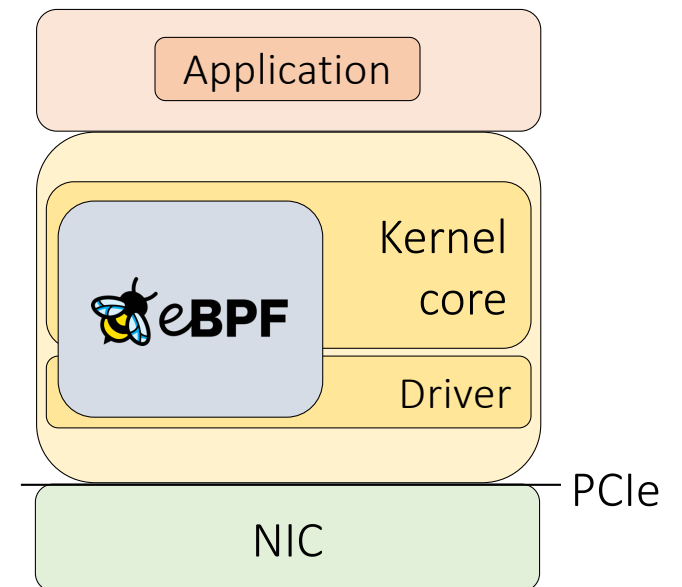
eBPF storyline

- One day, Alexei hit a bug that was causing the host to crash after hours of networking traffic
- He recognized that we cannot trust the compiler, we need something more



eBPF features

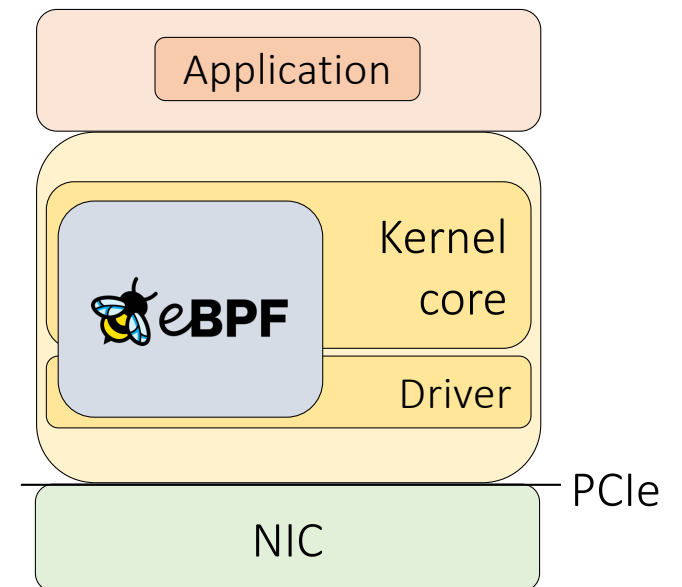
- Feature 2: Safetiness
- Linux kernel must be protected from erroneous or malicious injected programs
 - Achieved with a sandbox that prevents critical conditions at run-time
- A verifier checks the code before it gets sandboxed to ensure
 - No invalid memory accesses
 - Bounded program size
 - Bounded max number of instructions



eBPF features

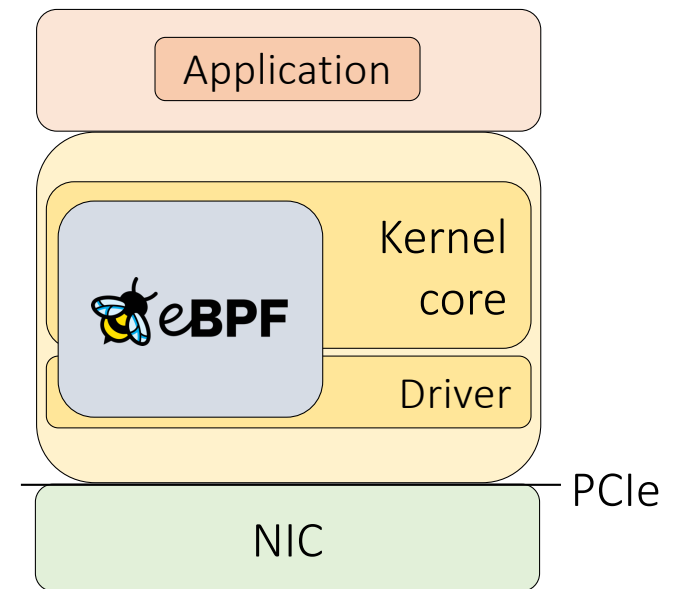
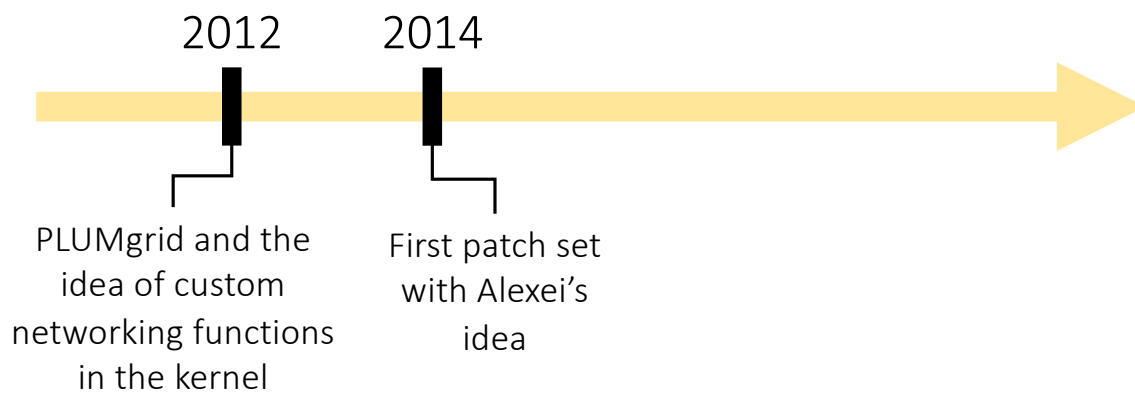
- Feature 2: Safetiness
- Linux kernel must be protected from erroneous or malicious injected programs
 - Achieved with a sandbox that prevents critical conditions at run-time
- A verifier checks the code before it gets sandboxed to ensure
 - No invalid memory accesses
 - Bounded program size
 - Bounded max number of instructions

Consequence: we cannot push arbitrary programs in the kernel!



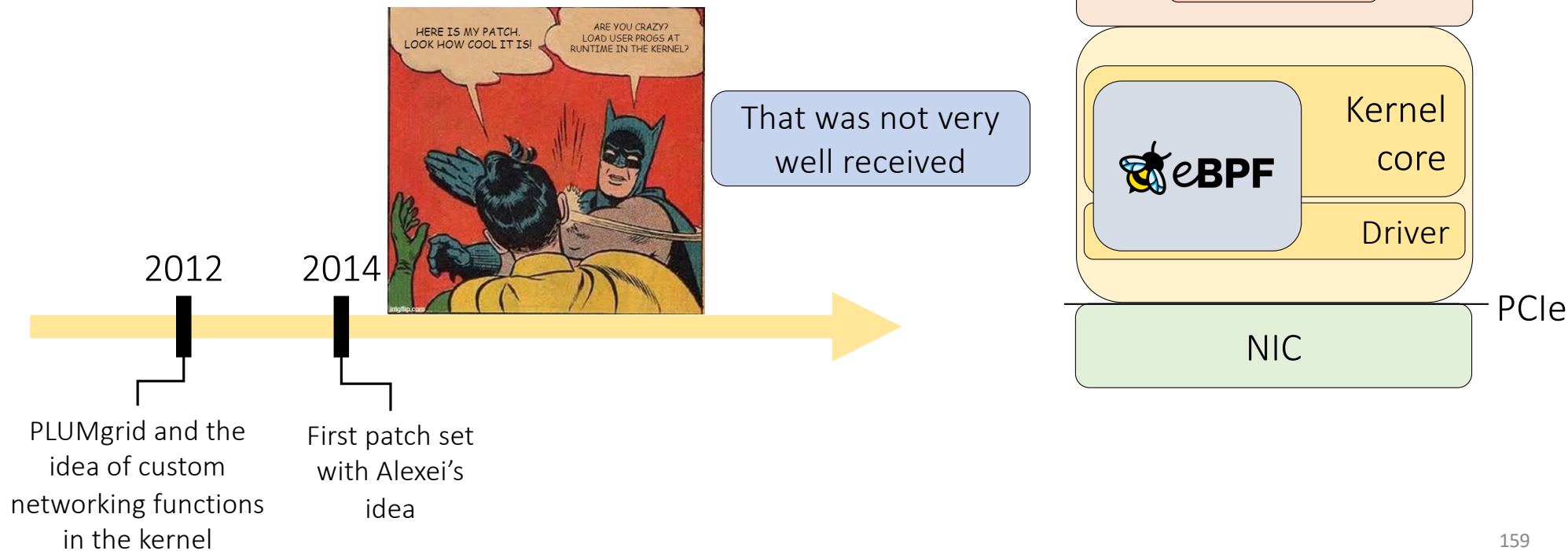
eBPF storyline

- The only way to get this widely adopted was to create a patch upstream it to the Linux kernel



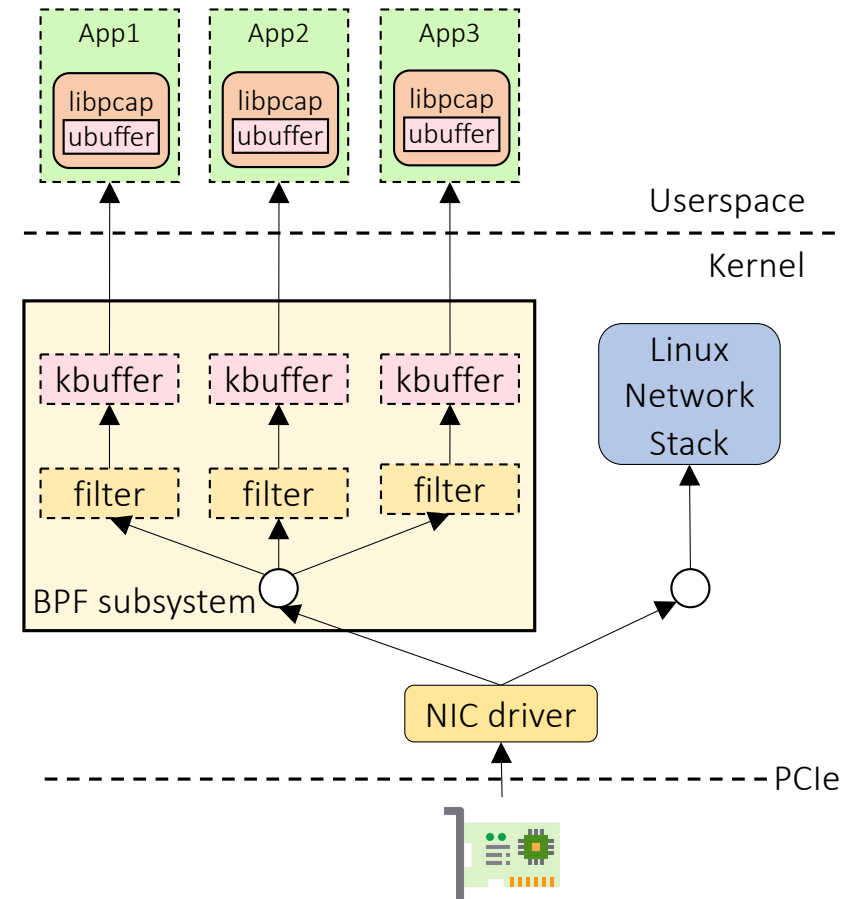
eBPF storyline

- The only way to get this widely adopted was to create a patch upstream it to the Linux kernel



The Berkeley Packet Filter (BPF)

- Generic in-kernel, event-based virtual CPU
 - Introduced in 1993 from Lawrence Berkeley National Laboratory
 - Available in Linux kernel 2.1.75 (1997)
 - Initially used as packet filter by packet capture tool tcpdump (via libpcap)
- Ad-hoc execution environment for packet filtering
 - Specific memory for packets (separated from the main RAM)
 - Specific vCPU interpreter



eBPF storyline

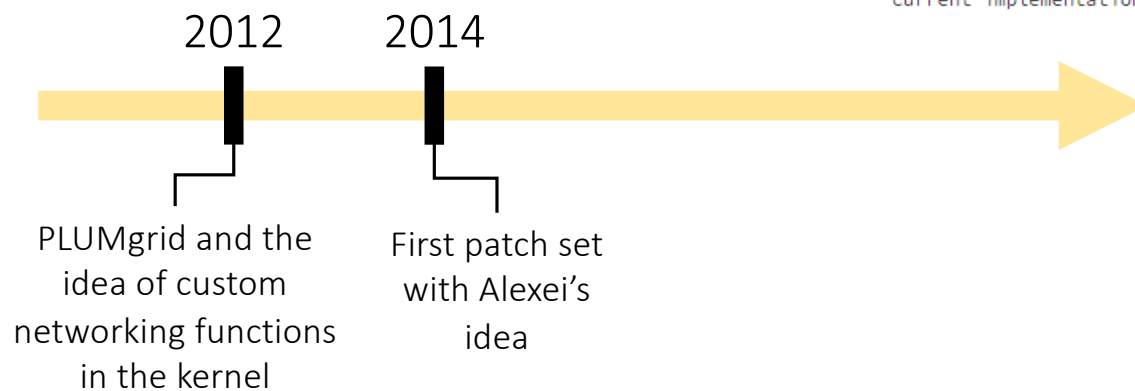
tracing: accelerate tracing filters with BPF

From: Alexei Starovoitov <ast@plumgrid.com>
To: Ingo Molnar <mingo@kernel.org>
Subject: [PATCH RFC net-next] tracing: accelerate tracing filters with BPF
Date: Tue, 13 May 2014 19:55:11 -0700
Message-ID: <1400036111-7803-1-git-send-email-ast@plumgrid.com>
Cc: "David S. Miller" <davem@davemloft.net>, Eric Dumazet <edumazet@google.com>, Daniel Borkmann <dborkman@redhat.com>, Steven Rostedt <rostedt@goodmis.org>, Peter Zijlstra <a.p.zijlstra@chello.nl>, Arnaldo Carvalho de Melo <acme@infradead.org>, Jiri Olsa <jolsa@redhat.com>, Thomas Gleixner <tglx@linutronix.de>, "H. Peter Anvin" <hpa@zytor.com>, netdev@vger.kernel.org, linux-kernel@vger.kernel.org
Archive-link: [Article](#)

author Alexei Starovoitov <ast@plumgrid.com> 2014-03-28 18:58:25 +0100
committer David S. Miller <davem@davemloft.net> 2014-03-31 00:45:09 -0400
commit bd4cf0ed331a275e9bf5a49e6d0fd55dffc551b8 (patch)
tree 6ffb15296ce4cdc1f272e31bd43a5804b8da588c
parent 77e0114ae9ae08685c503772a57af21d299c6701 (diff)
download [linux-bd4cf0ed331a275e9bf5a49e6d0fd55dffc551b8.tar.gz](#)

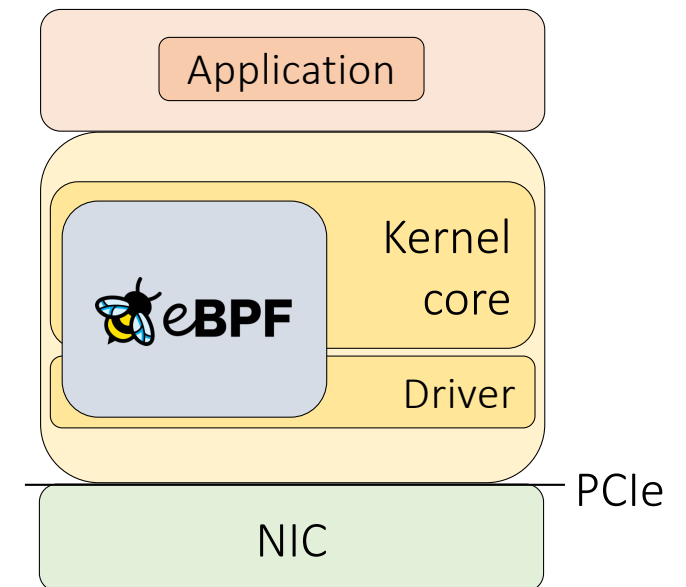
net: filter: rework/optimize internal BPF interpreter's instruction set

This patch replaces/reworks the kernel-internal BPF interpreter with an optimized BPF instruction set format that is modelled closer to mimic native instruction sets and is designed to be JITed with one to one mapping. Thus, the new interpreter is noticeably faster than the current implementation of `sk_run_filter()`; mainly for two reasons:



eBPF features

- Feature 3: Efficiency
- eBPF programs consumes a little amount of resources
- They executes in kernel space, potentially close to when packets are received (no need to copy packets as when we move them from kernel to user)

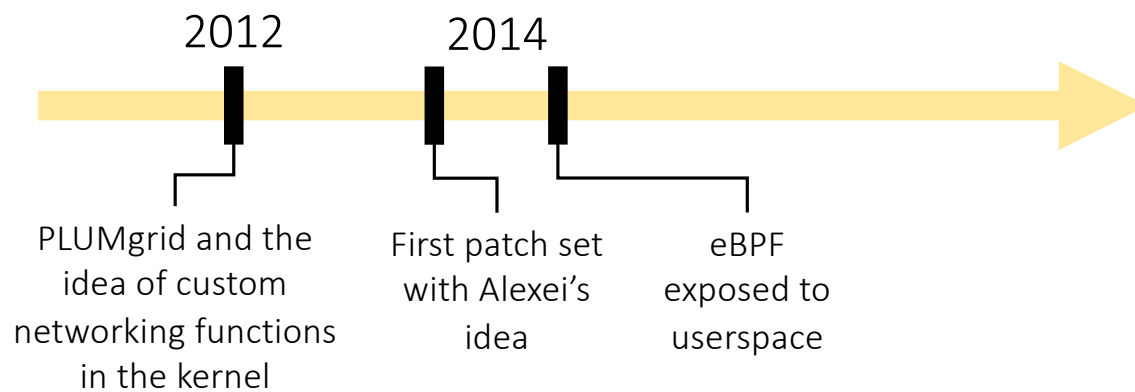


eBPF storyline

```
author    Alexei Starovoitov <ast@plumgrid.com> 2014-09-04 22:17:18 -0700
committer David S. Miller <davem@davemloft.net> 2014-09-09 10:26:47 -0700
commit    daedfb22451dd02b35c0549566cbb7cc06bdd53b (patch)
tree      f990840a7b9e6afe48ea73a5fdafe1cdc50f936d
parent    02ab695bb37ee9ad515df0d0790d5977505dd04a (diff)
download  linux-daedfb22451dd02b35c0549566cbb7cc06bdd53b.tar.gz
```

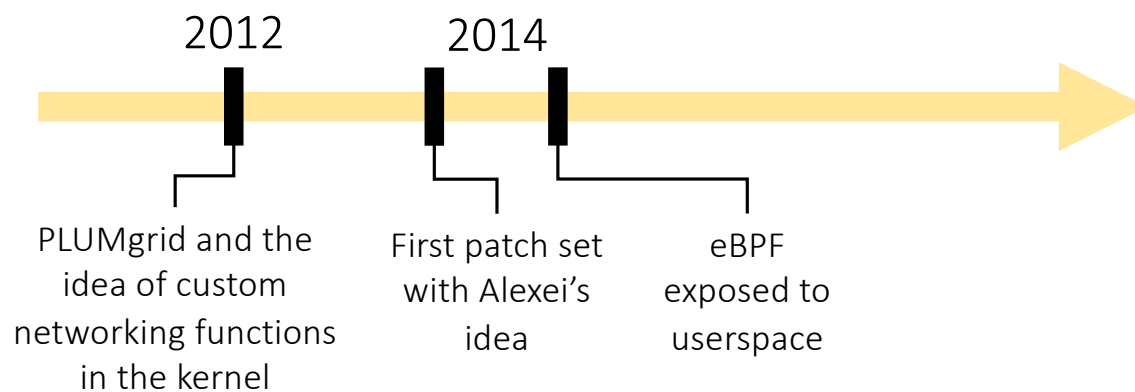
net: filter: split filter.h and expose eBPF to user space

allow user space to generate eBPF programs



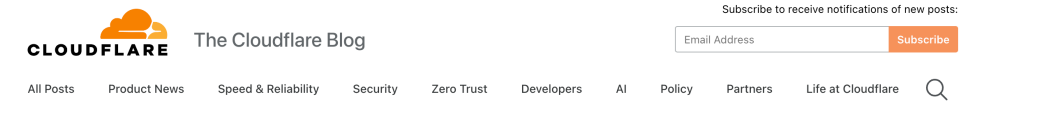
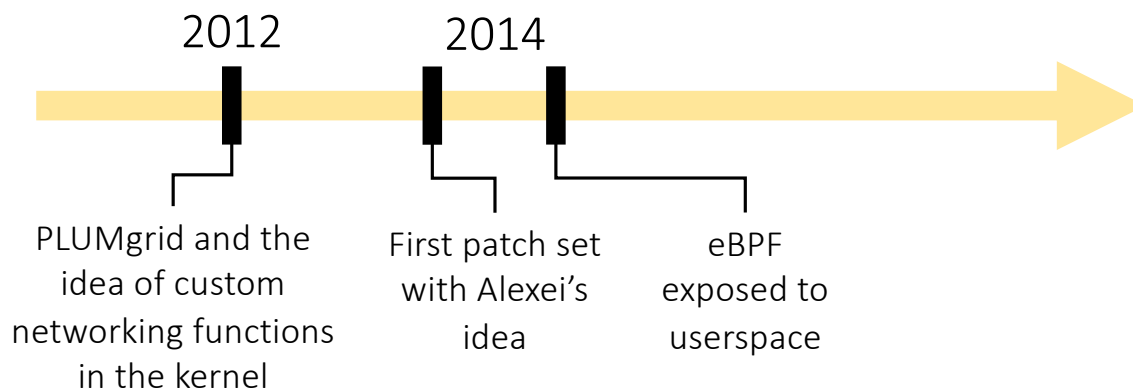
eBPF storyline

- Officially part of the Linux kernel since 3.15
- Initially, this was identified as eBPF and the old BPF called classic BPF or cBPF
- Now, people tend to refer to eBPF with just BPF



eBPF storyline

- Realization that this opened up many opportunities
 - Kernel tracing
 - Packet-processing
 - CPU scheduling



Cloudflare architecture and how BPF eats the world

05/18/2019

How Netflix uses eBPF flow logs at scale for network insight



Netflix Technology Blog · Follow

Published in Netflix TechBlog · 4 min read · Jun 7, 2021

823 7



Controlling the CPU scheduler with BPF

LWN.net needs you!

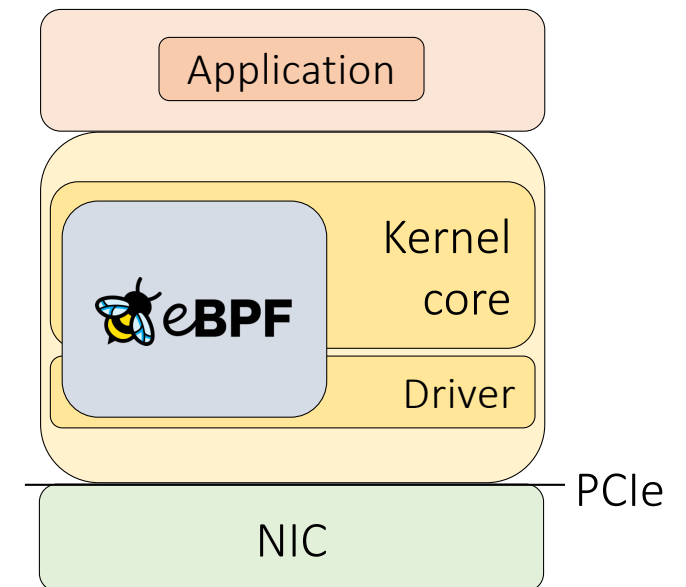
Without subscribers, LWN would simply not exist. Please consider [signing up for a subscription](#) and helping to keep LWN publishing

By Jonathan Corbet
October 21, 2021

While the BPF virtual machine has been supported by Linux for most of the kernel's existence, its role for much of that time was limited to, as its full name (Berkeley packet filter) would suggest, filtering packets. That [began to change](#) in 2012 with the introduction of `seccomp()` filtering, and [the pace picked up](#) in 2014 with the arrival of the extended BPF virtual machine. At this point, BPF hooks have found their way into many kernel subsystems. One area that has remained BPF-free, though, is the CPU scheduler; that could change if some version of [this patch set](#) from Roman Gushchin finds its way into the mainline.

eBPF features

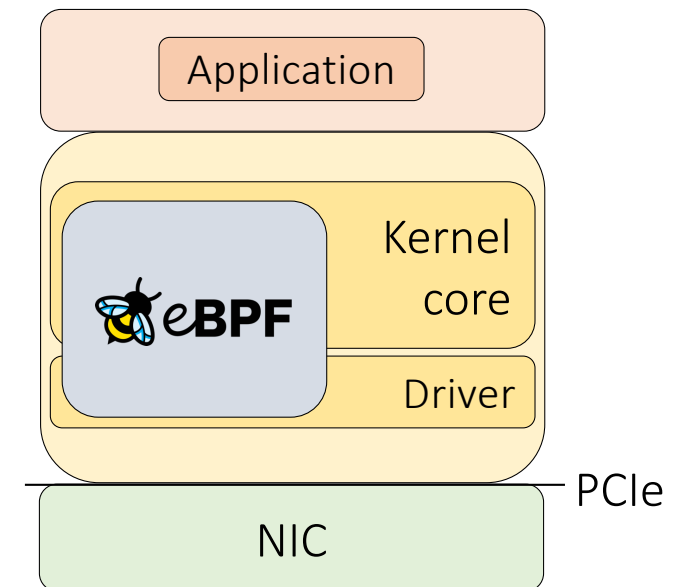
- **Feature 4:** Kernel events reaction
- eBPF code is hooked to a kernel event
 - When fired, your code (associated to an event handler) is executed
- Examples of possible events:
 - Network packet received
 - Message (socket-layer) received
 - Data written to disk
 - Page fault in memory
 - File in /etc folder being modified



eBPF features

- In general, any event can be potentially intercepted
- Depending on the event where the program is attached to, the context change

Example 1: a program attached to the TCP/IP stack is executed when a packet is received, and the context is the packet buffer

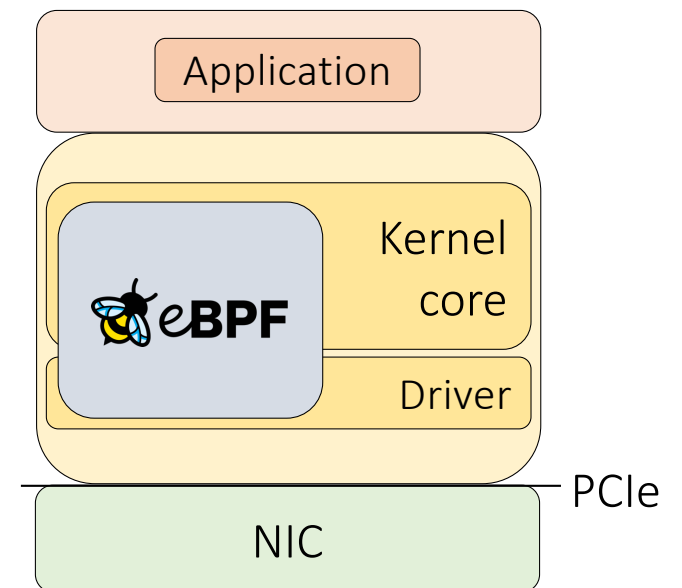


eBPF features

- In general, any event can be potentially intercepted
- Depending on the event where the program is attached to, the context change

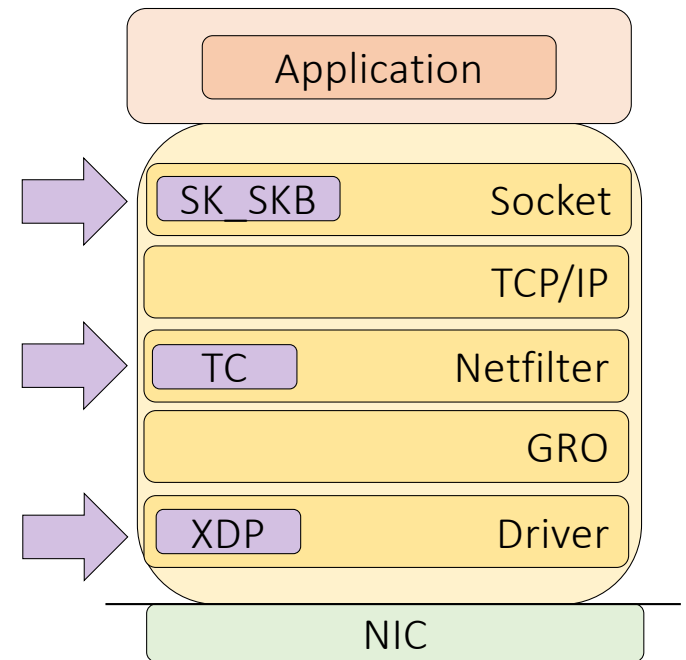
Example 1: a program attached to the TCP/IP stack is executed when a packet is received, and the context is the packet buffer

Example 2: a program attached to a syscall can be executed before or after the syscall execution and the context is the list of parameters of the syscall



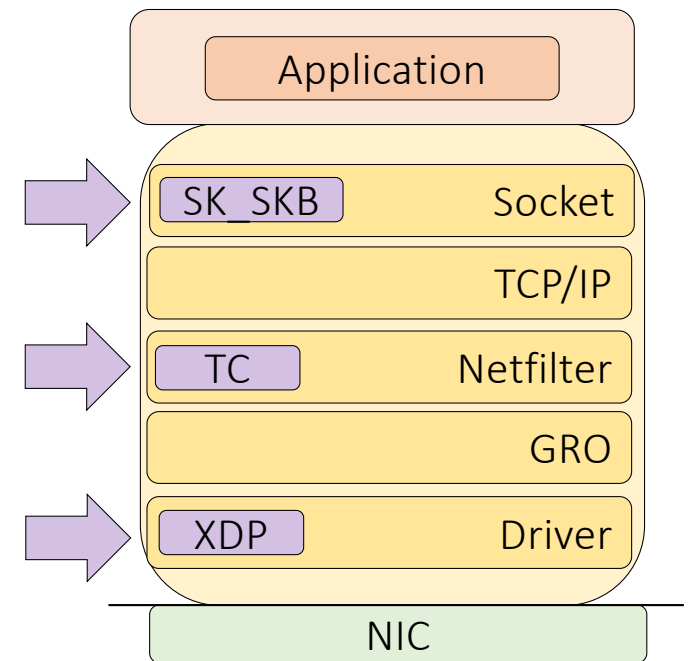
Hook points

- Several hook points (kernel events) for networking
 - Located at different levels of the stack
 - Opens up the possibility to implement packet-processing programs at different layers of the stack
- Some of interest:
 - eXpress Data Path (XDP)
 - Traffic Control (TC)
 - Socket SKB (SK_SKB)
 - There are more...



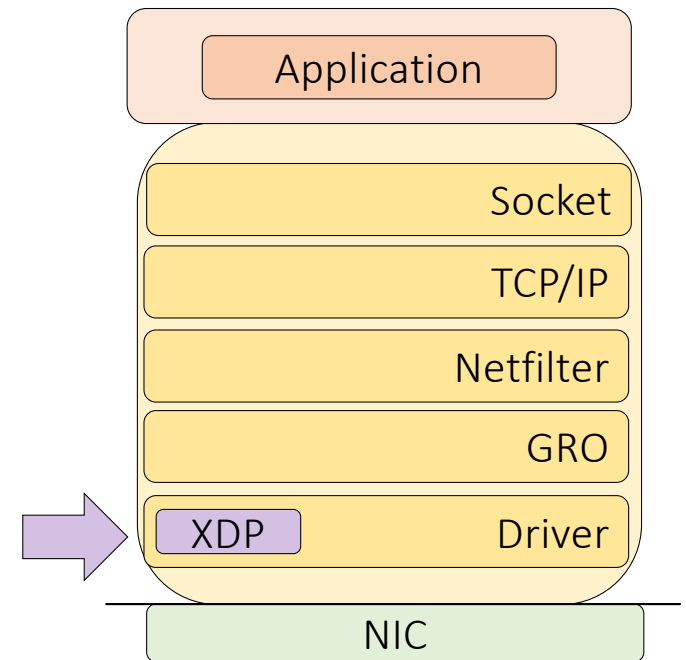
Hook points

- The eBPF program returns a verdict to tell the OS how to process the packet next
 - Redirect (redirect the packet to another net device of index ifindex)
 - Pass (continue the path of the packet in the stack)
 - Drop (drop the packet)



XDP

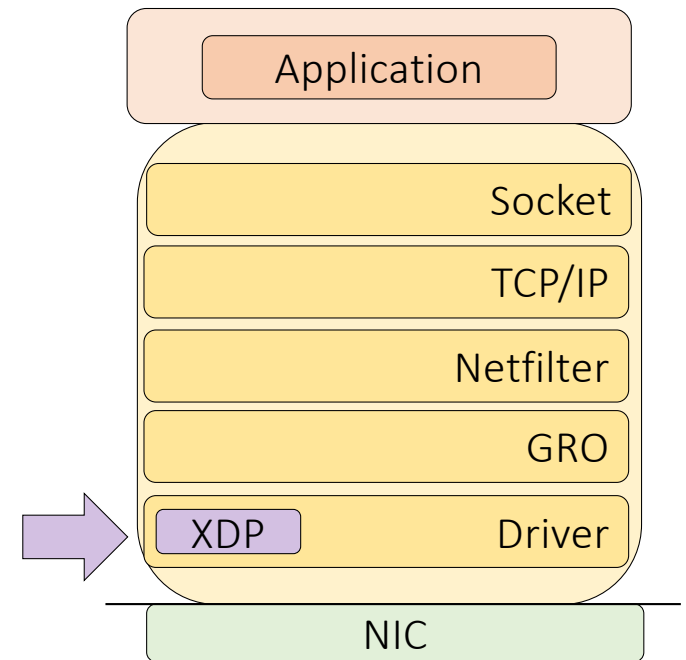
- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing



XDP

- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing

Why?

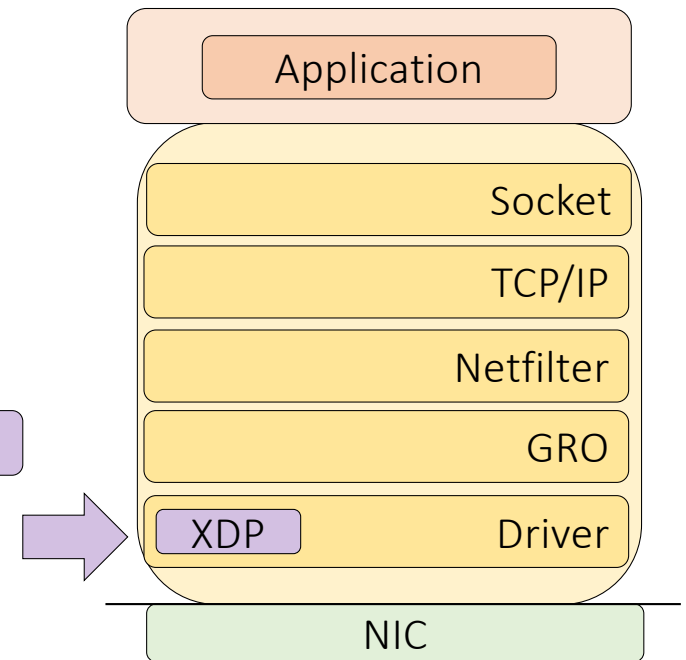


XDP

- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing

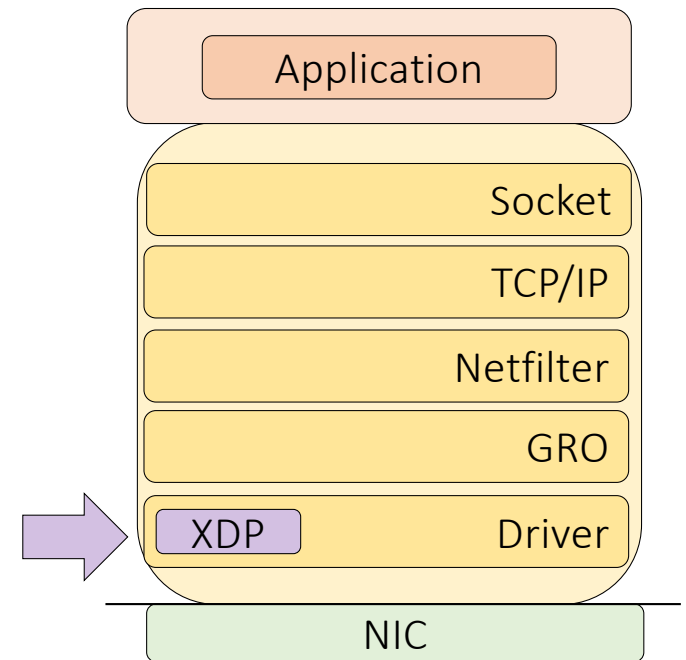
Why?

skb allocation is expensive and kernel processing too



XDP

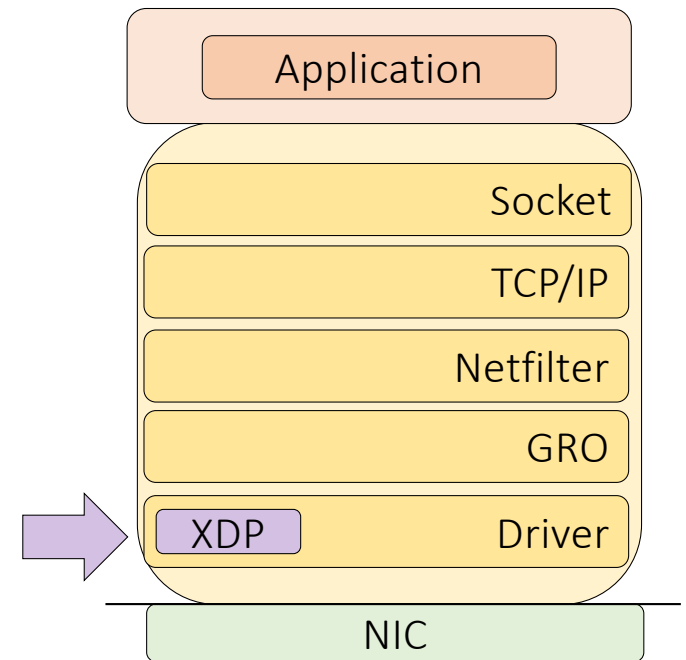
- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing
- XDP is good for offloading stateless protocols like UDP



XDP

- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing
- XDP is good for offloading stateless protocols like UDP

Why?

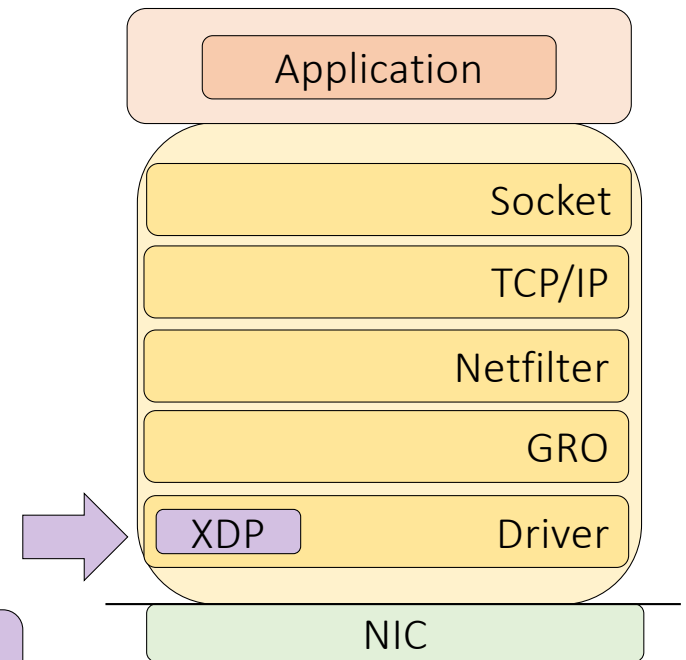


XDP

- It allows to run eBPF programs at the driver level, before skb allocation
- This is the earliest hook-point, suitable for high-performance processing
- XDP is good for offloading stateless protocols like UDP

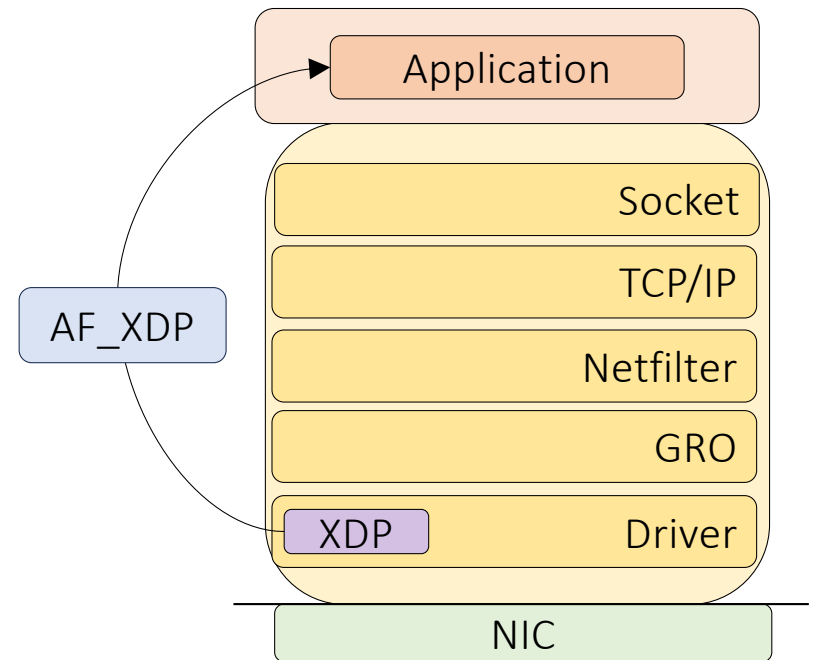
Why?

dropping or modifying TCP packets can corrupt the connection state in the kernel



AF_XDP

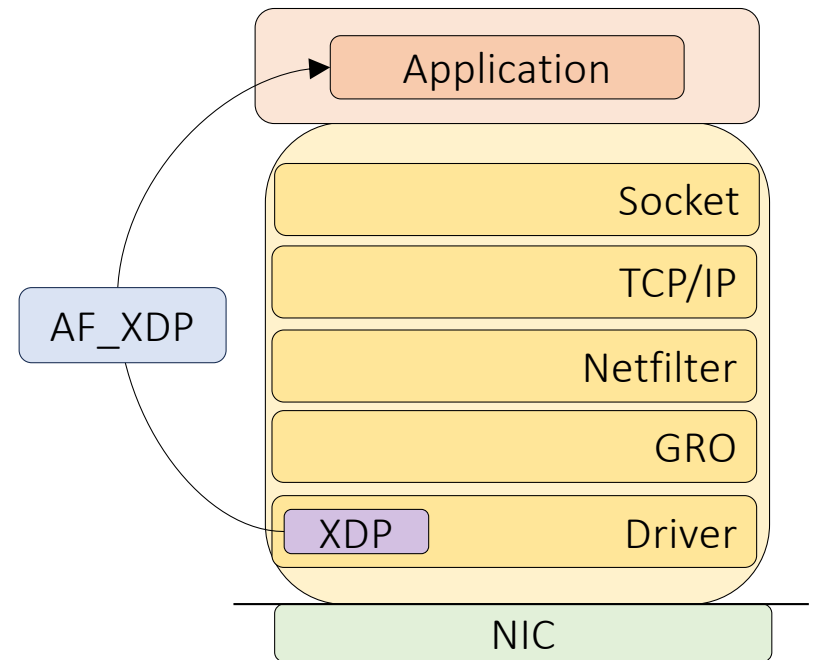
- AF_XDP sockets allow XDP programs to redirect packets directly into user-space memory buffers for fast packet delivery
- This mechanism bypasses the traditional kernel networking stack, including GRO, Netfilter, and the TCP/IP layers
- AF_XDP provides Linux's native approach to kernel-bypass packet processing, comparable to frameworks such as DPDK



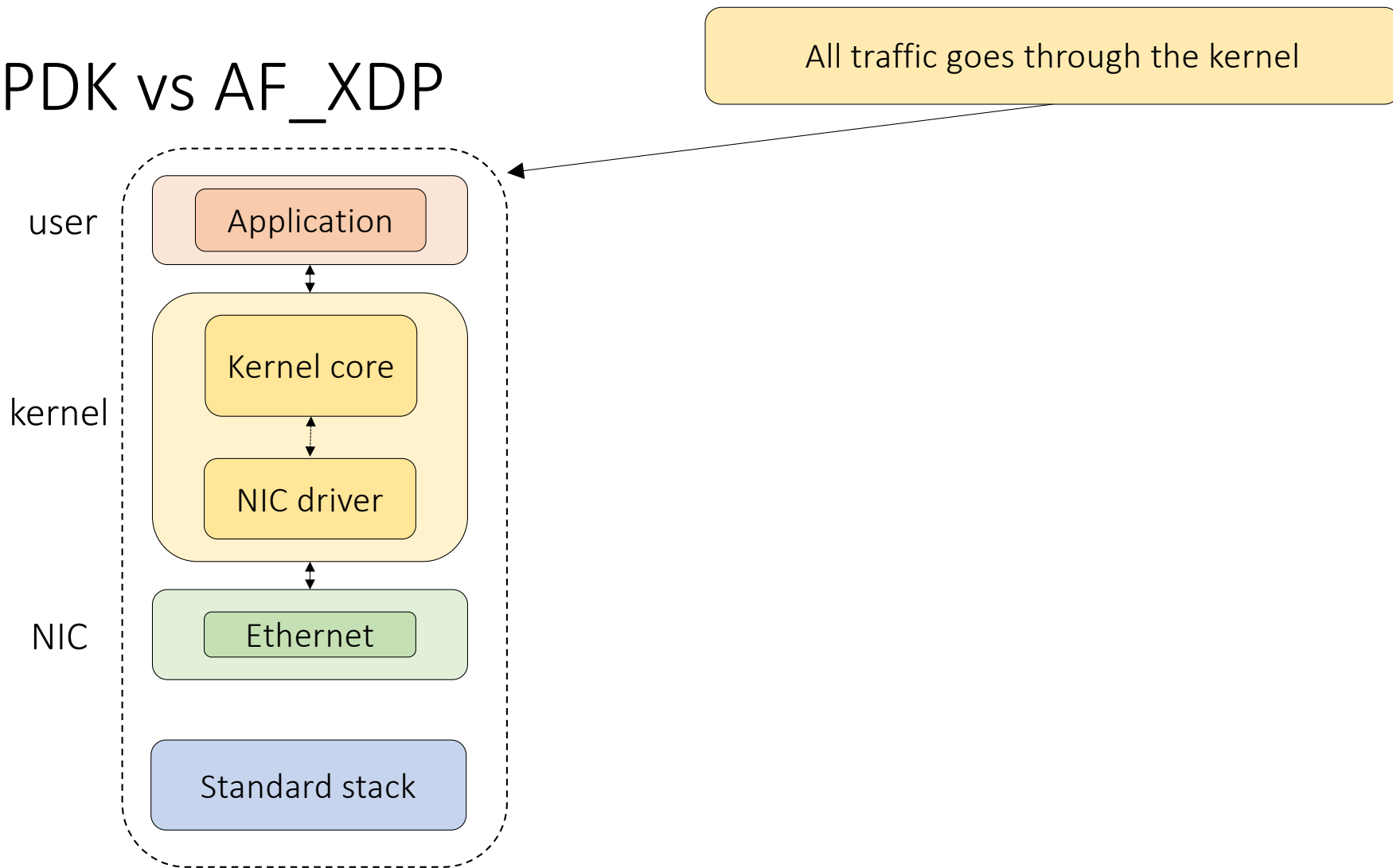
AF_XDP

- AF_XDP sockets allow XDP programs to redirect packets directly into user-space memory buffers for fast packet delivery
- This mechanism bypasses the traditional kernel networking stack, including GRO, Netfilter, and the TCP/IP layers
- AF_XDP provides Linux's native approach to kernel-bypass packet processing, comparable to frameworks such as DPDK

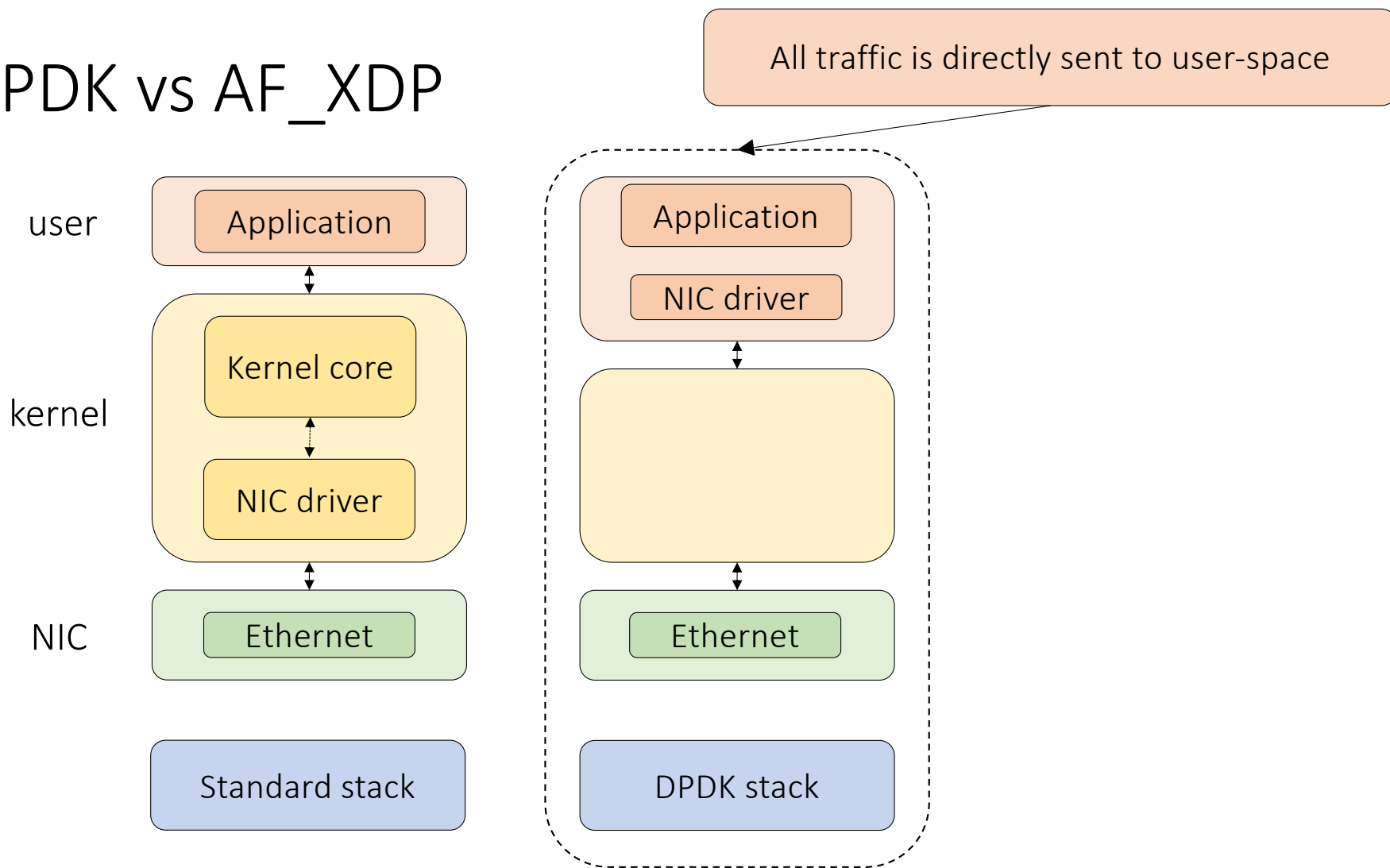
DPDK gives up all the kernel features, is this the same with AF_XDP?



DPDK vs AF_XDP

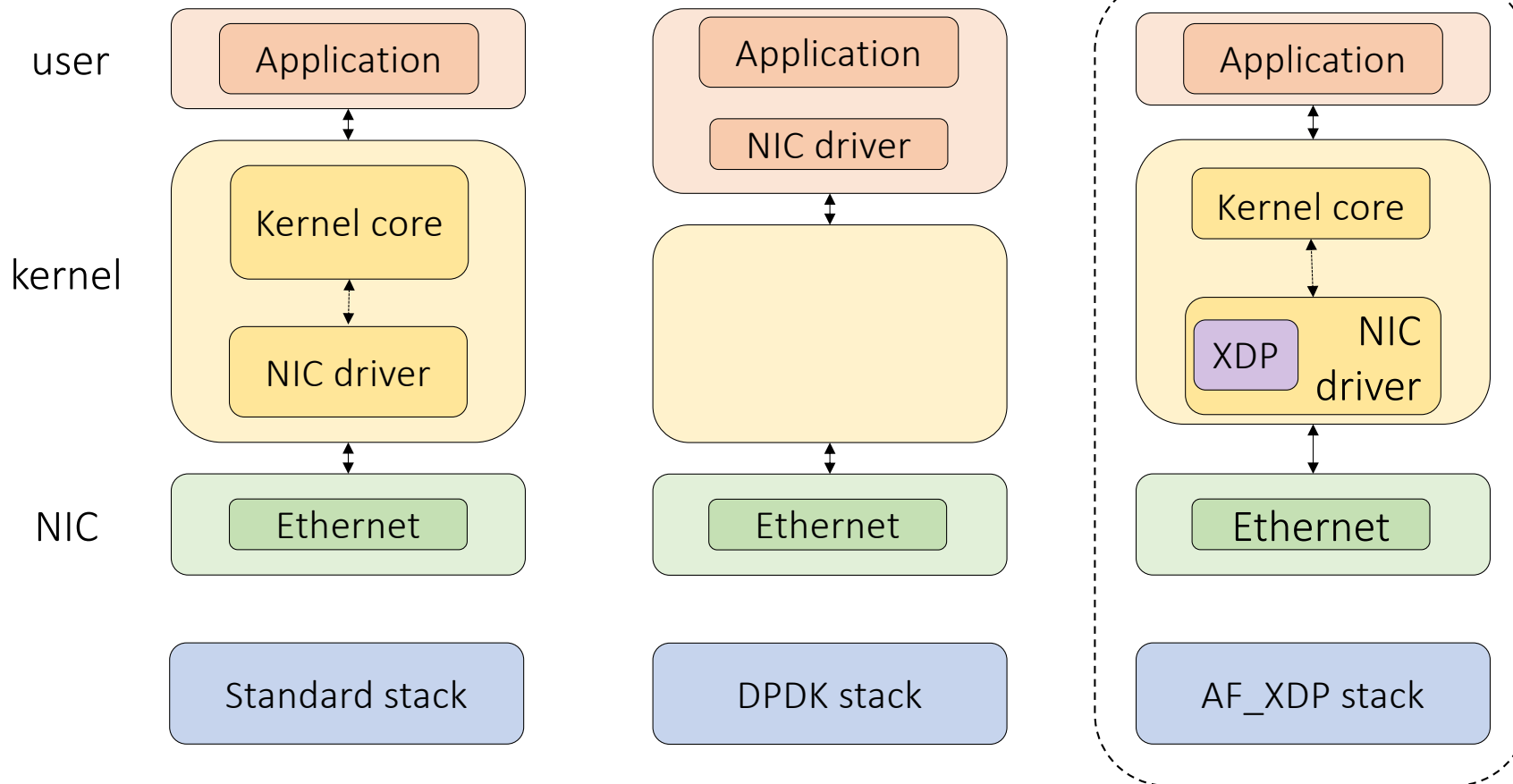


DPDK vs AF_XDP

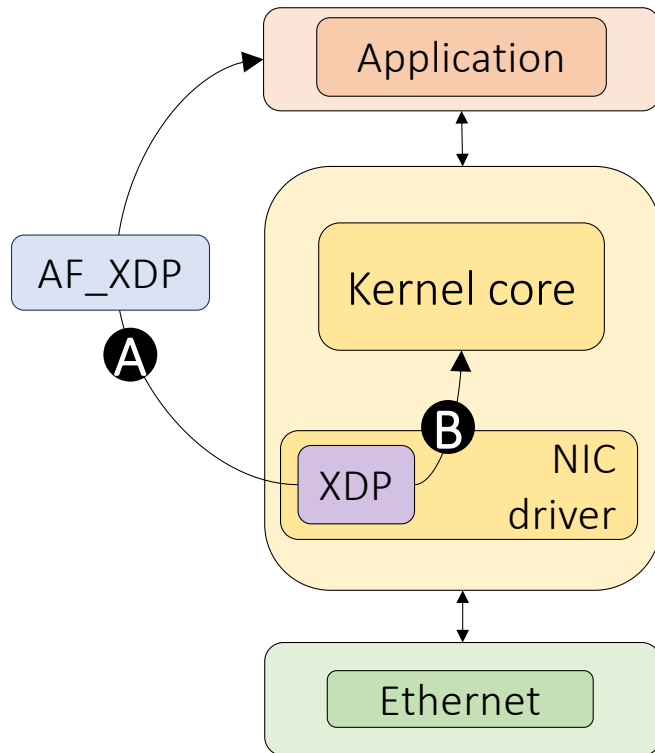


DPDK vs AF_XDP

Depending on the XDP program, traffic can go either through the kernel or directly sent to userspace



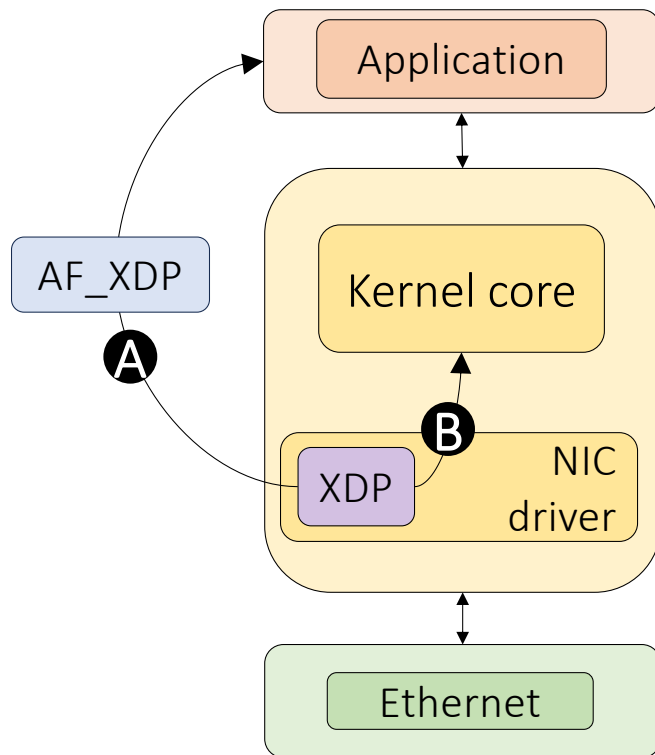
AF_XDP split processing



- The XDP program can steer packets through two different paths
- Path A: packets bypass the kernel and show at the application through the AF_XDP socket interface
- Path B: packets will go through the kernel

Consequence: I can get kernel features for some packets and high-speed processing for others

AF_XDP split processing



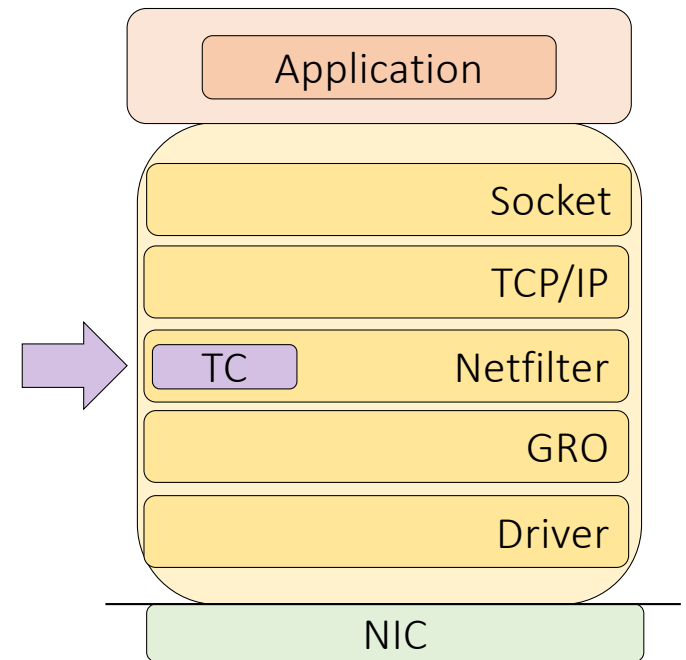
- The XDP program can steer packets through two different paths
- Path A: packets bypass the kernel and show at the application through the AF_XDP socket interface
- Path B: packets will go through the kernel

Consequence: I can get kernel features for some packets and high-speed processing for others

Example: I can steer ICMP packets through the kernel and get ping working 😊

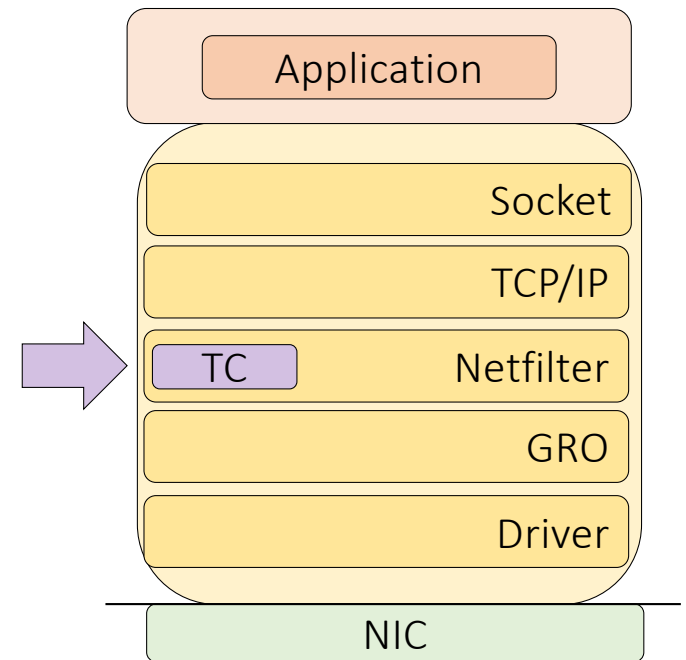
tc hook

- tc hook is located in the Traffic Control subsystem that helps in policing, classifying, shaping, and scheduling network traffic.
- An eBPF program attached to the tc hook has access to the `sk_buff` structure and can change things such as:
 - Priority
 - Queue mapping
 - Classid (to define a queue discipline)



tc hook

- Access to `sk_buff` is useful, but comes with an associated cost of the stack performing this allocation and metadata extraction
- The input context is a `sk_buff` and not a `xdp_buff` (packet) as for XDP

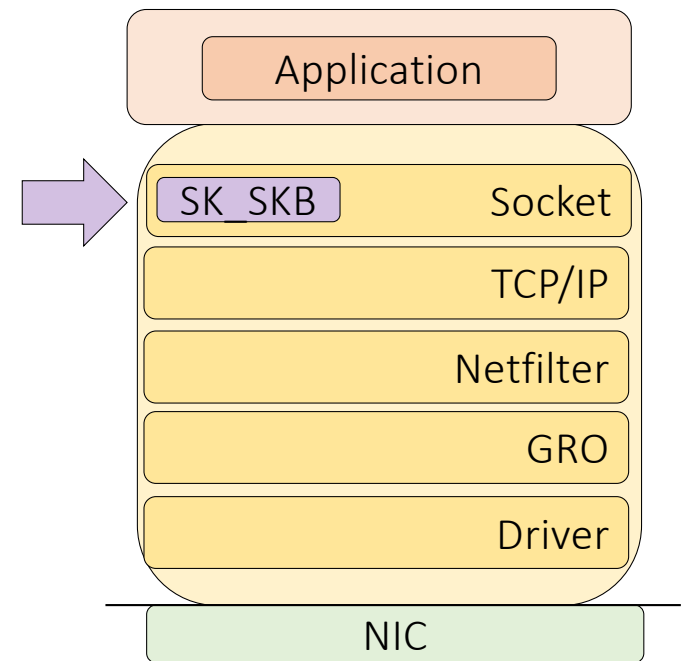


SK_SKB hook

- SK_SKB is located at the socket layer after the TCP/IP processing

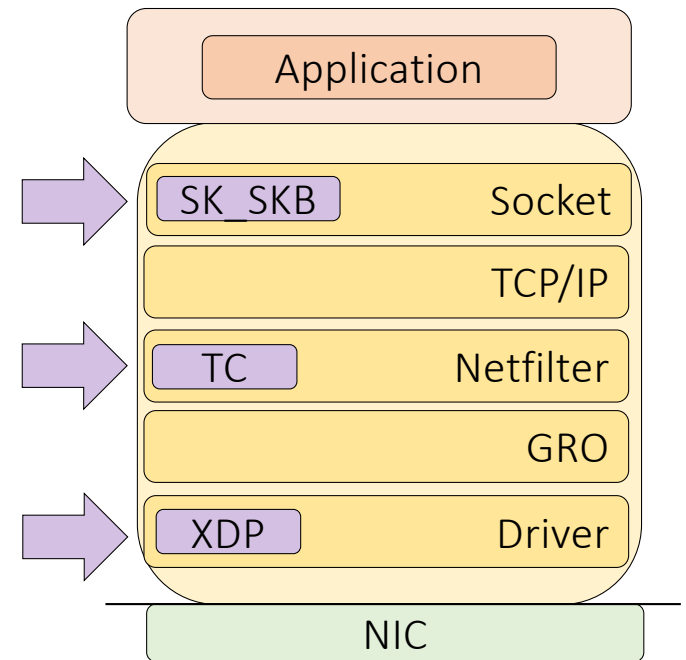
Consequence: eBPF programs attached to this hook access to segments (not packets)

- Allows to access skb and socket-related details
- This is a good hook for application-acceleration as you do not break TCP



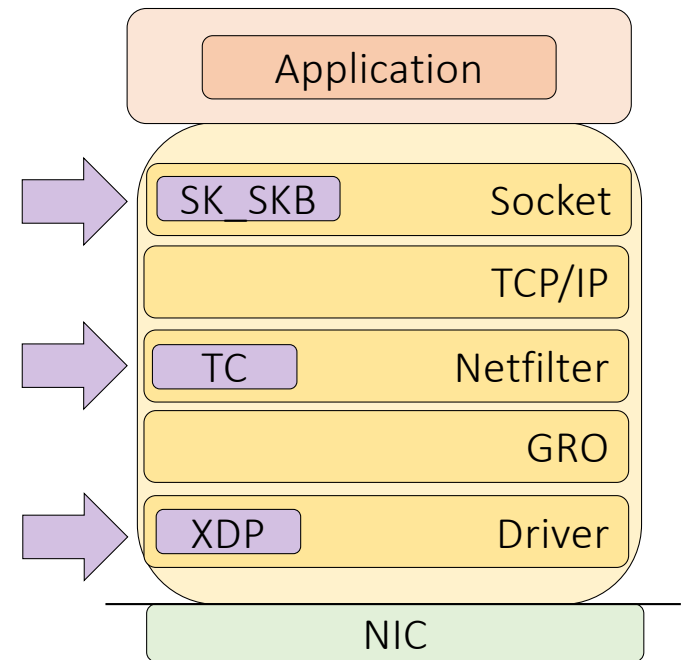
XDP vs tc vs SK_SKB

- XDP process packets before the network stack
 - Max performance as this is before skb allocation
 - Dropping or modifying TCP can corrupt connection state



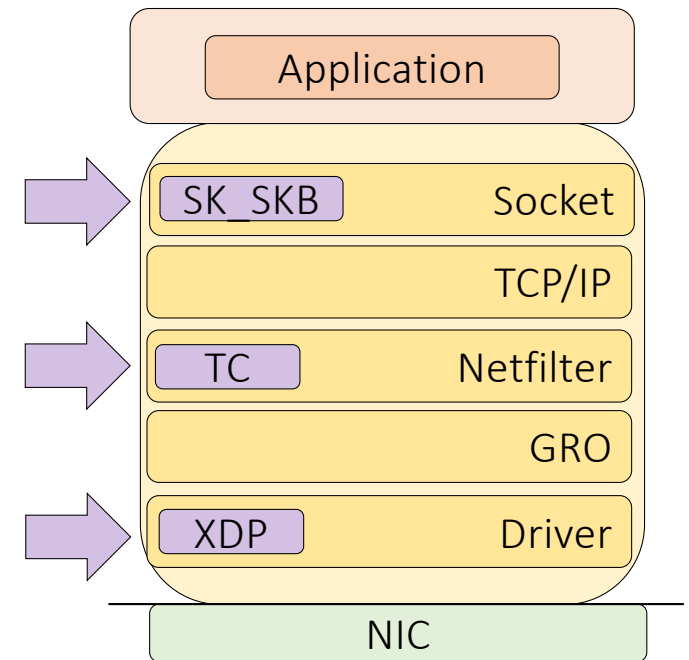
XDP vs tc vs SK_SKB

- XDP process packets before the network stack
 - Max performance as this is before skb allocation
 - Dropping or modifying TCP can corrupt connection state
- tc process packets after skb allocation
 - Access to all skb information
 - Not as performant as XDP



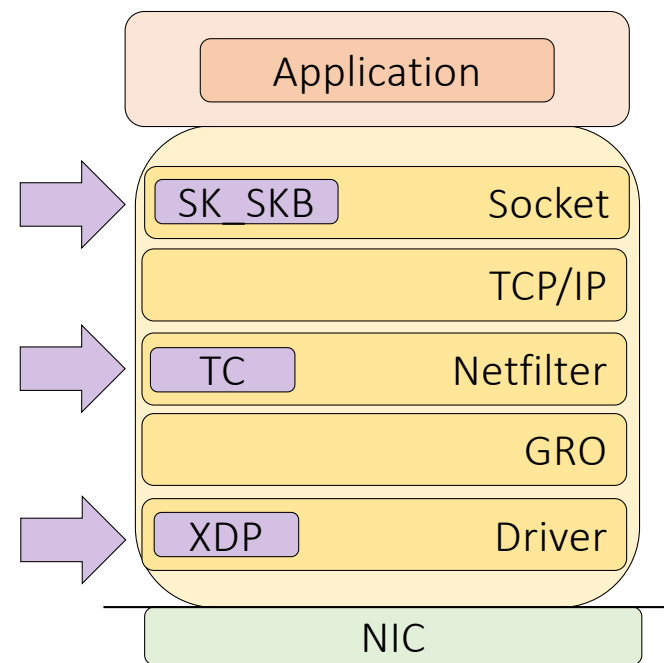
XDP vs tc vs SK_SKB

- XDP process packets before the network stack
 - Max performance as this is before skb allocation
 - Dropping or modifying TCP can corrupt connection state
- tc process packets after skb allocation
 - Access to all skb information
 - Not as performant as XDP
- SK_SKB acts on segments after TCP/IP processing
 - This is before the memory copy to user-space
 - Good fit for application acceleration



XDP vs tc vs SK_SKB

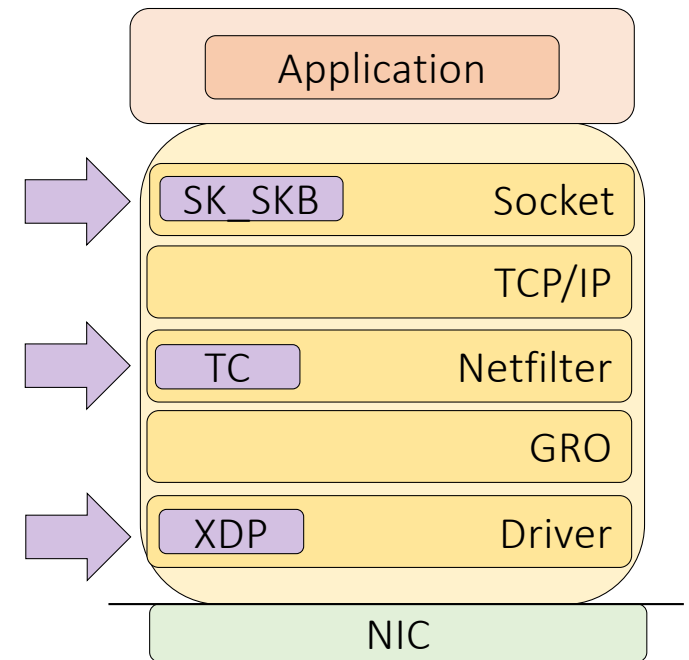
Hook	Time to reach (ns)	Prep. (ns)
XDP	0	38
TC	273 ± 51	35
SK_SKB	1072 ± 65	1350
Socket	5738 ± 779	-



XDP vs tc vs SK_SKB

Hook	Time to reach (ns)	Prep. (ns)
XDP	0	38
TC	273 ± 51	35
SK_SKB	1072 ± 65	1350
Socket	5738 ± 779	-

Takeaway: the time to reach a hook scales with the complexity of the OS network data path before the hook itself. The time to prepare the eBPF environment is, instead, hook-dependent



eBPF helpers

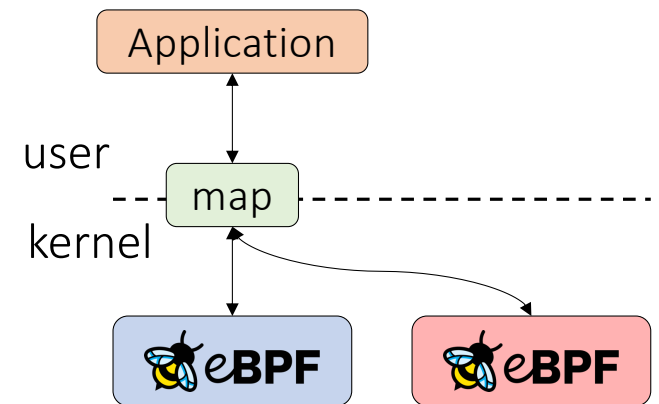
- eBPF assembly instructions are limited for safety reasons
 - We may need to perform some tasks not natively allowed in eBPF
- Solution: helpers!
 - Functions that are implemented natively in the Linux kernel, which are available as an assembly call
 - <https://github.com/iovisor/bcc/blob/master/src/cc/export/helpers.h>
- Examples: hash, longest-prefix-match tries, printk

Storing data in eBPF

- eBPF adopts preformatted memory, instead of unstructured vanilla memory pages
- Data access arbitrated by structures called *maps*
 - Key/value storage of different types
 - Array, HashMap, LRUMap..
- Interaction with maps is possible through helpers, such as
 - *bpf_map_lookup_elem()*
 - *bpf_map_update_elem()*
 - *bpf_map_delete_elem()*

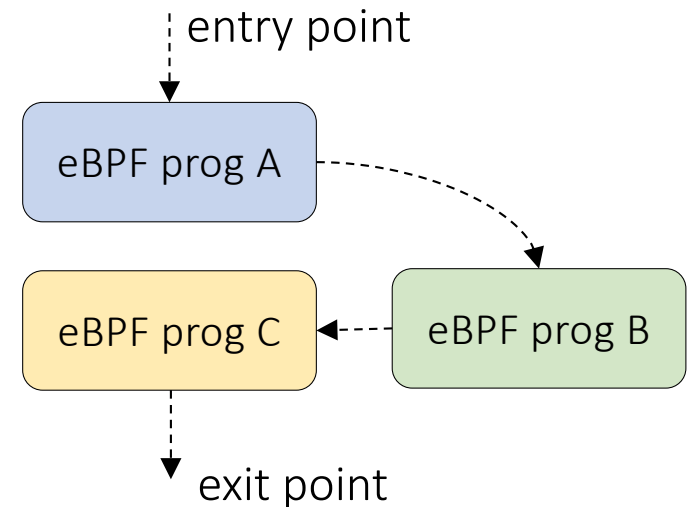
Storing data in eBPF

- Maps can be used to:
 - Export data from kernel to user space (e.g., kernel updates statistics about network traffic)
 - Data pushed by user space to kernel (e.g., user application configures and eBPF program behavior, for example a routing table)
 - Data shared between different eBPF programs (e.g., packet parser, sharing pointers to relevant protocol fields to all following module)
- Note: there are other shared memory communication channels (i.e., ring buffers, perf buffers)



Tail calls

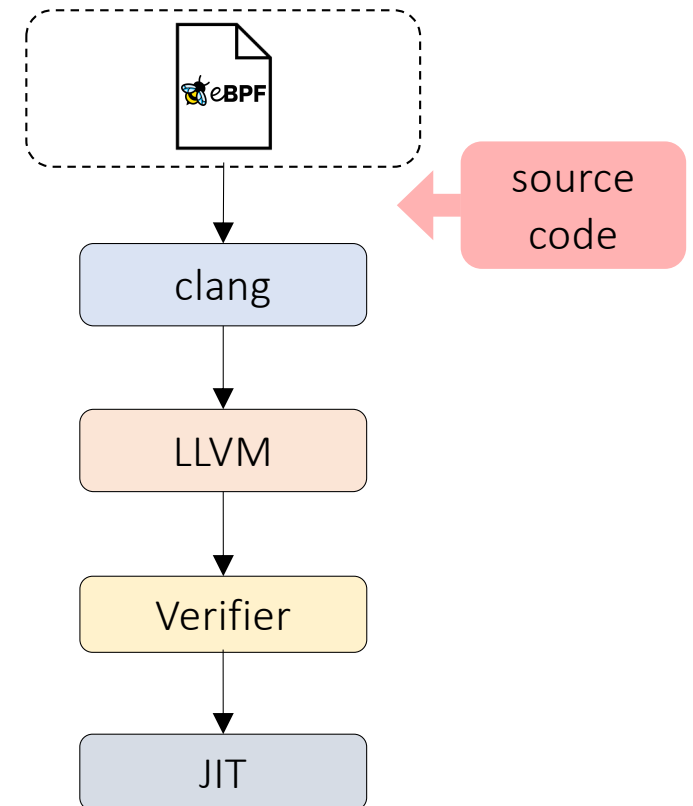
- This is a mechanism to allow an eBPF program to call another one
- This imposes minimal overhead, and it is implemented as a long jump, reusing the same stack frame
- Programs are verified independently
- Only programs of the same type can be tail called



eBPF toolchain

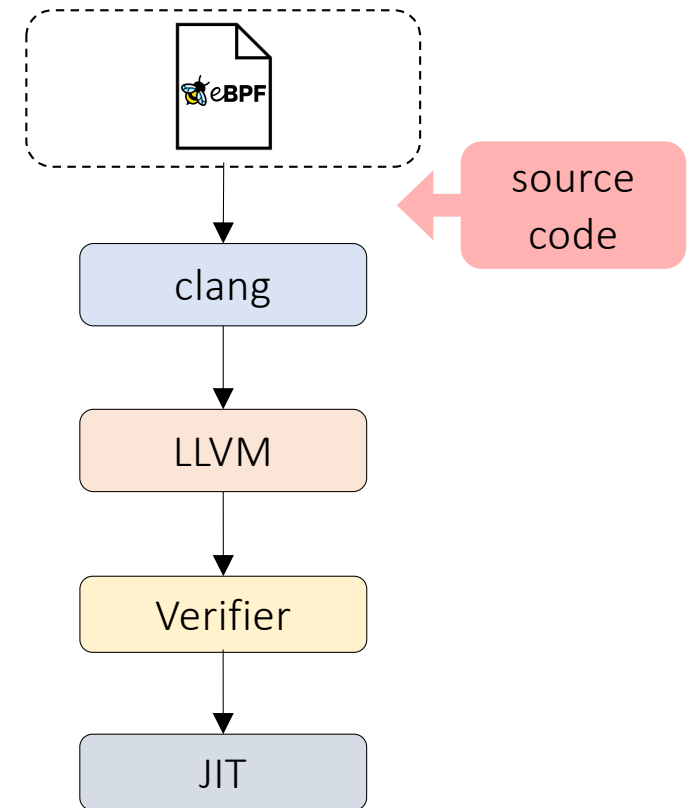
- eBPF code is natively written in restricted C

```
1 static __always_inline int parse_ethhdr(void *data, void *data_end,  
2                                         __u16 *nh_off,  
3                                         struct ethhdr **ethhdr) {  
4     struct ethhdr *eth = (struct ethhdr *)data;  
5     int hdr_size = sizeof(*eth);  
6  
7     /* Byte-count bounds check; check if current pointer + size of header  
8      * is after data_end.  
9      */  
10    if ((void *)eth + hdr_size > data_end)  
11        return -1;  
12  
13    *nh_off += hdr_size  
14    *ethhdr = eth;  
15  
16    return eth->h_proto; /* network-byte-order */  
17 }
```



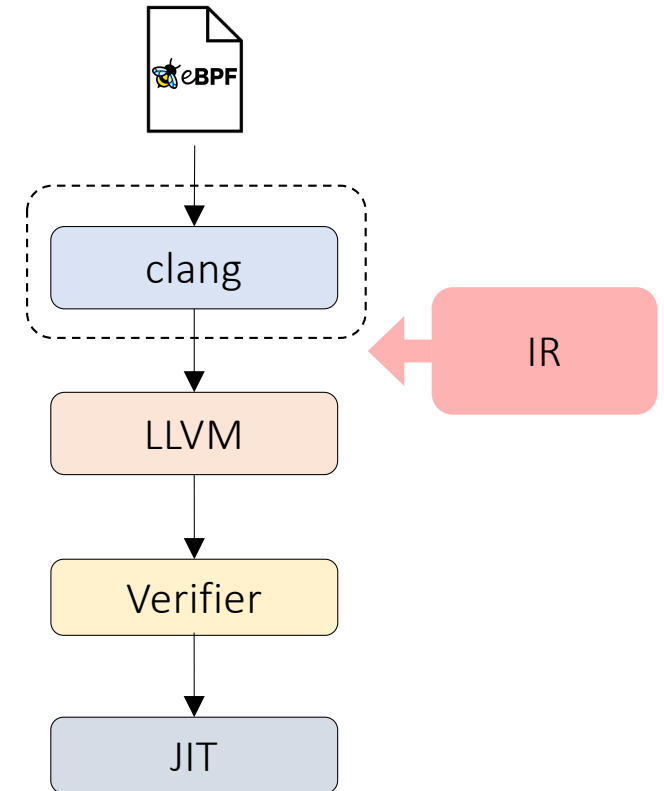
eBPF toolchain

- eBPF code is natively written in restricted C
- Compilers for other languages exists
 - To generate eBPF bytecode
 - To interact with the running eBPF program



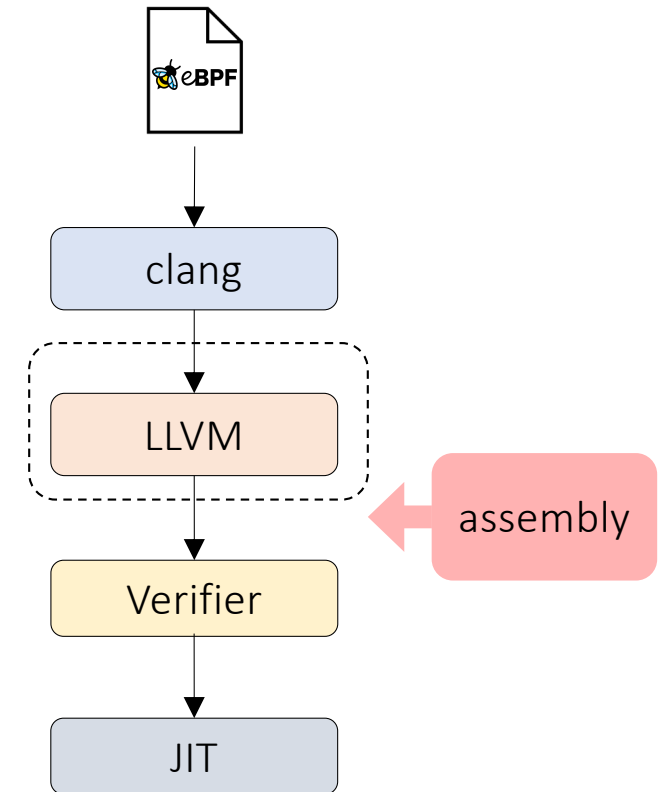
eBPF toolchain

- The code gets processed by Clang, a compiler front end for C-style programming languages
 - clang is a drop-in replacement of GCC
 - eBPF is also supported by GCC, but clang is the most commonly used



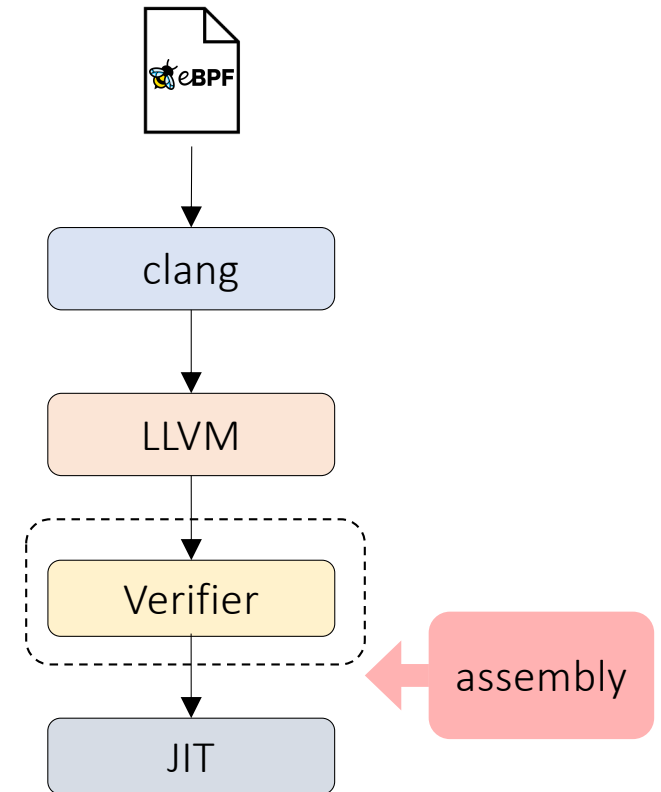
eBPF toolchain

- The code gets processed by Clang, a compiler front end for C-style programming languages
 - clang is a drop-in replacement of GCC
 - eBPF is also supported by GCC, but clang is the most commonly used
- The Intermediate Representation (IR) generated by Clang goes to the Low Level Virtual Machine (LLVM), a backend for assembly code generation and optimization



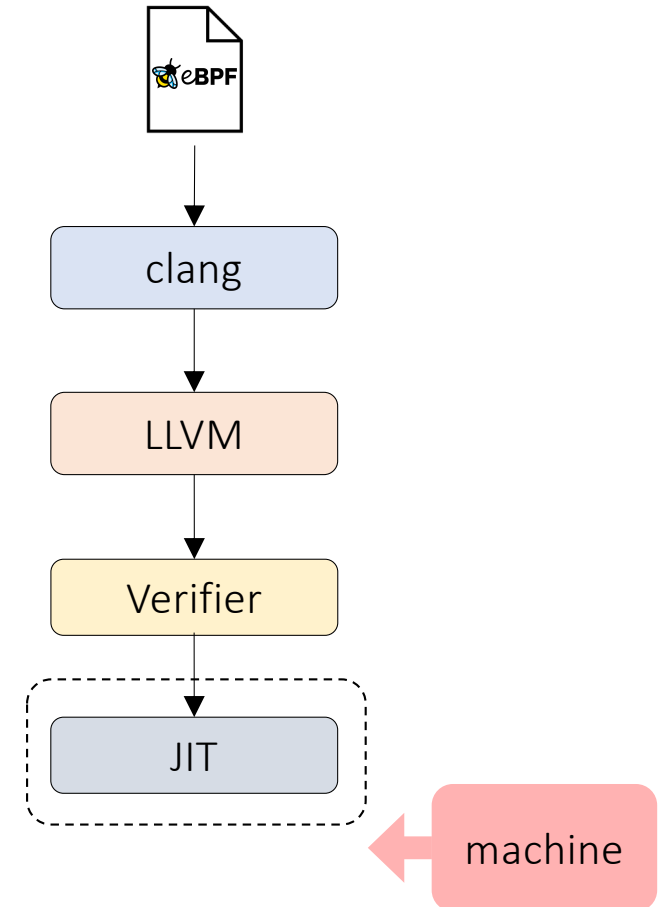
eBPF toolchain

- The assembly code is then checked for safety (we do not want the code to harm kernel behavior)



eBPF toolchain

- The assembly code is then checked for safety (we do not want the code to harm kernel behavior)
- As a final step, the verified assembly code goes through a Just-In-Time compiler for machine code generation
 - JIT compiler is the default option nowadays
 - The JIT compiler is more an ahead-of-time compiler rather than a JIT



Facts for the most curious ones

- The eBPF foundation is the entity that control eBPF

Platinum



Silver

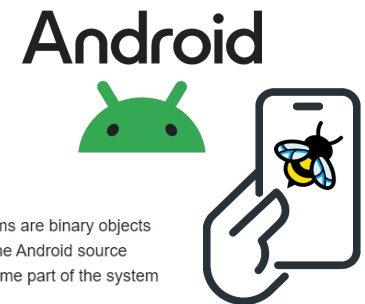


Facts for the most curious ones

- The eBPF foundation is the entity that control eBPF
- There is also an Android BPF loader

Android BPF loader

During Android boot, all eBPF programs located at `/system/etc/bpf/` are loaded. These programs are binary objects built by the Android build system from C programs and are accompanied by `Android.bp` files in the Android source tree. The build system stores the generated objects at `/system/etc/bpf`, and those objects become part of the system image.



Facts for the most curious ones

- The eBPF foundation is the entity that control eBPF
- There is also an Android BPF loader
- BPF is not just a Linux thing



Facts for the most curious ones

- The eBPF foundation is the entity that control eBPF
- There is also an Android BPF loader
- BPF is not just a Linux thing
- You can also build Tetris in BPF!

